

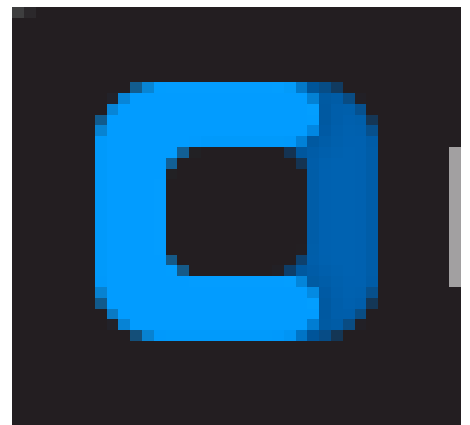
ספר פרויקט

תיכון אלון לאמנויות ולמדעים רמת השרון



- שם בית ספר: תיכון אלון רמת השרון
- שם העבודה: FlowChat – WhatsApp
- שם התלמיד: יונתן ברק
- ת.ז. התלמיד: 328146725
- שם המנחה: עידן פלדברג
- שם החלופה: הגנת סייבר
- תאריך הגשה: 01.06.25

FlowChat



תוכן

3.....	מבוא:
3.....	ייזום:
5.....	פירוט תיאור המערכת(אפיון):
9.....	תיאור תחום הידע – פרק מילולי:
9.....	פירוט מעמיק של היכולות שהוצגו בשלב הקודם ומעבר (ניתוח):
12.....	מבנה / ארכיטקטורה של הפרויקט:
12.....	תיאור הארכיטקטורה של המערכת המוצעת:
13.....	תיאור הטכנולוגיות הרלוונטיות:
15.....	תיאור זרימת המידע במערכת:
19.....	תיאור האלגוריתמים המרכזיים בפרויקט:
26.....	תיאור סביבת הפיתוח:
28.....	תיאור פרוטוקול התקשורת:
40.....	תיאור מסכי המערכת:
47.....	תיאור מבני הנתונים:
52.....	סקירת חולשות ואיומים:
55.....	מימוש הפרויקט:
68.....	חלק ב' - פירוט יכולות וקטעי קוד מרכזיים:
71.....	מסמך בדיקות מלא:
74.....	מדריך למשתמש:
74.....	פירוט כלל קבצי המערכת – עץ קבצים:
75.....	התקנת המערכת:
76.....	משתמשי המערכת:
81.....	סיכום אישי / רפלקציה:
83.....	ביבליוגרפיה:
84.....	נספחים:

מבוא:

ייזום:

תיאור ראשוני של המערכת: המערכת שפותחה במסגרת פרויקט זה הינה אפליקציית מסרים מיידיים מאובטחת, המדמה בפונקציונליות שלה שירותים מוכרים כמו WhatsApp. המוצר המוגמר מאפשר למשתמשים להירשם, להתחבר, לנהל פרופיל אישי (כולל תמונת פרופיל), לשלוח ולקבל הודעות טקסט מוצפנות מקצה לקצה, להשתתף בשיחות פרטיות וקבוצתיות, וליצור ולנהל קבוצות.

הבחירה בפרויקט זה נבעה ממספר גורמים. ראשית, אפליקציית "WhatsApp" או אפליקציות צ'אט אחרות הן האפליקציות הכי שימושיות וחשובות בטלפון. לכן עניין אותי לחקור ולהבין עד כמה מסובך לממש אפליקציה כזאת בעצמי. יתר על כן, האפשרות לבנות מערכת מלאה, הכוללת הן צד לקוח והן צד שרת, הציבה אתגר משמעותי ומעניין. הרלוונטיות של אפליקציות מסרים מיידיים בעולם המודרני, וההזדמנות להתנסות בבניית גרסה מפושטת אך פונקציונלית שלהן, היוו מניע נוסף. האתגרים העיקריים שאני צופה בפרויקט כוללים:

- מימוש נכון ויעיל של הצפנה מקצה לקצה (E2EE) עבור שיחות פרטיות וקבוצתיות, תוך שימוש במנגנוני RSA להחלפת מפתחות AES להצפנת ההודעות.
- ניהול תקין של מספר לקוחות מחוברים בו-זמנית בצד השרת, תוך שימוש בתהליכים והבטחת אמינות העברת הודעות בין הלקוחות והשרת.
- פיתוח ממשק משתמש אינטואיטיבי וידידותי באמצעות customtkinter.
- ניהול מסד נתונים (SQLite) לאחסון פרטי משתמשים, הודעות, קבוצות ומפתחות ציבוריים.
- נדרשה התמודדות עם מורכבות הקוד.
- מניעת פרצות אבטחה.

הגדרת הלקוח: מערכת FlowChat מיועדת למשתמשים פרטיים המחפשים פלטפורמת תקשורת אישית ומאובטחת. המערכת שואפת לשרת קהל יעד המעוניין בתקשורת ממוקדת עם חברים, משפחה או קולגות, תוך שימת דגש על פרטיות ואבטחת השיחה. כל אדם המעריך את פרטיותו ומעוניין בחוויית צ'אט מוכרת אך מבוקרת יותר, יכול למצוא בה עניין.

הגדרת יעדים/מטרות: היעדים המרכזיים שלי בפיתוח הפרויקט היו- פיתוח אפליקציית מסרים מיידיים פונקציונלית במודל שרת-לקוח. מימוש תקשורת מאובטחת באמצעות הצפנה מקצה לקצה להודעות פרטיות וקבוצתיות. יצירת ממשק משתמש גרפי (GUI) נוח וקל לשימוש. אפשרות לניהול משתמשים (הרשמה, התחברות), אנשי קשר וקבוצות צ'אט. הבטחת העברת הודעות אמינה בין המשתמשים. עמידה בדרישות הפרויקט הכוללות שימוש בתכנות מונחה עצמים, תקשורת מבוססת סוקטים, תהליכים, גישה למערכת הקבצים ואבטחה.

בעיות תועלת וחסכוניות: מערכת זו אינה באה לפתור בעיה שאין לה פתרונות קיימים בשוק (קיימות אפליקציות מסרים רבות). עם זאת, התועלת המרכזית בפיתוח המערכת היא ההתנסות המעשית והלימודית בבניית מערכת מורכבת, תוך יישום עקרונות של תכנות רשת, אבטחת מידע והצפנה. התועלות למשתמש הקצה הפוטנציאלי הן: פלטפורמת תקשורת מאובטחת עם דגש על פרטיות. ממשק משתמש פשוט ומוכר המאפשר התמצאות מהירה. השירותים שהמערכת מספקת כוללים: הרשמה ויצירת חשבון, התחברות מאובטחת, שליחה וקבלה של הודעות טקסט מוצפנות בשיחות פרטיות וקבוצתיות, יצירת קבוצות והצטרפות לקבוצות, ניהול תמונות פרופיל ואייקונים לקבוצות. ניהול הגדרת הקבוצות כמו: הגדרת מנהלים, הוצאת משתמשים מהקבוצה, הוספת משתמשים לקבוצה ויצאה מהקבוצה.

סקירת פתרונות קיימים: שוק המסרים המידיים רווי באפליקציות פופולריות כגון, WhatsApp, Telegram ו-Facebook Messenger. אפליקציות אלו מציעות מגוון רחב של תכונות, כולל הצפנה מקצה לקצה (בדרגות שונות של יישום כברירת מחדל), שיחות קוליות ווידאו, שיתוף מדיה מתקדם ועוד. הפרויקט שלי לוקח השראה מפונקציונליות הליבה של יישומים אלו, בעיקר WhatsApp ומתמקד במימוש מערכת הודעות טקסט מאובטחת עם תמיכה בקבוצות. המטרה אינה להתחרות במוצרים מסחריים אלו, אלא לממש מערכת לימודית המדגימה הבנה ויישום של הטכנולוגיות שנדרשו.

סקירת טכנולוגית הפרויקט וקיום סייגים בהגדרתה: הטכנולוגיות בהן נעשה שימוש בפרויקט שלי הן טכנולוגיות מוכרות ומבוססות בעולם התוכנה. ליבת המערכת מפותחת בשפת Python הן בצד השרת והן בצד הלקוח. סייגים ומגבלות הפרויקט נובעות בעיקר מהיקפו כפרויקט אישי בית ספרי, ולכן אינו כולל את כל התכונות המתקדמות הקיימות במוצרים מסחריים.

תיחום הפרויקט: הפרויקט עוסק במספר תחומים מרכזיים, בהתאם לדרישות הקורס ובדומה לפירוט בפרויקט: "DaVinci Mail"

- תכנות מונחה עצמים: המערכת בנויה ממחלקות שונות, NetworkNode, ChatServer, ClientHandler, ChatClientApp ו-DatabaseManager ומחלקות GUI נוספות בצד הלקוח (המנהלות היבטים שונים של הפרויקט).
- תקשורת רשת: מימוש שרת מרובה לקוחות ולקוח המתקשרים באמצעות סוקטים בפרוטוקול TCP פותח פרוטוקול הודעות פנימי מבוסס JSON להעברת פקודות ונתונים בין השרת ללקוח.
- מערכות הפעלה בהיבט של תהליכונים וקבצים: שימוש בתהליכונים בצד השרת לטיפול מקבילי בלקוחות. גישה למערכת הקבצים לשמירה וטעינה של מפתחות RSA תמונות פרופיל ואייקונים של קבוצות.
- אבטחה: הצפנת סיסמאות משתמשים במסד הנתונים באמצעות גיבוב. הצפנה מקצה לקצה (E2EE) של הודעות פרטיות וקבוצתיות תוך שימוש ב-RSA להעברת מפתחות AES, וב-AES להצפנת תוכן ההודעות. טיפול בסיסי בפרצות אבטחה פוטנציאליות.
- ממשק משתמש: פיתוח ממשק משתמש גרפי אינטראקטיבי באמצעות customtkinter.
- מודי נתונים: שימוש ב-SQLite לאחסון נתונים מובנים של משתמשים, הודעות וקבוצות.

הפרויקט אינו עוסק בתחומים הבאים:

- פיתוח Web כגון HTML, JavaScript, CSS.

- תכונות מתקדמות של אפליקציות מסרים כמו שיחות קול/וידאו, העברת קבצים מורכבת (מלבד תמונות פרופיל/קבוצה), מנגנוני סנכרון מתקדמים בין מכשירים מרובים, או אינטגרציה עם שירותים חיצוניים.
- מערכות ניהול תוכן מורכבות או אלגוריתמים מתקדמים לניתוח התנהגות משתמשים.

פירוט תיאור המערכת(אפיון):

תיאור מפורט של המערכת, מה היא אמורה לעשות? המערכת בפרויקט שלי הינה אפליקציית מסרים מיידיים, המאפשרת למשתמשים לתקשר זה עם זה באופן מאובטח באמצעות הודעות טקסט מוצפנות מקצה לקצה. המערכת מבוססת על ארכיטקטורת שרת-לקוח, כאשר השרת מנהל את הקישוריות בין הלקוחות, אימות המשתמשים ואחסון מידע חיוני, ואילו אפליקציית הלקוח מספקת את ממשק המשתמש הגרפי (GUI) ומטפלת בהצפנה ופענוח ההודעות בצד המשתמש. המערכת מאפשרת תקשורת מאובטחת, אם כי היא הינה מוגבלת לרשת פנימית. האפליקציה מאפשרת למשתמשים לבצע את הפעולות הבאות: הרשמה ויצירת חשבון: משתמשים חדשים יכולים ליצור חשבון על ידי בחירת שם משתמש וסיסמה. הודעות אישיות בין משתמשים וגם פתיחת קבוצות של מספר משתמשים. ניהול הגדרות משתמש והגדרות קבוצה. שמירת על פרטיות הסיסמה וההודעות.

פירוט היכולות שהיא תעניק לכל סוג משתמש במערכת:

- הרשמה למערכת ויצירת חשבון אישי מאובטח.
- התחברות מאובטחת למערכת עם שם משתמש וסיסמה.
- התנתקות מהמערכת.
- העלאה ושינוי של תמונת פרופיל אישית.
- איתור משתמשים אחרים במערכת.
- יצירת שיחה פרטית מוצפנת מקצה לקצה עם משתמש אחר.
- שליחה וקבלה של הודעות טקסט מוצפנות בשיחות פרטיות.
- יצירת קבוצת צ'אט חדשה והגדרת שמה.
- הוספת חברים לקבוצה (על ידי יוצר/מנהל הקבוצה).
- הסרת חברים מקבוצה (על ידי יוצר/מנהל הקבוצה).
- שינוי תפקיד של חבר בקבוצה (הפיכה למנהל/הורדה ממנהל, על ידי מנהל).
- שינוי שם הקבוצה (על ידי מנהל).
- העלאה ושינוי של אייקון לקבוצה (על ידי מנהל).
- עזיבת קבוצה.
- שליחה וקבלה של הודעות טקסט מוצפנות בשיחות קבוצתיות.
- הודעות מערכת שמראות על התרחשויות בקבוצה (שינוי שם, יצירה, שינוי אייקון הקבוצה ועוד..)
- צפייה בהיסטוריית השיחות (פרטיות וקבוצתיות).

פירוט הבדיקות (קופסא שחורה) שמתוכננות לפרויקט:

1. **בדיקת הרשמה:** וידוא יכולת הרשמה של משתמש חדש למערכת, שמירת פרטיו באופן מאובטח (סיסמה מגובבת, מפתח ציבורי), ויצירת מפתחות RSA מקומיים ללקוח. נבצע את

הבדיקה כך: ניסיון הרשמה עם פרטים תקינים (שם משתמש פנוי, סיסמה). מצופה: הרשמה מוצלחת, הודעה מתאימה, יכולת להתחבר עם המשתמש החדש. בדיקה שנוצרו קבצי מפתחות (client_public_key.pem, client_private_key.pem) אצל הלקוח. ניסיון הרשמה עם שם משתמש שכבר קיים במערכת. מצופה: הודעת שגיאה המציינת שהשם תפוס. ניסיון הרשמה עם שדות חסרים. מצופה: הודעת שגיאה המציינת אילו שדות נדרשים. בדיקה נוספת: לאחר הרשמה, בדיקה במסד הנתונים של השרת שהמשתמש נוצר, סיסמתו אוחסנה בצורה מגובבת והמפתח הציבורי שלו נשמר.

2. **בדיקת התחברות (Login):** וידוא יכולת התחברות של משתמש רשום, נדרש, וקבלת היסטוריית הודעות. את הבדיקה נבצע כך: ניסיון התחברות עם שם משתמש וסיסמה נכונים. מצופה: התחברות מוצלחת, מעבר למסך הראשי, טעינת היסטוריית הודעות רלוונטית (ופענוחה במידת הצורך). ניסיון התחברות עם שם משתמש נכון וסיסמה שגויה. מצופה: הודעת שגיאה "שם משתמש או סיסמה שגויים". ניסיון התחברות עם שם משתמש לא קיים. מצופה: הודעת שגיאה "שם משתמש או סיסמה שגויים".

3. **בדיקת שליחת וקבלת הודעות פרטיות מאובטחות:** מהות הבדיקה: וידוא תקינות תהליך אתחול ששן מאובטח באמצעות ושליחה/קבלה של הודעות מוצפנות AES בין שני משתמשים. אופן הביצוע: שליחת הודעות בין משתמשים כאשר שניהם מחוברים ובדיקה אם רואים את ההודעות והן מתעדכנות בזמן אמת. בדיקה כאשר אחד המשתמשים לא מחובר למערכת. בנוסף ניתן לבדוק בקובץ בסיס הנתונים שהודות לא נשמרות כטקסט רגיל אלא כמוצפן כך שבסיס הנתונים לא יוכל לדעת את משמעותן. בדיקת ההודעות בעזרת כלי הסנפה שיבדוק אם ניתן לוודא שתוכן ההודעה מוצפן.

4. **בדיקת שליחת הודעות קבוצות מאובטחות:** מהות הבדיקה - וידוא תקינות יצירת קבוצות ושליחה/קבלה של הודעות מוצפנות בקבוצה. אופן הביצוע: משתמש א' יוצר קבוצה ומוסיף את משתמש ב' ו-ג'. מצופה: מפתח AES קבוצתי נוצר ומופץ באופן מאובטח לחברים. משתמש א' שולח הודעה לקבוצה. מצופה: כל החברים מקבלים ומפענחים את ההודעה.

5. **בדיקת הגדרות הקבוצה:** מהות הבדיקה – בדיקת כל ההגדרות השונות לקבוצה. אופן ביצוע: בדיקת כל פעולה ובדיקה אם הודעות המערכת שמדווחות על פעולות הקבוצה קיימות.

6. **בדיקת ניהול פרופיל ותמונות:** מהות הבדיקה - וידוא יכולת העלאה ועדכון של תמונת פרופיל ותמונת קבוצה. אופן הביצוע: משתמש מעלה תמונת פרופיל. מצופה: התמונה מתעדכנת ומוצגת נכון. מנהל קבוצה מעלה אייקון לקבוצה. מצופה: האייקון מתעדכן. ניסיון להעלות קובץ שאינו תמונה. מצופה: הודעת שגיאה.

7. **בדיקת טיפול בשגיאות וקישוריות:** מהות הבדיקה- וידוא שהמערכת מטפלת באופן הולם בתקלות תקשורת, התנתקויות ושגיאות נפוצות. התנתקות פתאומית של לקוח מהרשת.

מצופה: הודעה מתאימה, שמירת מצב אם אפשרי. כיבוי השרת בזמן שלקוחות מחוברים.
מצופה: הודעת שגיאה בלקוחות. ניסיון לשלוח הודעה למשתמש לא מקוון. מצופה: הודעה מאוחסנת בשרת ונמסרת כשהמשתמש מתחבר.

8. **בדיקת ממשק משתמש: (GUI)** מהות הבדיקה- וידוא שהממשק הגרפי פועל כשורה, קל לניווט ואינטואיטיבי. אופן הביצוע- בדיקת כל הכפתורים, שדות הקלט, רשימות נגללות ומעברים בין מסכים. וידוא שהמידע (הודעות, שמות, תמונות) מוצג בצורה ברורה ובמקום הנכון. בדיקת תגובתיות הממשק ועדכונים דינמיים (למשל, קבלת הודעה חדשה).

תכנון וניהול לו"ז זמנים לפיתוח המערכת:

משימה	פירוט תמציתי	זמן מתוכנן	זמן ביצוע בפועל (למילוי)
שלב א: תכנון ומחקר ראשוני	בחירת נושא הפרויקט, הגדרת דרישות ותכנון ארכיטקטורה ראשונית.	שבוע	שבוע
שלב ב: פיתוח ליבות מערכת ומסד נתונים	מימוש ליבות שרת ולקוח בסיסיים (תקשורת, GUI, הקמת מסד נתונים (SQLite)).	3 שבועות	3 שבועות
שלב ג: מימוש הצפנה (E2EE)	שילוב RSA ו AES-להצפנת הודעות פרטיות וניהול מפתחות.	שבוע	שבוע וחצי
שלב ד: פונקציונליות קבוצות	מימוש צ'אט קבוצתי, ניהול חברים בסיסי והצפנת E2EE קבוצתית.	שבועיים	שבוע וחצי
שלב ה: שיפורי GUI ותכונות נוספות	ליטושי ממשק משתמש, הוספת ניהול תמונות פרופיל ואייקוני קבוצות.	שבוע	שבוע
שלב ו: בדיקות ותיקונים	ביצוע בדיקות מקיפות לאיתור ותיקון באגים ושיפור יציבות המערכת.	שבוע	שבוע
שלב ז: כתיבת ספר פרויקט	כתיבת כל חלקי ספר הפרויקט, הכנת תרשימים ותיעוד הקוד.	שבועיים	שבוע

ניהול הסיכונים בפרויקט והדרכים להתמודד איתם: במהלך הפרויקט זוהו מספר סיכונים מרכזיים, והוגדרו דרכי התמודדות עבורם. כולם חלק מדרכי ההתמודדות היו

1. מורכבות מימוש הצפנה. קושי ביישום נכון של ניהול מפתחות והצפנה אסימטרית וסימטרית, במיוחד עבור קבוצות.

○ דרכי התמודדות מתוכננות: מחקר מעמיק של RSA ו AES.

- ביצוע בפועל: תחילה היה קשה וקרו לי מספר בעיות בפיענוח הודעות כאשר אחד מהמשתמשים לא מחובר. חשבתי עוד והתייעצתי עם חבר שעזר לי לפתור את הבעיה.
- 2. טיפול בשגיאות ובתרחישי קצה. קושי לצפות ולטפל בכל שגיאות התקשורת, ניתוקים פתאומיים או קלט לא תקין מהמשתמש.
- דרכי התמודדות מתוכננות: שימוש במנגנוני try-except לטיפול בחריגות, מתן משוב ברור למשתמש על שגיאות.
- ביצוע בפועל: בחינת המקרים ומתי צריך לבצע בדיוקת, חשיבה לעומק על מקרי קצה.
- 3. אי עמידה בזמני הפרויקט ותכנון הזמנים.
- דרכי התמודדות מתוכננות: הקדשת זמן לחשיבה לזמני כל חלק בפרויקט.
- ביצוע בפועל: חשיב על מה הדברים החשובים שהכי הכרחיים לפרויקט ואם יש זמן להוסיף אפשרויות נוספות נוספים

תיאור תחום הידע – פרק מילולי

פירוט מעמיק של היכולות שהוצגו בשלב הקודם ומעבר (ניתוח):

יכולות צד לקוח:

שם היכולת	מהות היכולת – מה היא מבצעת בפועל	אוסף היכולות/פעולות הנדרשות למימוש היכולת	אובייקטים נחוצים
הרשמה למערכת	רישום משתמש חדש במערכת, כולל יצירת זהות קריפטוגרפית אישית ושליחת הפרטים לאישור השרת.	הצגת מסך הרשמה, קליטת פרטי משתמש (שם וסיסמה), יצירת מפתחות הצפנה אישיים (זוג מפתחות א-סימטרי), שמירת מפתחות מאובטחת במחשב הלקוח, הכנת בקשת הרשמה לשרת (כולל מפתח ציבורי), שליחת הבקשה לשרת, קבלת תשובת השרת (הצלחה או כישלון), הצגת משב למשתמש על תוצאת ההרשמה.	ממשק משתמש, מנגנון הצפנה, רכיב תקשורת, רכיב ניהול קבצים
התחברות למערכת	אימות משתמש קיים מול השרת, טעינת סביבת העבודה האישית שלו וכניסה למערכת.	הצגת מסך התחברות, קליטת פרטי משתמש (שם וסיסמה), טעינת מפתחות הצפנה אישיים מהאחסון המקומי, הכנת בקשת התחברות לשרת (כולל מפתח ציבורי), שליחת הבקשה לשרת, קבלת תשובת השרת, עיבוד תשובת השרת וטעינת נתוני משתמש (כגון היסטוריית שיחות) במקרה של הצלחה, הצגת משב למשתמש.	ממשק משתמש, רכיב תקשורת, רכיב ניהול קבצים, מנגנון הצפנה
שליחת הודעה פרטית מאובטחת	שליחת הודעת טקסט פרטית ומוצפנת מקצה לקצה למשתמש אחר.	ממשק משתמש – בחירת נמען והקלדת הודעה, אחזור או בקשת מפתח הצפנה של הנמען, יצירה והחלפה מאובטחת של מפתח שיחה (סימטרי), הצפנת תוכן ההודעה, שליחת ההודעה המוצפנת לשרת, עדכון תצוגת השיחה אצל השולח.	ממשק משתמש, מנגנון הצפנה, רכיב תקשורת, ניהול מפתחות
קבלת הודעה פרטית מאובטחת	קבלת הודעה פרטית מוצפנת, פענוחה באמצעות המפתחות המתאימים, והצגתה למשתמש.	קבלת הודעה מוצפנת מהשרת, זיהוי השולח וסוג ההודעה (אתחול שיחה או הודעה קיימת), פענוח מפתח השיחה (במקרה של אתחול שיחה) באמצעות מפתח פרטי, פענוח תוכן ההודעה באמצעות מפתח השיחה, הצגת ההודעה המפוענחת למשתמש.	רכיב תקשורת, מנגנון הצפנה, ניהול מפתחות, ממשק משתמש
יצירת קבוצת צ'אט חדשה	יצירת קבוצת צ'אט חדשה, כולל הגדרות ראשוניות ויצירת מפתח הצפנה קבוצתי והפצתו לחברים.	ממשק משתמש – הגדרת שם קבוצה ובחירת חברים, שליחת בקשת יצירת קבוצה לשרת, קבלת אישור ומזדה קבוצה מהשרת, יצירת מפתח הצפנה קבוצתי (סימטרי), שמירת המפתח הקבוצתי מקומית, הפצת המפתח הקבוצתי (באופן מוצפן) לחברים דרך השרת, עדכון ממשק המשתמש של היוצר.	ממשק משתמש, רכיב תקשורת, מנגנון הצפנה, ניהול מפתחות
שליחת הודעה	שליחת הודעת טקסט מוצפנת מקצה לקצה לחברי קבוצה ספציפית.	ממשק משתמש – בחירת קבוצה וכתובת הודעה, שליפת מפתח ההצפנה הקבוצתי מאחסון, הצפנת תוכן ההודעה באמצעות המפתח הקבוצתי, שליחת ההודעה המוצפנת	ממשק משתמש, מנגנון הצפנה, ניהול מפתחות, רכיב תקשורת

קבוצתית מאובטחת	לשרת (עם ציון הקבוצה), עדכון ממשק המשתמש של השולח.		
העלאת תמונת פרופיל	מאפשר למשתמש לבחור ולהעלות תמונת פרופיל אישית לשרת.	ממשק משתמש – בחירת קובץ תמונה מהמחשב, עיבוד ראשוני של התמונה (כגון שינוי גודל), קידוד נתוני התמונה (למשל, לפורמט טקסטואלי), שליחת נתוני התמונה המעובדים לשרת, קבלת סטטוס מהשרת והצגת משוב למשתמש.	ממשק משתמש, רכיב עיבוד תמונה, מנגנון קידוד, רכיב תקשורת
ניהול הגדרות הקבוצה	מאפשר למשתמש לנהל את הקבוצה במגוון אפשרויות כגון: שינוי שם, שינוי אייקון הקבוצה, הוספה והסרה של אנשים.	ממשק משתמש המציע בהגדרות לבצע פעולות אלה. בקשות שונות שיודעות להתייחס לכל אפשרות שונה בהגדרות הקבוצה. עדכון הרשאות של חברי הקבוצה.	רכיב תקשורת, ממשק משתמש

יכולות צד שרת:

שם היכולת	מהות היכולת – מה היא מבצעת בפועל	אוסף היכולות/פעולות הנדרשות למימוש היכולת	אובייקטים נחוצים
טיפול בבקשת הרשמה	קליטה ואימות של בקשת הרשמה ממשתמש חדש, יצירת רשומת משתמש מאובטחת במסד הנתונים.	קבלת בקשת הרשמה מהלקוח (שם, סיסמה, מפתח ציבורי), בדיקת תקינות וזמינות שם המשתמש במסד הנתונים, גיבוב הסיסמה עם "מלח" שמירת פרטי המשתמש החדש במסד הנתונים, שליחת תשובת הצלחה או כישלון ללקוח.	רכיב טיפול בלקוח, גישה למסד נתונים, מנגנון גיבוב סיסמאות, לוגיקת שרת
טיפול בבקשת התחברות	אימות פרטי משתמש קיים מול מסד הנתונים, ניהול מצב החיבור שלו, ושליחת נתונים ראשוניים הדרושים ללקוח.	קבלת בקשת התחברות מהלקוח (שם, סיסמה, מפתח ציבורי עדכני), אחזור פרטי המשתמש המאוחסנים (גיבוב סיסמה, מלח) ממסד הנתונים, אימות הסיסמה שנשלחה מול הגיבוב השמור, במקרה של אימות מוצלח: עדכון מפתח ציבורי של המשתמש (במידת הצורך), רישום הלקוח כמחובר, אחזור היסטוריית הודעות ראשונית, שליחת תשובת הצלחה (כולל היסטוריה) ללקוח, במקרה של אימות כושל: שליחת תשובת כישלון ללקוח.	רכיב טיפול בלקוח, גישה למסד נתונים, מנגנון אימות סיסמאות, לוגיקת שרת (ניהול משתמשים מחוברים)

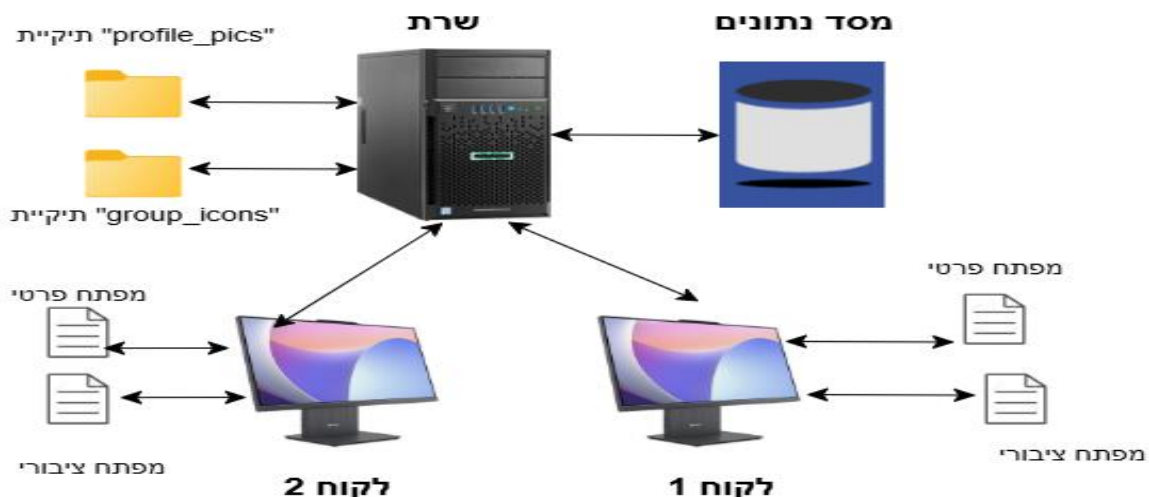
קבלת הודעה מוצפנת מלקוח שולח, זיהוי השולח והנמען המיועד, אחסון ההודעה המוצפנת (כפי שהיא) במסד הנתונים, כולל שמירת מפתח AES מוצפן-עצמית של השולח (להיסטוריה שלו) במקרה של אתחול ששן, בדיקה אם הנמען מחובר, אם הנמען מחובר: העברת ההודעה אליו, אם הנמען אינו מחובר: ההודעה תישמר למסירה עתידית (בעת התחברותו), משלוח אישור לשולח על מצב ההודעה (אופציונלי).	קבלת הודעה פרטית מוצפנת, אחסון מאובטח שלה, וניתוב לנמען המתאים או שמירתה למסירה מאוחרת.	אחסון וניתוב הודעה פרטית מאובטחת
קבלת בקשה ליצירת קבוצה מהלקוח (שם קבוצה, רשימת חברים ראשוניים), בדיקת תקינות הנתונים, יצירת רשומת קבוצה חדשה במסד הנתונים וקבלת מזהה קבוצה ייחודי, הוספת היוצר כמנהל הקבוצה וחברים נוספים למאגר הקבוצה במסד הנתונים, שליחת אישור יצירת קבוצה ללקוח היוצר.	טיפול בבקשת לקוח ליצירת קבוצת צ'אט חדשה, כולל רישום הקבוצה וחבריה הראשוניים במסד הנתונים.	טיפול בבקשת יצירת קבוצה
קבלת בקשה מלקוח (מנהל קבוצה) להפצת מפתח הצפנה קבוצתי (מוצפן) לחבר קבוצה ספציפי, אימות שהשולח הוא מנהל הקבוצה ושהנמען חבר בקבוצה, בדיקה אם הנמען מחובר, אם מחובר: העברת המפתח הקבוצתי המוצפן ללקוח הנמען, משוב לשולח לגבי הצלחת ההפצה (אופציונלי).	קבלת מפתח קבוצתי מוצפן המיועד לחבר קבוצה מסוים, וניסיונו להעבירו לאותו חבר במידה והוא מחובר.	טיפול בהפצת מפתח קבוצתי
קבלת הודעה קבוצתית מוצפנת (AES) מלקוח שולח, אימות שהשולח חבר בקבוצה המיועדת, אחסון ההודעה המוצפנת (כפי שהיא) במסד הנתונים, אחזור רשימת כל חברי הקבוצה ממסד הנתונים, שידור ההודעה המוצפנת לכל חברי הקבוצה המחברים כעת.	קבלת הודעה פרטית מוצפנת, המיועדת לקבוצה, אחסונה, ושידורה לכלל חברי הקבוצה המחברים.	אחסון ושידור הודעה קבוצתית מוצפנת
קבלת נתוני תמונה (בפורמט מקודד) ושם קובץ מקורי מהלקוח, פענוח נתוני התמונה, שמירת קובץ התמונה במערכת הקבצים של השרת (בתיקייה ייעודית) תוך יצירת שם קובץ ייחודי, עדכון מסד הנתונים (טבלת משתמשים) עם שם הקובץ החדש של תמונת הפרופיל, שליחת סטטוס ההעלאה ללקוח.	טיפול בבקשת לקוח להעלאת תמונת פרופיל, כולל שמירת הקובץ ועדכון הרשומה במסד הנתונים.	טיפול בהעלאת תמונת פרופיל
קבלת בקשה לביצוע פעולת ניהול על קבוצה (כגון הוספת/הסרת חבר, שינוי שם/אייקון קבוצה, שינוי תפקיד חבר), אימות הרשאות המשתמש השולח (האם הוא מנהל הקבוצה הרלוונטית), ביצוע השינויים הנדרשים במסד הנתונים, במקרה של העלאת אייקון: טיפול בשמירת הקובץ ועדכון מסד הנתונים, שליחת הודעת סטטוס ללקוח שביצע את הפעולה, (אופציונלי) שידור הודעת מערכת רלוונטית לכלל חברי הקבוצה לעדכון על השינוי שבוצע.	טיפול בפעולות ניהול שונות הקשורות לקבוצות צ'אט, כגון ניהול חברים, שינוי פרטי קבוצה, תוך אימות הרשאות ועדכון מסד הנתונים.	טיפול בפעולות ניהול קבוצה

מבנה / ארכיטקטורה של הפרויקט

תיאור הארכיטקטורה של המערכת המוצעת:

תיאור החומרה – רכיבים שונים והקשרים ביניהם: הפרויקט פועל על בסיס מספר רכיבי חומרה ותוכנה המקיימים אינטראקציה זה עם זה:

- מחשב השרת: זהו המחשב עליו רץ קוד השרת של האפליקציה. הוא מחובר לרשת LAN ומאזין לבקשות חיבור מלקוחות בכתובת IP ופורט מוגדרים. השרת מנהל את מסד הנתונים המרכזי (chat_app.db) ואת קבצי התמונות (תמונות פרופיל בתיקיית profile_pics וקבוצות בתיקיית group_icons).
- מחשבי הלקוח: אלו הם המחשבים עליהם מורצת אפליקציית הלקוח. הלקוח יוצר חיבור רשת אל השרת, שולח בקשות (כגון התחברות, שליחת הודעה) ומקבל תגובות. בצד הלקוח נשמרים מקומית המפתחות הפרטיים של המשתמש (client_private_key.pem) ומפתחות ההצפנה הקבוצתיים (group_aes_keys.json).
- רשת התקשורת: הרכיבים מתקשרים ביניהם באמצעות כתובת IP ופורט בדרך כלל על הרשת המקומית. בנוסף הפרוטוקול בו אני משתמש הוא TCP כיוון שמידע של הודעות חייב להגיע באופן מדויק ואמין. כך שנעדיף אמינות על מהירות במקרה זה. יתר על כן הקשרים בין הלקוחות לשרת הם קשרי בקשה-תגובה (request-response) וכן תקשורת דו-כיוונית להעברת הודעות בזמן אמת.
- הקשרים העיקריים בין הרכיבים:
 - לקוח <-> שרת: תקשורת דו-כיוונית להעברת בקשות, תגובות, הודעות צ'אט, מפתחות הצפנה ונתוני פרופיל/קבוצה.
 - שרת <-> מסד נתונים: השרת קורא וכותב נתונים למסד הנתונים המרכזי (פרטי משתמשים, הודעות, קבוצות וכו').
 - שרת <-> מערכת קבצים (שרת): השרת שומר וקורא קבצי תמונות.
 - לקוח <-> מערכת קבצים (לקוח): הלקוח שומר וקורא קבצי מפתחות הצפנה פרטיים וקבוצתיים.



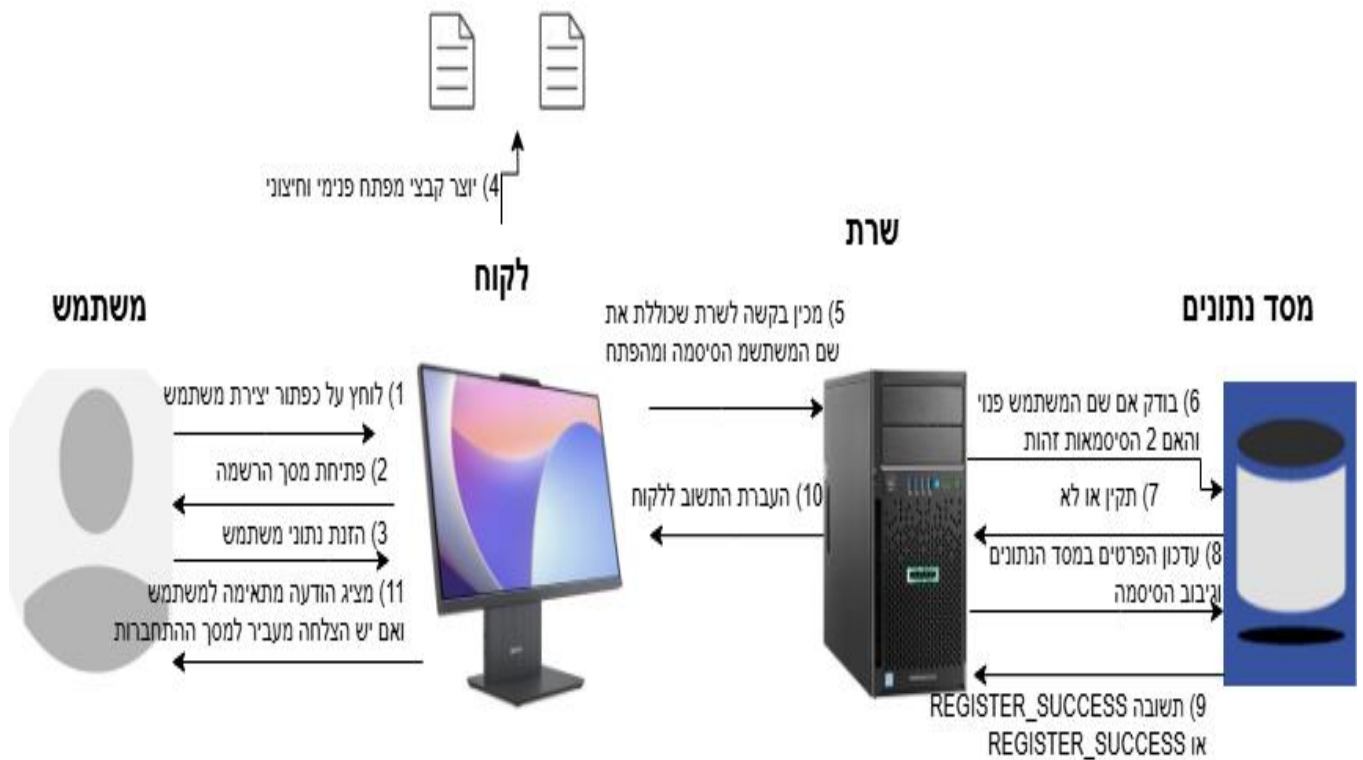
תיאור הטכנולוגיות הרלוונטיות:

- **שפת תכנות:** פייתון היא שפת התכנות המרכזית ששימשה לפיתוח הן של צד השרת והן של צד הלקוח של האפליקציה. פייתון נבחרה בזכות הפשטות היחסית של התחביר שלה, הקריאות הגבוהה של הקוד, והמגוון הרחב של ספריות זמינות המקלות על הפיתוח בתחומים כגון תקשורת רשת, קריפטוגרפיה, עבודה עם מסדי נתונים ופיתוח ממשקי משתמש.
- **מערכות הפעלה (מ"ה):**
 - **שרת:** קוד השרת של " יכול לרוץ על מגוון רחב של מערכות הפעלה התומכות בסביבת ריצה של פייתון, כולל Windows, Linux, macOS. הדבר מאפשר גמישות בבחירת סביבת ההרצה של השרת.
 - **לקוח:** אפליקציית הלקוח, שנכתבה באמצעות ספריית customtkinter המבוססת על Tkinter מיועדת לרוץ על מערכות הפעלה שולחניות נפוצות כגון Windows, macOS, וחלק מהפצות Linux התומכות בספריות הגרפיות הנדרשות.
- **תקשורת:**
 - **סוקטים:** התקשורת בין השרת ללקוחות מבוססת על מודול socket הסטנדרטי של פייתון. מודול זה מאפשר יצירת חיבורי רשת וניהול העברת נתונים.
 - **פרוטוקול TCP (Transmission Control Protocol):** נבחר כשכבת התעבורה להבטחת אמינות בהעברת הנתונים בין השרת ללקוח TCP. מבטיח שהודעות יגיעו ליעדן בסדר הנכון וללא אובדן מידע (באמצעות מנגנוני אישור ושליחה חוזרת). זאת מכיוון שאמינות המידע חשובה והכרחית לאפליקציה.
 - **JSON (JavaScript Object Notation):** פורמט ההודעות המוחלפות בין השרת ללקוח. כל הודעה, על סוגה והנתונים הנלווים אליה, נארזת כאובייקט JSON, מומרת למחרוזת, מקודדת לבתים ונשלחת. בצד המקבל, התהליך הפוך. JSON נבחר בשל קריאותו, פשטותו ותמיכתו הרחבה.
- **תחומי עניין מרכזיים בפרויקט:**
 - **תכנות רשתות:** הפרויקט כולל מימוש שרת מרובה לקוחות, ניהול חיבורים מקביליים (באמצעות threading בצד השרת), ושליחה וקבלה של נתונים ברשת באמצעות סוקטים.
 - **אבטחת מידע והצפנה:** תחום זה מהווה חלק מרכזי בפרויקט, וכולל מימוש הצפנה מקצה לקצה (E2EE) באמצעות שילוב של הצפנה א-סימטרית RSA להחלפת מפתחות (והצפנה סימטרית AES) להצפנת תוכן ההודעות. בנוסף מתבצע גיבוב סיסמאות מאובטח וניהול מפתחות הצפנה.
 - **מסדי נתונים:** שימוש במסד נתונים SQLite בצד השרת לאחסון ושליפה של נתונים מובנים כגון פרטי משתמשים, הודעות, פרטי קבוצות וחברויות.
 - **ממשקי משתמש גרפיים (GUI):** פיתוח אפליקציית לקוח שולחנית עם ממשק משתמש ויזואלי ואינטראקטיבי באמצעות ספריית customtkinter.

- **תכנות מונחה עצמים (OOP):** המערכת תוכננה ונבנתה תוך שימוש בעקרונות תכנות מונחה עצמים, עם חלוקה למחלקות בעלות אחריות מוגדרת כגון `NetworkNode`, `DatabaseManager`, `ChatServer`, `ClientHandler`, `ChatClientApp` וירשה שמתבצעת מ-`NetworkNode`.

תיאור זרימת המידע במערכת:

תרשים יכולת הרשמה למערכת:

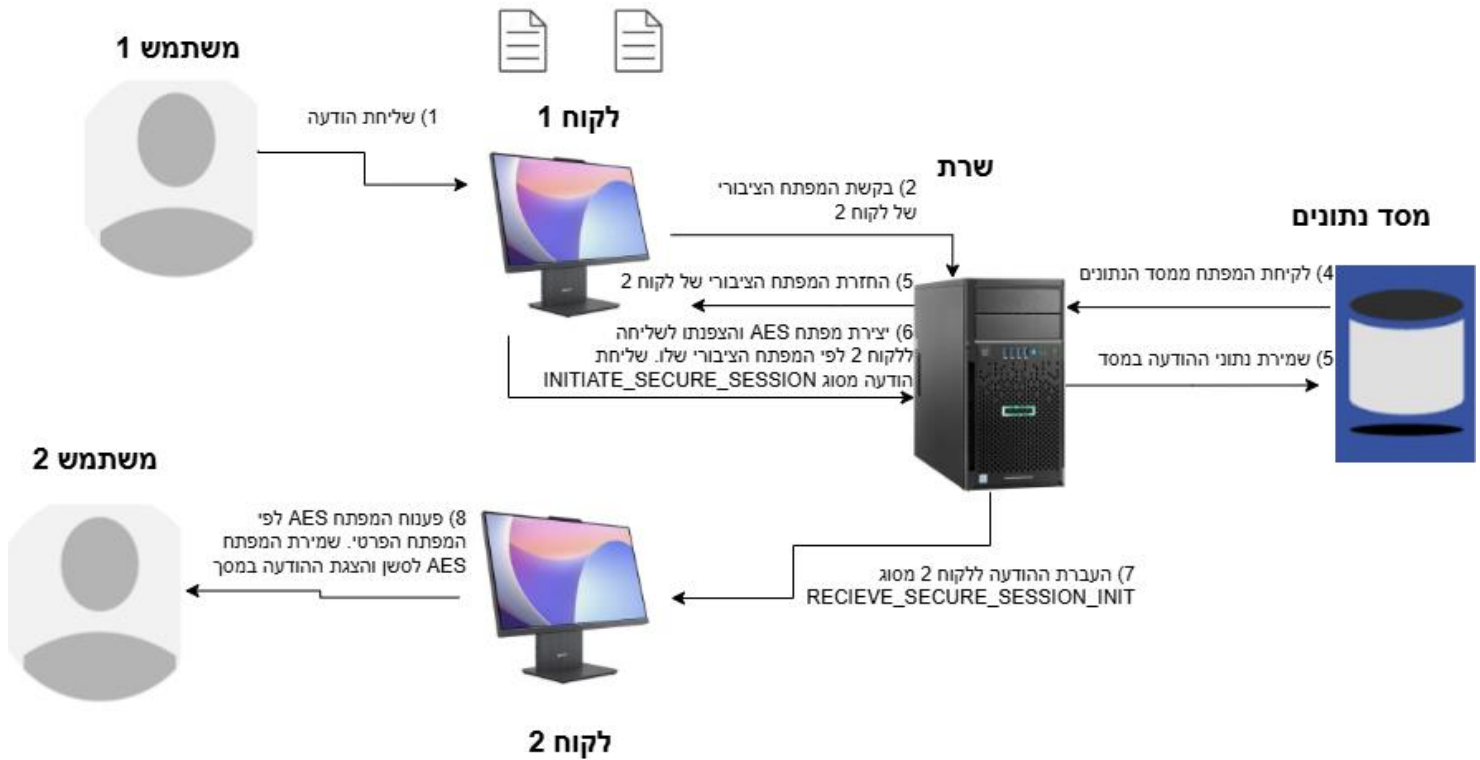


תרשים יכולת ההתחברות למערכת

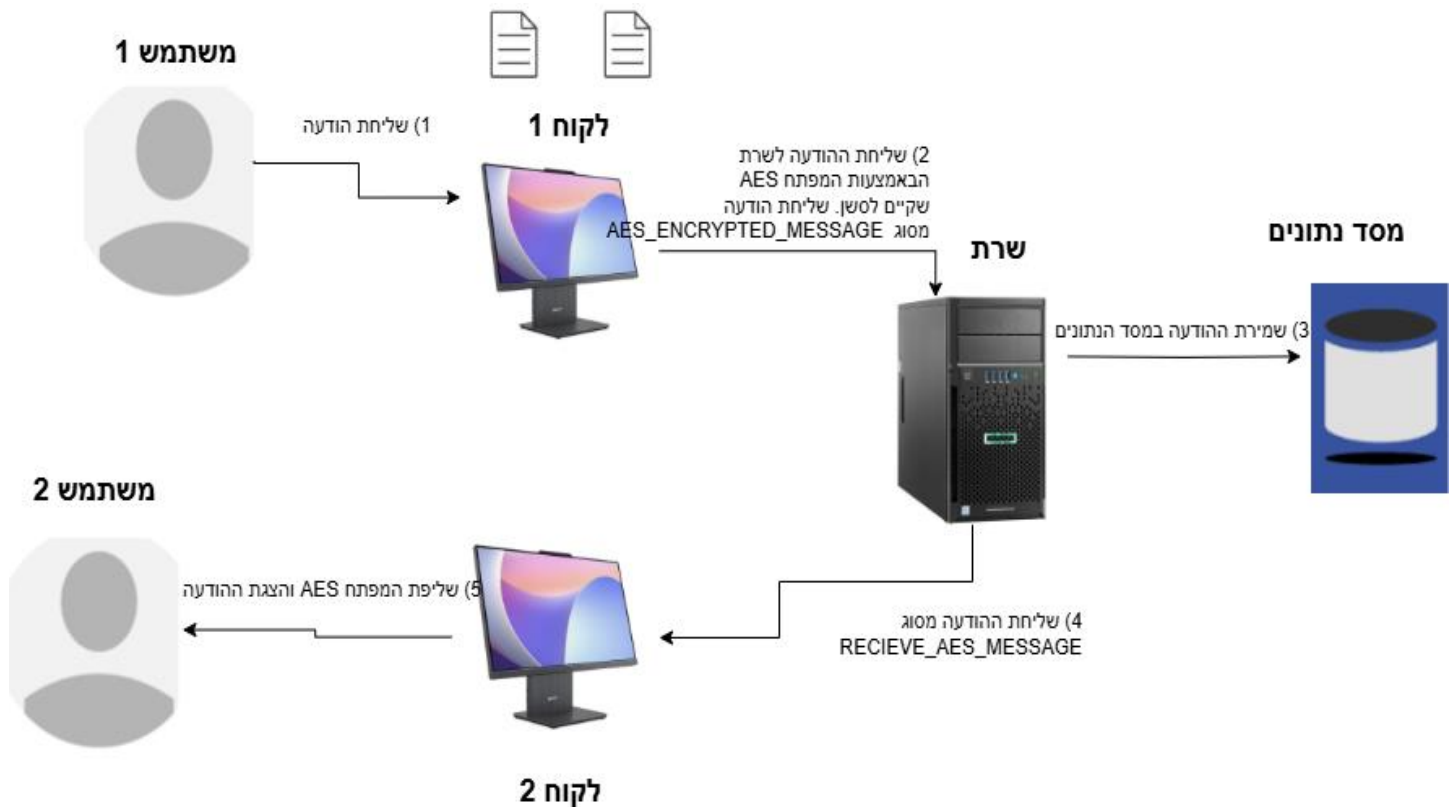


יכולת שליחת הודעה מאובטחת מקצה לקצה:

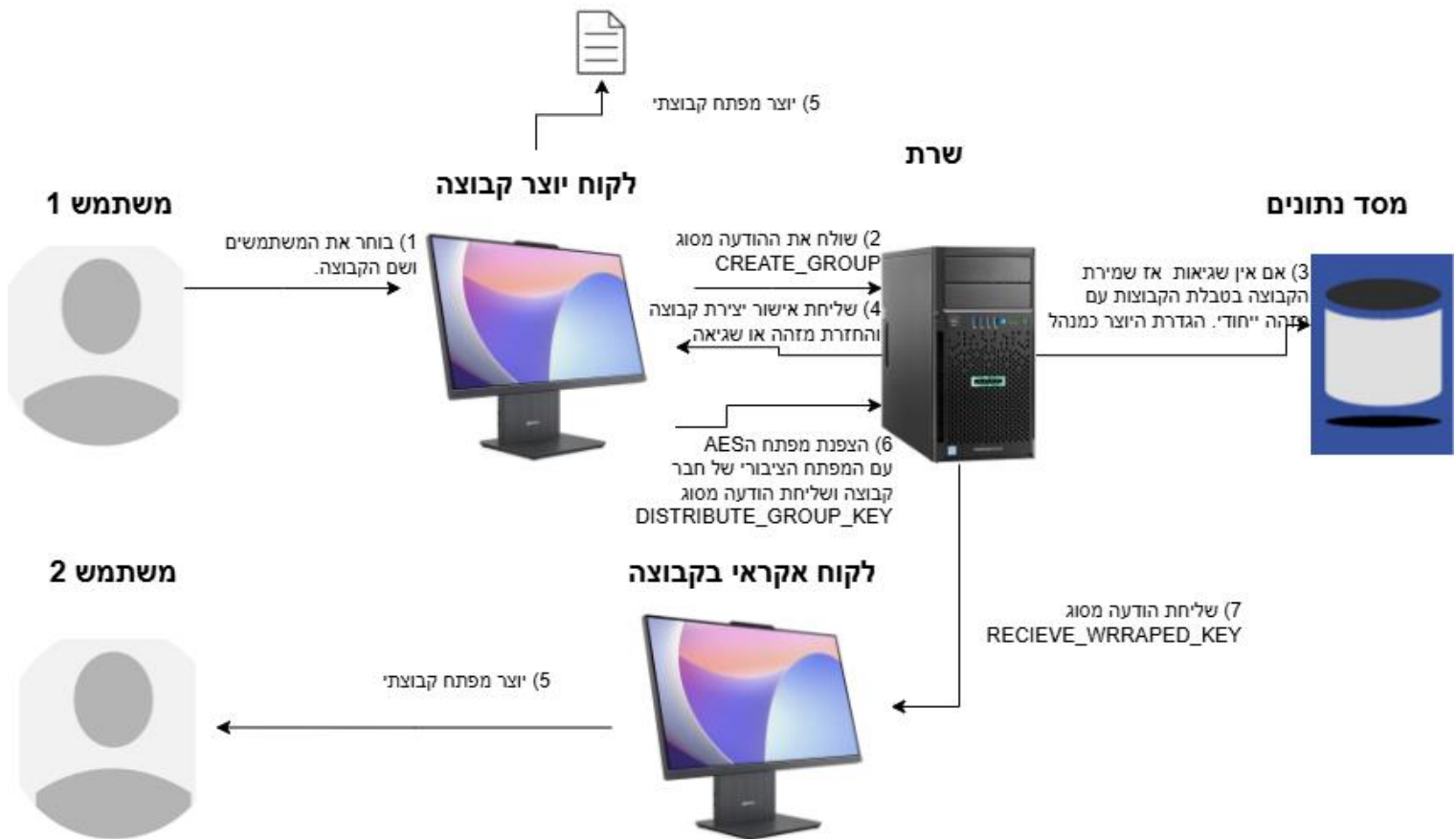
תרשים ראשון מתאר את התקשורת בתחילת כל סשן כלומר התחברות:



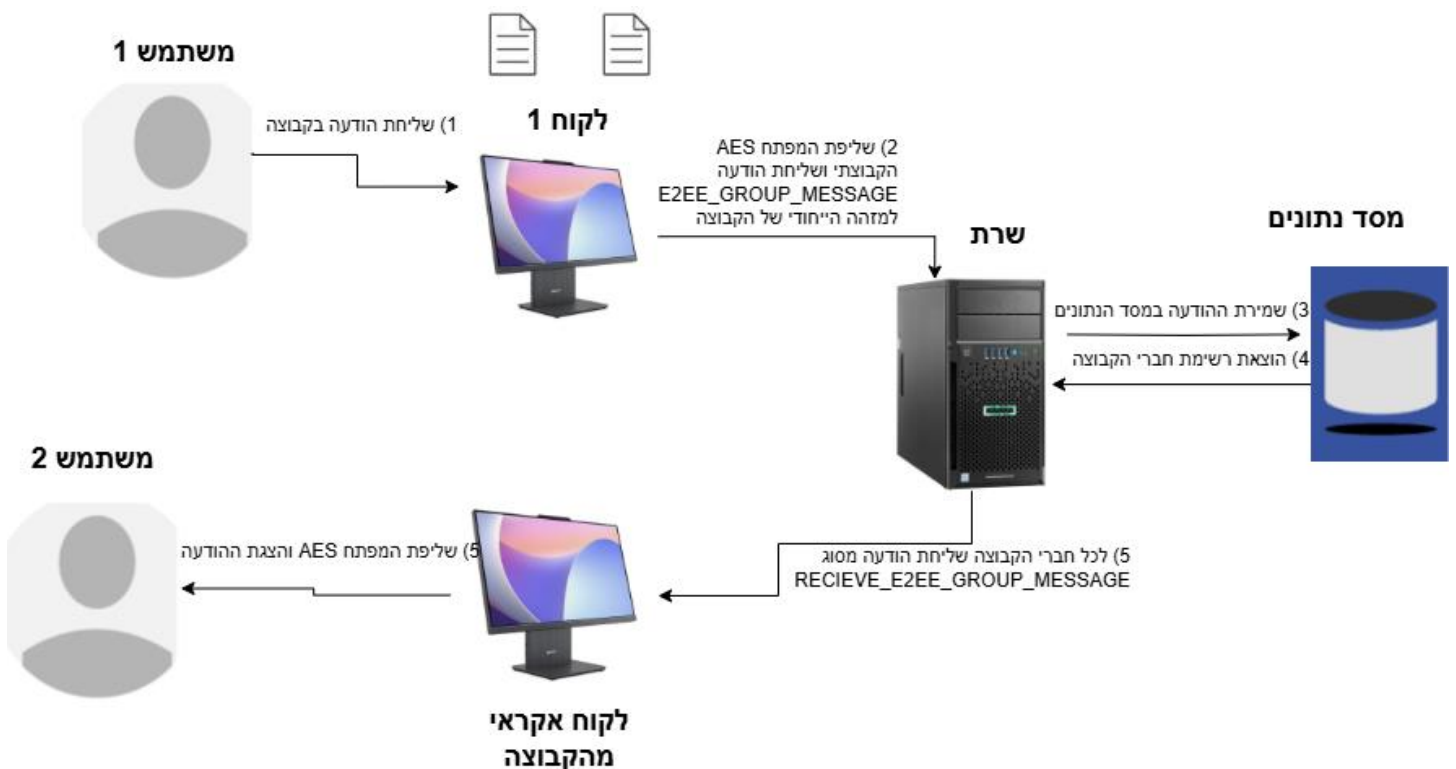
תרשים שני הודעות לאחר תחילת הסשן:



יכולת יצירת קבוצה חדשה:



יכולת שליחת הודעה בקבוצה:



יכולת שינוי הגדרות של הקבוצה:

שרטוט ייצוגי עבור הוספת משתמש לקבוצה:



יכולת להוסיף תמונת פרופיל(בדומה אייקון לקבוצה):



תיאור האלגוריתמים המרכזיים בפרויקט:

אלגוריתם 1: גיבוב סיסמאות מאובטח

ניסוח וניתוח של הבעיה האלגוריתמית: הבעיה המרכזית בהתמודדות עם סיסמאות משתמשים היא הצורך לאחסן אותן באופן כזה שלא יאפשר שחזור של הסיסמה המקורית, גם במקרה של פריצה למסד הנתונים של המערכת. בעת הרשמת משתמש לשירות ניתן תיאורטית להצפין את הסיסמה שהוגדרה לפני אחסונה במסד הנתונים. אולם, הצפנה היא תהליך דו-כיווני כלומר ניתן בעזרת המפתח המתאים לפענח את הערך המוצפן ולקבל חזרה את הערך המקורי. לכן, באופן תיאורטי, כל גורם תוקף שיצליח להשיג גישה למסד הנתונים (או לחלקו) ואולי גם למפתח ההצפנה, יוכל לפענח את כלל סיסמאות המשתמשים. חשיפה כזו של סיסמאות עלולה להוביל לבעיות אבטחת מידע חמורות ולפגיעה קשה במשתמשים, במיוחד אם הם משתמשים באותה סיסמה גם בשירותים אחרים.

מסיבה זו, הגישה המקובלת והמומלצת כיום בעולם אבטחת המידע היא שימוש ב**גיבוב (Hashing)** לאחסון סיסמאות. בניגוד להצפנה, גיבוב הוא תהליך חד-כיווני במהותו. לאחר שהמידע עבר גיבוב, לא ניתן לשחזר ממנו את הערך המקורי ("הנקי"). עם זאת, גם השימוש בגיבוב דורש התייחסות מיוחדת. אם נשתמש באותה פונקציית גיבוב פשוטה עבור כל הסיסמאות, אזי לסיסמאות זהות יהיה ערך מגובב זהה. עובדה זו פותחת פתח להתקפות המבוססות בשם "Rainbow Tables" או על גיבוב מראש של סיסמאות נפוצות. תוקף יכול להכין מראש טבלאות המכילות סיסמאות נפוצות ואת ערכי הגיבוב שלהן (באמצעות פונקציות גיבוב ידועות). לאחר מכן, אם התוקף משיג את מסד הנתונים של הסיסמאות המגובבות, הוא יכול להצליב את הערכים שבידיו עם הטבלאות שהכין, ובכך לחשוף סיסמאות נפוצות.

יתרה מכך, קיימות התקפות נוספות ושיטות מתוחכמות לניסיון "לעקוף" את החד-כיווניות של פונקציות גיבוב, כמו ניתוח דפוסים או שימוש בכוח חישוב רב. לכן, הכרחי להשתמש בפונקציות גיבוב מתקדמות וחזקות, שתוכננו במיוחד להתמודד עם איומים אלו. פונקציות גיבוב מתקדמות משתמשות בטכניקה של "מלח" (Salt) – ערך אקראי ויחודי המוצמד לכל סיסמה לפני תהליך הגיבוב. ה"מלח" גורם לכך שגם אם לשני משתמשים יש את אותה הסיסמה, ערכי הגיבוב שלהם (השמורים במסד הנתונים) יהיו שונים לחלוטין, מכיוון שהם גובבו עם "מלח" שונה. הדבר מנטרל את היעילות של טבלאות קשת מוכנות מראש. חשוב שה"מלח" יהיה אקראי מספיק ולא קצר, כדי להקשות על ניסיונות הנדסה לאחור של הערך המגובב. ה"מלח" למעשה משתלב עם פעולת הגיבוב הבסיסית כתוסף המשתנה עבור כל משתמש, וכך מוביל לתוצאה מגובבת סופית שונה.

בנוסף ל"מלח", ישנה לעיתים המלצה להשתמש גם ב"פלפל" (Pepper) – ערך סודי נוסף, שהוא זהה לכלל הסיסמאות במערכת, אך נשמר בנפרד ממסד הנתונים (למשל, בקוד השרת או בקובץ תצורה מאובטח). ה"פלפל" מוסיף שכבת הגנה נוספת למקרה שמסד הנתונים (כולל הגיבובים והמלחים) נפרץ, אך התוקף לא השיג גישה ל"פלפל". טכניקה נוספת להקשחת הגיבוב היא ביצוע מספר רב של איטרציות של פונקציית הגיבוב. בעוד שעבור אימות לגיטימי של משתמש בודד, תוספת הזמן של מספר מילי-שניות היא זניחה, עבור תוקף המנסה מיליוני או מיליארדי קומבינציות, הכפלת זמן החישוב עבור כל ניסיון הופכת את ההתקפה ללא מעשית. שילוב של "מלח" איכותי, מספר איטרציות גבוה, ואולי גם "פלפל", יוצר מנגנון גיבוב חזק מאוד, המקשה משמעותית על פריצת סיסמאות ומגן על פרטיות המשתמשים.

- **תיאור אלגוריתמים קיימים לפתרון הבעיה:** כפי שצוין, קיימות פונקציות גיבוב ייעודיות שניתן למצוא עליהם מידע בגוגל כמו `bcrypt`, `scrypt`, `PBKDF2` ו-`Argon2` כל אחת מהן מציעה גישה מעט שונה להשגת אבטחה.
- **סקירת הפתרון הנבחר בפרויקט:** בפרויקט שלי נבחר להשתמש באלגוריתם **PBKDF2** עם **HMAC-SHA256** לאבטחת הסיסמאות. המימוש מתבצע באמצעות הפונקציה `hashlib.pbkdf2_hmac` בספריית `hashlib` הסטנדרטית של פייתון, כפי שמופיע במחלקה `DatabaseManager` בקובץ: `shared_modules.py` בעת הרשמת משתמש חדש, נוצר עבורו "מלח" (Salt) "אקראי וייחודי באורך 16 בתים". הסיסמה שהזין המשתמש עוברת לאחר מכן תהליך גיבוב באמצעות `hashlib.pbkdf2_hmac`. פונקציה זו מקבלת כפרמטרים את אלגוריתם הגיבוב הפנימי, הסיסמה, המלח ומספר איטרציות גבוה (בפרויקט נקבע ל-100,000). במסד הנתונים בטבלת `users` נשמרים עבור כל משתמש שם המשתמש, תוצאת הגיבוב של הסיסמה (כמחרוזת הקסדצימלית), וה"מלח" הייחודי. הסיסמה המקורית אינה נשמרת כלל. בעת התחברות, המערכת שולפת את ה"מלח" השמור, מבצעת את אותו תהליך גיבוב על הסיסמה שהוזנה, ומשווה את התוצאה לגיבוב השמור. הגיבוב חח"ע ולכן רק עבור הסיסמה הנכונה נקבל תוצאת גיבוב זהה. יתר על כן הבחירה ב-PBKDF2 נובעת מהיותו תקן מוכח המשלב "מלח" ואיטרציות להגברת האבטחה, וזמינותו בספרייה סטנדרטית של פייתון.

דוגמה מהפרויקט:

id	username	passwo...
1	SYSTEM...	99c256e...
2	1	096823e...
3	2	a7613d6...
4	4	581d537...

ניתן לראות שהסיסמאות נשמרות כמחרוזת ארוכה. אם נלחץ על אחד הסיסמאות נקבל את המחרוזת:

"ee67f7ce7f44b43d27f5227ae95e4afa33be3f4c329f26e3df375ef5a609682364". זו היא כמובן התוצאה של הפעלת פונקציית ההאש על הסיסמה המקורית.

אלגוריתם 2: אבטחת העברת מידע מקצה לקצה (End-to-End Encryption - E2EE)

ניסוח וניתוח של הבעיה האלגוריתמית הכוללת: דמיינו שאתם רוצים לשלוח מכתב סודי לחבר. אתם לא רוצים שאיש בדרך לא הדוור, לא מישוה שמציץ לתיבת הדואר – יוכל לקרוא את מה שכתבתם. זו בדיוק הבעיה שאנחנו מנסים לפתור כאן. כלומר הבעיה היא איך להבטיח ששיחות בין משתמשים יישארו פרטיות לחלוטין, כך שרק השולח והנמען (או חברי הקבוצה) יוכלו להבין את תוכן ההודעות. קוראים לזה "הצפנה מקצה לקצה", (E2EE) "כי ההודעה מוצפנת בקצה אחד (אצל

השולח) ומפוענחת רק בקצה השני (אצל הנמען). אפילו השרת שלנו שמעביר את ההודעות לא יכול לדעת מה כתוב בהן.

הסבר מתמטי: בפרויקט זה השתמשתי בהצפנת RSA ו AES נתחיל מהסבר מתמטי של איך זה עובד:

אלגוריתם RSA מבוסס על הקושי החישובי הרב הכרוך בפירוק לגורמים של מספרים גדולים מאוד . יצירת מפתחות. **את המפתח הציבורי נמצא כך:**

1. בחירת שני מספרים ראשוניים גדולים ושונים p ו q מספרים אלו נשמרים בסוד.

2. חישוב המודולו: $n = p * q$

3. חישוב פונקציית אוילר עבור n : $\phi(n) = (p - 1)(q - 1)$

4. בחירת מספר שלם e כך שהוא בין 3 לערך פונקציית האוילר. בנוסף ערך פונקציית האוילר והמספר הם מספרים זרים (כלומר, המחלק המשותף המקסימלי שלהם הוא 1. ערך נפוץ עבור e הוא 65537 המפתח הציבורי הוא הזוג (e, n) .

5. כעת כאשר נרצה להצפין ערך m נצפין כך:

$$m^e \pmod n = c$$

את המפתח הפרטי נסמן ב d בשל כך שאנו עובדים עם מספרים ראשוניים נוכל לדעת כי d שונה מ e וראשוני לו, נוכל להשתמש במשפט אוילר ולקבוע כי:

$$e * d = 1 \pmod{\phi(n)}$$

ואז כאשר יש לנו את הערך המוצפן c , את המפתח הציבורי (e, n) והמפתח הפרטי d . נקבל את הנוסחה הבאה שתביא לנו את הערך המקורי m .

$$c^d \pmod n = m$$

מימוש אבטחת ההודעות בפרויקט: המערכת משלבת את RSA ו AES-להצפנה היברידית.

האתגר מורכב מכמה חלקים: ראשית, איך שני אנשים יכולים להסכים על "צופן סודי" משותף אם הם מתקשרים דרך האינטרנט. שנית, באיזה "צופן" להשתמש כדי שההודעות יהיו גם בטוחות וגם יישלחו ויגיעו מהר? ושלישית, איך כל זה עובד כשמדובר בשיחה קבוצתית עם הרבה משתתפים? בפרויקט זה החלטתי להשתמש בהצפנה היברידית שמשלבת הצפנת RSA אסימטרית והצפנת ASE סימטרית. ההצפנה עובדת ככה:

1. מנעול עם מפתח ציבורי ומפתח פרטי הצפנה א-סימטרית, כמו RSA לכל אחד מאיתנו יש צרור מפתחות מיוחד. מפתח אחד הוא "ציבורי" – אפשר לתת עותק שלו לכל מי שרוצים. מפתח שני הוא "פרטי" – אותו שומרים היטב בצד של הלקוח ואסור לאף אחד מלבד הלקוח להיות גישה אליו. אם מישהו רוצה לשלוח לנו משהו נעול שרק אנחנו נוכל לפתוח, הוא משתמש במפתח הציבורי שלנו כדי לנעול. רק אנחנו, עם המפתח הפרטי שלנו, נוכל לפתוח את הנעילה. דרך הצפנה זאת חזקה מאוד אבל איטית לשימוש רבה.

2. מנעול עם מפתח יחיד הצפנה סימטרית, כמו AES או הצפה "רגילה" לפי תבנית כלשהי שמפתח ה-AES נותן. דרך הצפנה זאת היא הרבה יותר מהירה ונוחה לשימוש למידע רב, אבל הבעיה היא שצריך איכשהו להעביר את המפתח הזה בצורה בטוחה לחבר שאיתו רוצים להתכתב, מבלי שאחרים ישיגו עותק.

השילוב ההיברידי אומר שנשתמש במנעול הציבורי-פרט (RSA) "המסורבל אך הבטוח כדי להעביר את המפתח הקטן והיעיל של המנעול הסימטרי (AES) אחרי שהמפתח הקטן והסודי עבר בבטחה, נשתמש רק בו כדי לנעול ולפתוח את כל ההודעות השוטפות.

בשיחות קבוצתיות, הרעיון דומה, אבל צריך לדאוג לכלל חברי הקבוצה יהיה עותק של אותו "מפתח קבוצתי סודי, ושרק להם תהיה גישה אליו. כך פשוט נעביר אותו בעזרת מפתח ציבורי ואז נפענח

סקירת הפתרון הנבחר בפרויקט: בפרויקט לקחתי את הרעיון של הצפנה היברידית ובנינו מערכת שמנסה להיות גם בטוחה וגם מעשית.

ראשית, כל משתמש והמפתחות האישיים שלו: כאשר משתמש חדש מצטרף לאפליקציה, המחשב שלו מייצר עבורו זוג מפתחות RSA ייחודיים. המפתח הפרטי נשמר בסוד מוחלט בקובץ מאובטח במחשב של המשתמש (client_private_key.pem) ואת המפתח הציבורי הוא שולח לשרת שלנו, והשרת שומר אותו יחד עם שם המשתמש. כך, אם מישהו ירצה לשלוח הודעה מוצפנת למשתמש הזה, הוא יוכל לבקש מהשרת את המפתח הציבורי שלו.

עבור הודעות פרטיות (1-ל-1): כאשר לקוח א' שולח הודעה ראשונה ללקוח ב' בסשן מאובטח, לקוח א' מייצר מפתח AES-256 חדש. מפתח AES זה מוצפן באמצעות המפתח הציבורי (RSA) של לקוח ב' ונשלח אליו. בנוסף, מפתח AES זה מוצפן גם באמצעות המפתח הציבורי (RSA) של לקוח א' עצמו (לצורך שמירת יכולת פענוח היסטוריה). ההודעה הראשונה מוצפנת עם מפתח ה-AES החדש. (השרת מקבל את כל החבילה, מאחסן את החלקים הרלוונטיים כולל מפתח ה-AES המוצפן-עצמית של השולח בשדה sender_e2ee_session_data, ומעביר לנמען את מפתח ה-AES המוצפן עבורו ואת ההודעה המוצפנת. הנמען מפענח את מפתח ה-AES באמצעות מפתח ה-RSA הפרטי שלו. כל ההודעות העוקבות באותו סשן מוצפנות ישירות עם מפתח ה-AES-המשותף.

[עבור הודעות קבוצתיות: בעת יצירת קבוצה, הלקוח של יוצר הקבוצה מייצר מפתח AES-256 קבוצתי. מפתח זה נשמר מקומית אצל היוצר. כדי להפיצו לחברי הקבוצה, היוצר מצפין את מפתח ה-AES-הקבוצתי עם המפתח הציבורי (RSA) של כל חבר בנפרד. המפתחות המוצפנים הללו נשלחים לשרת בהודעות DISTRIBUTE_GROUP_KEY והשרת מעביר אותם לכל חבר רלוונטי. כל חבר מפענח את המפתח הקבוצתי באמצעות המפתח הפרטי (RSA) שלו ושומר אותו מקומית בקובץ group_aes_keys.json כל הודעה הנשלחת לקבוצה מוצפנת על ידי השולח באמצעות המפתח הקבוצתי והשרת משדר אותה לכל חברי הקבוצה המחוברים, אשר מפענחים אותה באמצעות המפתח הקבוצתי שברשותם.

המימוש הטכני של פעולות ההצפנה והפענוח RSA ו-AES-יצירת מפתחות, ריפוד וטיפול מתבצע באמצעות הפונקציות המתאימות מספריית cryptography.hazmat.primitive בפיתון, כפי שמומש במחלקות ChatClientAppi NetworkNode

אלגוריתם 3: פרוטוקול תקשורת מבוסס JSON עם תו מפריד:

ניסוח וניתוח של הבעיה האלגוריתמית: במערכת כמו בפרויקט זה המורכבת משרת ומלקוחות מרובים, חיוני להגדיר "שפה" משותפת וברורה להעברת נתונים ובקשות. התקשורת מתבצעת על גבי פרוטוקול TCP המספק תקשורת אמינה אך מתייחס למידע כאל זרם רציף של בתים המשמעות היא שאם צד אחד שולח שתי הודעות נפרדות ברצף מהיר, הצד המקבל עלול לקבל אותן כחבילת נתונים אחת, ללא הפרדה ברורה ביניהן. לכן, נדרש מנגנון פרוטוקול ברמת האפליקציה שיגדיר את מבנה ההודעות ויאפשר למקבל לפענח בצורה נכונה כל הודעה בודדת מתוך זרם הנתונים, לזהות את מטרתה ואת הנתונים הנלווים אליה.

תיאור אלגוריתמים וגישות קיימות לפתרון הבעיה: קיימות מספר גישות מקובלות להגדרת פרוטוקולי הודעות.

1. שליחת אורך הודעה קבוע מראש. כלומר לפני כל הודעה ממשית, נשלח מספר קבוע של בתים (למשל, 2 או 4 בתים) המציין את אורכה הכולל של ההודעה הבאה. הצד המקבל קורא תחילה את שדה האורך, ולאחר מכן קורא בדיוק את מספר הבתים המצוין כדי לקבל את ההודעה כולה.

2. שימוש בתו מפריד (Delimiter): מגדירים תו או רצף תווים מיוחד ומוסכם שיסמן את סופה של כל הודעה. הצד המקבל קורא נתונים לתוך מאגר זמני (באפר) ומחפש באופן רציף את התו המפריד. כל הנתונים שהצטברו עד לתו המפריד (לא כולל) נחשבים להודעה אחת שלמה.

3. פרוטוקולים סטנדרטיים מבוססי טקסט או בינאריים: ניתן להשתמש בפרוטוקולים קיימים ומורכבים יותר.

סקירת הפתרון הנבחר בפרויקט: בפרויקט שלי נבחר לממש פרוטוקול תקשורת המבוסס על העברת הודעות בפורמט JSON כאשר כל הודעת JSON נשלחת כמחרוזת אחת המסתיימת בתו מפריד מוסכם – תו ירידת שורה.

אופן הפעולה:

1. מבנה ההודעה הלוגית: כל הודעה המועברת בין הלקוח לשרת (בשני הכיוונים) היא למעשה אובייקט JSON. מבנה בסיסי זה כולל בדרך כלל שני שדות עיקריים:

- "type" מחרוזת המגדירה את סוג הפעולה או ההודעה (לדוגמה: LOGIN).
- "payload" אובייקט JSON נוסף (יכול להיות גם ריק) המכיל את כל הנתונים הספציפיים הנדרשים לאותה פעולה או הודעה (למשל, שם משתמש וסיסמה עבור LOGIN או תוכן הודעה מוצפנת ונמען עבור AES_ENCRYPTED_MESSAGE).

2. שליחת ההודעה על גבי סוקט TCP: מערך הבתים הכולל מחרוזת JSON מקודדת + תו ירידת השורה נשלח דרך חיבור הסוקט הפתוח בין השולח למקבל באמצעות פונקציית `socket.sendall()`.

3. קבלת ההודעה ועיבוד: (Buffering, Delimiting, and Deserialization).

בצד המקבל הנתונים נקראים מהסוקט באופן רציף אל תוך מאגר זמני, באמצעות פונקציית `socket.recv()` הלוגיקה בצד המקבל סורקת את הבאפר באופן מתמשך בחיפוש אחר תו ירידת השורה, `(\n)` המשמש כתו מפריד מוסכם. ברגע שמתגלה תו ירידת שורה, כל הבתים שהצטברו בבאפר עד לנקודה זו (לא כולל התו המפריד עצמו) נחשבים להודעת JSON אחת שלמה.

לאחר חילוץ רצף הבתים המייצג את ההודעה, הוא מפוענח למחרוזת בקידוד UTF-8 `decode('utf-8')` מחרוזת JSON זו מומרת לאחר מכן לאובייקט פייתון (בדרך כלל מילון) באמצעות פונקציית `json.loads()` מרגע זה, האפליקציה בצד המקבל יכולה לגשת לשדות הסטנדרטיים של ההודעה, כגון "type" ו "payload" כדי להבין את מהותה ולבצע את הפעולה הנדרשת. מימוש לוגיקה זו קיים בפונקציה `receive_messages_thread` בקובץ `client.py` ובפונקציית `run` של המחלקה `ClientHandler` בקובץ `server.py`.

נימוקי הבחירה בגישה זו: הבחירה בפורמט JSON עם תו מפריד נעשתה בשל מספר יתרונות. ראשית, פשטות המימוש היחסית בפייתון, הודות לספריית `json` המובנית והטיפול הפשוט בתו המפריד, אשר חוסך מורכבות של חישוב אורך הודעה או ניהול מצב מסובך בצד המקבל. שנית, הקריאות ונוחות ניפוי השגיאות של הודעות JSON (כל עוד אינן מוצפנות) מקלות על הבנת התקשורת בזמן הפיתוח. שלישית, גמישות מבנה הנתונים של JSON מאפשרת העברת מבנים היררכיים ומגוונים בתוך שדה ה "payload"-המתאימה למגוון הפעולות הנדרשות במערכת. לבסוף, גישה זו מייתרת את הצורך בתיאום ושליחת אורך הודעה קבוע מראש, מה שמפשט את לוגיקת השליחה.

מגבלות אפשריות של הפתרון הנבחר: ישנן שתי מגבלות עיקריות שיש לקחת בחשבון. הראשונה היא הטיפול בתו המפריד `(\n)` בתוך תוכן ההודעה עצמו. ספריית `json` של פייתון מטפלת בכך באופן תקין על ידי "הימלטות" (escaping) של תווים מיוחדים במחרוזות, כך שהם לא יתפרשו בטעות כסוף ההודעה. השנייה נוגעת ליעילות עבור הודעות גדולות מאוד. העברת קבצים גדולים או נתונים בינאריים רבים לאחר קידודם ל JSON למשל, באמצעות (Base64) ולאחר מכן ל UTF-8-עלולה להיות פחות יעילה מבחינת נפח התעבורה בהשוואה לפרוטוקול בינארי יעודי. עם זאת, עבור הודעות טקסט ונתונים מובנים קטנים יחסית, כפי שקורה ברוב המקרים בפרויקט פתרון ה JSON-הוא יעיל מספיק.

אלגוריתם 4: טיפול בבקשות לקוח מרובות בשרת באמצעות Threading:

ניסוח וניתוח של הבעיה האלגוריתמית: השרת צריך לטפל בחיבורים ובקשות ממספר לקוחות במקביל, מבלי שלקוח אחד יצטרך להמתין עד שהטיפול בלקוח אחר יסתיים. נדרש מנגנון המאפשר טיפול מקבילי או מדמה טיפול מקבילי.

תיאור אלגוריתמים וגישות קיימות לפתרון הבעיה: הגישות הנפוצות כוללות ריבוי תהליכים (Multi-processing), ריבוי תהליכונים (Multi-threading) וקלט/פלט א-סינכרוני מבוסס אירועים (Asynchronous I/O).

אופן הפעולה: כאשר מתקבל חיבור חדש מלקוח, השרת הראשי יוצר ומפעיל תהליכון (thread) חדש המוקדש לטיפול באותו לקוח. כל תהליכון כזה מנוהל על ידי מופע של המחלקה `ClientHandler`, שאחראית על קריאת הודעות מאותו לקוח, עיבודן ושליחת תגובות אליו. גישה זו מאפשרת לשרת לטפל במספר לקוחות "במקביל".

ניהול משאבים משותפים: נעשה שימוש במנעולים (`threading.Lock`) כגון `active_clients_lock` ב `ChatServerCore`-כדי להגן על גישה למשאבים משותפים (כמו רשימת הלקוחות הפעילים) ולמנוע מצבי מרוץ.

- נימוק הבחירה: ריבוי תהליכים הוא מודל מקביליות נפוץ ואינטואיטיבי יחסית למשימות עתירות קלט/פלט (I/O-bound) כמו תקשורת רשת. הוא מאפשר שיתוף זיכרון בין התהליכים (בתוך אותו תהליך שרת), אם כי דורש סנכרון קפדני. מודול threading הוא חלק מהספרייה הסטנדרטית של פייתון.
- מגבלות: Global Interpreter Lock (GIL) - בפייתון מגביל ריצה מקבילית אמיתית של תהליכים על מספר ליבות CPU עבור קוד פייתון טהור.

מקורות מידע:

- אלגוריתם 1 מאמר - <https://supertokens.com/blog/password-hashing-salting>
- אלגוריתם 2 סרטון - <https://youtu.be/pOx2TYwR590?si=UffUF7yTXWUjmHQS>
- אלגוריתם 3 מאמר - <https://medium.com/data-science/working-with-json-data-in-python-45e25ff958ce>
- אלגוריתם 4 סרטון - <https://youtu.be/STEOavXqXkQ?si=H72Ceqis-TraTGFi>

תיאור סביבת הפיתוח:

פירוט כלי הפיתוח הדרושים לפיתוח

- שפת תכנות: פייתון גרסה 3
- סביבת פיתוח משולבת: בפרויקט זה השתמשתי בסיבת עבודה pycharm
- ספריות Python מרכזיות שנעשה בהן שימוש (בנוסף לספריות המובנות) :
 - customtkinter לבניית ממשק המשתמש הגרפי (GUI) של אפליקציית הלקוח.
 - cryptography למימוש פונקציות הצפנה א-סימטריות (RSA) וסימטריות (AES), יצירת מפתחות וטיפול בריפוד.
 - Pillow (PIL) לעיבוד תמונות בצד הלקוח (למשל, שינוי גודל תמונות פרופיל לפני העלאתן).
- ספריות Python מובנות שנעשה בהן שימוש נרחב :
 - socket למימוש תקשורת הרשת בין השרת ללקוחות.
 - Threading למימוש יכולת השרת לטפל במספר לקוחות במקביל.
 - sqlite3 לניהול מסד הנתונים בצד השרת.
 - json לטיפול בהודעות שנשלחות.
 - Hashlib לגיבוב סיסמאות באמצעות pbkdf2_hmac
 - os לפעולות הקשורות למערכת הקבצים (כגון יצירת נתיבים, בדיקת קיום קבצים).
 - base64 לקידוד נתונים בינאריים (כמו תמונות או מפתחות) לייצוג טקסטואלי עבור שמירה או שליחה בJSON.
 - datetime לניהול חותמות זמן.
 - queue לניהול תור הודעות נכנסות באפליקציית הלקוח.

פירוט הסביבה והכלים הנדרשים לבדיקות והרצה

- סביבת הרצה :
 - 🚦 מערכת הפעלה : את הפרויקט כתבתי בווינדוס אך ניתן להרי. את הקוד גם על מערכות הפעלה אחרות שמריצות פייתון.
 - 🚦 התקנת ספריות חיצוניות :יש לוודא התקנה של הספריות customtkinter, cryptography, Pillow בסיבת הפייתון של הלקוח. ניתן להתקין באמצעות pip install

• כלים לבדיקות :

DB Browser for SQLite:  כללי ויזואלי נוח לבדיקה וניתוח של תוכן מסד הנתונים של השרת (chat_app.db) בעת בדיקות.

Wireshark:  כלי להסנפת תעבורת רשת, שיכול לשמש לבדיקה שההצפנה אכן מיושמת ושהמידע הרגיש אינו עובר כטקסט גלוי ברשת.

• הגדרות רשת להרצה מבוזרת על מספר מחשבים באותה רשת:

 **איתור כתובת ה-IP של השרת:** במחשב המיועד להריץ את השרת, יש לפתוח את שורת הפקודה (CMD) ולהקליד: ipconfig יש לאתר את כתובת ה-IPv4 של מתאם הרשת הפעיל.

```
Wireless LAN adapter Wi-Fi:

Connection-specific DNS Suffix . : fritz.box
IPv6 Address. . . . . : 2a05:bb80:3c:a303:39b:47fa:9ba4:496c
IPv6 Address. . . . . : fd15:f626:d889:0:e524:a7d2:c36a:366b
Temporary IPv6 Address. . . . . : 2a05:bb80:3c:a303:c140:d555:acfc:efcf
Temporary IPv6 Address. . . . . : fd15:f626:d889:0:c140:d555:acfc:efcf
Link-local IPv6 Address . . . . . : fe80::2e10:7ea5:8a1c:eba9%14
IPv4 Address. . . . . : 192.168.178.25
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . : fe80::52e6:36ff:fed4:f90f%14
                             192.168.178.1
```

 עדכון כתובת השרת בקוד :

- בקובץ client.py וגם בקובץ server.py יש לעדכן את המשתנה HOST לכתובת ה IP של מחשב השרת שאותרה.
- בנוסף יש לוודא שהמשתנה PORT למשל 65432 בפרויקט זהה בקוד השרת ובקוד הלקוח ופנוי

```
HOST = '127.0.0.1'
PORT = 65432
```

 **הרצת המערכת :** יש להריץ תחילה את קובץ השרת (server.py) במחשב השרת. לאחר שהשרת פועל ומאזין, ניתן להריץ את קובץ הלקוח (client.py) בכל אחד ממחשבי הלקוח.

כדי להשתמש במערכת, יש להקפיד על התקנת כל הספריות הנדרשות בסביבת הפיתוח של כל מחשב (שרת ולקוח) ועל הגדרות הרשת הנכונות במידה ומריצים את המערכת על מחשבים שונים באותה רשת.

תיאור פרוטוקול התקשורת:

תיאור מילולי של פרוטוקול התקשורת(מבנה / כמות בתיים):

פרוטוקול התקשורת מגדיר את הכללים והפורמט להעברת נתונים ובקשות בין אפליקציית הלקוח לבין השרת. הפרוטוקול בנוי בשכבות, כאשר שכבת התעבורה מתבססת על TCP ומעליה, שכבת האפליקציה משתמשת בפורמט JSON להגדרת מבנה ההודעות הלוגיות.

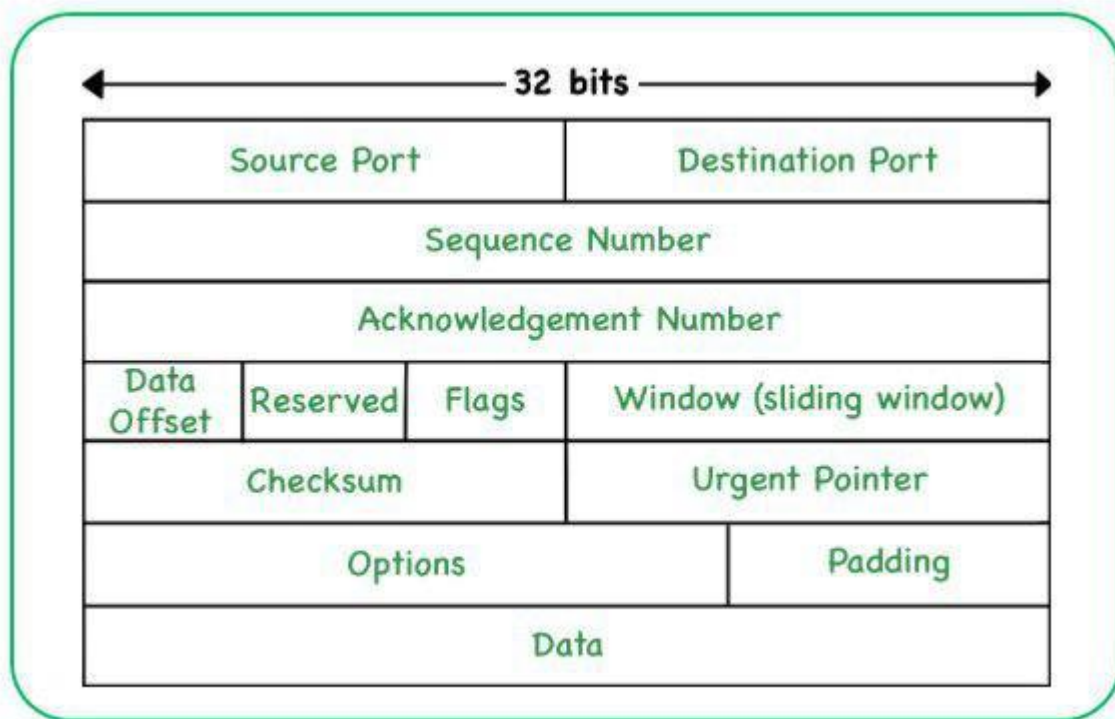
1. שכבת תעבורה – פרוטוקול (TCP (Transmission Control Protocol: התקשורת

הבסיסית בין הלקוח לשרת מתבצעת על גבי פרוטוקול TCP. TCP הוא פרוטוקול מכוון חיבור ואמין, הפועל בשכבת התעבורה (השכבה הרביעית במודל במודל חמש השכבות. הבחירה ב TCP נובעת מהצורך להבטיח שההודעות יגיעו ליעדן בשלמותן, בסדר הנכון, וללא אובדן מידע.

❖ **יצירת חיבור (Three-Way Handshake):** לפני העברת נתונים כלשהי, הלקוח והשרת מבצעים תהליך "לחיצת יד משולשת (Three-Way Handshake) " לביסוס החיבור. תהליך זה כולל שליחת הודעת סנכרון (SYN) מהלקוח לשרת, קבלת אישור סנכרון (SYN-ACK) מהשרת ללקוח, ולבסוף שליחת אישור (ACK) מהלקוח לשרת. רק לאחר ביסוס החיבור, מתחילה העברת הנתונים. כך יש ודאות בין הצדדים שהם יכולים לקבל ולשלוח הודעות לשני.

❖ **אמינות TCP** משתמש במנגנוני אישור קבלה (acknowledgements) ומספור רציף של חבילות מידע (sequence numbers) כדי לוודא שכל חבילה הגיעה ליעדה. אם חבילה אובדת או מגיעה פגומה, היא נשלחת מחדש. כך יש ודאות שהמידע יגיע בסדר הנכון. בנוסף זיהוי המידע כפגום נעשה על ידי מנגנון checksum שבדק האם ערכו זהה למה שהיה בצד של השולח. כך יש ודאות די גבוהה שהמידע הגיע באופן מלא.

❖ **שמירה על סדר TCP:** מבטיח שהנתונים יגיעו לנמען באותו הסדר שבו נשלחו, גם אם חבילות המידע הגיעו ברשת בסדר שונה.



התמונה מציגה את Header של חבילת TCP המשמש להעברת נתונים אמינה ברשת. פתח זה שאורכו בדרך כלל כ-20 בתים, מכיל מספר שדות קריטיים. פורט המקור (Source Port) ופורט היעד (Destination Port) כל אחד בגודל 16 ביט, מזהים את האפליקציות השולחת והמקבלת במחשבים המתקשרים. **מספר הרצף (Sequence Number)** בגודל 32 ביט, חיוני לסידור נכון של חבילות המידע בצד המקבל ולזיהוי חבילות שאבדו, בעוד **מספר אישור הקבלה (Acknowledgement Number)** גם הוא 32 ביט, משמש את הנמען לאשר קבלת נתונים ולציין את מספר הרצף הבא שהוא מצפה לקבל. **דגלי הבקרה (Flags)** כגון SYN, ACK, FIN-הם סיביות בודדות המנהלות את שלבי יצירת החיבור, אישור הנתונים וסיום החיבור. שדות נוספים כמו **גודל החלון (Window Size)** מסייעים בבקרת זרימה למניעת הצפה של המקבל, ו**סכום הביקורת (Checksum)** משמש לוודא שלמות הפתח והנתונים המועברים, ובכך מבטיח את אמינות התקשורת.

2. שכבת אפליקציה – פורמט JSON ותו מפריד: מגדיר את מבנה ההודעות הלוגיות באמצעות

❖ מבנה הודעת JSON: כל הודעה הנשלחת היא אובייקט JSON הכולל לפחות שני שדות עיקריים:

- "type": מחרוזת המגדירה את סוג הפעולה או ההודעה (לדוגמה "LOGIN", "REGISTER", "SEND_PRIVATE_MESSAGE").
- "payload": אובייקט JSON נוסף המכיל את הנתונים הספציפיים לאותה פעולה או הודעה.

❖ תהליך שליחה: אובייקט ה JSON-מומר למחרוזת, מקודד לבתים, (UTF-8) ולסופו מתווסף תו ירידת שורה (\n) המשמש כתו מפריד בין הודעות JSON שלמות.

❖ **תהליך קבלה:** הצד המקבל קורא נתונים מהסוקט לתוך באפר, מחפש את תו ירידת השורה (n) כדי לתחום הודעת JSON שלמה, מפענח את הבתים למחרוזת UTF-8 ולבסוף ממיר את מחרוזת ה JSON-בחזרה לאובייקט נתונים.

❖ **כמות בתים:** כמות הבתים להודעת JSON משתנה ותלויה בתוכן ההודעה ובגודל ה payload-אין אורך קבוע מראש להודעות JSON בפרוטוקול זה, בניגוד לשיטות אחרות שעשויות לשלוח את אורך ההודעה לפני ההודעה עצמה.

השילוב של TCP להעברה אמינה ו JSON-למבנה נתונים גמיש וקריא, יחד עם מנגנון הפרדת הודעות מבוסס תו מפריד, מהווה את מרכז פרוטוקול התקשורת בפרויקט.

פירוט כלל ההודעות הזורמות במערכת:

הודעות הקשורות לאימות וניהול משתמש:

✓ **שם ההודעה:** SERVER_PUBLIC_KEY

- **נשלחת מ:** שרת
- **נשלחת אל:** לקוח (מיד לאחר חיבור ראשוני)
- **מבנה השדות בהודעה:**
 - key (string) המפתח הציבורי (RSA) של השרת, בפורמט PEM.

✓ **שם ההודעה:** REGISTER

- **נשלחת מ:** לקוח
- **נשלחת אל:** שרת
- **מבנה השדות בהודעה (payload):**
 - username (string) שם המשתמש המבוקש.
 - password (string) הסיסמה שנבחרה על ידי המשתמש.
 - client_public_key (string) המפתח הציבורי (RSA) של הלקוח, בפורמט PEM.

✓ **שם ההודעה:** REGISTER_SUCCESS

- **נשלחת מ:** שרת
- **נשלחת אל:** לקוח
- **מבנה השדות בהודעה (payload):**
 - message (string) הודעת אישור על הצלחת ההרשמה.

✓ **שם ההודעה:** REGISTER_FAIL

- **נשלחת מ:** שרת

- **נשלחת אל :-לקוח**
- **מבנה השדות בהודעה: (payload)**
 - **message (string)** הודעת שגיאה המפרטת את סיבת כישלון ההרשמה.
- ✓ **שם ההודעה: LOGIN**
 - **נשלחת מ :-לקוח**
 - **נשלחת אל :-שרת**
 - **מבנה השדות בהודעה: (payload)**
 - **username (string)** שם המשתמש.
 - **password (string)** הסיסמה.
 - **client_public_key (string)** המפתח הציבורי (RSA) של הלקוח, בפורמט PEM.
- ✓ **שם ההודעה: LOGIN_SUCCESS**
 - **נשלחת מ :-שרת**
 - **נשלחת אל :-לקוח**
 - **מבנה השדות בהודעה: (payload)**
 - **username (string)** שם המשתמש שאושר.
 - בנוסף, השרת ישלח לאחר מכן הודעות MESSAGE עם היסטוריית השיחות
- ✓ **שם ההודעה: LOGIN_FAIL**
 - **נשלחת מ :-שרת**
 - **נשלחת אל :-לקוח**
 - **מבנה השדות בהודעה: (payload)**
 - **message (string)** הודעת שגיאה המפרטת את סיבת כישלון ההתחברות.
- ✓ **שם ההודעה: REQUEST_USER_PUBLIC_KEY**
 - **נשלחת מ :-לקוח**
 - **נשלחת אל :-שרת**
 - **מבנה השדות בהודעה: (payload)**
 - **username (string)** שם המשתמש שלגביו מתבקש המפתח הציבורי.
- ✓ **שם ההודעה: USER_PUBLIC_KEY_INFO**
 - **נשלחת מ :-שרת**

○ נשלחת אל :-לקוח

○ מבנה השדות בהודעה: (payload)

- username (string) שם המשתמש שהמפתח שייך לו.
- key (string) המפתח הציבורי (RSA) של המשתמש המבוקש, בפורמט PEM.

✓ שם ההודעה: USER_PUBLIC_KEY_FAIL

○ נשלחת מ :-שרת

○ נשלחת אל :-לקוח

○ מבנה השדות בהודעה: (payload)

- username (string) שם המשתמש שהמפתח עבורו לא נמצא.
- message (string) סיבת הכישלון.

הודעות הקשורות לתקשורת פרטית מאובטחת:

✓ שם ההודעה: INITIATE_SECURE_SESSION

○ נשלחת מ :-לקוח

○ נשלחת אל :-שרת

○ מבנה השדות בהודעה: (payload)

- recipient (string) שם המשתמש הנמען.
- encrypted_session_key_b64 (string) מפתח ה AES-לסשן, מוצפן RSA עם המפתח הציבורי של הנמען, ומקודד Base64
- sender_encrypted_session_key_b64 (string) מפתח ה AES-לסשן, מוצפן RSA עם המפתח הציבורי של השולח, ומקודד Base64 עבור היסטוריה.
- initial_message (object): אובייקט המכיל את ההודעה הראשונה המוצפנת AES

✓ שם ההודעה: RECEIVE_SECURE_SESSION_INIT

○ נשלחת מ :-שרת

○ נשלחת אל :-לקוח

○ מבנה השדות בהודעה: (payload)

- sender_username (string) שם המשתמש השולח.
- encrypted_session_key_b64 (string) מפתח ה AES-לסשן, מוצפן RSA עבור הנמען מקודד Base64

- initial_message (object) ההודעה הראשונה המוצפנת AES כמו ב- INITIATE_SECURE_SESSION).
- timestamp (string) חותמת הזמן של ההודעה.
- ✓ **שם ההודעה: AES_ENCRYPTED_MESSAGE**
 - **נשלחת מ: -** לקוח
 - **נשלחת אל: -** שרת
 - **מבנה השדות בהודעה: (payload)**
- recipient (string) שם המשתמש הנמען.
- aes_payload (object) אוביייקט המכיל את ההודעה המוצפנת AES :
 - iv_b64 (string) ה-IV ששימש להצפנה, מקודד Base64
 - ciphertext_b64 (string) תוכן ההודעה המוצפן AES מקודד Base64.

✓ **שם ההודעה: RECEIVE_AES_MESSAGE**

- **נשלחת מ: -** שרת
- **נשלחת אל: -** לקוח
- **מבנה השדות בהודעה: (payload)**
- sender_username (string) שם המשתמש השולח.
- aes_payload (object) ההודעה המוצפנת AES כמו ב- AES_ENCRYPTED_MESSAGE).
- timestamp (string) חותמת הזמן של ההודעה.

הודעות כלליות והיסטוריה:

✓ **שם ההודעה: MESSAGE**

משמשת בעיקר להעברת היסטוריה והודעות מערכת

- **נשלחת מ: -** שרת
- **נשלחת אל: -** לקוח
- **מבנה השדות בהודעה: (payload)**
- sender (string) שם המשתמש השולח.
- recipient (string, nullable) שם המשתמש הנמען (אם פרטית).
- text (string) תוכן ההודעה. עבור הודעות מוצפנות בהיסטוריה, זה יהיה JSON של המטען המוצפן המקורי.

- timestamp (string) חותמת הזמן.
- recipient_group_id (integer, nullable) מזהה הקבוצה (אם קבוצתית).
- is_system_message (boolean) האם זו הודעת מערכת.
- e2ee_type (string, nullable) סוג ההצפנה שבוצעה.
- sender_e2ee_session_data (string, nullable) עבור הודעות AES_SESSION_INIT בהיסטוריה, זהו מפתח ה-AES-המוצפן-עצמית של השולח.

✓ שם ההודעה: MESSAGE_STORED_FOR_OFFLINE_DELIVERY

- נשלחת מ:- שרת
- נשלחת אל:- לקוח
- מבנה השדות בהודעה: (payload)
 - recipient_username (string) שם הנמען שההודעה עבורו אוחסנה.
 - timestamp_of_storage (string) חותמת הזמן שבה השרת אחסן את ההודעה.
 - message_preview (string) תצוגה מקדימה קצרה של ההודעה למשל "Encrypted AES Message" או "Secure Session Started"

הודעות הקשורות לניהול קבוצות ותקשורת קבוצתית מאובטחת:

✓ שם ההודעה: CREATE_GROUP

- נשלחת מ:- לקוח
- נשלחת אל:- שרת
- מבנה השדות בהודעה: (payload)
 - group_name (string) שם הקבוצה הרצוי.
 - member_usernames (list of strings) רשימת שמות משתמשים של חברים ראשוניים (כולל היוצר).

✓ שם ההודעה: GROUP_CREATE_SUCCESS

- נשלחת מ:- שרת
- נשלחת אל:- לקוח (יוצר הקבוצה)
- מבנה השדות בהודעה: (payload)
 - group_id (integer) מזהה הקבוצה החדשה שנוצרה.
 - group_name (string) שם הקבוצה.
 - members (list of objects) רשימת החברים בקבוצה

✓ שם ההודעה : `DISTRIBUTE_GROUP_KEY`

- נשלחת מ :-לקוח (מנהל קבוצה)
- נשלחת אל :-שרת (לניתוב לחבר קבוצה)
- מבנה השדות בהודעה: (payload)
 - `group_id` (integer) מזהה הקבוצה.
 - `target_username` (string) שם המשתמש שאליו מיועד המפתח.
 - `wrapped_group_key_b64` (string) מפתח ה AES הקבוצתי, מוצפן RSA עם המפתח הציבורי של `target_username` ומקודד Base64.

✓ שם ההודעה : `RECEIVE_WRAPPED_GROUP_KEY`

- נשלחת מ :-שרת
- נשלחת אל :-לקוח (חבר קבוצה)
- מבנה השדות בהודעה: (payload)
 - `group_id` (integer) מזהה הקבוצה.
 - `sender_username` (string) שם המשתמש שהפיץ את המפתח.
 - `wrapped_group_key_b64` (string) מפתח ה AES-הקבוצתי המוצפן.
 - `timestamp` (string) חותמת הזמן של ההפצה.

✓ שם ההודעה : `E2EE_GROUP_MESSAGE`

- נשלחת מ :-לקוח (חבר קבוצה)
- נשלחת אל :-שרת (לשידור לקבוצה)
- מבנה השדות בהודעה: (payload)
 - `group_id` (integer) מזהה הקבוצה.
 - `aes_payload` (object) אובייקט המכיל את ההודעה המוצפנת AES
 - `iv_b64` (string) ה IV ששימש להצפנה מקודד Base64
 - `ciphertext_b64` (string) תוכן ההודעה המוצפן AES מקודד Base64

✓ שם ההודעה : `RECEIVE_E2EE_GROUP_MESSAGE`

- נשלחת מ :-שרת
- נשלחת אל :-לקוח (חבר קבוצה)
- מבנה השדות בהודעה: (payload)
 - `group_id` (integer) מזהה הקבוצה.

- sender_username (string) שם המשתמש ששלח את ההודעה.
- aes_payload (object) ההודעה המוצפנת AES
- timestamp (string) חותמת הזמן של ההודעה.

✓ שם ההודעה : GET_ALL_USERS

○ נשלחת מ :-לקוח

○ נשלחת אל :-שרת

○ אין שדות

✓ שם ההודעה : ALL_USERS_LIST

○ נשלחת מ :-שרת

○ נשלחת אל :-לקוח

○ מבנה השדות בהודעה : (payload)

- users (list of strings) רשימה של שמות המשתמשים הרשומים.

✓ שם ההודעה : GET_MY_GROUPS

○ נשלחת מ :-לקוח

○ נשלחת אל :-שרת

○ אין שדות

✓ שם ההודעה : MY_GROUPS_LIST

○ נשלחת מ :-שרת

○ נשלחת אל :-לקוח

○ מבנה השדות בהודעה : (payload)

- groups (list of objects) רשימה של הקבוצות שהמשתמש חבר בהן, כל אובייקט מכיל group_id, group_name, group_icon_filename

✓ שם ההודעה : GET_GROUP_DETAILS

○ נשלחת מ :-לקוח

○ נשלחת אל :-שרת

○ מבנה השדות בהודעה : (payload)

- group_id (integer) מזהה הקבוצה המבוקשת.

✓ שם ההודעה : GROUP_DETAILS_DATA

○ נשלחת מ :-שרת

○ נשלחת אל :-לקוח

○ מבנה השדות בהודעה: (payload)

- group_id (integer) מזהה הקבוצה.
- group_name (string) שם הקבוצה.
- creator_username (string) יוצר הקבוצה.
- group_icon_filename (string) שם קובץ האייקון.
- members (list of objects) רשימת חברי הקבוצה, כל אובייקט מכיל role.username

קיימים עוד מספר סוגים של הודעות לטיפול בקבוצות והגדרות שונות אך הן די דומות ליכולות שתיארתי כבר.

הודעות הקשורות לניהול תמונות:

✓ שם ההודעה: UPLOAD_PROFILE_PICTURE

○ נשלחת מ :-לקוח

○ נשלחת אל :-שרת

○ מבנה השדות בהודעה: (payload)

- image_data_base64 (string) נתוני התמונה מקודדים בBase64
- original_filename (string) שם הקובץ המקורי של התמונה.

✓ שם ההודעה: PROFILE_PICTURE_UPLOAD_STATUS

○ נשלחת מ :-שרת

○ נשלחת אל :-לקוח

○ מבנה השדות בהודעה: (payload)

- success (boolean) האם ההעלאה הצליחה.
- message (string) הודעת סטטוס.
- new_filename (string, optional) שם הקובץ החדש כפי שנשמר בשרת.

✓ שם ההודעה: GET_PROFILE_PICTURE

○ נשלחת מ :-לקוח

○ נשלחת אל :-שרת

○ מבנה השדות בהודעה: (payload)

- username (string) שם המשתמש שתמונתו מתבקשת.

✓ שם ההודעה : PROFILE_PICTURE_DATA

○ נשלחת מ :- שרת

○ נשלחת אל :- לקוח

○ מבנה השדות בהודעה : (payload)

- username (string) שם המשתמש.
- filename (string, nullable) שם קובץ התמונה.
- image_data_base64 (string, nullable) נתוני התמונה מקודדים ב-Base64.
- message (string, optional) הודעת שגיאה אם התמונה לא נמצאה.

✓ שם ההודעה : UPLOAD_GROUP_ICON

○ נשלחת מ :- לקוח (מנהל קבוצה)

○ נשלחת אל :- שרת

○ מבנה השדות בהודעה : (payload)

- group_id (integer) מזהה הקבוצה.
- image_data_base64 (string) נתוני האייקון מקודדים ב-Base64
- original_filename (string) שם הקובץ המקורי של האייקון.

✓ שם ההודעה : GROUP_PICTURE_UPLOAD_STATUS

○ נשלחת מ :- שרת

○ נשלחת אל :- לקוח

○ מבנה השדות בהודעה : (payload)

- success (boolean) האם ההעלאה הצליחה.
- group_id (integer) מזהה הקבוצה.
- new_filename (string, optional) שם קובץ האייקון החדש.
- message (string) הודעת סטטוס.

✓ שם ההודעה : GET_GROUP_ICON

○ נשלחת מ :- לקוח

○ נשלחת אל :- שרת

○ מבנה השדות בהודעה : (payload)

▪ group_id (integer) מזהה הקבוצה.

✓ שם ההודעה : GROUP_ICON_DATA

○ נשלחת מ :- שרת

○ נשלחת אל :- לקוח

○ מבנה השדות בהודעה : (payload)

▪ group_id (integer) מזהה הקבוצה.

▪ filename (string, nullable) שם קובץ האייקון.

▪ image_data_base64 (string, nullable) נתוני האייקון מקודדים ב-Base64.

▪ message (string, optional) הודעת שגיאה.

הודעות שגיאה וסטטוס כלליות:

✓ שם ההודעה : ERROR / SERVER_ERROR / SYSTEM_ERROR

○ נשלחת מ :- שרת

○ נשלחת אל :- לקוח

○ מבנה השדות בהודעה : (payload)

▪ message (string) תיאור השגיאה.

תיאור מסכי המערכת:

לכל מסך מיועד בפרויקט:

➤ מסך התחברות (Login Screen)

תיאור המסך (איזה מידע מציג, מה ניתן לבצע): זהו המסך הראשון שמופיע בהרצת הלקוח. המסך מציג שדות להזנת "שם משתמש" ו"סיסמה". המשתמש יכול להזין את פרטיו וללחוץ על כפתור "התחבר" כדי לשלוח בקשת התחברות לשרת. בנוסף, קיימים קישורים או כפתורים למעבר למסך "הרשמה" (עבור משתמשים חדשים) ולעיתים למסך "שכחתי סיסמה" (אם ממומש). במקרה של שגיאת התחברות (למשל, פרטים שגויים), תוצג הודעת שגיאה מתאימה במסך זה.

תיאור המסך: מסך זה מיועד למשתמשים חדשים המעוניינים ליצור חשבון במערכת. הוא מציג שדות להזנת "שם משתמש" רצוי ו"סיסמה" ו"אימות סיסמה". לאחר מילוי הפרטים, המשתמש לוחץ על כפתור "הירשם". הלקוח שולח את הפרטים לשרת, ולאחר קבלת תשובה מהשרת, מוצגת הודעת הצלחה או כישלון. במקרה של הצלחה, המשתמש מופנה למסך ההתחברות.

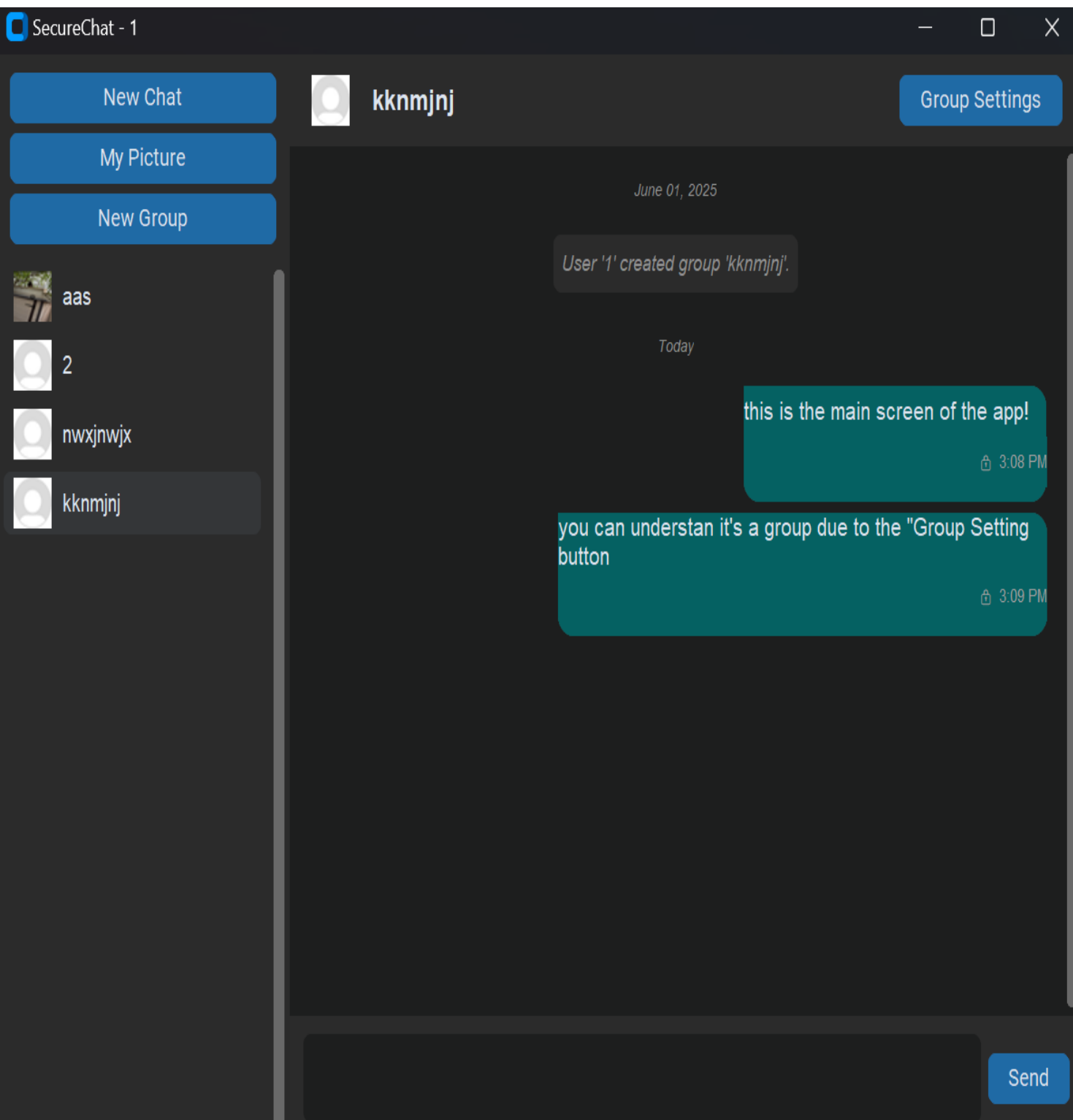
➤ המסך הראשי:

תיאור המסך (איזה מידע מציג, מה ניתן לבצע): זהו המסך המרכזי של האפליקציה לאחר שהמשתמש התחבר בהצלחה. מסך זה מחולק בדרך כלל למספר אזורים:

- **רשימת צ'אטים:** מציגה את שם איש הקשר או הקבוצה שאיתם מתנהלת השיחה הנוכחית, ותמונת פרופיל/אייקון קבוצה. בנוסף למעלה יש אפשרויות של: התחלת צ'אט חדש, יצירת קבוצה ושינוי תמונת פרופיל.
- **אזור השיחה הפעילה:**

מציגה את השיחה בצ'אט הנוכחי. מוצגת כל ההיסטוריה של ההודעות. בנוסף למעלה מוצג שם המשתמש ולידו תמונת הפרופיל או בקבוצות מוצג שם הקבוצה ולידו אייקון הקבוצה. בנוסף בקבוצות יש אפשרות בצד למעלה מימין לעבור למסך הגדרות הקבוצה וגם מוצגות הודעות המערכת של

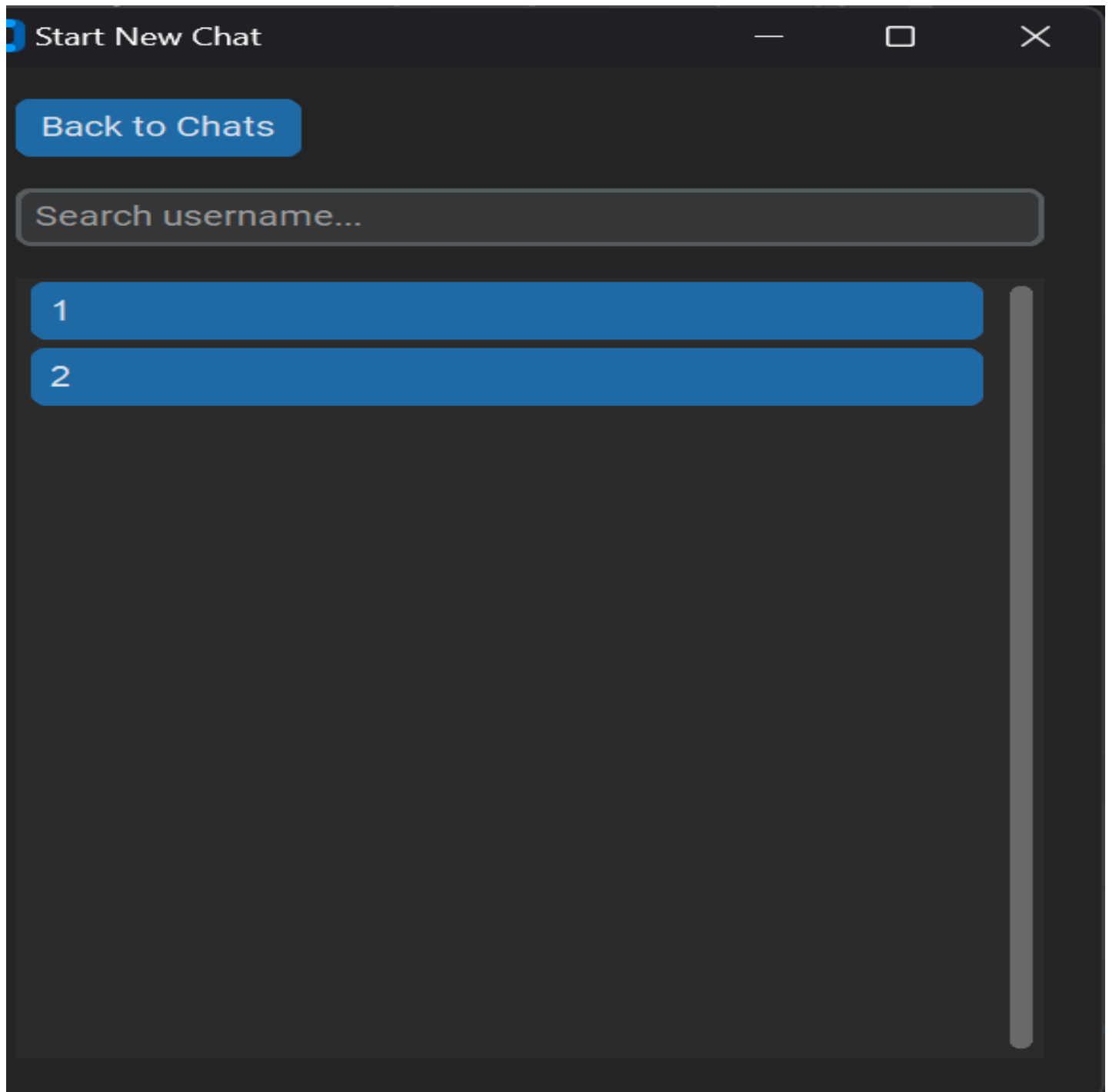
הקבוצה. יתר על כן למטה מימין יש אזור לכתיבת הודעה חשובה ולידו כפתור שליחת הודעה חדשה.



➤ בחירת משתמש חדש לשיחה חדשה.

תיאור המסך: מסך זה מאפשר למשתמש לחפש משתמשים אחרים הרשומים במערכת ולהתחיל איתם שיחה פרטית חדשה. הוא כולל שדה לחיפוש שם משתמש

ורשימה של תוצאות החיפוש (או רשימה של כל המשתמשים אם החיפוש ריק).
לחיצה על משתמש מהרשימה תפתח חלון שיחה חדש איתו במסך הראשי.



➤ מסך יצירת קבוצה חדשה:

תיאור המסך (איזה מידע מציג, מה ניתן לבצע): מסך זה מאפשר למשתמש ליצור קבוצת צ'אט חדשה. הוא יכול שדה להזנת שם הקבוצה הרצוי, ואפשרות לבחור חברים ראשוניים להוספה לקבוצה (מרשימת המשתמשים הקיימים). לאחר אישור, הקבוצה נוצרת והמשתמש (כמנהל ראשוני) יכול להתחיל לשלוח בה הודעות.

Create New Group

Create Group

Back to Chats

Group Name:

Select Members:

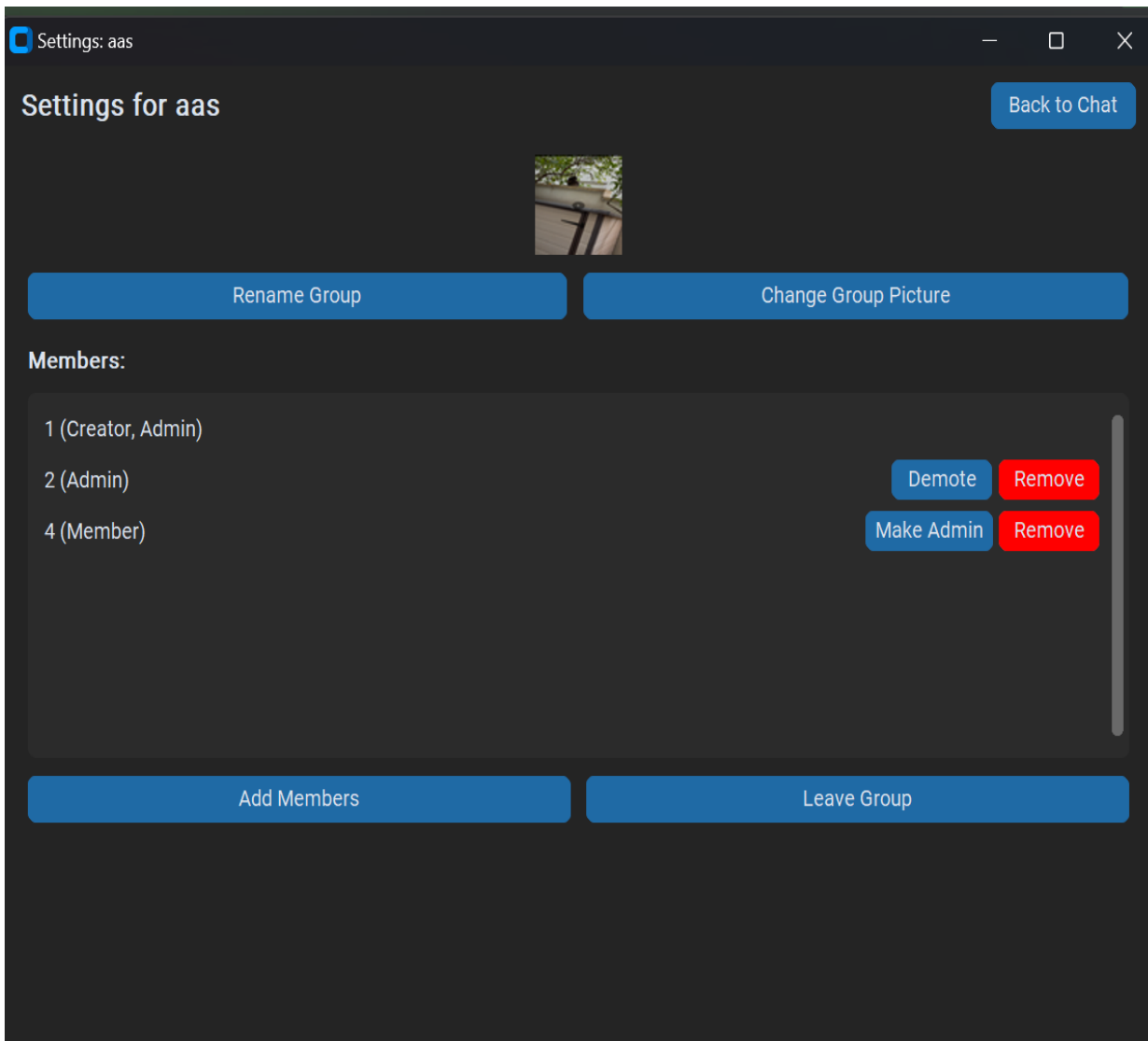
☐ 1

☐ 2

Create Group

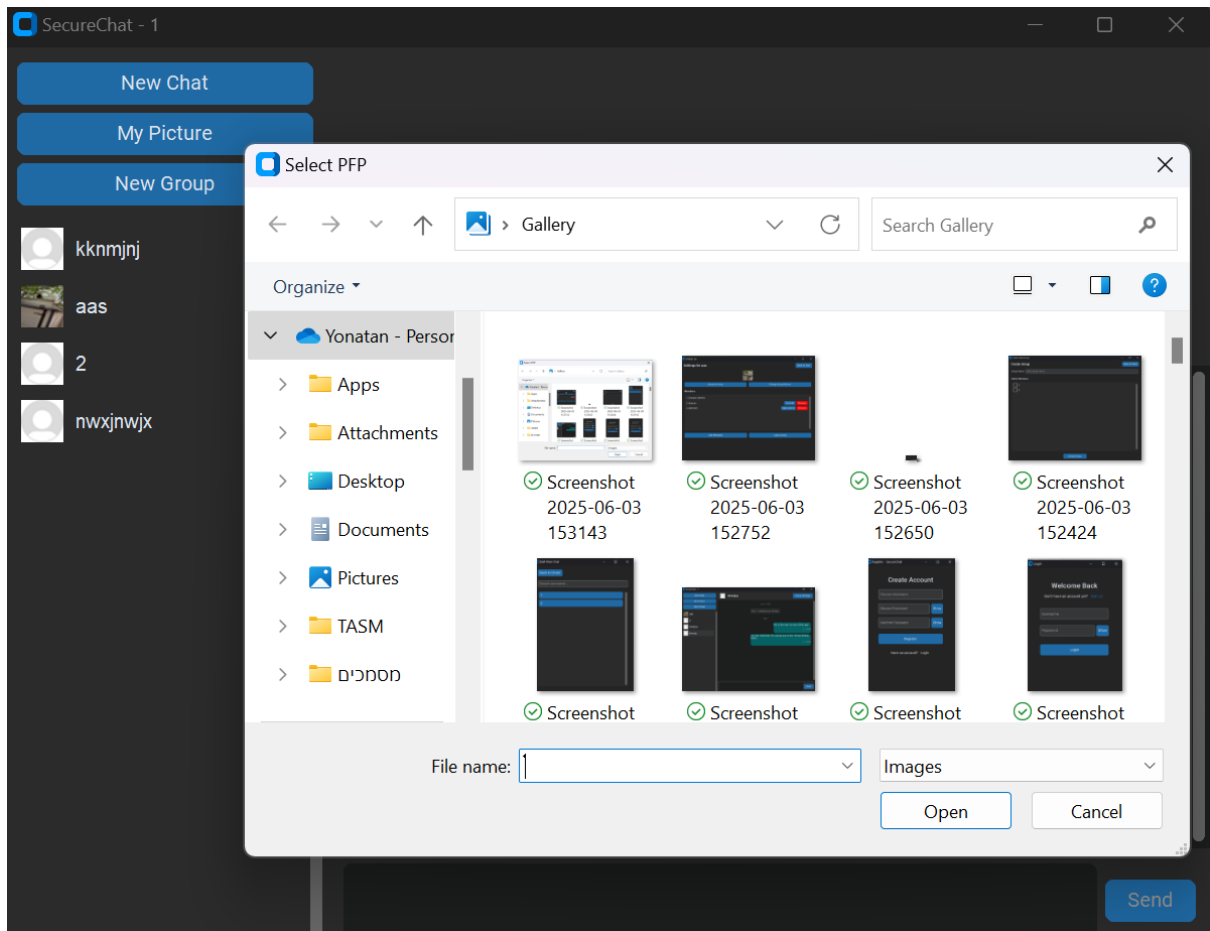
מסך הגדרות קבוצה:

תיאור המסך (איזה מידע מציג, מה ניתן לבצע): למסך זה ניתן להיכנס משיחה קבוצתית ומאפשר למנהלי הקבוצה לנהל את הגדרותיה. הוא יציג את שם הקבוצה הנוכחי, האייקון שלה, ורשימת החברים עם תפקידיהם. הפעולות שניתן לבצע (עבור מנהלים) כוללות: שינוי שם הקבוצה, העלאת/שינוי אייקון קבוצה, הוספת חברים חדשים, הסרת חברים קיימים, ושינוי תפקידי חברים (הפיכה למנהל/הורדה ממנהל). משתמשים שאינם מנהלים יוכלו לצפות בפרטי הקבוצה ולעזוב אותה.



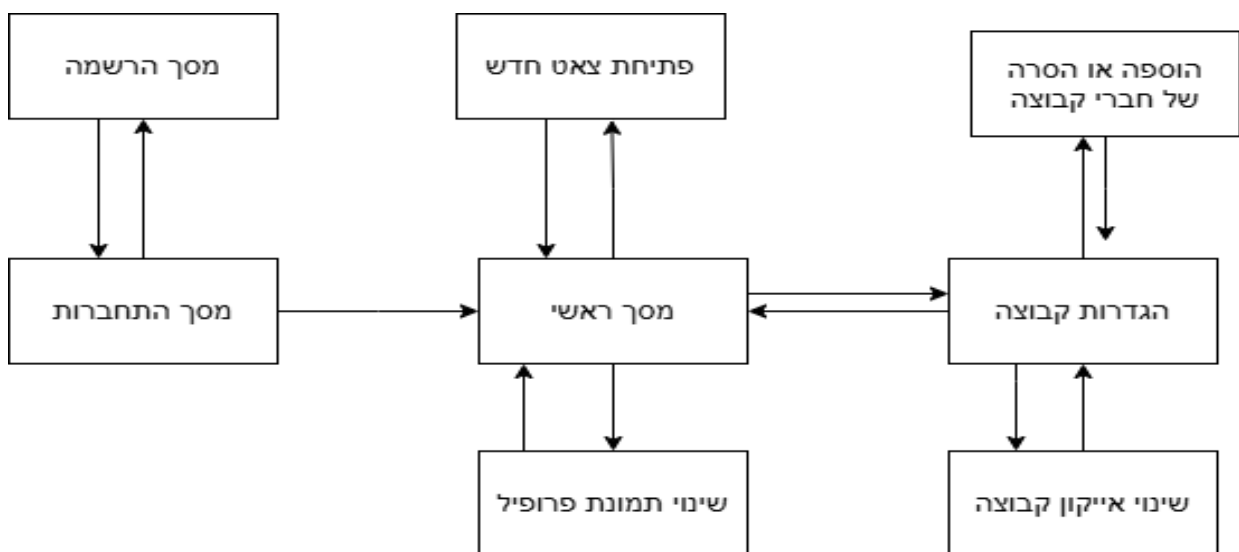
➤ מסך העלאת תמונת פרופיל/אייקון קבוצה:

תיאור המסך (איזה מידע מציג, מה ניתן לבצע): זהו אינו מסך מלא אלא חלון דיאלוג סטנדרטי של מערכת ההפעלה (filedialog) הנפתח כאשר המשתמש בוחר להעלות תמונת פרופיל או אייקון קבוצה. החלון מאפשר למשתמש לנווט במערכת הקבצים שלו ולבחור קובץ תמונה (בפורמטים נתמכים כמו: JPG, PNG) לאחר בחירת הקובץ, הלקוח מעבד ושולח אותו לשרת.



בנוסף קיים מסך של משתמשים שמסירים ומשתמשים שמוסיפים לקבוצה שנראה זהה לזה של יצירת הקבוצה רק שמציג את מי שלא בקבוצה כבר עכשיו או את מי שבקבוצה.

תרשים מסכים המתאר את היררכיית המסכים והמעברים ביניהם (diagram flow Screen) :



תיאור מבני הנתונים:

פירוט מבני הנתונים (מסד נתונים, קבצים, מקומיים וכו'):

הפרויקט שלי עושה שימוש במספר סוגים של מבני נתונים לאחסון וניהול המידע שלה. המרכיב המרכזי הוא מסד נתונים בצד השרת, המאוחסן בקובץ chat_app.db ומנוהל באמצעות SQLite3. מסד נתונים זה אחראי על שמירת כל המידע הקבוע והמשתנה של המערכת, כגון פרטי משתמשים, תוכן הודעות, פרטי קבוצות, והקשרים בין משתמשים לקבוצות. הוא מהווה את בסיס הנתונים עבור נתונים אלו.

בנוסף למסד הנתונים המרכזי, המערכת משתמשת בקבצים מקומיים בצד הלקוח לצורך אחסון מידע רגיש הקשור למשתמש הספציפי. קבצים אלו כוללים את קובץ המפתח הפרטי (RSA) של המשתמש, הנשמר תחת השם "client_private_key.pem" ומשמש לפענוח מידע שהוצפן עם המפתח הציבורי התואם של המשתמש. כמו כן, נשמר מקומית קובץ המכיל את המפתח הציבורי (RSA) של המשתמש ("client_public_key.pem"), לנוחות ולשליחה לשרת. לבסוף, מפתחות ה-AES המשותפים לקבוצות שהמשתמש חבר בהן נשמרים מקומית בקובץ JSON בשם: "group_aes_keys.json" כאשר המפתחות עצמם מקודדים ב-Base64.

בצד השרת, מלבד מסד הנתונים, מתבצע גם **אחסון קבצים במערכת הקבצים** עבור מדיה. תמונות פרופיל שהועלו על ידי משתמשים נשמרות בתיקייה ייעודית בשם "profile_pics": ואייקוני קבוצות שהועלו נשמרים בתיקייה בשם "group_icons": שמות הקבצים בתיקיות אלו מקושרים לרשומות המתאימות במסד הנתונים. בנוסף נשמרים בתיקיות קבצים בשם: "default_icon" ו"default_pic".

מסד נתונים:

טבלאות:

טבלה 1 - users

- תפקיד: אחסון מידע על כל המשתמשים הרשומים במערכת.
- מפתח ראשי: id

שם השדה	תיאור	טיפוס השדה	דוגמה לערך	מאפיינים מיוחדים
id	מזהה ייחודי של המשתמש (רץ באופן אוטומטי)	INTEGER	1, 2, 0105	PRIMARY KEY AUTOINCREMENT
username	שם המשתמש הייחודי, כפי שנבחר על ידי המשתמש בעת ההרשמה.	TEXT	"Yonatan"	UNIQUE NOT NULL

password_hash	גיבוב של סיסמת המשתמש, מאוחסן לאחר תהליך גיבוב מאובטח.	TEXT	מחרוזת (הקסדימלית) ארוכה "a1b2c3d4..."	NOT NULL
salt	ה"מלח" (salt) "האקראי" ששימש לגיבוב הסיסמה של משתמש זה.	TEXT	מחרוזת (הקסדימלית) "e5f6g7h8..."	NOT NULL
profile_picture_filename	שם הקובץ של תמונת הפרופיל של המשתמש (הקובץ עצמו נשמר בתיקיית profile_pics).	TEXT	"yonatan.png", "default_pic.jpg"	DEFAULT 'default_pic.jpg'
rsa_public_key_pem	המפתח הציבורי (RSA) של המשתמש בפורמט PEM, המשמש להצפנת מפתחות המיועדים לו.	TEXT	מחרוזת PEM ארוכה "-----BEGIN PUBLIC KEY..."	DEFAULT NULL

טבלה 2 – groups

תפקיד: אחסון מידע על קבוצות שנוצרו

מפתח ראשי: group_id

שם השדה	תיאור	טיפוס השדה	דוגמה לערך	מאפיינים מיוחדים
group_id	מזהה ייחודי של הקבוצה (רץ באופן אוטומטי).	INTEGER	1, 5, 22	PRIMARY KEY AUTOINCREMENT
group_name	שם הקבוצה, כפי שהוגדר על ידי יוצר הקבוצה.	TEXT	"חברים ללימודים", "צוות פיתוח"	NOT NULL
creator_username	שם המשתמש שיצר את הקבוצה.	TEXT	"alice"	NOT NULL, FOREIGN KEY (creator_username) REFERENCES

users(username) ON DELETE SET NULL				
DEFAULT CURRENT_TIMESTAMP	"2023-05-15 10:30:00.123"	DATETIME	חותמת הזמן של מועד יצירת הקבוצה.	created_at
DEFAULT 'default_icon.jpg'	"group_1_icon.png", "default_icon.jpg"	TEXT	שם הקובץ של אייקון הקבוצה (הקובץ עצמו נשמר בתיקיית group_icons).	group_icon_filename

טבלה 3 – group members

תפקיד: אחסון מידע על קבוצות

מפתח – group_id

שם השדה	תיאור	טיפוס השדה	דוגמה לערך	מאפיינים מיוחדים
group_id	מזהה הקבוצה אליה המשתמש משויך.	INTEGER	1	NOT NULL, FOREIGN KEY (group_id) REFERENCES groups(group_id) ON DELETE CASCADE
username	שם המשתמש החבר בקבוצה.	TEXT	"bob123"	NOT NULL, FOREIGN KEY (username) REFERENCES users(username) ON DELETE CASCADE
role	תפקיד המשתמש בקבוצה (למשל, חבר רגיל או מנהל).	TEXT	"member", "admin"	NOT NULL DEFAULT 'member'
joined_at	חותמת הזמן של מועד הצטרפות	DATETIME	"2023-05-16 11:00:00.456"	DEFAULT CURRENT_TIMESTAMP

			המשתמש לקבוצה.	
--	--	--	----------------	--

טבלה 4 – messages

שם השדה	תיאור	טיפוס השדה	דוגמה לערך	מאפיינים מיוחדים
id	מזהה ייחודי של ההודעה.	INTEGER	1, 1001, 5023	PRIMARY KEY AUTOINCREMENT
sender	שם המשתמש ששלח את ההודעה.	TEXT	"alice"	NOT NULL
recipient	שם המשתמש הנמען (עבור הודעות פרטיות).	TEXT	"bob123", NULL עבור הודעה קבוצתי	FOREIGN KEY (recipient) REFERENCES users(username) ON DELETE CASCADE
recipient_group_id	מזהה הקבוצה הנמענת (עבור הודעות קבוצתיות).	INTEGER	1, NULL הודעה פרטית	FOREIGN KEY (recipient_group_id) REFERENCES groups(group_id) ON DELETE CASCADE
message	תוכן ההודעה. עבור הודעות מוצפנות מקצה לקצה, שדה זה יכיל את הטקסט המוצפן (לרוב כ-JSON המכיל IV וצופן, או נתונים בינאריים מקודדים).	TEXT	"שלום לך, {\"iv_b64\": \"...\", \"ciphertext_b64\": \"...\"} (E2EE)"	NOT NULL

DEFAULT CURRENT_TIMESTAMP	"2023-05-17 12:34:56.789"	DATE TIME	חותמת הזמן של מועד שליחת/קבלת ההודעה.	timestamp
DEFAULT FALSE NOT NULL	0 (FALSE), 1 (TRUE)	BOOLE AN	ציון האם ההודעה היא הודעת מערכת (למשל, "משתמש הצטרף לקבוצה") או הודעת משתמש רגילה.	is_system_message
DEFAULT NULL	"AES_SESSION_INIT", "AES_MSG", "GROUP_AES"	TEXT	סוג ההצפנה מקצה לקצה שנעשה בה שימוש עבור הודעה זו, אם רלוונטי.	e2ee_type
DEFAULT NULL	מחרוזת של Base64 מפתח AES (RSA) מוצפן "AbCdEfGh..."	TEXT	עבור הודעות אתחול סשן (AES_SESSION_INIT), מכיל את מפתח ה- AES של הסשן כשהוא מוצפן עם המפתח הציבורי של השולח עצמו (לצורך פענוח ההיסטוריה שלו).	sender_e2ee_session_data

סקירת חולשות ואיומים

✓ שכבת האפליקציה:

המשתמש חשוף למספר איומים בשכבת האפליקציה אותם אני אציג:

🚩 **עבודה עם בסיס נתונים sql injection:** הזרקת SQL היא טכניקת תקיפה נפוצה שבה תוקף מנסה להחדיר פקודות SQL זדוניות לתוך שאילתות המופנות למסד הנתונים, בדרך כלל דרך שדות קלט באפליקציה. הצלחה של הזרקה כזו עלולה לאפשר לתוקף לקרוא מידע רגיש, לשנות נתונים, למחוק נתונים, ואף להשתלט על מסד הנתונים כולו. לדוגמה, בשדה הזנת סיסמה, תוקף עלול להזין קלט כמו OR '1'='1' בניסיון לעקוף את מנגנון האימות.

🚩 **הפתרון אצלי:** בפרויקט נעשה שימוש ב **Parameterized Queries** - שאילתות פרמטריות בעת גישה למסד הנתונים באמצעות ספריית sqlite3 של פייתון. בשיטה זו, הקלט מהמשתמש מועבר לשאילתת ה-SQL כפרמטר נפרד ולא כחלק ממחרוזת השאילתה עצמה (למשל `cursor.execute("SELECT * FROM users WHERE username = ? (username_input)")`). כן מסד הנתונים מתייחס לקלט כנתון בלבד ולא כחלק מהפקודה, מה שמונע את האפשרות להזריק קוד SQL זדוני. בנוסף מתבצעות בדיקות תקינות בסיסיות בצד הלקוח על קלטים מסוימים כדי למנוע הכנסת תווים שעלולים לגרום לבעיות.

• עבודה עם אתרי Web:

🚩 פרויקט זה הוא אפליקציית Desktop שולחנית ואינו מערב עבודה ישירה עם טכנולוגיות Web בצד השרת או הלקוח (כגון הרצת קוד בצד הדפדפן). לפיכך איומים נפוצים לאתרי Web לא יוכלו לעבוד פה.

• תהליך ה Login אימות ווידוא :

🚩 **האיום:** ניחוש סיסמאות, גניבת פרטי הזדהות, והתחזות למשתמשים אחרים.

🚩 **הפתרון:**

- **גיבוב סיסמאות מאובטח:** כפי שפורט בסעיף האלגוריתמים, סיסמאות המשתמשים אינן נשמרות כטקסט רגיל אלא מגובבות באמצעות אלגוריתם PBKDF2 עם HMAC-SHA256 ו"מלח" ייחודי לכל משתמש. זה מקשה מאוד על שחזור הסיסמה המקורית גם אם מסד הנתונים נפרץ.
- **שיוך מפתח ציבורי למשתמש:** בעת ההתחברות הלקוח שולח את המפתח הציבורי (RSA) שלו לשרת, והשרת שומר או מעדכן אותו. מפתח זה משמש לאחר מכן בתהליכי הצפנה מקצה לקצה.

• מתקפת אדם באמצע (MITM – Man-in-the-Middle):

🚩 **האיום:** תוקף המצליח ליירט את התקשורת בין שני לקוחות, או בין לקוח לשרת, ועלול לקרוא או לשנות את המידע המועבר.

🚩 **הפתרון בפרויקט עבור תוכן ההודעות (הצפנה מקצה לקצה (E2EE)):**

- **סוג ההצפנה:** נעשה שימוש בהצפנה היברידית. אלגוריתם RSA משמש להחלפה מאובטחת של מפתחות סימטריים (AES) ואלגוריתם AES עם מפתח 256 ביט, וריפוד משמש להצפנת תוכן ההודעות עצמן (הן פרטיות והן קבוצתיות).
 - **עיקרון הפעולה:** השרת משמש כמתווך להעברת ההודעות המוצפנות אך אינו מחזיק במפתחות הפרטיים של המשתמשים או במפתחות AES של הסשנים/קבוצות בצורתם הגלויה, ולכן אינו יכול לפענח את תוכן ההודעות. רק הנמענים המורשים, המחזיקים במפתחות המתאימים, יכולים לפענח את ההודעות.
 - **הגנה על החלפת מפתחות:** המפתח הציבורי של השרת נשלח ללקוח בעת החיבור הראשוני. החלפת מפתחות AES בין לקוחות נעשית על ידי הצפנתם עם מפתחות RSA ציבוריים של הנמענים.
- **מתקפות מניעת שירות: (DoS/DDoS)**
 - 🚩 **האיום:** הצפת השרת במספר רב של בקשות במטרה לגרום לקריסתו או לחוסר זמינותו עבור משתמשים לגיטימיים.
 - 🚩 **הפתרון בפרויקט בסיסי:** השרת ממומש עם מנגנון ריבוי תהליכונים (threading) כאשר כל לקוח מחובר מטופל על ידי תהליכון נפרד. קיימת הגבלה מובנית ב `socket.listen()` על מספר החיבורים הממתינים בתור. עם זאת, לא יושמו מנגנונים מתקדמים להגנה מפני DoS/DDoS כגון הגבלת קצב בקשות (rate limiting) פר לקוח/כתובת IP, חסימה דינמית של כתובות חשודות, או שימוש בשירותי הגנה חיצוניים. זוהי נקודה לשיפור אפשרי במערכת.
 - **העלאת קבצים (תמונות פרופיל ואייקוני קבוצות):**
 - 🚩 **האיום:** העלאת קבצים זדוניים (למשל, המכילים קוד מזיק שיופעל בעת ניסיון להציגם) או קבצים גדולים מאוד שיכולים להעמיס על שטח האחסון או על רוחב הפס של השרת.
 - 🚩 **הפתרון בפרויקט בסיסי:** בצד הלקוח, קיימת הגבלה על סוגי הקבצים המותרים להעלאה בדרך כלל קובצי תמונה כמו PNG, JPG וכן עיבוד ראשוני של התמונה (כגון שינוי גודל). בצד השרת, הקבצים נשמרים בתיקיות ייעודיות. עם זאת, לא מבוצעת בשרת בדיקה מעמיקה של תוכן הקובץ (למשל, סריקת וירוסים) או אימות קפדני של סוג הקובץ מעבר לסיומת. כמו כן, לא מיושמת הגבלת גודל קובץ מחמירה בצד השרת.
- שכבת התעבורה:**
- **פרוטוקול TCP ולחיצת יד משולשת:**
 - בפרויקט זה התקשורת משתמשת בפרוטוקול TCP להעברת נתונים. מספק אמינות שמירה על סדר ההודעות, ובקרת זרימה. תהליך לחיצת היד המשולשת, SYN, SYN-ACK, ACK מבסס את החיבור בין הלקוח לשרת לפני העברת נתונים.

- **האיום TCP**: עצמו אינו פרוטוקול מוצפן. ללא שכבת אבטחה נוספת, תוקף ברשת עלול ליירט את התעבורה.
- **הפתרון**: כפי שתואר לעיל, הצפנה מקצה לקצה (E2EE) ברמת האפליקציה מגנה על *תוכן* ההודעות המועברות על גבי TCP. במערכות ייצור, נהוג להוסיף גם הצפנה של כל ערוץ התקשורת באמצעות TLS (Transport Layer Security) שלא מומש בפרויקט זה.

הפעלת המערכת

• חולשות אפשריות :

- **הזרקת קוד** : תיאורטית, אם קלט מהמשתמש היה משמש לבניית פקודות מערכת או נתיבי קבצים בצורה לא מבוקרת, הדבר עלול היה לאפשר הזרקת קוד. בפרויקט הנוכחי, עיקר הקלט משמש לנתוני אפליקציה (שמות משתמש, הודעות) או מועבר בצורה מובנית, (JSON) מה שמצמצם סיכון זה.
- **אחסון לא מאובטח של מפתחות פרטיים וקבוצתיים בצד הלקוח**: קבצי המפתח הפרטי של (client_private_key.pem) RSA ומפתחות ה AES הקבוצתיים (group_aes_keys.json) נשמרים מקומית ללא שכבת הצפנה נוספת המבוססת על סיסמת משתמש. גישה לא מורשית לקבצים אלו במחשב הלקוח עלולה לחשוף את המפתחות.
- **איך טופלו**: הזרקת קוד : נמנע שימוש ישיר בקלט משתמש לבניית פקודות מערכת. אבטחת מפתחות לקוח : במסגרת הפרויקט לא מומשה הצפנה נוספת לקבצי המפתחות המקומיים. שיפור אפשרי היה דרישת סיסמה מהמשתמש בעת הפעלת האפליקציה, ושימוש בה לגזירת מפתח שיצפין ויפענח את קבצי המפתחות הללו.

מימוש הפרויקט:

סקירת כל המודולים / מחלקות המרכיבים את המערכת וקשרי הגומלין ביניהם:

שם המודול המיובא	למה מיועד המודול
socket	יצירה וניהול של חיבורי רשת (TCP/IP) בין השרת ללקוחות.
threading	יצירה וניהול של תהליכונים (Threads) לטיפול מקבילי בלקוחות מרובים בצד השרת.
json	דחיסה וסידור של אובייקטי Python למחרוזות JSON ולהפך, לצורך העברת נתונים בפרוטוקול התקשורת.
sqlite3	התחברות וביצוע פעולות על מסד נתונים מסוג SQLite
hashlib	יצירת גיבוב מאובטח לסיסמאות באמצעות אלגוריתם PBKDF2
cryptography	פונקציות קריפטוגרפיות מתקדמות: יצירת מפתחות RSA ו AES הצפנה ופענוח א-סימטריים (RSA) וסימטריים (AES), ריפוד קריפטוגרפי.
customtkinter	בניית ממשק משתמש גרפי (GUI) מודרני ואינטראקטיבי עבור אפליקציית הלקוח.
os	פונקציות הקשורות למערכת ההפעלה, כגון עבודה עם נתיבי קבצים, יצירת תיקיות ובדיקת קיום קבצים.
base64	קידוד נתונים בינאריים (כמו תמונות) למחרוזת Base64 ופענוח חזרה, לצורך שמירה או שליחה בפורמט טקסטואלי (JSON).
datetime	עבודה עם חותמות זמן ותאריכים.
PIL (Pillow)	עיבוד תמונות, כגון שינוי גודל ופורמט של תמונות פרופיל ואייקוני קבוצות.
queue	יצירת תור (Queue) לניהול הודעות נכנסות מהשרת באופן א-סינכרוני באפליקציית הלקוח.

שם המחלקה 1: NetworkNode קובץ shared_modules.py :

- **תפקיד המחלקה:** מחלקה בסיסית המספקת פונקציונליות קריפטוגרפית לניהול, יצירה, טעינה ושמירה של זוג מפתחות RSA (ציבורי ופרטי), וכן לביצוע פעולות הצפנה ופענוח באמצעות מפתחות RSA. משמשת כיסוד הן לשרת והן ללקוח בפעולות הדורשות יכולות הצפנה א-סימטרית.

- **תכונות המחלקה:**

שם תכונה	תפקיד ושימוש
private_key	משמש לפענוח הודעות מוצפנות ע"י המפתח הציבורי
public_key	משמש להצפנת הודעות RSA

• פעולות המחלקה:

שם פעולה	טענת כניסה (פרמטרים אותם מקבלת)	טענת יציאה (מה הפעולה מבצעת ומה מחזירה)
__init__	self	מאתחלת את המחלקה על ידי טעינה או יצירה של זוג מפתחות RSA.
_save_private_key	self, pk, filename=CLIENT_PRIVATE_KEY_FILE	שומרת את המפתח הפרטי הנתון לקובץ PEM.
_save_public_key	self, pubk, filename=CLIENT_PUBLIC_KEY_FILE	שומרת את המפתח הציבורי הנתון לקובץ PEM.
_load_private_key	self, filename=CLIENT_PRIVATE_KEY_FILE	טוענת מפתח פרטי מקובץ PEM מחזירה אובייקט מפתח פרטי או None אם נכשל.
_generate_new_rsa_keys	self	יוצרת זוג מפתחות RSA חדש (פרטי וציבורי) ושומרת אותם לקבצים.
_load_or_generate_rsa_keys	self	מנסה לטעון זוג מפתחות RSA קיים. אם הטעינה נכשלת, יוצרת זוג מפתחות חדש. מבטיחה שלמופע יהיו מפתחות זמינים.
get_public_key_pem	self	מחזירה את המפתח הציבורי של הצומת בפורמט PEM כמחרוזת, או None אם המפתח לא זמין.
encrypt_message	self, message_bytes, recipient_public_key_pem_str	מצפינה את message_bytes באמצעות המפתח הציבורי של הנמען. מחזירה את הבתים המוצפנים או None במקרה של שגיאה.
decrypt_message	self, encrypted_message_bytes	מפענחת את encrypted_message_bytes באמצעות המפתח הפרטי של הצומת. מחזירה את הבתים המפוענחים (בדרך כלל מחרוזת הודעה) או None במקרה של שגיאה (למשל, אם המפתח לא מתאים).

שם המחלקה 2: DatabaseManager קובץ shared_modules.py :

- **תפקיד המחלקה:** מנהלת את כל האינטראקציות עם מסד הנתונים SQLite של האפליקציה. זה כולל יצירת טבלאות, הוספת משתמשים, אחסון ושליפה של הודעות, ניהול קבוצות, אחסון מפתחות ציבוריים של משתמשים ועוד.

- **תכונות המחלקה:**

שם תכונה	תפקיד ושימוש
db_name	מחרוזת המכילה את שם קובץ מסד הנתונים (למשל , 'chat_app.db').

פעולות המחלקה:

שם פעולה	טענת כניסה (פרמטרים אותם מקבלת)	טענת יציאה (מה הפעולה מבצעת ומה מחזירה)
<code>__init__</code>	<code>self, db_name=DB_NAME</code>	מאתחלת את מנהל מסד הנתונים עם שם קובץ מסד הנתונים.
<code>hash_password</code>	<code>password_str, salt_bytes=None</code>	מבצעת גיבוב (hashing) לסיסמה הנתונה באמצעות PBKDF2-HMAC-SHA256 עם salt. מחזירה tuple של הסיסמה המגובבת (hex) וה-salt (hex).
<code>verify_password</code>	<code>stored_password_hash_hex, provided_password_str, salt_hex</code>	מאמתת סיסמה שסופקה מול סיסמה מגובבת שמורה, תוך שימוש ב salt-המתאים. מחזירה True אם הסיסמאות תואמות False, אחרת.
<code>initialize_database</code>	Self	יוצרת את הטבלאות (users, groups, group_members, messages) במסד הנתונים אם אינן קיימות. מוסיפה עמודות חדשות) כמו rsa_public_key_pem, (e2ee_type במידת הצורך. יוצרת אינדקסים ומשתמש מערכת. SYSTEM_EVENT מבטיחה שתיקיות המדיה (profile_pics, group_icons) קיימות.
<code>add_user_to_db</code>	<code>self, cursor, username, password_hash, salt, profile_picture_filename=None, rsa_public_key_pem=None</code>	מוסיפה משתמש חדש למסד הנתונים עם הפרטים הנתונים. מחזירה True בהצלחה, False אחרת (למשל, אם שם המשתמש כבר קיים).
<code>get_user_credentials_from_db</code>	<code>self, cursor, username</code>	שולפת את הגיבוב של הסיסמה וה salt עבור משתמש נתון. מחזירה tuple של (password_hash, salt) אם המשתמש לא נמצא.
<code>store_user_rsa_public_key</code>	<code>self, cursor, username, rsa_public_key_pem_str</code>	שומרת או מעדכנת את המפתח הציבורי RSA (בפורמט PEM של משתמש נתון במסד הנתונים. מחזירה True אם העדכון הצליח.
<code>get_user_rsa_public_key_from_db</code>	<code>self, cursor, username</code>	שולפת את המפתח הציבורי RSA (PEM) של משתמש נתון ממסד הנתונים. מחזירה None אם לא נמצא.
<code>update_user_profile_picture_in_db</code>	<code>self, cursor, username, new_profile_picture_filename</code>	מעדכנת את שם קובץ תמונת הפרופיל של משתמש נתון

במסד הנתונים. מחזירה True אם העדכון הצליח.		
שולפת את שם קובץ תמונת הפרופיל של משתמש נתון. מחזירה את שם הקובץ או None.	self, cursor, username	get_profile_picture_filename_from_db
שולפת רשימה של כל שמות המשתמשים הרשומים למעט (SYSTEM_EVENT) מחזירה רשימה של מחרוזות.	self, cursor	get_all_registered_users_from_db
בודקת אם משתמש עם שם נתון קיים במסד הנתונים. מחזירה True אם קיים, False אחרת.	self, cursor, username	check_user_exists
שומרת הודעה חדשה במסד הנתונים עם כל הפרטים הרלוונטיים. מחזירה את ה-ID של ההודעה החדשה.	self, cursor, sender, message_text, recipient_user=None, recipient_group_id=None, timestamp=None, is_system=False, e2ee_type=None, sender_e2ee_session_data=None	store_message_in_db
שולפת את כל ההודעות הפרטיות והקבוצתיות הרלוונטיות למשתמש נתון, ממוינות לפי חותמת זמן. מחזירה רשימה של מילונים, כל מילון מייצג הודעה.	self, cursor, username	get_all_messages_for_user_from_db
יוצרת קבוצה חדשה במסד הנתונים. מחזירה את ה-ID של הקבוצה החדשה או None בכישלון.	self, cursor, group_name, creator_username, created_at_ts=None, group_icon_filename=None	create_group_in_db
מוסיפה משתמש כחבר לקבוצה נתונה. מחזירה True בהצלחה, False אחרת.	self, cursor, group_id, username, role='member', joined_at_ts=None	add_member_to_group_in_db
שולפת את כל החברים בקבוצה נתונה. מחזירה רשימה של מילונים (username, role)	self, cursor, group_id	get_group_members_from_db
שולפת את כל הקבוצות שמשתמש נתון חבר בהן. מחזירה רשימה של מילונים (group_id, group_name, group_icon_filename).	self, cursor, username	get_user_groups_from_db
שולפת פרטים מלאים על קבוצה נתונה, כולל רשימת	self, cursor, group_id	get_group_details_from_db

החברים. מחזירה מילון עם פרטי הקבוצה או None		
בודקת אם המשתמש חבר בקבוצה. מחזירה True אם כן False, אחרת.	self, cursor, username, group_id	is_user_member_of_group
שולפת את תפקיד המשתמש בקבוצה. מחזירה את התפקיד (מחרוזת) או None	self, cursor, username, group_id	get_group_member_role
בודקת אם המשתמש הוא מנהל בקבוצה. מחזירה True אם כן False, אחרת.	self, cursor, username, group_id	is_user_admin_in_group
מסירה חבר מהקבוצה. מחזירה True אם ההסרה הצליחה.	self, cursor, group_id, username_to_remove	remove_member_from_group_in_db
מעדכנת את תפקיד החבר בקבוצה. מחזירה True אם העדכון הצליח.	self, cursor, group_id, target_username, new_role	update_group_member_role_in_db
מעדכנת את סמל הקבוצה. מחזירה True אם העדכון הצליח.	self, cursor, group_id, new_icon_filename	update_group_icon_in_db
שולפת את שם קובץ סמל הקבוצה. מחזירה את שם הקובץ או None	self, cursor, group_id	get_group_icon_filename_from_db
משנה את שם הקבוצה. מחזירה True אם השינוי הצליח.	self, cursor, group_id, new_group_name	rename_group_in_db
שולפת את שם משתמש יוצר הקבוצה. מחזירה את שם המשתמש או None	self, cursor, group_id	get_group_creator_from_db
סופרת את מספר החברים בקבוצה. מחזירה מספר שלם.	self, cursor, group_id	count_group_members_in_db
סופרת את מספר המנהלים בקבוצה. מחזירה מספר שלם.	self, cursor, group_id	count_group_admins_in_db
מוחקת קבוצה ממסד הנתונים. מחזירה True אם המחיקה הצליחה.	self, cursor, group_id	delete_group_from_db

שם המחלקה 3: ChatServer קובץ chat_server_core :

- **תפקיד המחלקה:** המחלקה הראשית של השרת. היא מאזינה לחיבורי לקוחות חדשים, מנהלת את הלקוחות הפעילים, ואחראית על אתחול וסגירה מסודרים של השרת. יורשת מ-NetworkNode כדי לנהל את מפתחות ה-RSA של השרת עצמו.

• תכונות המחלקה:

שם תכונה	תפקיד ושימוש
host	כתובת ה IP-שהשרת מאזין אליה.
port	מספר הפורט שהשרת מאזין אליו.
db_manager	מופע של DatabaseManager לגישה למסד הנתונים.
server_socket	אובייקט ה socket-הראשי של השרת המאזין לחיבורים.
active_clients	מילון הממפה שמות משתמשים למופעי ClientHandler הפעילים שלהם.
active_clients_lock	מנעול (threading.Lock) להבטחת גישה בטוחה למילון active_clients מתהליכונים שונים.
is_running	דגל בוליאני המציין אם השרת אמור להמשיך לרוץ.
client_handler_class	שמירת המחלקה (לא מופע) שתשמש ליצירת ClientHandler חדשים (מוזרק בזמן יצירת השרת).

• פעולות המחלקה:

שם פעולה	טענת כניסה (פרמטרים אותם מקבלת)	טענת יציאה (מה הפעולה מבצעת ומה מחזירה)
__init__	self, host, port, db_name=DB_NAME, client_handler_class=None	מאתחלת את השרת עם הגדרות הרשת, מנהל מסד הנתונים, ומחלקה לטיפול בלקוחות. קוראת ל __init__ של NetworkNode לאתחול מפתחות השרת.
start	self	מפעילה את השרת: מאתחלת את מסד הנתונים, יוצרת socket קושרת אותו לכתובת והפורט, מאזינה לחיבורים נכנסים, ומקבלת אותם בלולאה. עבור כל חיבור, יוצרת ומתחילה תהליכון ClientHandler חדש. מטפלת בשגיאות וב- KeyboardInterrupt.
shutdown	self	סוגרת את השרת באופן מסודר: מפסיקה לקבל חיבורים חדשים, סוגרת את ה socket-הראשי, מנתקת את כל הלקוחות הפעילים תוך שליחת הודעת סגירה, ומנקה משאבים.
register_client	self, username, client_handler_instance	רושמת לקוח פעיל חדש במילון active_clients אם המשתמש כבר מחובר ממקום אחר, מנתקת את החיבור הישן.
unregister_client	self, username, client_handler_instance	מסירה לקוח ממילון active_clients (בדרך כלל בעת התנתקות)

מחזירה את מופע ClientHandler המשויך לשם משתמש נתון, אם קיים ופעיל.	self, username	get_client_handler
מנסה לשלוח את המפתח הציבורי (PEM) של משתמש יעד, תחילה מלקוחות פעילים (אם הם שלחו אותו) ולאחר מכן ממסד הנתונים. מחזירה את מחרוזת המפתח או None	self, target_username	get_user_public_key_pem
שולחת את message_obj לכל החברים הפעילים (אונליין) בקבוצה נתונה, למעט המשתמש ב-exclude_username אם סופק, בדרך כלל השולח	self, group_id, message_obj, db_cursor, exclude_username=None	broadcast_message_to_group

שם המחלקה 4: ClientHandler קובץ server.py :

- **תפקיד המחלקה:** מטפלת בתקשורת עם לקוח בודד המחובר לשרת. כל מופע של ClientHandler רץ כתהליכון (thread) נפרד, ומאפשר לשרת לטפל במספר לקוחות במקביל. היא אחראית על קבלת הודעות מהלקוח, עיבודן, ושליחת תגובות או הודעות אחרות בחזרה ללקוח.
- **תכונות במחלקה:**

שם תכונה	תפקיד ושימוש
client_socket	אובייקט socket של הלקוח הספציפי הזה.
client_address	כתובת ה-IP והפורט של הלקוח.
server	הפניה למופע הראשי של ChatServer המאפשר גישה למשאבים משותפים ולפעולות שרת.
username	שם המשתמש של הלקוח לאחר התחברות מוצלחת None לפני.
client_public_key_pem	המפתח הציבורי (PEM) של הלקוח, נשלח על ידו בעת התחברות.
receive_buffer	מאגר (buffer) לאיסוף נתונים המתקבלים מהלקוח, עד שמתקבלת הודעה מלאה מופרדת ב'ח'.
is_connected	דגל בוליאני המציין אם החיבור עם הלקוח עדיין פעיל.

פעולות במחלקה:

שם פעולה	טענת כניסה (פרמטרים אותם מקבלת)	טענת יציאה (מה הפעולה מבצעת ומה מחזירה)
<code>__init__</code>	<code>self, client_socket, client_address, server_instance, ChatServer</code>	מאתחלת את <code>handlers</code> עם פרטי הלקוח והפניה לשרת.
<code>_get_db_connection</code>	<code>self</code>	יוצרת ומחזירה חיבור חדש למסד הנתונים של השרת. מחזירה אובייקט חיבור או <code>None</code> בשגיאה.
<code>send_message_to_client</code>	<code>self, message_obj</code>	ממירה את <code>message_obj</code> ל-JSON ושולחת אותו ללקוח דרך ה-socket מחזירה <code>True</code> בהצלחה, <code>False</code> , בכישלון.
<code>cleanup</code>	<code>self</code>	מנקה את משאבי ה- <code>handler</code> -בעת התנתקות: מסירה את הלקוח מהשרת, סוגרת את ה- <code>socket</code> .
<code>run</code>	<code>self</code>	הולאה הראשית של התהליכון. שולחת תחילה את המפתח הציבורי של השרת ללקוח. לאחר מכן, קוראת הודעות מהלקוח, מפענחת אותן מ-JSON וקוראת לפונקציית <code>_handle...</code> המתאימה לסוג ההודעה. מטפלת בשגיאות חיבור ועיבוד.
<code>_handle_register</code>	<code>self, payload</code>	מטפלת בבקשת הרשמה של לקוח: בודקת תקינות נתונים, יוצרת <code>hash</code> לסיסמה, מוסיפה את המשתמש למסד הנתונים דרך <code>DatabaseManager</code> . כולל המפתח הציבורי שלו, ושולחת תגובה מתאימה ללקוח.
<code>_handle_login</code>	<code>self, payload</code>	מטפלת בבקשת התחברות: מאמתת שם משתמש וסיסמה מול מסד הנתונים, שומרת/מעדכנת את המפתח הציבורי של הלקוח, רושמת את הלקוח כפעיל בשרת, ושולחת לו היסטוריית הודעות רלוונטית.
<code>_handle_request_user_public_key</code>	<code>self, payload</code>	מטפלת בבקשה לקבלת מפתח ציבורי של משתמש אחר. שולפת את המפתח דרך <code>ChatServer</code> ושולחת אותו ללקוח המבקש.
<code>_handle_initiate_secure_session</code>	<code>self, payload</code>	מטפלת בבקשה ליצירת סשן מאובטח (E2EE 1-to-1) שומרת את פרטי הסשן במסד הנתונים (כולל המפתח המוצפן עבור השולח לצורך היסטוריה), ומעבירה את המפתח המוצפן עבור הנמען וההודעה הראשונית אליו אם הוא מחובר.
<code>_handle_aes_encrypted_message</code>	<code>self, payload</code>	מטפלת בקבלת הודעת AES מוצפנת בסשן קיים. שומרת את ההודעה המוצפנת במסד הנתונים ומעבירה אותה לנמען אם הוא מחובר.
<code>_handle_e2ee_group_message</code>	<code>self, payload</code>	מטפלת בקבלת הודעת E2EE קבוצתית. שומרת את ההודעה המוצפנת במסד הנתונים (עם סימון <code>e2ee_type='GROUP_AES'</code> ומשדרת אותה לחברי הקבוצה המחוברים).
<code>_handle_distribute_group_key</code>	<code>self, payload</code>	מטפלת בהעברת מפתח AES קבוצתי שעטוף במפתח ה-RSA של משתמש יעד לאותו משתמש יעד אם הוא מחובר. זהו חלק ממנגנון הפצת מפתחות לקבוצות E2EE.
<code>_handle_get_all_users</code>	<code>self, payload</code>	שולפת את רשימת כל המשתמשים הרשומים מה DB ושולחת אותה ללקוח.
<code>_handle_get_my_groups</code>	<code>self, payload</code>	שולפת את רשימת הקבוצות שהלקוח חבר בהן מה DB ושולחת אותה ללקוח.

מטפלת בבקשה ליצירת קבוצה חדשה, כולל הוספת היוצר וחברים ראשוניים.	self, payload	_handle_create_group
מטפלת בבקשה להוסיף חברים חדשים לקבוצה קיימת (דורש הרשאות מנהל).	self, payload	_handle_add_group_members
מטפלת בבקשה להסרת חבר מקבוצה (דורש הרשאות מנהל).	self, payload	_handle_remove_group_member
מטפלת בבקשת העלאת תמונת פרופיל חדשה.	self, payload	_handle_upload_profile_picture
מטפלת בבקשה לקבלת נתוני תמונת פרופיל של משתמש.	self, payload	_handle_get_profile_picture
מטפלת בבקשת העלאת תמונת סמל קבוצה חדשה (דורש הרשאות מנהל).	self, payload	_handle_upload_group_picture
מטפלת בבקשה לקבלת נתוני סמל קבוצה.	self, payload	_handle_get_group_icon

שם המחלקה 5: ChatClientApp קובץ client.py :

- **תפקיד המחלקה:** המחלקה הראשית של צד הלקוח. היא אחראית על ממשק המשתמש הגרפי (GUI) באמצעות CustomTkinter, ניהול התקשורת עם השרת, שליחה וקבלה של הודעות, הצפנה ופענוח של הודעות, E2EE וניהול המצב הלוקלי של המשתמש (היסטוריית צ'אטים, מפתחות וכו'). יורשת מ NetworkNode-לניהול מפתחות ה-RSA של הלקוח.

• תכונות המחלקה:

שם תכונה	תפקיד ושימוש
client_socket	אובייקט ה socket לתקשורת עם השרת.
receive_thread	תהליכון (thread) האחראי על קבלת הודעות מהשרת ברקע.
username	שם המשתמש של הלקוח לאחר התחברות מוצלחת.
message_queue	תור (Queue) המשמש להעברת הודעות שהתקבלו מהשרת (ב-receive_thread לטיפול על ידי ה thread-הראשי של ה-GUI).
server_public_key_pem	המפתח הציבורי (PEM) של השרת, המתקבל בעת החיבור.
user_public_keys_cache	מילון (cache) המאחסן מפתחות ציבוריים (PEM) של משתמשים אחרים, כפי שהתקבלו מהשרת.
chat_histories	מילון המאחסן את היסטוריית ההודעות עבור כל צ'אט (פרטי או קבוצתי). המפתח הוא מזהה הצ'אט (שם משתמש או ID קבוצה) והערך הוא רשימת הודעות.

chat_session_keys	מילון המאחסן מפתחות AES פעילים עבור סשנים של 1 E2EE-to- המפתח הוא שם המשתמש האחר, והערך הוא מילון המכיל את מפתח ה-AES.
group_aes_keys	מילון המאחסן מפתחות AES עבור קבוצות. E2EE המפתח הוא ID הקבוצה, והערך הוא מפתח ה-AES-נטען ונשמר לקובץ.
current_chat_idenfifier	מזהה הצ'אט הפעיל כרגע) שם משתמש או ID קבוצה.
current_chat_is_group	דגל בוליאני המציין אם הצ'אט הפעיל הוא קבוצתי.
GROUP_KEYS_FILE	שם הקובץ לשמירת מפתחות AES קבוצתיים.
profile_picture_cache	מטמון (cache) לתמונות פרופיל שנטענו.
group_icon_cache	מטמון (cache) לסמלי קבוצות שנטענו.
auth_frame	מסגרת (frame) ב GUI-לתצוגות אימות (כניסה/הרשמה).
main_chat_frame	מסגרת (frame) ב GUI-לתצוגת הצ'אט הראשית.
new_chat_frame	מסגרת (frame) ב GUI-לתצוגות יצירת צ'אט/קבוצה והגדרות.
message_handlers	מילון הממפה סוגי הודעות מהשרת לפונקציות טיפול (handlers) מתאימות.

• פעולות במחלקה:

שם פעולה	טענת כניסה (פרמטרים אותם מקבלת)	טענת יציאה (מה הפעולה מבצעת ומה מחזירה)
<code>__init__</code>	<code>self, *args, **kwargs</code>	מאתחלת את אפליקציית ה-GUI קוראת ל <code>__init__</code> -של <code>NetworkNode</code> לאתחול מפתחות RSA של הלקוח מאתחלת משתני מצב, יוצרת ווידג'טים בסיסיים, מגדירה <code>message handlers</code> , מתחברת לשרת ומציגה את מסך ההתחברות.
<code>connect_to_server</code>	<code>self</code>	יוצרת חיבור <code>socket</code> לשרת ומפעילה את <code>receive_messages_thread</code> .
<code>receive_messages_thread</code>	<code>self</code>	פועלת בלולאה ברקע, קוראת נתונים מה <code>socket</code> -מפענחת הודעות, JSON ומכניסה אותן ל- <code>message_queue</code> .
<code>send_message</code>	<code>self, message_type, payload</code>	בונה אובייקט הודעה, ממירה אותו ל-JSON ושולחת אותו לשרת. מחזירה <code>True</code> בהצלחה, <code>False</code> בכישלון.

נקראת תקופתית על ידי לולאת ה-GUI הראשית. בודקת הודעות חדשות ב message_queue ומעבירה אותן לטיפול של ה-handler המתאים) מתוך self.message_handlers).	self	check_queue
מופעלת בעת לחיצה על כפתור "שלח". אוספת את הטקסט מהמשתמש. אם הצ'אט הוא פרטי, E2EE יוצרת סשן AES (אם לא קיים) על ידי החלפת מפתח AES מוצפן RSA עם הנמען, ואז מצפינה ושולחת את ההודעה באמצעות AES. אם הצ'אט הוא קבוצתי, E2EE משתמשת במפתח AES הקבוצתי השמור להצפנת ההודעה. מוסיפה את ההודעה (בטקסט רגיל) להיסטוריה הלוקלית ומנקה את תיבת הקלט.	self, event=None	send_message_event
מטפלת בקבלת הודעות היסטוריות או הודעות מערכת מהשרת. מפענחת הודעות E2EE (פרטיות וקבוצתיות) מההיסטוריה באמצעות המפתחות השמורים RSA לפענוח מפתח AES שנשמר עצמית, ומפתח AES מתאים להודעה עצמה. (מוסיפה את ההודעה המפוענחת/הרגילה להיסטוריית הצ'אט הרלוונטית ומעדכנת את התצוגה).	self, payload	_handle_message
מטפלת בקבלת בקשה חיה ליצירת סשן E2EE פרטי. מפענחת את מפתח ה-AES באמצעות המפתח הפרטי של הלקוח, שומרת את מפתח ה-AES, מפענחת את ההודעה הראשונית באמצעות מפתח ה-AES, ומוסיפה אותה להיסטוריה.	self, payload	_handle_receive_secure_session_init
מטפלת בקבלת הודעת AES מוצפנת חיה בסשן 1-to-1 קיים. מפענחת את ההודעה באמצעות מפתח ה-AES השמור עבור השולח, ומוסיפה אותה להיסטוריה.	self, payload	_handle_receive_aes_message
מטפלת בקבלת מפתח AES קבוצתי חדש עטוף במפתח ה-	self, payload	_handle_receive_wrapped_group_key

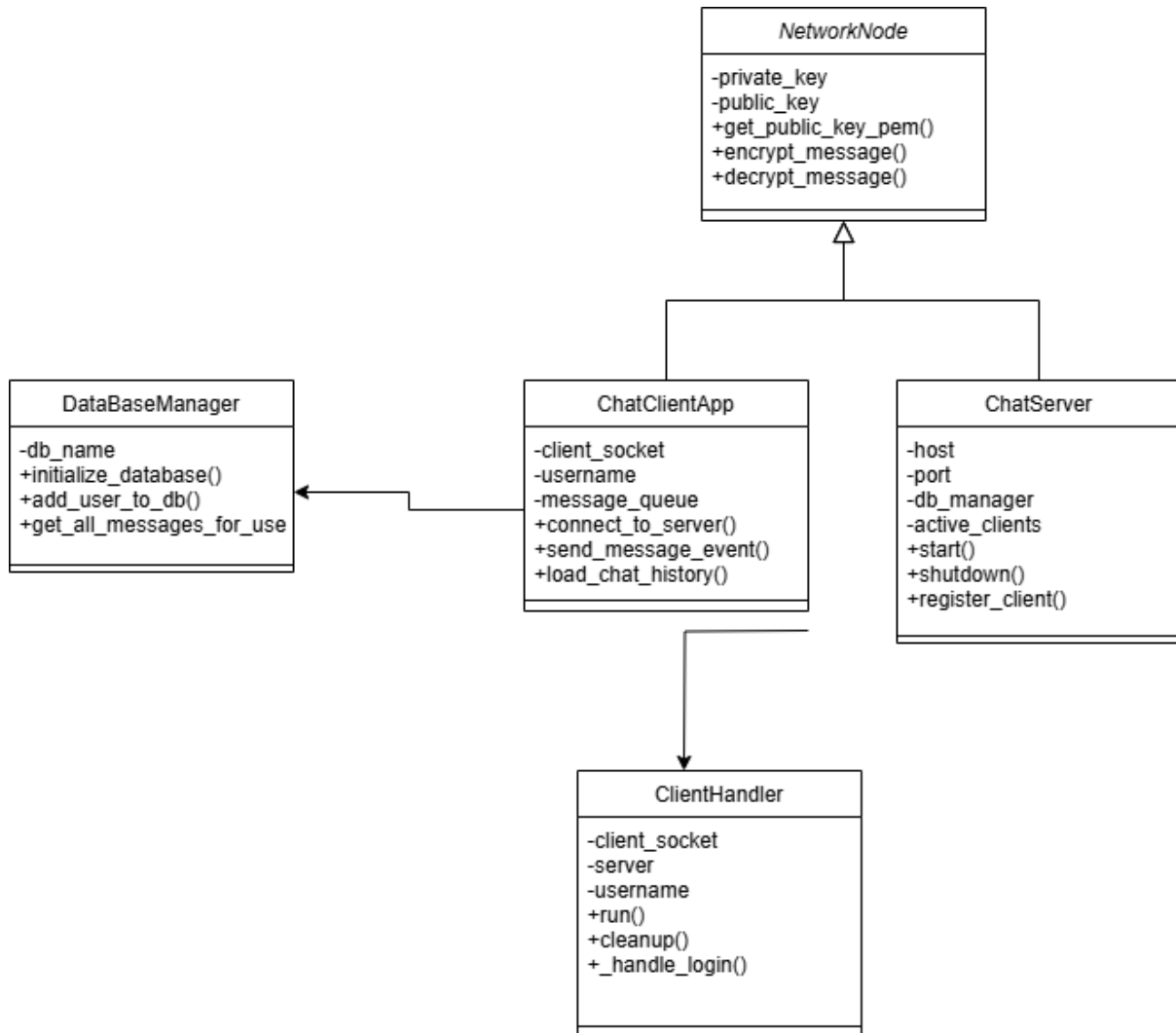
RSA של הלקוח הנוכחי מפענחת את מפתח ה-AES- הקבוצתי, שומרת אותו (בזיכרון ולקובץ), ואם הקבוצה פעילה, מרעננת את היסטוריית ההודעות שלה.		
מטפלת בקבלת הודעת E2EE קבוצתית חיה (שנשלחה על ידי משתמש אחר). מפענחת את ההודעה באמצעות מפתח ה- AES הקבוצתי השמור, ומוסיפה אותה להיסטוריה.	self, payload	_handle_receive_e2ee_group_message
פונקציות המנהלות את המעבר בין המסכים השונים באפליקציה (התחברות, הרשמה, מסך צ'אט ראשי, יצירת צ'אט/קבוצה, הגדרות קבוצה וכו')	self, (פרמטרים תלויי תצוגה)	show_login_view, show_chat_screen, ...
מאכלסת את רשימת הצ'אטים (משתמשים וקבוצות) בחלונית השמאלית של המסך הראשי.	self	populate_chat_list
טוענת ומרנדרת את היסטוריית ההודעות עבור הצ'אט הנבחר בחלונית הראשית, כולל טיפול בתאריכים, סטטוסים של הודעות, וגלילה.	self, identifier, is_group, scroll_to_bottom, preserve_scroll_info	load_chat_history
מופעלת בעת בחירת צ'אט. מעדכנת את הכותרת, מאתחלת/מבקשת מפתחות הצפנה רלוונטיים) עבור E2EE פרטי, (וטוענת את היסטוריית הצ'אט.	self, identifier, is_group, group_name_for_header	select_chat
מצפינה נתונים באמצעות מפתח AES נתון ו-IV חדש. מחזירה מילון עם IV וטקסט מוצפן ב-Base64.	self, plaintext_bytes, key	encrypt_aes
מפענחת נתונים מוצפני AES באמצעות מפתח ו-IV נתונים. מחזירה את הנתונים המפוענחים כבתים או None בכישלון.	self, ciphertext_b64, iv_b64, key	decrypt_aes
טוענת מפתחות AES קבוצתיים מקובץ JSON מקומי.	self	_load_group_aes_keys
שומרת את מפתחות AES הקבוצתיים הנוכחיים לקובץ JSON מקומי.	self	_save_group_aes_keys
שולחת בקשה לשרת לקבלת המפתח הציבורי של משתמש אחר.	self, target_username	request_other_user_public_key
פותחת דיאלוג לבחירת קובץ תמונת פרופיל, מעבדת (משנה גודל) ושולחת אותה לשרת.	self	prompt_upload_profile_picture

פותרת דיאלוג לבחירת קובץ
סמל קבוצה, מעבדת ושולחת
אותה לשרת.

self, group_id,
group_name_for_title

prompt_upload_gro
up_picture

תרשים מחלקות:



חלק ב' - פירוט יכולות וקטעי קוד מרכזיים

גיבוב סיסמאות לאחסון מאובטח

- הסבר על היכולת: במקום לאחסן סיסמאות כטקסט חשוף, הפרויקט משתמש בגיבוב (Hashing) חד-כיווני. לכל סיסמה מוצמד "מלח" (salt) "אקראי וייחודי לפני שהיא עוברת

גיבוב, מה שמבטיח שגם סיסמאות זהות ייראו שונה במסד הנתונים. תהליך זה, המבוצע עם מספר גבוה של איטרציות, מונע התקפות נפוצות ומבטיח שגם במקרה של דליפת מסד הנתונים, לא ניתן יהיה לשחזר את הסיסמאות המקוריות.

• קוד רלוונטי:

```
• @staticmethod
def hash_password(password_str, salt_bytes=None):
    if salt_bytes is None:
        salt_bytes = os.urandom(16)
    salt_hex = salt_bytes.hex()
    hashed_password = hashlib.pbkdf2_hmac(
        'sha256',
        password_str.encode('utf-8'),
        salt_bytes,
        100000 # Iteration count
    )
    return hashed_password.hex(), salt_hex

@staticmethod
def verify_password(stored_password_hash_hex, provided_password_str,
salt_hex):
    salt_bytes = bytes.fromhex(salt_hex)
    rehashed_password_hex, _ =
    DatabaseManager.hash_password(provided_password_str, salt_bytes)
    return rehashed_password_hex == stored_password_hash_hex
```

הצפנה היברידית מקצה לקצה (E2EE)

• הסבר על היכולת: כדי לאבטח שיחות מקצה לקצה, הפרויקט משתמש בהצפנה היברידית המשלבת את היתרונות של שתי שיטות. הצפנה אסימטרית (RSA) משמשת כדי להעביר באופן מאובטח מפתח הצפנה סימטרי (AES) חד-פעמי בין המשתתפים. לאחר שהמפתח המשותף הועבר, כל המשך השיחה מוצפן במהירות וביעילות באמצעות AES. באופן זה, התקשורת מאובטחת וחסונה בפני האזנה, גם מצד השרת. פתרון ייחודי שמומש הוא הצפנה עצמית של מפתח ה-AES על ידי השולח, מה שמאפשר לו לפענח את הודעותיו שלו מתוך היסטוריית הצ'אט

• קוד רלוונטי:

```
• @staticmethod
def hash_password(password_str, salt_bytes=None):
    # Generate a new salt if one isn't provided
    if salt_bytes is None:
        salt_bytes = os.urandom(16)
    salt_hex = salt_bytes.hex()

    # Hash the password using PBKDF2 with a high iteration count for
    security
    hashed_password = hashlib.pbkdf2_hmac(
        'sha256',
        password_str.encode('utf-8'),
        salt_bytes,
        100000 # Iteration count
    )
    return hashed_password.hex(), salt_hex
```

```
@staticmethod
def verify_password(stored_password_hash_hex, provided_password_str,
salt_hex):
    # Use the stored salt to re-hash the provided password
    salt_bytes = bytes.fromhex(salt_hex)
    rehashed_password_hex, _ =
DatabaseManager.hash_password(provided_password_str, salt_bytes)

    # Compare the new hash with the stored hash
    return rehashed_password_hex == stored_password_hash_hex

Python

# In file: client.py (within send_message_event method)
# Initiating a new secure session

# 1. Generate a new AES session key
aes_key = self.generate_aes_key()

# 2. Encrypt the AES key for the recipient using their public RSA key
recipient_encrypted_aes_key_bytes = self.encrypt_message(aes_key,
recipient_rsa_public_key_pem)
recipient_encrypted_aes_key_b64 =
base64.b64encode(recipient_encrypted_aes_key_bytes).decode('utf-8')

# 3. Self-encrypt the AES key for the sender's own history decryption
own_public_key_pem = self.get_public_key_pem()
sender_self_encrypted_aes_key_bytes = self.encrypt_message(aes_key,
own_public_key_pem)
sender_self_encrypted_aes_key_b64 =
base64.b64encode(sender_self_encrypted_aes_key_bytes).decode('utf-8')

# 4. Encrypt the initial message with the new AES key
message_bytes_to_send = message_text.encode('utf-8')
aes_encrypted_payload_for_initial_message =
self.encrypt_aes(message_bytes_to_send, aes_key)

# 5. Build the payload and send it to the server
session_init_payload = {
    "recipient": recipient,
    "encrypted_session_key_b64": recipient_encrypted_aes_key_b64,
    "sender_encrypted_session_key_b64":
sender_self_encrypted_aes_key_b64,
    "initial_message": aes_encrypted_payload_for_initial_message
}
self.send_message("INITIATE_SECURE_SESSION", session_init_payload)
```

פרוטוקול תקשורת מבוסס JSON

- **הסבר על היכולת:** התקשורת בין הלקוח לשרת מתבססת על פרוטוקול מובנה וברור המשתמש בפורמט JSON. כל הודעה היא אובייקט JSON המכיל שדה "type" המגדיר את סוג הפעולה (למשל, התחברות או שליחת הודעה), ושדה "payload" המכיל את הנתונים הנלווים. מבנה זה מאפשר תקשורת אמינה, קלה לעיבוד, וגמישה להוספת יכולות חדשות בעתיד. כדי להפריד בין הודעות JSON שונות, כל הודעה נשלחת כשהיא מופרדת על ידי תו רידת שורה.(\n)

- קוד רלוונטי:

```
• self.client_socket.sendall(json.dumps(message_obj).encode('utf-8') +  
b'\n')  
• message_obj = json.loads(message_str)
```

4.ניהול בקשות מרובות באמצעות תהליכונים (Multi-Threading)

- **הסבר על היכולת:** כדי לטפל בחיבורים של מספר לקוחות במקביל מבלי שאחד יעכב את השני, השרת פועל במודל של ריבוי תהליכונים. כאשר לקוח חדש מתחבר, התהליכון הראשי של השרת מקבל את החיבור ומייצר עיבוד תהליכון ייעודי (מופע של מחלקת ClientHandler). תהליכון זה מנהל את כל התקשורת עם אותו לקוח באופן עצמאי, ומאפשר לשרת להישאר פנוי לקבל לקוחות נוספים ולהגיב לכולם ביעילות.

• קוד עצמו:

```
• class ClientHandler(threading.Thread):  
• client_thread = self.client_handler_class(client_socket,  
client_address, self)  
client_thread.start()  
•
```

מסמך בדיקות מלא:

הבדיקות שתכננתם בשלב האפיון

בדיקה 1: הרשמת משתמש חדש וכניסה ראשונית למערכת

- **מה מטרת הבדיקה?** המטרה הייתה לוודא שתהליך ההרשמה למערכת פועל כשורה מקצה לקצה. היה עלינו לבדוק שמשתמש חדש יכול להירשם עם שם משתמש וסיסמה, שפרטיו נשמרים באופן מאובטח במסד הנתונים (כולל גיבוב הסיסמה ושמירת המפתח הציבורי שלו), ושהוא יכול להתחבר למערכת באופן מיידי באמצעות הפרטים שהזין.
- **מה בוצע בפועל?** הבדיקה בוצעה באופן ידני. תחילה, הפעלנו את הלקוח ומילאנו את טופס ההרשמה עם פרטים תקינים. לאחר ההרשמה, בדקנו ישירות במסד הנתונים של השרת כדי לוודא שנוספה רשומה חדשה בטבלת המשתמשים, ושהסיסמה אוחסנה כערך מגובב ולא

כטקסט רגיל. לאחר מכן, ניסינו להתחבר עם שם המשתמש והסיסמה החדשים. כחלק מהבדיקה, ניסינו גם להירשם פעם נוספת עם אותו שם משתמש כדי לוודא שהמערכת מונעת כפילויות.

- **מה היו תוצאות הבדיקה?** התוצאות היו חיוביות ברובן. ההרשמה הראשונית הצליחה, המשתמש נוסף למסד הנתונים, והצלחנו להתחבר איתו למערכת. הניסיון להירשם עם שם משתמש שכבר היה קיים נכשל כמצופה, והמערכת החזירה הודעת שגיאה מתאימה למשתמש.
- **במידה והתגלו בעיות – כיצד נפתרו?** בגרסה מוקדמת של הקוד, התגלתה בעיה: כאשר ניסינו להירשם עם שם משתמש קיים, השרת היה קורס. הסיבה לכך הייתה שגיאת IntegrityError ממסד הנתונים שלא טופלה כראוי בקוד השרת. הפתרון היה להוסיף בלוק של try...except בפונקציה המטפלת בהרשמה (_handle_register) בצד השרת. הבלוק תפס את השגיאה הספציפית הזו, מנע את קריסת התהליכון, ובמקום זאת שלח ללקוח הודעת שגיאה מסודרת המודיעה לו ששם המשתמש כבר תפוס.

בדיקה 2: שליחת וקבלת הודעה פרטית מוצפנת בין שני משתמשים

- **מה מטרת הבדיקה?** מטרת הבדיקה הייתה לאמת את התקינות והאבטחה של מנגנון ההצפנה מקצה לקצה (E2EE) בשיחה פרטית. רצינו לוודא שתהליך ההצפנה ההיברידי, הכולל החלפת מפתח סימטרי (AES) באמצעות הצפנה אסימטרית (RSA) פועל כהלכה, ושתוכן ההודעות נשאר חסוי ואינו ניתן לקריאה על ידי גורם שלישי, כולל השרת.
- **מה בוצע בפועל?** לצורך הבדיקה, הפעלנו שני לקוחות במקביל והתחברנו עם שני משתמשים שונים, נקרא להם "יוסי" ו-"דנה". יוסי התחיל שיחה חדשה עם דנה ושלח לה הודעה. בדקנו בצד של דנה אם ההודעה התקבלה ואם התוכן שלה פוענח והוצג נכון. לאחר מכן, דנה שלחה הודעה בחזרה ליוסי כדי לוודא שהסשן המאובטח ממשיך לפעול לשני הכיוונים. במקביל, בדקנו את מסד הנתונים בשרת כדי לוודא שתוכן ההודעות אכן שמור בו בצורתו המוצפנת בלבד.
- **מה היו תוצאות הבדיקה?** הבדיקה הצליחה. ההודעה הראשונה מיוסי לדנה יצרה סשן מאובטח חדש, וההודעה הגיעה ליעדה ופוענחה בהצלחה. כל ההודעות הבאות נשלחו ופוענחו במהירות באמצעות המפתח הסימטרי שנוצר. במסד הנתונים של השרת, כל ההודעות אכן נשמרו כמוצפנות ולא היה ניתן לקרוא את תוכן.
- **במידה והתגלו בעיות – כיצד נפתרו?** במהלך הבדיקות התגלתה בעיה משמעותית: המשתמש השולח (יוסי) לא היה מסוגל לצפות בהיסטוריית ההודעות שלו לאחר שהתנתק והתחבר מחדש. הסיבה הייתה שמפתח ה-AES של השיחה הוצפן רק עבור הנמען (דנה), ולכן ליוסי לא הייתה דרך לפענח אותו בעתיד. הפתרון היה לשכלל את תהליך יצירת הסשן: בזמן השליחה, יוסי מצפין את מפתח ה-AES לא רק עבור דנה, אלא גם פעם נוספת עבור עצמו, באמצעות המפתח הציבורי של עצמו. השרת שומר את שני המפתחות המוצפנים הללו. כך, כאשר יוסי טוען את היסטוריית השיחה, הוא יכול להשתמש במפתח הפרטי שלו כדי לפענח את מפתח הסשן ולקרוא את ההודעות ששלח.

בדיקה 3: יצירת קבוצה והפצת מפתח קבוצתי מאובטח

- **מה מטרת הבדיקה?** המטרה הייתה לבדוק את תהליך יצירת קבוצה חדשה ואת מנגנון האבטחה הייחודי שלה. היה עלינו לוודא שיוצר הקבוצה יכול לייצר מפתח AES קבוצתי,

להצפין אותו באופן אישי עבור כל אחד מחברי הקבוצה באמצעות מפתחות ה-RSA- הציבוריים שלהם, ושהשרת מפיץ את המפתחות "העטופים" האלה בהצלחה לחברים הנכונים.

- **מה בוצע בפועל? בדיקה זו כללה שלושה משתמשים: "משה" (היוצר), "לאה" ו"יעקב".**
משה התחבר, יצר קבוצה חדשה והזמין אליה את לאה ויעקב. עקבנו אחר התהליך בצד הלקוח של משה כדי לוודא שהוא מפיץ מפתח AES אחד, ולאחר מכן מצפין אותו פעמיים – פעם אחת עם המפתח הציבורי של לאה ופעם שנייה עם המפתח הציבורי של יעקב. לאחר מכן, וידאנו שהשרת מקבל את הבקשות להפצת המפתחות. לבסוף, התחברנו כלקוחות של לאה ויעקב ובדקנו אם הם יכולים לשלוח ולקבל הודעות מוצפנות בקבוצה, מה שמעיד על קבלה ופענוח מוצלח של המפתח הקבוצתי.
- **מה היו תוצאות הבדיקה? התהליך עבד כמתוכנן.** משה הצליח ליצור את הקבוצה, והשרת העביר בהצלחה את מפתח ה-AES-המוצפן לכל אחד מהחברים. לאה ויעקב הצליחו לפענח את המפתח הקבוצתי כל אחד באמצעות מפתח ה-RSA-הפרטי שלו, והשיחה הקבוצתית התנהלה באופן מוצפן ומאובטח.
- **במידה והתגלו בעיות – כיצד נפתרו?** בשלב מוקדם, אם אחד החברים שהוזמנו לקבוצה לא היה מחובר באותו רגע, הוא לא קיבל את המפתח הקבוצתי ונותר "מחוי" לשיחה המוצפנת. הפתרון לכך דרש חשיבה מחודשת על מנגנון ההפצה. כרגע, הפתרון הוא שהמפתח מועבר רק לחברים מחוברים. בעתיד, ניתן לשפר זאת על ידי שמירת המפתח המוצפן עבור המשתמש הלא מחובר במסד הנתונים, והשרת יעביר לו את המפתח בפעם הבאה שהוא יתחבר למערכת.

פירוט בדיקות נוספות אותן בצעתם למערכת

בדיקה 4: עמידות השרת בהתנתקות פתאומית של לקוח

- **מה מטרת הבדיקה?** רצינו לוודא שהשרת יציב ולא קורס במקרה שלקוח מתנתק באופן פתאומי (למשל, סגירת חלון האפליקציה או איבוד חיבור האינטרנט) מבלי לבצע תהליך התנתקות מסודר.
- **מה בוצע בפועל?** חיברנו לקוח לשרת והתחברנו עם משתמש. לאחר מכן, סגרנו את תהליך הלקוח באופן פתאומי דרך מנהל המשימות. במקביל, עקבנו אחר יומן השרת (הלוגים) ובדקנו האם הוא ממשיך לפעול כרגיל ומסוגל לקבל חיבורים חדשים מלקוחות אחרים.
- **מה היו תוצאות הבדיקה?** השרת זיהה את ניתוק הקשר. בצד השרת, התהליכון שהיה אחראי על הלקוח שנותק נתקל בשגיאת `ConnectionResetError` והודות למנגנון טיפול בשגיאות, הוא הפעיל את פונקציית הניקוי (`cleanup`) שסגרה את המשאבים וסיימה את פעולתו בצורה מסודרת. השרת הראשי המשיך לפעול ללא כל הפרעה.
- **במידה והתגלו בעיות – כיצד נפתרו?** בגרסה ראשונית, לולאת קבלת ההודעות ב-`ClientHandler` לא הייתה עטופה כולה בבלוק `try...except` ונדרש מספיק. ניתוק פתאומי גרם לקריסת התהליכון, אך הותיר "רשומת רפאים" של המשתמש במילון הלקוחות הפעילים של השרת. הפתרון היה לעטוף את כל הלולאה הראשית של התהליכון בבלוק `try...except` שתופס שגיאות חיבור כלליות ומבטיח שבכל מקרה של ניתוק, פונקציית ה-`cleanup`-נקראת והמשתמש מוסר כראוי מרשימת הפעילים.

בדיקה 5: טיפול בתמונות פרופיל

- **מה מטרת הבדיקה?** לוודא את תקינות תהליך העלאת תמונת פרופיל, אחסונה בצד השרת, ושליפתה והצגתה אצל משתמשים אחרים.
- **מה בוצע בפועל?** משתמש א' התחבר והעלה תמונת פרופיל חדשה. בדקנו בתיקיית profile_pics שבשרת שקובץ התמונה אכן נוצר שם. לאחר מכן, וידאנו שבמסד הנתונים, הרשומה של משתמש א' עודכנה עם שם הקובץ החדש. לבסוף, התחברנו עם משתמש ב', התחלנו איתו שיחה, ואימתנו שתמונת הפרופיל החדשה של משתמש א' מוצגת כראוי בכותרת חלון השיחה.
- **מה היו תוצאות הבדיקה?** התהליך כולו – העלאה, אחסון ושליפה – עבד כמצופה.
- **במידה והתגלו בעיות – כיצד נפתרו?** שמנו לב שתמונות גדולות במיוחד גרמו לאפליקציית הלקוח "לקפוא" לרגע בזמן ההעלאה, והעלו את צריכת הזיכרון בשרת. הפתרון יושם בצד הלקוח: לפני השליחה לשרת, הקוד מבצע כעת שינוי גודל לתמונה ומגביל אותה לממדים מקסימיים (למשל 300 300 אפיקסלים), (וגם דוחס אותה קלות. שינוי זה הפחית משמעותית את גודל הקבצים הנשלחים ברשת ושיפר את חווית המשתמש והיעילות, מבלי לפגוע באופן ניכר באיכות התמונה המוצגת.

מדריך למשתמש

פירוט כלל קבצי המערכת – עץ קבצים

המערכת מחולקת לשני חלקים עיקריים: צד שרת וצד לקוח. להפעלה תקינה, יש לשמור על מבנה הקבצים הבא.

מבנה התיקיות בצד השרת :

```
Server/
|
|— server.py
|— chat_server_core.py
|— shared_modules.py
|— chat_app.db           (נוצר אוטומטית בהרצה ראשונה)
|— profile_pics/         (נוצרת אוטומטית)
|— group_icons/          (נוצרת אוטומטית)
```

מבנה התיקיות בצד הלקוח:

```
Client/
|
|— client.py
|— shared_modules.py
|— client_private_key.pem (נוצר אוטומטית בהרצה ראשונה)
|— client_public_key.pem  (נוצר אוטומטית בהרצה ראשונה)
|— group_aes_keys.json    (נוצר אוטומטית לאחר קבלת מפתח קבוצתי)
```

התקנת המערכת:

פירוט הסביבה הנדרשת:

- **מערכת הפעלה:** המערכת פותחה ונבדקה על מערכת הפעלה, Windows, אך צפויה לעבוד על מערכות הפעלה מודרניות אחרות התומכות ב Python-כמו MacOS או Linux.
- **Python** נדרשת גרסה 3.8 ומעלה של python

פירוט הכלים הנדרשים **ספריות Python** יש להתקין את הספריות הבאות באמצעות מנהל החבילות pip ניתן לפתוח שורת פקודה (cmd/terminal) ולהריץ את הפקודות הבאות:

```
pip install customtkinter
pip install Pillow
pip install cryptography
```

- **מיקומי קבצים:** יש ליצור שתי תיקיות נפרדות, אחת עבור השרת ואחת עבור הלקוח. יש למקם את כל קבצי השרת (server.py, chat_server_core.py, shared_modules.py) בתוך תיקיית השרת, ואת כל קבצי הלקוח (client.py, shared_modules.py) בתוך תיקיית הלקוח. אין צורך ליצור את קבצי מסד הנתונים או המפתחות באופן ידני; הם ייווצרו אוטומטית בהרצה הראשונה.
- **נתונים התחלתיים:** אין צורך בנתונים התחלתיים. המערכת אינה מגיעה עם משתמשים קיימים. המשתמש הראשון (וכל משתמש אחריו) יצטרך להירשם דרך ממשק ההרשמה של האפליקציה.
- **רשת:** כדי שהמערכת תעבוד, השרת וכל הלקוחות חייבים להיות מחוברים לאותה רשת מקומית (LAN). בנוסף, יש להגדיר את כתובת ה-IP של המחשב המארח את השרת בקוד. יש לפתוח את הקובץ client.py ואת הקובץ server.py ולשנות את המשתנה HOST לכתובת ה-IP הפנימית של מחשב השרת.
 - **כיצד למצוא את כתובת ה-IP במערכת Windows,** פתח את שורת הפקודה (cmd) והקלד ipconfig מצא את השורה "IPv4 Address" תחת חיבור הרשת הפעיל שלך לדוגמה "Ethernet adapter", או "Wireless LAN adapter"
- **ארכיטקטורה מינימלית נדרשת:**
 - **שרת:** מחשב עם מעבד מודרני
 - **לקוח:** מחשב עם מעבד מודרני
 - ניתן להריץ את השרת ואת הלקוח על אותו המחשב למטרות בדיקה, אך מומלץ להריץ אותם על מחשבים נפרדים ברשת כדי לדמות שימוש אמיתי.

משתמשי המערכת

במערכת זו קיים סוג משתמש אחד: **משתמש קצה**. כל הפונקציונליות של האפליקציה, כולל הרשמה, התחברות, יצירת קבוצות ושיחה, זמינה לכל מי שפותח חשבון. אין במערכת תפקיד "מנהל ראשי" חיצוני; ניהול קבוצות (כמו הוספה או הסרה של חברים) מתבצע על ידי משתמשים שהוגדרו כמנהלי אותה קבוצה.

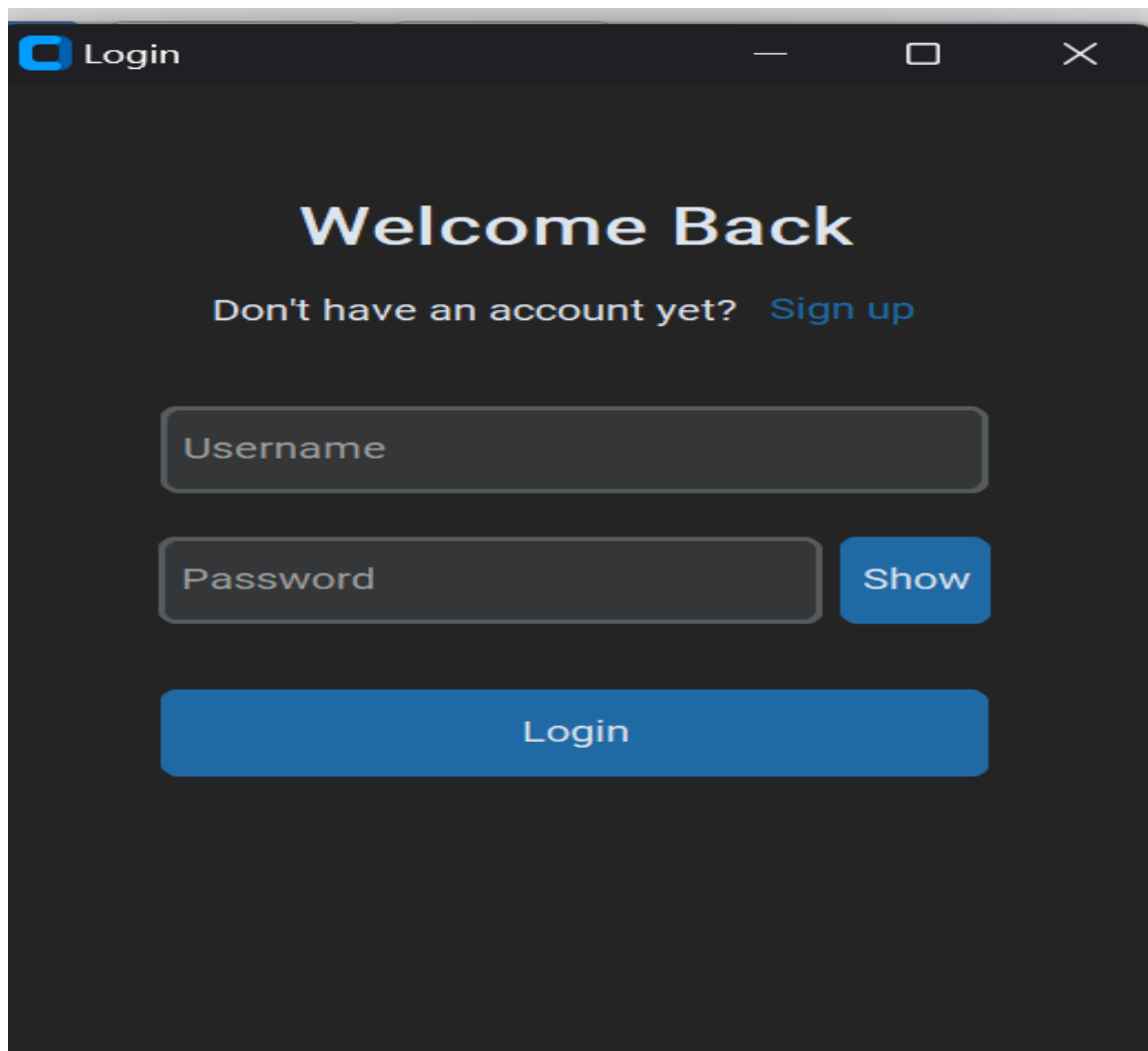
- **אופן הפעלת המערכת:**

1. **הפעלת השרת:** נווט אל תיקיית השרת בשורת הפקודה והרץ את הפקודה python : server.py חלון שורת פקודה יישאר פתוח ויציג הודעות על פעילות השרת. יש להשאיר אותו פועל כל עוד רוצים שהשירות יהיה זמין.

2. **הפעלת הלקוח:** לאחר שהשרת פועל, נווט אל תיקיית הלקוח במחשב אחר (או באותו מחשב בחלון שורת פקודה נפרד) והרץ את הפקודה `python client.py`: חלון האפליקציה הגרפי אמור להופיע. ניתן להפעיל מספר לקוחות במקביל.

- **צילומי מסכי הפרויקט הרלוונטיים:**

מסך הכניסה וההרשמה עם הפעלת האפליקציה, יופיע מסך הכניסה. במסך זה ניתן להזין שם משתמש וסיסמה כדי להתחבר לחשבון קיים. אם אין לך חשבון, תוכל ללחוץ על הקישור "Sign up" כדי לעבור למסך ההרשמה, שם תתבקש למלא שם משתמש, סיסמה ואימות סיסמה.

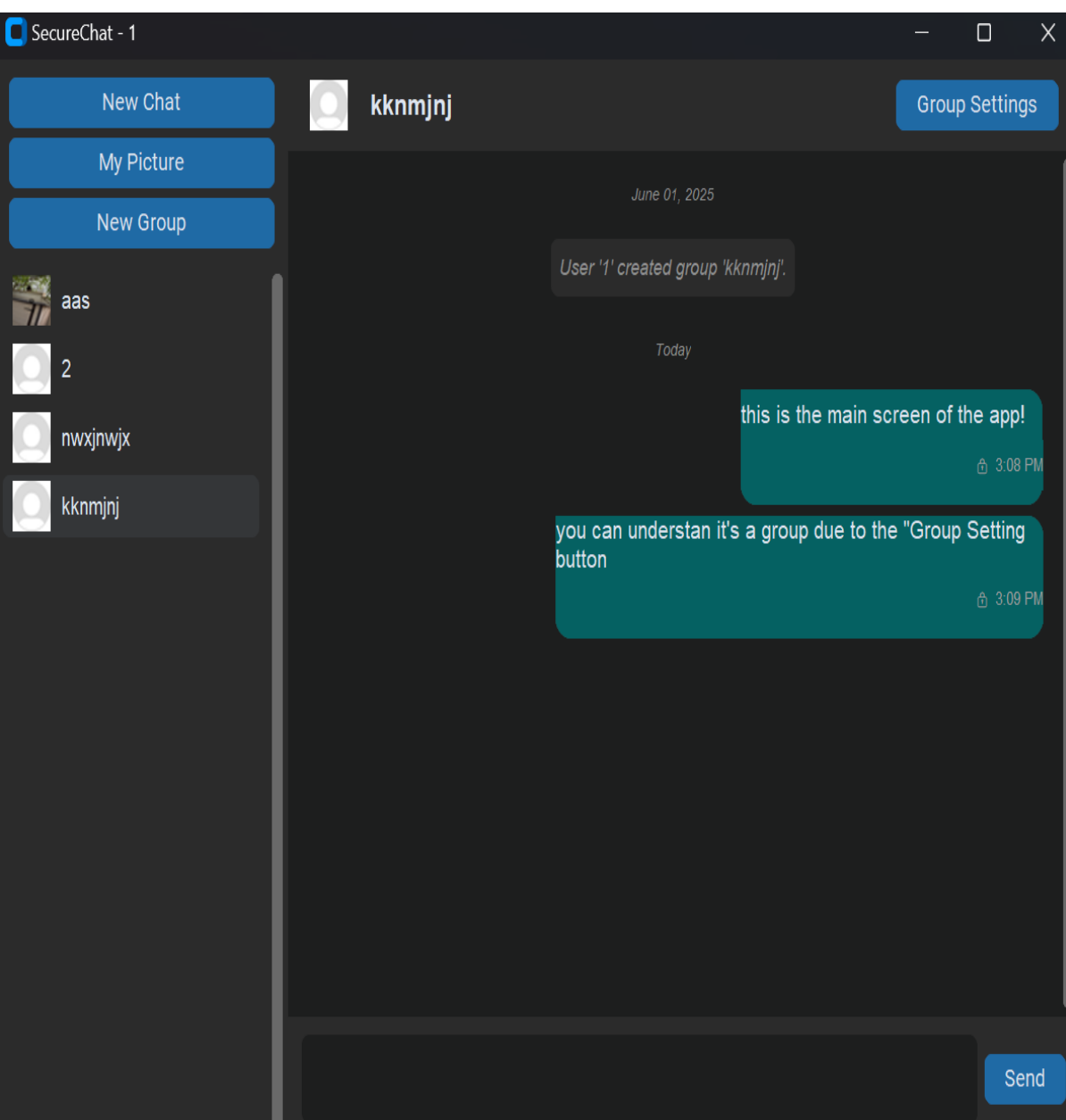


המסך הראשי של האפליקציה לאחר התחברות מוצלחת, יופיע המסך הראשי. המסך מחולק לשני חלקים עיקריים:

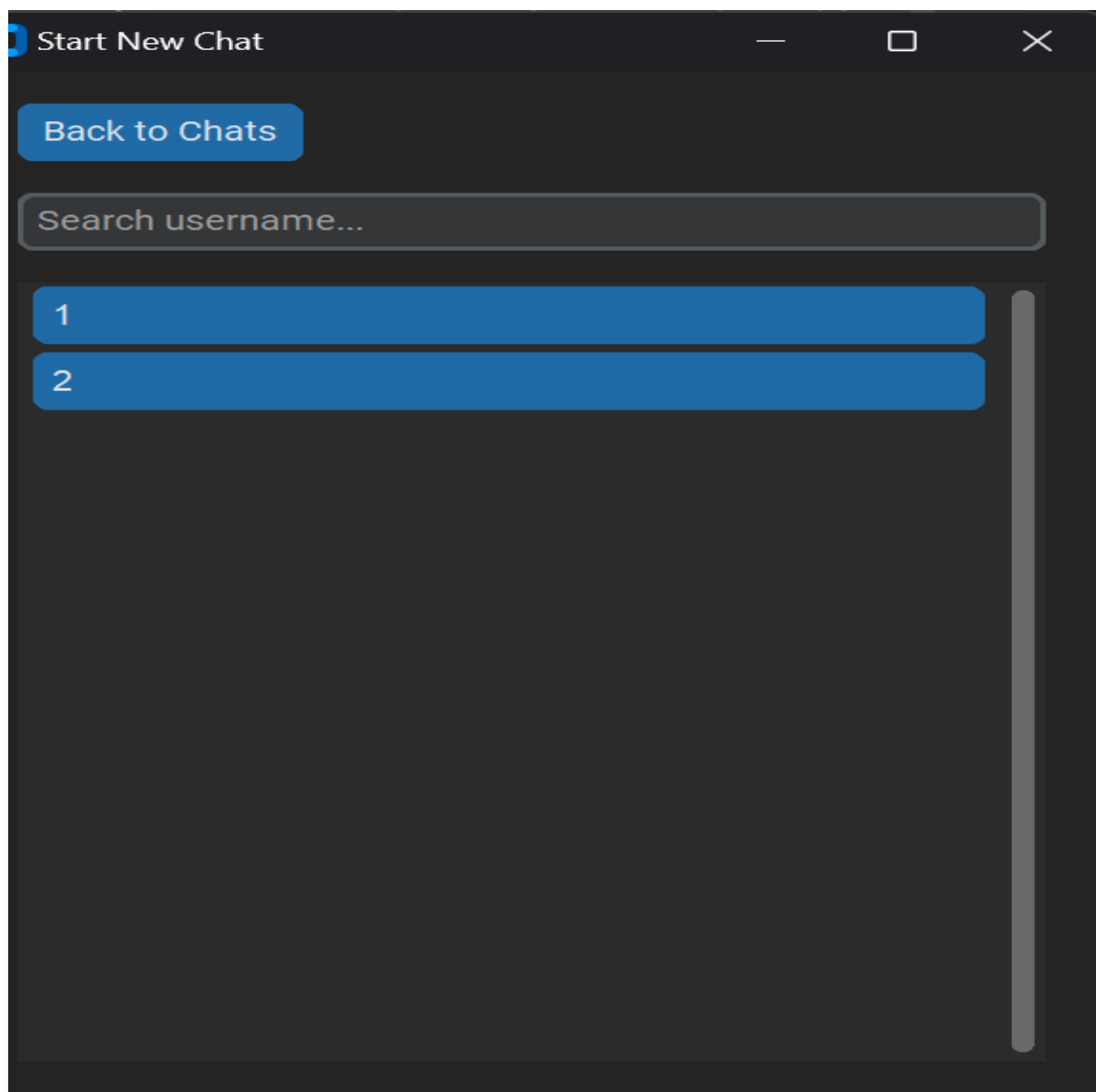
- **צד ימין (רשימת הצ'אטים):** כאן מופיעות כל השיחות הפרטיות והקבוצתיות שלך. לחיצה על שיחה תטען אותה בחלק המרכזי.
- **צד שמאל (חלון השיחה):** כאן מוצג תוכן השיחה שנבחרה. למעלה מופיע שם השיחה (שם המשתמש או שם הקבוצה), ובתחתית נמצאת תיבת הטקסט להזנת

יונתן ברק – FlowChat

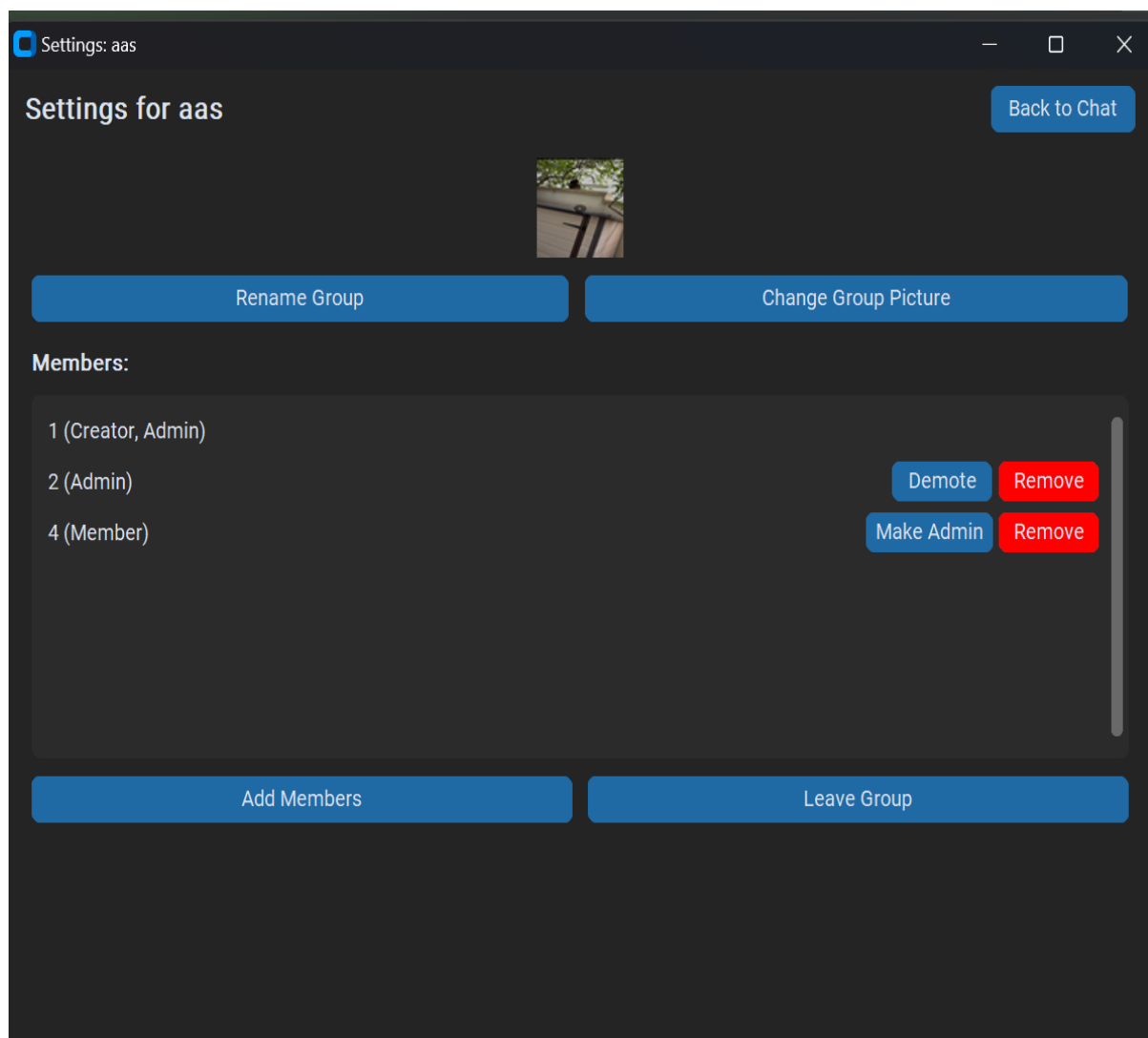
הודעות חדשות. בפינה הימנית העליונה של רשימת הצ'טים נמצאים כפתורים להתחלת שיחה חדשה, יצירת קבוצה חדשה, והעלאת תמונת פרופיל.



3. **התחלת שיחה חדשה ויצירת קבוצה** לחיצה על כפתור "New Chat" תפתח חלון עם רשימת כל המשתמשים הרשומים במערכת, עם אפשרות חיפוש. בחירה במשתמש תתחיל איתו שיחה פרטית. לחיצה על כפתור "New Group" תפתח חלון בו יש להזין שם לקבוצה ולבחור את החברים שברצונך לצרף מתוך רשימת המשתמשים.



ניהול קבוצה בשיחה קבוצתית, אם אתה מנהל הקבוצה, יופיע כפתור "Group Settings" בכותרת השיחה. לחיצה עליו תפתח חלון ניהול המאפשר להוסיף או להסיר חברים, לשנות את שם הקבוצה, או להחליף את סמל הקבוצה.



סיכום אישי / רפלקציה

✓ תיאור תהליך העבודה על הפרויקט, ההצלחות, האתגרים, הקשיים ודרכי הפתרון

תהליך העבודה על הפרויקט התנהל באופן איטרטיבי, החל מבניית יכולות הליבה של המערכת ועד להוספת השכבות המורכבות של אבטחה ופונקציונליות מתקדמת. תהליך זה כלל מספר הצלחות שהיוו אבני דרך משמעותיות, כגון מימוש התקשורת הראשונית בין השרת ללקוח, הצלחת מנגנון ההצפנה מקצה לקצה בפעם הראשונה, וייצוב השרת כך שיוכל לטפל במספר לקוחות במקביל.

האתגר הטכני המרכזי היה ניהול התקשורת הא-סינכרונית בין תהליכון הרשת לתהליכון ממשק המשתמש בלקוח. קושי זה נפתר באמצעות מימוש של תור (Queue) להעברת הודעות בין התהליכונים באופן בטוח. אתגר נוסף היה איתור ותיקון שגיאות במנגנון ההצפנה, אשר דרש פירוק של התהליך לחלקים קטנים ובדיקה שיטתית של כל שלב בנפרד. כמו כן, ניהול המשאבים המשותפים בשרת, כגון רשימת הלקוחות הפעילים, דרש שימוש במנעולים (threading.Lock) כדי למנוע מצבי מרוץ (race conditions) ולהבטיח את יציבות המערכת.

✓ תהליך למידה - תיאור הלמידה שהתקיימה, תכולות חדשות שנלמדו באופן עצמאי

הפרויקט דרש למידה עצמאית נרחבת של נושאים מתקדמים. עיקר הלמידה התמקדה בתחומים הבאים:

- **תכנות רשתות:** עבודה מעמיקה עם ספריית sockets ליצירת תקשורת מבוססת TCP, והבנת המורכבות של ארכיטקטורת שרת-לקוח.
- **ריבוי תהליכונים (Multi-Threading):** תכנון ומימוש שרת המסוגל לטפל במספר לקוחות במקביל, תוך שימוש בספריית threading וניהול משאבים משותפים.
- **קריפטוגרפיה יישומית:** מעבר להבנה תיאורטית, הפרויקט דרש יישום מעשי של מודל הצפנה היברידי, המשלב הצפנה אסימטרית (RSA) להחלפת מפתחות והצפנה סימטרית (AES) להעברת המידע.
- **פיתוח ממשק משתמש:** עבודה עם ספריית customtkinter לבניית ממשק גרפי מודרני ורספונסיבי.

✓ אילו כלים נלקחים להמשך

מעבר לידע הטכני, הפרויקט הקנה לי כלים ומתודולוגיות עבודה חשובים. למדתי כיצד לתכנן ארכיטקטורה של מערכת מורכבת, לפרק בעיות גדולות למודולים קטנים יותר, ולכתוב קוד נקי וניתן לתחזוקה. פיתחתי מיומנויות מתקדמות באיתור ופתרון תקלות במערכות מבוזרות ומקביליות. כמו כן, רכשתי ניסיון מעשי ביישום עקרונות אבטחת מידע, אשר מהווה כלי חיוני בעולם הפיתוח המודרני.

✓ תובנות מהתהליך, לרבות עזרה ממומחים, שיתוף מידע ולמידת עמיתים

התובנה המרכזית מהתהליך היא חשיבותו של תכנון מוקדם. השקעת זמן בתכנון הארכיטקטורה והפרוטוקולים בשלב הראשון חסכה זמן רב בשלבי הפיתוח המאוחרים יותר. בנוסף, הבנתי את הערך הרב בקבלת נקודת מבט חיצונית; התייעצויות עם המנחה ודיונים עם עמיתים לכיתה סייעו לפתור בעיות מורכבות ולחשוב על פתרונות יצירתיים שלא הייתי מגיע אליהם לבד.

✓ בראייה לאחור, האם היית מיישם אחרת חלקים בפרויקט

באופן כללי, אני שבע רצון מההחלטות הארכיטקטוניות המרכזיות. מודל ההצפנה ההיברידי וארכיטקטורת ריבוי התהליכים הוכיחו את עצמם כיעילים ויציבים. עם זאת, בראייה לאחור, הייתי משקיע יותר זמן בתכנון סכמת מסד הנתונים הראשונית, בעיקר בכל הנוגע לניהול קבוצות. תכנון גמיש יותר היה יכול להפוך שאליות מסוימות לפשוטות ויעילות יותר, ולחסוך שינויים שנדרשו בשלבים מאוחרים של הפיתוח.

✓ במידה והיו ברשותך משאבים נוספים, האם וכיצד ניתן לשפר את הפרויקט

בהינתן זמן ומשאבים נוספים, ניתן להרחיב את הפרויקט באופן משמעותי. השלבים הבאים היו כוללים:

1. **העברת קבצים:** מימוש יכולת לשלוח ולקבל קבצים מכל סוג, ולא רק הודעות טקסט.
2. **שיחות קול/וידאו:** שילוב טכנולוגיית WebRTC או דומה להוספת יכולות תקשורת בזמן אמת.
3. **מנגנון מתקדם להודעות אופליין:** פיתוח מערכת שתאפשר שמירה והעברה אמינה של מפתחות הצפנה למשתמשים שלא היו מחוברים בזמן יצירת הקבוצה.
4. **תמיכה בפרוטוקולים נוספים:** פיתוח גרסת ווב או אפליקציה למובייל כדי להרחיב את גישות השירות.

✓ שאלות חקר עצמי לשיקול התלמידים

- כיצד ניתן להתאים את ארכיטקטורת השרת הנוכחית כדי לתמוך בעשרות אלפי משתמשים בו-זמנית?
- אילו מנגנוני אבטחה נוספים ניתן היה לשלב, כגון אימות דו-שלבי (2FA) בכניסה למערכת?
- מהם היתרונות והחסרונות של שימוש בפרוטוקול UDP במקום TCP עבור תכונות עתידיות כמו שיחות טלפון קוליות?

תודות

ברצוני להודות למנחה הפרויקט, עידן פלדברג, על הליווי המקצועי, ההכוונה והזמינות לאורך כל תהליך הפיתוח. תודה נוספת לחבריי לכיתה על העזרה ההדדית, שיתוף הידע והדיונים הפוריים שתרמו רבות להתקדמות הפרויקט. לבסוף, תודה למשפחתי על התמיכה, הסבלנות והעידוד לאורך כל הדרך.

ביבליוגרפיה

מקורות שבהם נעזרתי לפתרון בעיות:

+ [Geeks for geeks](#)

+ [GitHub](#)

תוכנות שבהן נעזרתי לביצוע בדיקות ושרטוט גרפים:

+ [DB Browser for SQLite](#)

+ [Diagrams](#)

נספחים:

בקובץ: "shared_modules.py"

```
# shared_modules.py

import sqlite3
import os
import hashlib
import json
from datetime import datetime
from cryptography.hazmat.primitives.asymmetric import rsa, padding as rsa_padding # Renamed to
avoid confusion
import os

# --- Cryptography Imports (for NetworkNode) ---
from cryptography.hazmat.primitives import serialization, hashes
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.backends import default_backend

# --- Shared Constants ---
DB_NAME = 'chat_app.db'
PROFILE_PICS_DIR = 'profile_pics'
GROUP_ICONS_DIR = 'group_icons'

CLIENT_PRIVATE_KEY_FILE = "client_private_key.pem" # Store in a known, writable location
CLIENT_PUBLIC_KEY_FILE = "client_public_key.pem" # Store in a known, writable location

# NetworkNode Class

class NetworkNode:
    def __init__(self):
        self.private_key = None
        self.public_key = None
        self._load_or_generate_rsa_keys()

    def _save_private_key(self, pk, filename=CLIENT_PRIVATE_KEY_FILE):
        try:
            pem = pk.private_bytes(
                encoding=serialization.Encoding.PEM,
                format=serialization.PrivateFormat.PKCS8,
                encryption_algorithm=serialization.NoEncryption() # For simplicity
            )
            with open(filename, 'wb') as f:
                f.write(pem)
            print(f"[NetworkNode] Private key saved to {filename}")
        except Exception as e:
            print(f"[NetworkNode_ERROR] Failed to save private key: {e}")
            # Consider how to handle this error; it's critical for persistence

    def _save_public_key(self, pubk, filename=CLIENT_PUBLIC_KEY_FILE):
        try:
            pem = pubk.public_bytes(
                encoding=serialization.Encoding.PEM,
                format=serialization.PublicFormat.SubjectPublicKeyInfo
            )
            with open(filename, 'wb') as f:
                f.write(pem)
            print(f"[NetworkNode] Public key saved to {filename}")
        except Exception as e:
            print(f"[NetworkNode_ERROR] Failed to save public key: {e}")

    def _load_private_key(self, filename=CLIENT_PRIVATE_KEY_FILE):
        if not os.path.exists(filename):
            print(f"[NetworkNode] Private key file '{filename}' not found.")
            return None
        try:
            with open(filename, 'rb') as f:
                private_key = serialization.load_pem_private_key(
                    f.read(),
                    password=None, # Adjust if you add password encryption

```

```

        backend=default_backend()
    )
    print(f"[NetworkNode] Private key loaded from {filename}")
    return private_key
except FileNotFoundError: # Should be caught by os.path.exists, but good to have
    print(
        f"[NetworkNode] Private key file '{filename}' not found during load attempt
(should not happen if exists check passed).")
    return None
except Exception as e:
    print(f"[NetworkNode_ERROR] Error loading private key from {filename}: {e}")
    return None

def _generate_new_rsa_keys(self):
    """Generates a new RSA key pair and saves them."""
    print("[NetworkNode] Generating new RSA key pair...")
    self.private_key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=2048, # Standard key size
        backend=default_backend()
    )
    self.public_key = self.private_key.public_key()
    print("[NetworkNode] New RSA key pair generated.")

    # Save the newly generated keys
    self._save_private_key(self.private_key)
    self._save_public_key(self.public_key)

def _load_or_generate_rsa_keys(self):
    # Try to load existing private key
    loaded_private_key = self._load_private_key() # Uses default filename

    if loaded_private_key:
        self.private_key = loaded_private_key
        self.public_key = self.private_key.public_key()
        # Optionally, you could load the public key from its file too for verification,
        # but deriving from private key is standard.
        print("[NetworkNode] Successfully loaded existing RSA key pair.")
    else:
        # If loading failed (e.g., file not found or corrupt), generate new keys
        print("[NetworkNode] Failed to load existing RSA private key or file not found.
Will generate new keys.")
        self._generate_new_rsa_keys()

def get_public_key_pem(self):
    if self.public_key:
        try:
            return self.public_key.public_bytes(
                encoding=serialization.Encoding.PEM,
                format=serialization.PublicFormat.SubjectPublicKeyInfo
            ).decode('utf-8')
        except Exception as e:
            print(f"[NetworkNode_ERROR] Could not get public key PEM: {e}")
            return None
    print("[NetworkNode_WARN] Public key not available in get_public_key_pem.")
    return None

def encrypt_message(self, message_bytes, recipient_public_key_pem_str):
    if not isinstance(message_bytes, bytes):
        print("[NetworkNode_TYPE_ERROR] Message for encryption must be bytes.")
        # Consider raising TypeError for stricter handling
        return None # Or raise error
    if not recipient_public_key_pem_str:
        print("[NetworkNode_ENCRYPT_ERROR] Recipient public key PEM is missing.")
        return None
    if not self.public_key: # Own public key for context, not directly used here but
implies init failed
        print("[NetworkNode_ENCRYPT_ERROR] Own keys not initialized properly.")
        return None
    try:
        recipient_public_key = serialization.load_pem_public_key(
            recipient_public_key_pem_str.encode('utf-8'),
            backend=default_backend()
        )
        return recipient_public_key.encrypt(
            message_bytes,

```

```

        rsa_padding.OAEP( # Use the aliased rsa_padding
            mgf=rsa_padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )
except Exception as e:
    print(f"[NetworkNode_ENCRYPT_ERROR] RSA Encryption failed: {e}")
    return None

def decrypt_message(self, encrypted_message_bytes):
    if not self.private_key:
        print("[NetworkNode_DECRYPT_ERROR] Private key not available for decryption.")
        return None
    if not isinstance(encrypted_message_bytes, bytes):
        print("[NetworkNode_TYPE_ERROR] Encrypted message for decryption must be bytes.")
        # Consider raising TypeError
        return None
    try:
        return self.private_key.decrypt(
            encrypted_message_bytes,
            rsa_padding.OAEP( # Use the aliased rsa_padding
                mgf=rsa_padding.MGF1(algorithm=hashes.SHA256()),
                algorithm=hashes.SHA256(),
                label=None
            )
        )
    except Exception as e:
        print(f"[NetworkNode_DECRYPT_ERROR] RSA Decryption failed: {e}")
        # This is where the "Self-Key RSA Decrypt Fail" would originate if keys don't
match
        return None

# DatabaseManager Class
class DatabaseManager:
    def __init__(self, db_name=DB_NAME):
        self.db_name = db_name

    @staticmethod
    def hash_password(password_str, salt_bytes=None):
        if salt_bytes is None:
            salt_bytes = os.urandom(16)
        salt_hex = salt_bytes.hex()
        hashed_password = hashlib.pbkdf2_hmac(
            'sha256',
            password_str.encode('utf-8'),
            salt_bytes,
            100000 # Iteration count
        )
        return hashed_password.hex(), salt_hex

    @staticmethod
    def verify_password(stored_password_hash_hex, provided_password_str, salt_hex):
        salt_bytes = bytes.fromhex(salt_hex)
        rehashed_password_hex, _ = DatabaseManager.hash_password(provided_password_str,
salt_bytes)
        return rehashed_password_hex == stored_password_hash_hex

    def initialize_database(self):
        """
        Initializes the database schema, creates necessary tables if they don't exist,
        adds an 'rsa_public_key_pem' column to the users table if missing,
        adds an 'e2ee_type' column to the messages table if missing,
        and ensures necessary media directories exist.
        Manages its own database connection for setup.
        """
        if not os.path.exists(PROFILE_PICS_DIR):
            os.makedirs(PROFILE_PICS_DIR)
            print(f"[DB MANAGER] Created directory: {PROFILE_PICS_DIR}")
        if not os.path.exists(GROUP_ICONS_DIR):
            os.makedirs(GROUP_ICONS_DIR)
            print(f"[DB MANAGER] Created directory: {GROUP_ICONS_DIR}")

        conn = sqlite3.connect(self.db_name)
        conn.execute("PRAGMA foreign_keys = ON;")

```

```

cursor = conn.cursor()

# --- Create Tables (IF NOT EXISTS should run first for all primary tables) ---
# Users Table
cursor.execute('''
    CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        username TEXT UNIQUE NOT NULL,
        password_hash TEXT NOT NULL,
        salt TEXT NOT NULL,
        profile_picture_filename TEXT DEFAULT 'default_pic.jpg'
        /* rsa_public_key_pem will be added below if not present */
    )
''')

# Groups Table
cursor.execute('''
    CREATE TABLE IF NOT EXISTS groups (
        group_id INTEGER PRIMARY KEY AUTOINCREMENT,
        group_name TEXT NOT NULL,
        creator_username TEXT NOT NULL,
        created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
        group_icon_filename TEXT DEFAULT 'default_icon.jpg',
        FOREIGN KEY (creator_username) REFERENCES users(username) ON DELETE SET NULL
    )
''')

# Group Members Table
cursor.execute('''
    CREATE TABLE IF NOT EXISTS group_members (
        group_id INTEGER NOT NULL,
        username TEXT NOT NULL,
        role TEXT NOT NULL DEFAULT 'member',
        joined_at DATETIME DEFAULT CURRENT_TIMESTAMP,
        PRIMARY KEY (group_id, username),
        FOREIGN KEY (group_id) REFERENCES groups(group_id) ON DELETE CASCADE,
        FOREIGN KEY (username) REFERENCES users(username) ON DELETE CASCADE
    )
''')

# Messages Table
cursor.execute('''
    CREATE TABLE IF NOT EXISTS messages (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        sender TEXT NOT NULL,
        recipient TEXT,
        recipient_group_id INTEGER,
        message TEXT NOT NULL,
        timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
        is_system_message BOOLEAN DEFAULT FALSE NOT NULL,
        /* e2ee_type will be added below if not present */
        FOREIGN KEY (recipient_group_id) REFERENCES groups(group_id) ON DELETE
CASCADE,
        FOREIGN KEY (recipient) REFERENCES users(username) ON DELETE CASCADE
    )
''')

# --- Now, attempt to add new columns to existing tables if they don't exist ---
# Add rsa_public_key_pem column to users table
try:
    cursor.execute("SELECT rsa_public_key_pem FROM users LIMIT 1;")
except sqlite3.OperationalError: # Column likely doesn't exist
    print("[DB_MANAGER] Adding 'rsa_public_key_pem' column to 'users' table.")
    try:
        cursor.execute("ALTER TABLE users ADD COLUMN rsa_public_key_pem TEXT DEFAULT
NULL;")
        conn.commit() # Commit alter table immediately
    except sqlite3.OperationalError as e_alter_user: # Handle if alter still fails
        (e.g. syntax on older sqlite)
        print(f"[DB_MANAGER_ERROR] Failed to add 'rsa_public_key_pem' to users:
{e_alter_user}")

# Add e2ee_type column to messages table
try:
    cursor.execute("SELECT e2ee_type FROM messages LIMIT 1;")
except sqlite3.OperationalError: # Column likely doesn't exist
    print("[DB_MANAGER] Adding 'e2ee_type' column to 'messages' table.")

```

```

        try:
            cursor.execute("ALTER TABLE messages ADD COLUMN e2ee_type TEXT DEFAULT NULL;")
            conn.commit() # Commit alter table immediately
        except sqlite3.OperationalError as e_alter_msg:
            print(f"[DB_MANAGER_ERROR] Failed to add 'e2ee_type' to messages:
{e_alter_msg}")

    # Add sender_e2ee_session_data column to messages table (NEW)
    try:
        cursor.execute("SELECT sender_e2ee_session_data FROM messages LIMIT 1;")
    except sqlite3.OperationalError: # Column likely doesn't exist
        print("[DB_MANAGER] Adding 'sender_e2ee_session_data' column to 'messages'
table.")

    try:
        # This column will store the AES session key, RSA-encrypted with the sender's
own public key.
        # It's primarily for AES_SESSION_INIT type messages.
        cursor.execute("ALTER TABLE messages ADD COLUMN sender_e2ee_session_data TEXT
DEFAULT NULL;")
        conn.commit() # Commit alter table immediately
    except sqlite3.OperationalError as e_alter_msg_sender_key:
        print(
            f"[DB_MANAGER_ERROR] Failed to add 'sender_e2ee_session_data' to messages:
{e_alter_msg_sender_key}")

    # --- Indexes (can be created after tables and columns are set) ---
    cursor.execute("CREATE INDEX IF NOT EXISTS idx_users_username ON users (username)")
    cursor.execute("CREATE INDEX IF NOT EXISTS idx_messages_sender ON messages (sender)")
    cursor.execute("CREATE INDEX IF NOT EXISTS idx_messages_recipient ON messages
(recipient)")
    cursor.execute(
        "CREATE INDEX IF NOT EXISTS idx_messages_recipient_group_id ON messages
(recipient_group_id)")
    cursor.execute("CREATE INDEX IF NOT EXISTS idx_messages_timestamp ON messages
(timestamp)")
    cursor.execute("CREATE INDEX IF NOT EXISTS idx_group_members_group_id ON group_members
(group_id)")
    cursor.execute("CREATE INDEX IF NOT EXISTS idx_group_members_username ON group_members
(username)")
    cursor.execute("CREATE INDEX IF NOT EXISTS idx_messages_e2ee_type ON messages
(e2ee_type)")

    # --- SYSTEM_EVENT User ---
    try:
        cursor.execute("SELECT 1 FROM users WHERE username = 'SYSTEM_EVENT'")
        if cursor.fetchone() is None:
            system_password_hash, system_salt_hex = DatabaseManager.hash_password(
                "a_very_secure_system_event_password_!@#$_for_init_v3")
            cursor.execute(
                "INSERT INTO users (username, password_hash, salt) VALUES (?, ?, ?)",
                ('SYSTEM_EVENT', system_password_hash, system_salt_hex)
            )
            print("[DB_MANAGER] 'SYSTEM_EVENT' user created.")
    except sqlite3.Error as e: # Catch any error during SYSTEM_EVENT user creation
        print(f"[DB_MANAGER_ERROR] Could not create/verify 'SYSTEM_EVENT' user: {e}")

    conn.commit() # Final commit for any remaining changes (like index creation or user
insert)
    conn.close()
    print("[DB_MANAGER] Database schema initialization/verification process completed.")
# --- User Management Methods ---
    def add_user_to_db(self, cursor, username, password_hash, salt,
                        profile_picture_filename=None, rsa_public_key_pem=None): # Added
rsa_public_key_pem
        try:
            # Build the SQL query and parameters dynamically based on what's provided
            columns = ["username", "password_hash", "salt"]
            values_placeholders = ["?", "?", "?"]
            params = [username, password_hash, salt]

            if profile_picture_filename is not None: # Only include if a specific filename is
given
                columns.append("profile_picture_filename")
                values_placeholders.append("?")
                params.append(profile_picture_filename)
            # If profile picture filename is None, we DON'T add it to the SQL,
            # so the table's DEFAULT 'default pic.jpg' will apply.

```



```

        if rsa_public_key_pem is not None: # Only include if a specific key is given
            columns.append("rsa_public_key_pem")
            values_placeholders.append("?")
            params.append(rsa_public_key_pem)
        # If rsa_public_key_pem is None, and it has a DEFAULT NULL in schema, that will
        apply.
        # If it has a different SQL DEFAULT, that would apply.

        sql = f"INSERT INTO users ({', '.join(columns)}) VALUES ({', '
        '.join(values_placeholders)})"

        print(f"[DB_MANAGER] Executing add_user_to_db SQL: {sql} with params: {params}")
# For debugging
        cursor.execute(sql, tuple(params)) # Params must be a tuple for execute
        return True
    except sqlite3.IntegrityError: # Username likely already exists
        print(f"[DB_MANAGER_WARN] IntegrityError on add_user_to_db for {username}. User
may already exist.")
        return False
    except Exception as e:
        print(f"[DB_MANAGER_ERROR] Unexpected error in add_user_to_db for {username}:
{e}")
        return False
    def get_user_credentials_from_db(self, cursor, username):
        cursor.execute("SELECT password hash, salt FROM users WHERE username = ?",
        (username,))
        return cursor.fetchone()

    def store_user_rsa_public_key(self, cursor, username, rsa_public_key_pem_str):
        """Stores or updates a user's RSA public key."""
        # Ensure user exists, or handle insertion if users might not have standard accounts
        yet.
        # For now, assume user exists from registration.
        cursor.execute(
            "UPDATE users SET rsa_public_key_pem = ? WHERE username = ?",
            (rsa_public_key_pem_str, username)
        )
        if cursor.rowcount == 0:
            print(f"[DB_WARN] No user found to update RSA public key for: {username}, or key
was the same.")
            return cursor.rowcount > 0

    def get_user_rsa_public_key_from_db(self, cursor, username):
        """Fetches a user's stored RSA public key."""
        cursor.execute("SELECT rsa_public_key_pem FROM users WHERE username = ?", (username,))
        result = cursor.fetchone()
        return result[0] if result and result[0] else None

    def update_user_profile_picture_in_db(self, cursor, username,
        new_profile_picture_filename):
        cursor.execute("UPDATE users SET profile_picture_filename = ? WHERE username = ?",
            (new_profile_picture_filename, username))
        return cursor.rowcount > 0

    def get_profile_picture_filename_from_db(self, cursor, username):
        cursor.execute("SELECT profile_picture_filename FROM users WHERE username = ?",
        (username,))
        result = cursor.fetchone()
        return result[0] if result else None

    def get_all_registered_users_from_db(self, cursor):
        cursor.execute("SELECT username FROM users WHERE username != 'SYSTEM_EVENT' ORDER BY
        username ASC")
        return [row[0] for row in cursor.fetchall()]

    def check_user_exists(self, cursor, username):
        cursor.execute("SELECT 1 FROM users WHERE username = ?", (username,))
        return cursor.fetchone() is not None

# --- Message Management Methods ---

    def store_message_in_db(self, cursor, sender, message_text, recipient_user=None,
        recipient_group_id=None,
        timestamp=None, is_system=False, e2ee_type=None,

```

```

        sender_e2ee_session_data=None): # Added sender_e2ee_session_data
    if timestamp is None:
        timestamp = datetime.now().strftime('%Y-%m-%d %H:%M:%S.%f')[:-3]

    # Ensure the SQL query includes the new column
    sql = """INSERT INTO messages
        (sender, recipient, message, recipient_group_id, timestamp,
         is_system_message, e2ee_type, sender_e2ee_session_data)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?)"""
    params = (sender, recipient_user, message_text, recipient_group_id, timestamp,
              is_system, e2ee_type, sender_e2ee_session_data)

    cursor.execute(sql, params)
    return cursor.lastrowid

def get_all_messages_for_user_from_db(self, cursor, username):
    messages = []
    print(f"[DB_FETCH DEBUG] Fetching history for {username}...")
    try:
        # Query for private/direct messages for the user
        # Added sender e2ee session data to the SELECT
        cursor.execute(
            """SELECT id, sender, recipient, message AS text, timestamp,
                recipient_group_id, is_system_message, e2ee_type,
sender_e2ee_session_data
            FROM messages
            WHERE recipient_group_id IS NULL AND (sender = ? OR recipient = ?)""",
            (username, username)
        )
        private_rows = cursor.fetchall()
        col_names_private = [desc[0] for desc in cursor.description] if cursor.description
    except Exception as e:
        print(f"[DB_FETCH_ERROR] Private row {i} to dict: {row_data}. Error: {e}")
        continue

    for i, row_data in enumerate(private_rows):
        try:
            messages.append(dict(zip(col_names_private, row_data)))
        except Exception as e:
            print(f"[DB_FETCH_ERROR] Private row {i} to dict: {row_data}. Error: {e}")
            continue

    # Get IDs of groups the user is a member of
    cursor.execute("SELECT group_id FROM group_members WHERE username = ?",
                    (username,))
    user_group_ids = [row_gid[0] for row_gid in cursor.fetchall()]

    if user_group_ids:
        placeholders = ', '.join('?' for _ in user_group_ids)
        # Query for group messages for those groups
        # Added sender_e2ee_session_data to the SELECT
        group_messages_query = f"""
            SELECT id, sender, NULL AS recipient, message AS text, timestamp,
                recipient_group_id, is_system_message, e2ee_type,
sender_e2ee_session_data
            FROM messages
            WHERE recipient_group_id IN ({placeholders})
        """
        cursor.execute(group_messages_query, user_group_ids)
        group_rows = cursor.fetchall()
        col_names_group = [desc[0] for desc in cursor.description] if cursor.description
    except Exception as e:
        print(f"[DB_FETCH_ERROR] Group row {i} to dict: {row_data}. Error: {e}")
        continue

    for i, row_data in enumerate(group_rows):
        try:
            messages.append(dict(zip(col_names_group, row_data)))
        except Exception as e:
            print(f"[DB_FETCH_ERROR] Group row {i} to dict: {row_data}. Error: {e}")
            continue

    except sqlite3.Error as e_sql:
        print(f"[DB_FETCH_SQL_ERROR] For {username}: {e_sql}")
        return [] # Return empty list on SQL error

    # Robust timestamp parsing (this was part of your original method)
    def robust_parse_timestamp(ts_string):
        if not isinstance(ts_string, str):
            return datetime.min # Or handle as an error, or a default past date
        try:
            # Try parsing with milliseconds
            return datetime.strptime(ts_string, '%Y-%m-%d %H:%M:%S.%f')

```

```

except ValueError:
    try:
        # Try parsing without milliseconds
        return datetime.strptime(ts_string, '%Y-%m-%d %H:%M:%S')
    except ValueError:
        print(f"[DB_TS_ERR] Parse fail: '{ts_string}'")
        return datetime.min # Or handle as an error

# Sort all collected messages by timestamp (this was part of your original method)
try:
    # Add a sortable key, handling potential missing or malformed timestamps
    for m in messages:
        if 'timestamp' not in m or not isinstance(m['timestamp'], str):
            m['timestamp_sort_key'] = datetime.min
        else:
            m['timestamp_sort_key'] = robust_parse_timestamp(m['timestamp'])

    messages.sort(key=lambda item: item['timestamp_sort_key'])

    # Remove the temporary sort key
    for m in messages:
        if 'timestamp_sort_key' in m:
            del m['timestamp_sort_key']
except Exception as e_sort:
    print(f"[DB_SORT_ERROR] For {username}: {e_sort}")
    # Decide if you want to return unsorted messages or an empty list on sort error
    # Returning potentially unsorted list here:
    # return messages
    return [] # Or return empty on sort error for consistency with SQL error

print(f"[DB_FETCH DEBUG] Returning {len(messages)} sorted messages for {username}.")
return messages

def _handle_initiate_secure_session(self, payload):
    if not self.username:
        return self.send_message_to_client({"type": "AUTH_REQUIRED", "payload":
{"message": "Login required."}})

    recipient_username = payload.get("recipient")
    # This is the AES key encrypted for the RECIPIENT
    recipient_encrypted_session_key_b64 = payload.get("encrypted_session_key_b64")
    # NEW: This is the AES key encrypted for the SENDER (by the sender themselves)
    sender_encrypted_session_key_b64 = payload.get("sender_encrypted_session_key_b64")

    initial_message_payload = payload.get("initial_message") # AES-encrypted actual first
message

    if not recipient_username or not recipient_encrypted_session_key_b64 or \
        not initial_message_payload or not isinstance(initial_message_payload, dict):
        # sender_encrypted_session_key_b64 is optional for now from server's perspective
of *relaying*
        # but client will need to send it if it wants to decrypt its own history.
        # For basic operation, server doesn't strictly need it to forward to recipient.
        # However, for storing it, we should check. Let's make it expected for this
feature.
        return self.send_message_to_client(
            {"type": "ERROR", "payload": {"message": "Invalid session initiation payload
(missing fields)."}})

    conn = self._get_db_connection()
    if not conn: return
    cursor = conn.cursor()
    timestamp = datetime.now().strftime('%Y-%m-%d %H:%M:%S.%f')[:-3]

    # The "message" column still stores the payload relevant for the RECIPIENT
    message_content_for_db_recipient_view = json.dumps({
        "encrypted_session_key_b64": recipient_encrypted_session_key_b64, # Key for
recipient
        "initial_message": initial_message_payload
    })

    try:
        self.server.db_manager.store_message_in_db(
            cursor,
            sender=self.username,
            message_text=message_content_for_db_recipient_view, # For recipient

```

```

        recipient_user=recipient_username,
        timestamp=timestamp,
        is_system=False,
        e2ee_type='AES_SESSION_INIT',
        sender_e2ee_session_data=sender_encrypted_session_key_b64 # For sender's
history
    )
    conn.commit()
    print(f"[SERVER] Stored AES_SESSION_INIT from {self.username} for
{recipient_username} (with sender data).")

    recipient_handler = self.server.get_client_handler(recipient_username)
    if recipient_handler:
        # Send the recipient-specific data to the recipient
        recipient_handler.send_message_to_client({
            "type": "RECEIVE_SECURE_SESSION_INIT",
            "payload": {
                "sender_username": self.username,
                "encrypted_session_key_b64": recipient_encrypted_session_key_b64, #
Key for them

                "initial_message": initial_message_payload,
                "timestamp": timestamp
                # No need to send sender_encrypted_session_key_b64 to the recipient
            }
        })
    else:
        # Sender gets confirmation
        self.send_message_to_client({
            "type": "MESSAGE_STORED_FOR_OFFLINE_DELIVERY",
            "payload": {"recipient_username": recipient_username,
                "timestamp_of_storage": timestamp,
                "message_preview": "[Secure Session Started]"}
        })
    except sqlite3.Error as e_db:
        if conn: conn.rollback()
        print(f"[INIT_SESS_DB_ERROR] {self.username} to {recipient_username}: {e_db}")
        self.send_message_to_client(
            {"type": "ERROR", "payload": {"message": "Server DB error initiating
session."}})
    except Exception as e:
        if conn: conn.rollback() # Ensure rollback on generic error too before commit
        print(f"[INIT_SESS_UNEXPECTED_ERROR] {self.username} to {recipient_username}:
{e}")
        self.send_message_to_client(
            {"type": "ERROR", "payload": {"message": "Unexpected server error initiating
session."}})
    finally:
        if conn: conn.close()

# --- Group Management Methods (Ensure all these are present from your version) ---
def create_group_in_db(self, cursor, group_name, creator_username,
    created_at_ts=None, group_icon_filename=None): #
group_icon_filename defaults to None
    if created_at_ts is None:
        created_at_ts = datetime.now().strftime('%Y-%m-%d %H:%M:%S.%f')[:-3]

    # Build the SQL query and parameters dynamically
    columns = ["group_name", "creator_username", "created_at"]
    values_placeholders = ["?", "?", "?"]
    params = [group_name, creator_username, created_at_ts]

    # Only include group_icon_filename in the INSERT statement if a specific value is
provided.
    # If group_icon_filename is None (its default Python value if not passed by
_handle_create_group),
    # it will be omitted here, and the SQL schema's DEFAULT constraint will apply.
    if group_icon_filename is not None:
        columns.append("group_icon_filename")
        values_placeholders.append("?")
        params.append(group_icon_filename)

    sql = f"INSERT INTO groups ({', '.join(columns)}) VALUES ({',
'.join(values_placeholders)})"

    # For debugging, you can uncomment this to see the exact SQL being run
    # print(f"[DB MANAGER DEBUG] Executing create group in db SQL: {sql} with params:

```

```
{params}")

    try:
        cursor.execute(sql, tuple(params)) # Ensure params is a tuple
        return cursor.lastrowid
    except sqlite3.Error as e:
        print(f"[DB_MANAGER_ERROR] Failed to create group '{group_name}': {e}")
        return None

    def add_member_to_group_in_db(self, cursor, group_id, username, role='member',
joined_at_ts=None):
        if joined_at_ts is None: joined_at_ts = datetime.now().strftime('%Y-%m-%d
%H:%M:%S.%f')[:-3]
        try: cursor.execute("INSERT INTO group_members (group_id, username, role, joined_at)
VALUES (?, ?, ?, ?)", (group_id, username, role, joined_at_ts)); return True
        except sqlite3.IntegrityError: return False

    def get_group_members_from_db(self, cursor, group_id):
        cursor.execute("SELECT username, role FROM group_members WHERE group_id = ? ORDER BY
username ASC", (group_id,))
        return [{"username": row[0], "role": row[1]} for row in cursor.fetchall()]

    def get_user_groups_from_db(self, cursor, username):
        cursor.execute("SELECT g.group_id, g.group_name, g.group_icon_filename FROM groups g
JOIN group_members gm ON g.group_id = gm.group_id WHERE gm.username = ? ORDER BY g.group_name
ASC", (username,))
        return [{"group_id": row[0], "group_name": row[1], "group_icon_filename": row[2]} for
row in cursor.fetchall()]

    def get_group_details_from_db(self, cursor, group_id):
        cursor.execute("SELECT group_name, creator_username, group_icon_filename FROM groups
WHERE group_id = ?", (group_id,)); group_info = cursor.fetchone()
        if not group_info: return None
        members = self.get_group_members_from_db(cursor, group_id)
        return {"group_id": group_id, "group_name": group_info[0], "creator_username":
group_info[1], "group_icon_filename": group_info[2], "members": members }

    def is_user_member_of_group(self, cursor, username, group_id):
        cursor.execute("SELECT 1 FROM group_members WHERE group_id = ? AND username = ?",
(group_id, username)); return cursor.fetchone() is not None

    def get_group_member_role(self, cursor, username, group_id):
        cursor.execute("SELECT role FROM group_members WHERE group_id = ? AND username = ?",
(group_id, username)); result = cursor.fetchone(); return result[0] if result else None

    def is_user_admin_in_group(self, cursor, username, group_id):
        return self.get_group_member_role(cursor, username, group_id) == 'admin'

    def remove_member_from_group_in_db(self, cursor, group_id, username_to_remove):
        cursor.execute("DELETE FROM group_members WHERE group_id = ? AND username = ?",
(group_id, username_to_remove)); return cursor.rowcount > 0

    def update_group_member_role_in_db(self, cursor, group_id, target_username, new_role):
        cursor.execute("UPDATE group_members SET role = ? WHERE group_id = ? AND username =
?", (new_role, group_id, target_username)); return cursor.rowcount > 0

    def update_group_icon_in_db(self, cursor, group_id, new_icon_filename):
        cursor.execute("UPDATE groups SET group_icon_filename = ? WHERE group_id = ?",
(new_icon_filename, group_id)); return cursor.rowcount > 0

    def get_group_icon_filename_from_db(self, cursor, group_id):
        cursor.execute("SELECT group_icon_filename FROM groups WHERE group_id = ?",
(group_id,)); result = cursor.fetchone(); return result[0] if result else None

    def rename_group_in_db(self, cursor, group_id, new_group_name):
        cursor.execute("UPDATE groups SET group_name = ? WHERE group_id = ?", (new_group_name,
group_id)); return cursor.rowcount > 0

    def get_group_creator_from_db(self, cursor, group_id):
        cursor.execute("SELECT creator_username FROM groups WHERE group_id = ?", (group_id,));
result = cursor.fetchone(); return result[0] if result else None

    def count_group_members_in_db(self, cursor, group_id):
        cursor.execute("SELECT COUNT(*) FROM group_members WHERE group_id = ?", (group_id,));
result = cursor.fetchone(); return result[0] if result else 0

    def count_group_admins_in_db(self, cursor, group_id):
        cursor.execute("SELECT COUNT(*) FROM group_members WHERE group_id = ? AND role =
'admin'", (group_id,)); result = cursor.fetchone(); return result[0] if result else 0

    def delete_group_from_db(self, cursor, group_id):
        cursor.execute("DELETE FROM groups WHERE group_id = ?", (group_id,)); return
cursor.rowcount > 0
```

בקובץ: "chat_sever_core.py"

```
# chat_server_core.py

import socket
import threading
import sqlite3

from shared_modules import NetworkNode, DatabaseManager, DB_NAME # Use the new
shared_modules.py

# Constants if not imported from shared_modules or defined in the main runner
# HOST = '127.0.0.1'
# PORT = 65432

class ChatServer(NetworkNode):

    def __init__(self, host, port, db_name=DB_NAME, client_handler_class=None):
        super().__init__()
        self.host = host
        self.port = port
        self.db_manager = DatabaseManager(db_name)
        self.server_socket = None
        self.active_clients = {} # username: ClientHandler instance
        self.active_clients_lock = threading.Lock()
        self.is_running = False
        self.client_handler_class = client_handler_class # To pass the actual ClientHandler
class
        print(f"[SERVER] Initialized. PK (first 60char): {self.get_public_key_pem()[0:60]}...")

    def start(self):
        if not self.client_handler_class:
            print("[SERVER_FATAL] ClientHandler class not provided to ChatServer.")
            return

        self.is_running = True
        self.db_manager.initialize_database()
        self.server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        try:
            self.server_socket.bind((self.host, self.port))
            self.server_socket.listen(5)
            print(f"[SERVER] Listening on {self.host}:{self.port}")
            while self.is_running:
                try:
                    self.server_socket.settimeout(1.0)
                    client_socket, client_address = self.server_socket.accept()
                    self.server_socket.settimeout(None)
                    print(f"[SERVER] Accepted connection from {client_address}")
                    client_thread = self.client_handler_class(client_socket, client_address,
self)

                    client_thread.start()
                except socket.timeout:
                    continue
                except OSError as e:
                    if self.is_running: print(f"[SERVER_ERROR] Accept OSError: {e}")
                    self.is_running = False;
                    break
                except Exception as e:
                    print(f"[SERVER_ERROR] Accept Exception: {e}")
            except socket.error as e:
                print(f"[SERVER_FATAL] Socket bind/listen error: {e}"); self.is_running = False
            except KeyboardInterrupt:
                print("\n[SERVER] KeyboardInterrupt. Shutting down."); self.is_running = False
            finally:
                self.shutdown()

    def shutdown(self):
        # (Full implementation as previously provided)
        print("[SERVER] Shutdown sequence...");
        self.is_running = False
        if self.server_socket:
            try:
                self.server_socket.close()
            except Exception as e:
                print(f"[SERVER_ERR] Closing main socket: {e}")
```

```

        with self.active_clients_lock:
            handlers_copy = list(self.active_clients.values()); self.active_clients.clear()
        for handler in handlers_copy:
            try:
                handler.send_message_to_client({"type": "SYSTEM", "payload": {"text": "Server
is shutting down."}})
                handler.cleanup();
                if handler.is_alive(): handler.join(0.5)
            except Exception as e:
                print(f"[SERVER_ERR] Handler cleanup: {e}")
        print("[SERVER] Shutdown complete.")

    def register_client(self, username, client_handler_instance):
        with self.active_clients_lock:
            if username in self.active_clients:
                old_h = self.active_clients.get(username)
                if old_h and old_h != client_handler_instance:
                    old_h.send_message_to_client({"type": "SYSTEM", "payload": {"text":
"Logged out: new session."}});
                    old_h.cleanup()
                self.active_clients[username] = client_handler_instance;
                print(f"[SERVER] User '{username}' registered. Active:
{len(self.active_clients)}")

    def unregister_client(self, username, client_handler_instance):
        with self.active_clients_lock:
            if self.active_clients.get(username) == client_handler_instance:
                del self.active_clients[username];
                print(f"[SERVER] User '{username}' unregistered. Active:
{len(self.active_clients)}")

    def get_client_handler(self, username):
        with self.active_clients_lock: return self.active_clients.get(username)

    def get_user_public_key_pem(self, target_username):
        """
        Retrieves the PEM-encoded public key of a user.
        First checks active clients, then falls back to database.
        """
        # Check active clients first (might be fresher if recently updated and not yet in DB,
        though DB should be source of truth)
        target_handler = self.get_client_handler(target_username) # This uses
        active_clients_lock
        if target_handler and hasattr(target_handler, 'client_public_key_pem') and
        target_handler.client_public_key_pem:
            print(f"[SERVER] Found PK for '{target_username}' in active handlers.")
            return target_handler.client_public_key_pem

        # If not in active handlers (or handler doesn't have key yet), try database
        print(f"[SERVER] PK for '{target_username}' not in active handlers, trying DB.")
        conn = None
        try:
            # Need a short-lived connection for this read.
            # _get_db_connection is a ClientHandler method. ChatServer needs its own.
            conn = sqlite3.connect(self.db_manager.db_name, timeout=5.0)
            cursor = conn.cursor()
            key_pem = self.db_manager.get_user_rsa_public_key_from_db(cursor, target_username)
            if key_pem:
                print(f"[SERVER] Found PK for '{target_username}' in DB.")
            else:
                print(f"[SERVER_WARN] PK for '{target_username}' not found in DB either.")
            return key_pem
        except sqlite3.Error as e:
            print(f"[SERVER_ERROR] DB error fetching PK for {target_username}: {e}")
            return None
        finally:
            if conn:
                conn.close()

    def broadcast_message_to_group(self, group_id, message_obj, db_cursor,
        exclude_username=None):
        try:
            member_records = self.db_manager.get_group_members_from_db(db_cursor, group_id)
            for rec in member_records:
                uname = rec["username"]
                if exclude_username and uname == exclude_username: continue
                handler = self.get_client_handler(uname)
                if handler: handler.send_message_to_client(message_obj)

```

```
except sqlite3.Error as e:
    print(f"[BROADCAST_GRP_DB_ERR] GID {group_id}: {e}")
except Exception as e:
    print(f"[BROADCAST_GRP_UNEX_ERR] GID {group_id}: {e}")
```

בקובץ: "server.py":

```
# server.py

import socket # Used by ClientHandler (SHUT_RDWR)
import threading
import json
import sqlite3 # Used by ClientHandler for type hints and errors
import base64
import os
from datetime import datetime

# --- Imports from your custom modules ---
# As per your statement, NetworkNode is in shared modules (was db manager.py)
from shared_modules import DatabaseManager, NetworkNode, DB_NAME, PROFILE_PICS_DIR,
GROUP_ICONS_DIR
from chat_server_core import ChatServer # ChatServer class

# --- Server Configuration
HOST = '127.0.0.1'
PORT = 65432

# DB_NAME, PROFILE_PICS_DIR, GROUP_ICONS_DIR are imported from shared_modules

# ClientHandler Class Definition

class ClientHandler(threading.Thread):

    def __init__(self, client_socket, client_address, server_instance: ChatServer):
        super().__init__(daemon=True)
        self.client_socket = client_socket
        self.client_address = client_address
        self.server = server_instance
        self.username = None
        self.client_public_key_pem = None
        self.receive_buffer = b""
        self.is_connected = True

    def _get_db_connection(self):
        try:
            conn = sqlite3.connect(self.server.db_manager.db_name, timeout=7.5) # Use DB_NAME
            conn.execute("PRAGMA foreign_keys = ON;")
            return conn
        except sqlite3.Error as e:
            print(f"[DB_CONN_ERR] User {self.username or self.client_address}: {e}")
            self.send_message_to_client({"type": "SYSTEM_ERROR", "payload": {"message": "DB
connect error."}})
            return None

    def send_message_to_client(self, message_obj):
        if not self.is_connected or not self.client_socket:
            return False
        try:
            self.client_socket.sendall(json.dumps(message_obj).encode('utf-8') + b'\n')
            return True
        except (OSError, ConnectionResetError, BrokenPipeError) as e:
            print(f"[SEND_ERR] Client {self.client_address} (User: {self.username}): {e}.
Cleaning up.")
            self.cleanup()
            return False
        except Exception as e:
            print(f"[SEND_ERR_UNEX] Client {self.client_address} (User: {self.username}):
{e}")
            return False
```



```

def cleanup(self):
    if not self.is_connected and self.client_socket is None: return
    print(f"[CLEANUP] For {self.client_address} (User: {self.username})")
    self.is_connected = False
    if self.username:
        self.server.unregister_client(self.username, self)
        self.username = None
    socket_to_close = self.client_socket
    self.client_socket = None
    if socket_to_close:
        try:
            socket_to_close.shutdown(socket.SHUT_RDWR)
        except OSError:
            pass
        except Exception as e:
            print(f"[CLEANUP_ERR] Socket shutdown {self.client_address}: {e}")
        try:
            socket_to_close.close()
        except Exception as e:
            print(f"[CLEANUP_ERR] Socket close {self.client_address}: {e}")

def run(self):
    print(f"[HANDLER] Thread started for {self.client_address}")
    server_pk_pem = self.server.get_public_key_pem() # ChatServer inherits this from
NetworkNode
    if not server_pk_pem:
        self.send_message_to_client({"type": "SYSTEM_ERROR", "payload": {"text": "Server
key error."}})
        self.cleanup()
        return
    self.send_message_to_client({"type": "SERVER PUBLIC KEY", "payload": {"key":
server_pk_pem}})

    while self.is_connected:
        try:
            data = self.client_socket.recv(8192) # Increased buffer
            if not data:
                if self.is_connected:
                    print(f"[HANDLER_RECV] No data from {self.client_address} (User:
{self.username}). Client closed.")
                    self.is_connected = False
                    break
                self.receive_buffer += data
            while b'\n' in self.receive_buffer and self.is_connected:
                message_part, self.receive_buffer = self.receive_buffer.split(b'\n', 1)
                message_str = message_part.decode('utf-8').strip()
                if not message_str: continue
                try:
                    message_obj = json.loads(message_str)
                    msg_type = message_obj.get("type")
                    payload = message_obj.get("payload", {})

                    handler_method_name = f"_handle_{msg_type.lower() if msg_type else
'unknown_message'}"
                    handler_method = getattr(self, handler_method_name,
self._handle_unknown_message) # Fallback

                    if self.username or msg_type in ["REGISTER",
"LOGIN"]: # Allow register/login
                        handler_method(payload)
                    else:
                        self.send_message_to_client(
                            {"type": "AUTH_REQUIRED", "payload": {"message":
"Authentication required."}})

                except json.JSONDecodeError:
                    print(f"[JSON_ERR] User {self.username or self.client_address}. Data:
'{message_str[:100]}'")
                    self.send_message_to_client({"type": "ERROR", "payload": {"message":
"Invalid JSON."}})
                except Exception as e_proc:
                    print(f"[MSG_PROC_ERR] User {self.username or self.client_address}:
{e_proc} on type '{msg_type}'.")
                    self.send_message_to_client({"type": "SERVER ERROR", "payload":
{"message": "Error processing request."}})

```

```

        except (ConnectionResetError, ConnectionAbortedError, BrokenPipeError, OSError) as
e_conn:
            if self.is_connected: print(
                f"[HANDLER_CONN_ERR] Client {self.client_address} (User: {self.username})
disconnected: {e_conn}")
            self.is_connected = False
            except Exception as e_outer:
                if self.is_connected: print(
                    f"[HANDLER_CRITICAL_ERR] Loop for {self.client_address} (User:
{self.username}): {e_outer}")
                self.is_connected = False
            self.cleanup()
            # print(f"[HANDLER] Thread finished for {self.client_address} (Former User:
{self.username} or 'N/A')")

    def _handle_initiate_secure_session(self, payload):
        if not self.username:
            return self.send_message_to_client({"type": "AUTH_REQUIRED", "payload":
{"message": "Login required."}})

        recipient_username = payload.get("recipient")
        # This is the AES key encrypted for the RECIPIENT's processing
        recipient_encrypted_session_key_b64 = payload.get("encrypted_session_key_b64")

        # NEW: This is the AES key encrypted by the SENDER for their OWN history viewing
        sender_encrypted_session_key_b64 = payload.get("sender_encrypted_session_key_b64")

        initial_message_payload = payload.get("initial_message") # This is the AES-encrypted
actual first message

        # Basic validation
        if not recipient_username or \
            not recipient_encrypted_session_key_b64 or \
            not sender_encrypted_session_key_b64 or \
            not initial_message_payload or \
            not isinstance(initial_message_payload, dict) or \
            "iv_b64" not in initial_message_payload or \
            "ciphertext_b64" not in initial_message_payload:
            print(f"[SERVER_ERROR] _handle_initiate_secure_session: Invalid payload from
{self.username}. "
                f"Recipient: {recipient_username is not None}, "
                f"RecipientKey: {recipient_encrypted_session_key_b64 is not None}, "
                f"SenderKey: {sender_encrypted_session_key_b64 is not None}, "
                f"InitialMsg: {initial_message_payload is not None and
isinstance(initial_message_payload, dict)}")

            return self.send_message_to_client(
                {"type": "ERROR", "payload": {"message": "Invalid or incomplete session
initiation payload."}})

        conn = self._get_db_connection()
        if not conn:
            # _get_db_connection likely sends an error, but ensure flow stops
            return

        cursor = conn.cursor()
        timestamp = datetime.now().strftime('%Y-%m-%d %H:%M:%S.%f')[:-3]

        # This is the JSON string stored in the main 'message' column,
        # primarily for the recipient to process.
        message_content_for_db_recipient_view = json.dumps({
            "encrypted_session_key_b64": recipient_encrypted_session_key_b64, # Key for
recipient
            "initial_message": initial_message_payload
        })

        # --- DEBUGGING PRINT ---
        print(f"[SERVER_STORE_INIT_SESS] From {self.username} to {recipient_username}.")
        print(
            f"[SERVER_STORE_INIT_SESS] sender encrypted_session_key_b64 received from client:
{sender_encrypted_session_key_b64 is not None}")
        if sender_encrypted_session_key_b64:
            print(
                f"[SERVER_STORE_INIT_SESS] Value of sender_encrypted_session_key_b64
(preview): {sender_encrypted_session_key_b64[:50]}...")
        # --- END DEBUGGING PRINT ---

```

```

try:
    self.server.db_manager.store_message_in_db(
        cursor,
        sender=self.username,
        message_text=message_content_for_db_recipient_view, # Main payload for
recipient
        recipient_user=recipient_username,
        timestamp=timestamp,
        is_system=False,
        e2ee_type='AES_SESSION_INIT',
        sender_e2ee_session_data=sender_encrypted_session_key_b64 # Data for sender's
history
    )
    conn.commit()
    print(f"[SERVER] Stored AES_SESSION_INIT from {self.username} for
{recipient_username} (with sender self-encrypted key data).")

    # Attempt to deliver to recipient if they are online
    recipient_handler = self.server.get_client_handler(recipient_username)
    if recipient_handler:
        # Send only the recipient-relevant parts
        recipient_handler.send_message_to_client({
            "type": "RECEIVE_SECURE_SESSION_INIT",
            "payload": {
                "sender_username": self.username,
                "encrypted_session_key_b64": recipient_encrypted_session_key_b64, #
Key for recipient
                "initial_message": initial_message_payload,
                "timestamp": timestamp
                # DO NOT send sender_encrypted_session_key_b64 to the recipient
            }
        })
        print(f"[SERVER] Relayed AES_SESSION_INIT from {self.username} to online user
{recipient_username}.")
    else:
        # Notify sender that message is stored for offline delivery
        self.send_message_to_client({
            "type": "MESSAGE_STORED_FOR_OFFLINE_DELIVERY",
            "payload": {"recipient_username": recipient_username,
                "timestamp_of_storage": timestamp,
                "message_preview": "[Secure Session Started]"} # Generic
preview
        })
        print(f"[SERVER] AES_SESSION_INIT from {self.username} for offline user
{recipient_username} stored.")

    except sqlite3.Error as e_db:
        if conn: conn.rollback()
        print(f"[INIT_SESS_DB_ERROR] User {self.username} to {recipient_username}:
{e_db}")
        self.send_message_to_client(
            {"type": "ERROR", "payload": {"message": "Server database error while
initiating secure session."}})
        except Exception as e:
            # It's good practice to rollback if an error occurs before a successful commit
            if conn and conn.in_transaction: # Check if a transaction is active
                try:
                    conn.rollback()
                    print(
                        f"[INIT_SESS_UNEXPECTED_ERROR_RB] Rolled back transaction for
{self.username} to {recipient_username}.")
                except sqlite3.Error as e_rb:
                    print(f"[INIT_SESS_UNEXPECTED_ERROR_RB_FAIL] Failed to rollback
transaction: {e_rb}")

                print(f"[INIT_SESS_UNEXPECTED_ERROR] User {self.username} to {recipient_username}:
{e}")
                self.send_message_to_client(
                    {"type": "ERROR",
                     "payload": {"message": "An unexpected server error occurred while initiating
session."}})
            finally:
                if conn:
                    conn.close()

def _handle_aes_encrypted_message(self, payload):

```

```

        if not self.username:
            return self.send_message_to_client({"type": "AUTH_REQUIRED", "payload":
{"message": "Login required."}})

        recipient_username = payload.get("recipient")
        aes_payload = payload.get("aes_payload") # This is {'iv_b64': ..., 'ciphertext_b64':
...}

        if not recipient_username or not aes_payload or \
            not isinstance(aes_payload, dict) or \
            "iv_b64" not in aes_payload or "ciphertext_b64" not in aes_payload:
            return self.send_message_to_client({"type": "ERROR", "payload": {"message":
"Invalid AES message payload."}})

        conn = self._get_db_connection()
        if not conn: return
        cursor = conn.cursor()
        timestamp = datetime.now().strftime('%Y-%m-%d %H:%M:%S.%f')[:-3]
        # Store the AES payload (IV + ciphertext) as a JSON string in the message column
        message_content_for_db = json.dumps(aes_payload)

        try:
            self.server.db_manager.store_message_in_db(
                cursor, sender=self.username, message_text=message_content_for_db,
                recipient_user=recipient_username, timestamp=timestamp,
                is_system=False, e2ee_type='AES MSG' # Mark as AES message
            )
            conn.commit()
            print(f"[SERVER] Stored AES_MSG from {self.username} for {recipient_username}")

            recipient_handler = self.server.get_client_handler(recipient_username)
            if recipient_handler:
                recipient_handler.send_message_to_client({
                    "type": "RECEIVE_AES_MESSAGE",
                    "payload": {
                        "sender_username": self.username,
                        "aes_payload": aes_payload,
                        "timestamp": timestamp
                    }
                })
            else:
                self.send_message_to_client({
                    "type": "MESSAGE_STORED_FOR_OFFLINE_DELIVERY",
                    "payload": {"recipient_username": recipient_username,
"timestamp_of_storage": timestamp, "message_preview": "[Encrypted AES Message]"}
                })
            except sqlite3.Error as e_db:
                if conn: conn.rollback()
                print(f"[AES_MSG_DB_ERROR] {self.username} to {recipient_username}: {e_db}")
                self.send_message_to_client({"type": "ERROR", "payload": {"message": "Server DB
error storing AES message."}})
            except Exception as e:
                if conn: conn.rollback()
                print(f"[AES_MSG_UNEXPECTED_ERROR] {self.username} to {recipient_username}: {e}")
                self.send_message_to_client({"type": "ERROR", "payload": {"message": "Unexpected
server error storing AES message."}})
            finally:
                if conn: conn.close()

        def _handle_unknown_message(self, payload):
            self.send_message_to_client({"type": "ERROR", "payload": {"message": "Unknown or
unsupported message type."}})

        def _handle_register(self, payload):
            username = payload.get("username")
            password = payload.get("password")
            client_public_key_pem = payload.get("client_public_key") # Expecting this from client

            if not username or not password:
                return self.send_message_to_client(
                    {"type": "REGISTER_FAIL", "payload": {"message": "Username and password are
required."}})

            if not client_public_key_pem: # Make public key mandatory for registration now
                return self.send_message_to_client(
                    {"type": "REGISTER_FAIL", "payload": {"message": "Client public key is
required for registration."}})

```

```

hashed_pw, salt_hex = DatabaseManager.hash_password(password)

conn = self._get_db_connection()
if not conn:
    # _get_db_connection already sends an error if it fails to connect
    return self.send_message_to_client( # Defensive error
        {"type": "REGISTER_FAIL", "payload": {"message": "Server database connection
error."}})

    cursor = conn.cursor()
    try:
        # Pass the client_public_key_pem to add_user_to_db
        success = self.server.db_manager.add_user_to_db(
            cursor, username, hashed_pw, salt_hex,
rsa_public_key_pem=client_public_key_pem
        )
        if success:
            conn.commit()
            print(f"[SERVER] User '{username}' registered successfully with their public
key.")

            self.send_message_to_client(
                {"type": "REGISTER_SUCCESS", "payload": {"message": "Registration
successful! Please log in."}})
            else:
                # This rollback is good practice, though add user to db might not start a
transaction itself
                # if it's just one INSERT. If it did more, this is crucial.
                if conn.in_transaction: conn.rollback()
                self.send_message_to_client(
                    {"type": "REGISTER_FAIL",
                     "payload": {"message": "Username may already exist or another
registration error."}})
            except sqlite3.Error as e_db:
                if conn.in_transaction: conn.rollback()
                print(f"[REG_DB_ERR] Database error during registration for {username}: {e_db}")
                self.send_message_to_client(
                    {"type": "REGISTER_FAIL", "payload": {"message": "Server database error during
registration."}})
            except Exception as e_reg:
                if conn.in_transaction: conn.rollback()
                print(f"[REG_UNEXPECTED_ERR] Unexpected error during registration for {username}:
{e_reg}")

                self.send_message_to_client(
                    {"type": "REGISTER_FAIL", "payload": {"message": "Unexpected server error
during registration."}})
            finally:
                if conn:
                    conn.close()

    def _handle_login(self, payload):
        username = payload.get("username")
        password = payload.get("password")
        client_rsa_public_key_pem = payload.get("client_public_key") # Client sends its RSA
public key

        if not username or not password:
            return self.send_message_to_client(
                {"type": "LOGIN_FAIL", "payload": {"message": "Username/password required."}})
        if not client_rsa_public_key_pem:
            return self.send_message_to_client(
                {"type": "LOGIN_FAIL", "payload": {"message": "Client RSA public key
required."}})

        conn = self._get_db_connection()
        if not conn: return

        cursor = conn.cursor()
        try:
            user_creds = self.server.db_manager.get_user_credentials_from_db(cursor, username)
            if user_creds and DatabaseManager.verify_password(user_creds[0], password,
user_creds[1]):
                self.username = username
                self.client_public_key_pem = client_rsa_public_key_pem # Store client's RSA
key in the handler

                # Store/Update client's RSA public key in the database
                self.server.db_manager.store_user_rsa_public_key(cursor, self.username,

```

```

self.client_public_key_pem)
    conn.commit() # Commit RSA key storage before proceeding further

    self.server.register_client(self.username, self)
    self.send_message_to_client({"type": "LOGIN_SUCCESS", "payload": {"username":
self.username}})
    print(f"[SERVER] User '{self.username}' logged in. Their RSA PK
stored/updated. Sending history.")

    # Fetch and send historical messages (cursor is still valid after commit for
reads on same connection)
    all_msgs = self.server.db_manager.get_all_messages_for_user_from_db(cursor,
self.username)
    print(f"[SERVER] Fetched {len(all_msgs)} historical messages for
{self.username}.")
    for msg_data in all_msgs:
        try:
            hist_payload = msg_data.copy()
            if not isinstance(hist_payload, dict): continue # Skip malformed

            if hist_payload.get("recipient_group_id") is not None: # Group
message
                hist_payload["sender_username"] = msg_data.get("sender")

            # Ensure all necessary fields are present and correctly named
            # 'text' from DB is 'message' column, 'e2ee_type' is now also selected
            final_hist_payload = {
                "sender": msg_data.get("sender"),
                "recipient": msg_data.get("recipient"),
                "text": msg_data.get("text"), # This is the content (plain, or
JSON string for E2EE)
                "timestamp": msg_data.get("timestamp"),
                "recipient_group_id": msg_data.get("recipient_group_id"),
                "is_system_message": msg_data.get("is_system_message", False),
                "e2ee_type": msg_data.get("e2ee_type"), # Pass this to client
                "sender_e2ee_session_data":
msg_data.get("sender_e2ee_session_data"),
                # Ensure this is included
                "sender_username": hist_payload.get("sender_username")
                # Ensure this is set if it's a group message
            }
            print(f"[SERVER HIST_SEND] Sending historical message payload to
client: {final_hist_payload}")
            self.send_message_to_client({"type": "MESSAGE", "payload":
final_hist_payload})
        except Exception as e_send_hist:
            print(
                f"[SERVER_ERROR] _handle_login: Failed to process/send historical
message for {self.username}: {e_send_hist}")
            print(f"[SERVER] Finished sending history for {self.username}.")
        else:
            self.send_message_to_client(
                {"type": "LOGIN_FAIL", "payload": {"message": "Invalid username or
password."}})
    except sqlite3.Error as e_db:
        if conn: conn.rollback() # Rollback if storing RSA key failed
        print(f"[LOGIN_DB_ERROR] User {username}: {e_db}")
        self.send_message_to_client({"type": "LOGIN_FAIL", "payload": {"message": "Server
DB error during login."}})
    except Exception as e_login:
        if conn: conn.rollback()
        print(f"[LOGIN_UNEXPECTED_ERROR] User {username}: {e_login}")
        self.send_message_to_client(
            {"type": "LOGIN_FAIL", "payload": {"message": "Unexpected server error during
login."}})
    finally:
        if conn: conn.close()

    def _handle_request_user_public_key(self, payload):
        target_username = payload.get("username")
        if not target_username: return self.send_message_to_client(
            {"type": "USER_PUBLIC_KEY_FAIL", "payload": {"message": "Target username
missing."}})
        key_pem = self.server.get_user_public_key_pem(target_username)
        if key_pem:
            self.send_message_to_client(

```

```

        {"type": "USER_PUBLIC_KEY_INFO", "payload": {"username": target_username,
"key": key_pem}})
    else:
        self.send_message_to_client({"type": "USER_PUBLIC_KEY_FAIL", "payload":
{"username": target_username,
"message": "User offline or no key."}})

    def _handle_encrypted_private_message(self, payload):
        recipient_username = payload.get("recipient")
        encrypted_text_b64 = payload.get("encrypted_text_b64")

        if not self.username:
            return self.send_message_to_client({"type": "AUTH_REQUIRED", "payload":
{"message": "Login first."}})
        if not recipient_username or not encrypted_text_b64:
            return self.send_message_to_client(
                {"type": "ERROR", "payload": {"message": "Recipient or encrypted text
missing."}})

        conn = self.get_db_connection()
        if not conn: return

        cursor = conn.cursor()
        message_id = None # To store the ID of the saved message
        timestamp = datetime.now().strftime('%Y-%m-%d %H:%M:%S.%f')[:-3]
        try:
            print(f"[SERVER DEBUG] Attempting to store E2EE msg from {self.username} for
{recipient_username}")
            message_id = self.server.db_manager.store_message_in_db(
                cursor,
                sender=self.username,
                message_text=encrypted_text_b64, # Storing the encrypted blob
                recipient_user=recipient_username,
                timestamp=timestamp,
                is_system=False
            )
            conn.commit() # Commit the message to the database
            print(f"[SERVER] Stored E2EE message ID {message_id} from {self.username} for
{recipient_username}")

            # Now handle online/offline delivery notification
            recipient_handler = self.server.get_client_handler(recipient_username)
            if recipient_handler:
                recipient_handler.send_message_to_client({
                    "type": "INCOMING_ENCRYPTED_MESSAGE",
                    "payload": {
                        "sender_username": self.username,
                        "encrypted_text_b64": encrypted_text_b64,
                        "timestamp": timestamp
                    }
                })
            else:
                self.send_message_to_client({
                    "type": "MESSAGE_STORED_FOR_OFFLINE_DELIVERY",
                    "payload": {
                        "recipient_username": recipient_username,
                        "timestamp_of_storage": timestamp,
                        "message_preview": "[Encrypted Message]"
                    }
                })
        except sqlite3.Error as e_db:
            if conn: conn.rollback() # Rollback if commit failed or other DB error before
commit for this specific try
            print(f"[STORE_ENCRYPTED_MSG_ERROR] {self.username} to {recipient_username}:
{e_db}")

            self.send_message_to_client({"type": "ERROR", "payload": {
                "message": "Failed to store encrypted message due to server database
error."}})
        except Exception as e: # Catch other unexpected errors
            # If commit happened, this error is during notification. If not, rollback.
            # This generic except might be too broad if commit already happened.
            # It's safer to assume if we reach here and commit hasn't been explicitly
            confirmed as successful, a rollback is safer.
            if conn: conn.rollback()
            print(f"[ENCRYPTED MSG UNEXPECTED ERROR] {self.username} to {recipient username}:
{e}")

```

```

        self.send_message_to_client({"type": "ERROR", "payload": {
            "message": "An unexpected error occurred while processing your encrypted
message."}})
    finally:
        if conn: conn.close()

def _handle_get_all_users(self, payload=None):
    conn = self._get_db_connection()
    if not conn: return
    cursor = conn.cursor()
    try:
        users = self.server.db_manager.get_all_registered_users_from_db(cursor)
        self.send_message_to_client({"type": "ALL_USERS_LIST", "payload": {"users":
users}})
    except sqlite3.Error as e:
        print(f"[GET ALL USERS_ERR] {e}")
        self.send_message_to_client(
            {"type": "ERROR", "payload": {"message": "Could not get users."}})
    finally:
        conn.close()

def _handle_get_my_groups(self, payload=None):
    if not self.username: return self.send_message_to_client({"type": "AUTH_REQUIRED"})
    conn = self._get_db_connection()
    if not conn: return
    cursor = conn.cursor()
    try:
        groups = self.server.db_manager.get_user_groups_from_db(cursor, self.username)
        self.send_message_to_client({"type": "MY_GROUPS_LIST", "payload": {"groups":
groups}})
    except sqlite3.Error as e:
        print(f"[GET MY GRPS_ERR] User {self.username}: {e}")
        self.send_message_to_client(
            {"type": "ERROR", "payload": {"message": "Could not get groups."}})
    finally:
        conn.close()

def _handle_get_group_details(self, payload):
    group_id = payload.get("group_id")
    if group_id is None: return self.send_message_to_client(
        {"type": "ERROR", "payload": {"message": "Group ID missing."}})
    conn = self._get_db_connection()
    if not conn: return
    cursor = conn.cursor()
    try:
        details = self.server.db_manager.get_group_details_from_db(cursor, group_id)
        if details:
            self.send_message_to_client({"type": "GROUP_DETAILS_DATA", "payload":
details})
        else:
            self.send_message_to_client(
                {"type": "ERROR", "payload": {"message": f"Group ID {group_id} not
found."}})
    except sqlite3.Error as e:
        print(f"[GET_GRP_DETAILS_ERR] GID {group_id}: {e}")
        self.send_message_to_client(
            {"type": "ERROR", "payload": {"message": "Could not get group details."}})
    finally:
        conn.close()

def _handle_create_group(self, payload):
    group_name = payload.get("group_name", "").strip()
    member_usernames = payload.get("member_usernames", [])
    if not self.username: return self.send_message_to_client({"type": "AUTH_REQUIRED"})
    if not group_name: return self.send_message_to_client(
        {"type": "GROUP_CREATE_FAIL", "payload": {"message": "Group name empty."}})
    members_to_add = set(m.strip() for m in member_usernames if m.strip())
    members_to_add.add(self.username)
    conn = self._get_db_connection()
    if not conn: return
    cursor = conn.cursor()
    try:
        for member_uname in members_to_add:
            if not self.server.db_manager.check_user_exists(cursor, member_uname):
                conn.rollback()

```



```

        return self.send_message_to_client(
            {"type": "GROUP_CREATE_FAIL", "payload": {"message": f"User
'{member_username}' DNE."}})
        ts = datetime.now().strftime('%Y-%m-%d %H:%M:%S.%f')[:-3]
        group_id = self.server.db_manager.create_group_in_db(cursor, group_name,
self.username, created_at=ts)
        if not group_id:
            conn.rollback()
            return self.send_message_to_client(
                {"type": "GROUP_CREATE_FAIL", "payload": {"message": "Failed create group
row."}})
        for member_username in members_to_add:
            role = 'admin' if member_username == self.username else 'member'
            self.server.db_manager.add_member_to_group_in_db(cursor, group_id,
member_username, role, joined_at=ts)
            sys_msg = f"User '{self.username}' created group '{group_name}'."
            self.server.db_manager.store_message_in_db(cursor, "SYSTEM_EVENT", sys_msg,
recipient_group_id=group_id,
                                                    timestamp=ts, is_system=True)
            conn.commit()
            self.send_message_to_client({"type": "GROUP_CREATE_SUCCESS",
"payload": {"group_id": group_id, "group_name":
group_name,
"members": list(members_to_add)}})
            broadcast_payload = {"group_id": group_id, "sender_username": "SYSTEM_EVENT",
"text": sys_msg,
"timestamp": ts, "is_system_event": True}
            self.server.broadcast_message_to_group(group_id,
{"type": "RECEIVE_GROUP_MESSAGE",
"payload": broadcast_payload},
cursor)
        except sqlite3.Error as e:
            conn.rollback()
            print(
                f"[CREATE_GRP_ERR] User {self.username}, Group {group_name}: {e}")
            self.send_message_to_client(
                {"type": "GROUP_CREATE_FAIL", "payload": {"message": f"DB error: {e}"}})
        finally:
            conn.close()

    def _handle_add_group_members(self, payload):
        if not self.username: return self.send_message_to_client({"type": "AUTH_REQUIRED"})
        group_id = payload.get("group_id")
        new_member_usernames = payload.get("new_member_usernames", [])
        if group_id is None or not isinstance(new_member_usernames, list): return
        self.send_message_to_client(
            {"type": "ADD_MEMBERS_FAIL", "payload": {"group_id": group_id, "message": "Invalid
payload."}})
        if not new_member_usernames: return self.send_message_to_client(
            {"type": "ADD_MEMBERS_SUCCESS", "payload": {"group_id": group_id, "message": "No
new members."}})
        conn = self._get_db_connection()
        if not conn: return
        cursor = conn.cursor()
        added_list = []
        failed_map = {}
        try:
            if not self.server.db_manager.is_user_admin_in_group(cursor, self.username,
group_id):
                conn.rollback()
                return self.send_message_to_client(
                    {"type": "ADD_MEMBERS_FAIL", "payload": {"group_id": group_id, "message":
"Permission denied."}})
            for uname_raw in set(new_member_usernames):
                uname = uname_raw.strip()
                if not uname: continue
                if not self.server.db_manager.check_user_exists(cursor, uname):
                    failed_map[uname] = "User DNE."
                    continue
                if self.server.db_manager.is_user_member_of_group(cursor, uname, group_id):
                    failed_map[uname] = "Already member."
                    continue
                if self.server.db_manager.add_member_to_group_in_db(cursor, group_id, uname,
role='member'):
                    added_list.append(uname)
                else:
                    failed_map[uname] = "DB add failed."

```

```

        ts = ""
        sys_msg = ""
        if added_list:
            sys_msg = f"'{self.username}' added {'', '.join(added_list)} to the group.";
            ts = datetime.now().strftime('%Y-%m-%d %H:%M:%S.%f')[:-3]
            self.server.db_manager.store_message_in_db(cursor, "SYSTEM_EVENT", sys_msg,
recipient_group_id=group_id,
                                                    timestamp=ts, is_system=True)

        conn.commit()
        final_parts = []
        if added_list: final_parts.append(f"Added: {'', '.join(added_list)}".)
        if failed_map: final_parts.append(
            f"Failed: " + "; ".join([f"{u} ({r})" for u, r in failed_map.items()]) + ".")
        self.send_message_to_client({"type": "ADD_MEMBERS_SUCCESS", "payload":
{"group_id": group_id,
                                                    "message":
" ".join(
final_parts) if final_parts else "No changes.",
                                                    "added":
added_list,
                                                    "failed":
failed_map}})
        if added_list and sys_msg:
            broadcast_payload = {"group_id": group_id, "sender_username": "SYSTEM_EVENT",
"text": sys_msg,
                                "timestamp": ts, "is_system_event": True}
            self.server.broadcast_message_to_group(group_id,
                                                    {"type": "RECEIVE_GROUP_MESSAGE",
"payload": broadcast_payload},
                                                    cursor)

        except sqlite3.Error as e:
            conn.rollback()
            print(
                f"[ADD_MEM_DB_ERR] User {self.username}, GID {group_id}: {e}")
            self.send_message_to_client(
                {"type": "ADD_MEMBERS_FAIL", "payload": {"group_id": group_id, "message": f"DB
error: {e}"}})
        except Exception as e_unhandled:
            conn.rollback()
            print(
                f"[ADD_MEM_UNEX_ERR] User {self.username}, GID {group_id}: {e_unhandled}")
            self.send_message_to_client(
                {"type": "ADD_MEMBERS_FAIL", "payload": {"group_id": group_id, "message":
"Unexpected server error."}})
        finally:
            conn.close()

    def _handle_remove_group_member(self, payload):
        if not self.username: return self.send_message_to_client({"type": "AUTH_REQUIRED"})
        group_id = payload.get("group_id")
        username_to_remove = payload.get("username_to_remove")
        if group_id is None or not username_to_remove: return self.send_message_to_client(
            {"type": "REMOVE_MEMBER_FAIL", "payload": {"group_id": group_id, "message":
"Invalid payload."}})
        conn = self._get_db_connection()
        if not conn: return
        cursor = conn.cursor()
        try:
            if not self.server.db_manager.is_user_admin_in_group(cursor, self.username,
group_id):
                conn.rollback()
                return self.send_message_to_client(
                    {"type": "REMOVE_MEMBER_FAIL", "payload": {"group_id": group_id,
"message": "Permission denied."}})
            if not self.server.db_manager.is_user_member_of_group(cursor, username_to_remove,
group_id):
                conn.rollback()
                return self.send_message_to_client({"type": "REMOVE_MEMBER_FAIL", "payload":
{"group_id": group_id,
"message": f"User '{username_to_remove}' not member."}})
            if self.username == username_to_remove:
                conn.rollback()
                return self.send_message_to_client(
                    {"type": "REMOVE_MEMBER_FAIL",
"payload": {"group_id": group_id, "message": "Admins use 'Leave Group'."}})

```

```

        group_creator = self.server.db_manager.get_group_creator_from_db(cursor, group_id)
        if username_to_remove == group_creator:
            conn.rollback()
            return self.send_message_to_client(
                {"type": "REMOVE_MEMBER_FAIL",
                 "payload": {"group_id": group_id, "message": "Creator cannot be removed.}})
        role_of_removed = self.server.db_manager.get_group_member_role(cursor,
username_to_remove, group_id)
        if role_of_removed == 'admin' and
self.server.db_manager.count_group_admins_in_db(cursor, group_id) <= 1:
            conn.rollback()
            return self.send_message_to_client({"type": "REMOVE_MEMBER_FAIL", "payload":
{"group_id": group_id,
"message": "Cannot remove last admin.}}))
            ts = ""
            sys_msg = ""
            if self.server.db_manager.remove_member_from_group_in_db(cursor, group_id,
username_to_remove):
                sys_msg = f"'{self.username}' removed '{username_to_remove}' from the group."
                ts = datetime.now().strftime('%Y-%m-%d %H:%M:%S.%f')[:-3]
                self.server.db_manager.store_message_in_db(cursor, "SYSTEM_EVENT", sys_msg,
recipient_group_id=group_id,
                                                                    timestamp=ts, is_system=True)
                conn.commit()
                self.send_message_to_client({"type": "REMOVE_MEMBER_SUCCESS",
                                             "payload": {"group_id": group_id,
"removed_username": username_to_remove,
                                                                    "message": f"Removed
'{username_to_remove}'."}})
                broadcast_payload = {"group_id": group_id, "sender_username": "SYSTEM_EVENT",
"text": sys_msg,
                                                                    "timestamp": ts, "is_system_event": True,
                                                                    "removed_username_info": username_to_remove}
                self.server.broadcast_message_to_group(group_id,
{"type": "RECEIVE_GROUP_MESSAGE",
"payload": broadcast_payload},
                                                                    cursor)
            else:
                conn.rollback(); self.send_message_to_client({"type": "REMOVE_MEMBER_FAIL",
"payload": {"group_id":
group_id,
                                                                    "message": f"Failed
to remove '{username_to_remove}'."}})
            except sqlite3.Error as e:
                conn.rollback()
                print(f"[REM_MEM_DB_ERR] User {self.username}, GID {group_id}, Target
{username_to_remove}: {e}")
                self.send_message_to_client(
                    {"type": "REMOVE_MEMBER_FAIL", "payload": {"group_id": group_id, "message":
f"DB error: {e}"}})
            except Exception as e_unhandled:
                conn.rollback()
                print(
                    f"[REM_MEM_UNEX_ERR] User {self.username}, GID {group_id}: {e_unhandled}")
                self.send_message_to_client(
                    {"type": "REMOVE_MEMBER_FAIL",
                     "payload": {"group_id": group_id, "message": "Unexpected server error.}})
            finally:
                conn.close()

    def _handle_change_member_role(self, payload):
        if not self.username: return self.send_message_to_client({"type": "AUTH_REQUIRED"})
        group_id = payload.get("group_id")
        target_username = payload.get("target_username")
        new_role = payload.get("new_role")
        if group_id is None or not target_username or new_role not in ['admin',
                                                                    'member']: return
        self.send_message_to_client(
            {"type": "CHANGE_ROLE_FAIL", "payload": {"group_id": group_id, "message": "Invalid
payload.}})
        conn = self._get_db_connection()
        if not conn: return
        cursor = conn.cursor()
        try:
            if not self.server.db_manager.is_user_admin_in_group(cursor, self.username,
group_id):

```

```

        conn.rollback()
        return self.send_message_to_client(
            {"type": "CHANGE_ROLE_FAIL", "payload": {"group_id": group_id, "message":
"Permission denied."}})
        current_target_role = self.server.db_manager.get_group_member_role(cursor,
target_username, group_id)
        if not current_target_role:
            conn.rollback()
            return self.send_message_to_client({"type": "CHANGE_ROLE_FAIL", "payload":
{"group_id": group_id, "message": f"User '{target_username}' not member."}))
        if self.username == target_username:
            conn.rollback()
            return self.send_message_to_client(
                {"type": "CHANGE_ROLE_FAIL", "payload": {"group_id": group_id, "message":
"Cannot change own role."}})
        group_creator = self.server.db_manager.get_group_creator_from_db(cursor, group_id)
        if target_username == group_creator and new_role == 'member':
            conn.rollback()
            return self.send_message_to_client(
                {"type": "CHANGE_ROLE_FAIL",
                "payload": {"group_id": group_id, "message": "Creator role cannot be
revoked."}})
        if current_target_role == 'admin' and new_role == 'member' and
self.server.db_manager.count_group_admins_in_db(
            cursor, group_id) <= 1:
            conn.rollback()
            return self.send_message_to_client({"type": "CHANGE_ROLE_FAIL", "payload":
{"group_id": group_id,
"message": "Cannot demote last admin."}})
        ts = ""
        sys_msg = ""
        if self.server.db_manager.update_group_member_role_in_db(cursor, group_id,
target_username, new_role):
            sys_msg = f"'{self.username}' changed '{target_username}'s role to
{new_role}.";
            ts = datetime.now().strftime('%Y-%m-%d %H:%M:%S.%f')[:-3]
            self.server.db_manager.store_message_in_db(cursor, "SYSTEM_EVENT", sys_msg,
recipient_group_id=group_id,
                                                    timestamp=ts, is_system=True)
            conn.commit()
            self.send_message_to_client({"type": "MEMBER_ROLE_CHANGED",
"payload": {"group_id": group_id, "username":
target_username,
"new_role": new_role,
"message": f"Role of
'{target_username}' changed to {new_role}."}})
            broadcast_payload = {"group_id": group_id, "sender_username": "SYSTEM_EVENT",
"text": sys_msg,
"timestamp": ts, "is_system_event": True,
"affected_user": target_username,
"new_role": new_role}
            self.server.broadcast_message_to_group(group_id,
                                                    {"type": "RECEIVE_GROUP_MESSAGE",
"payload": broadcast_payload},
                                                    cursor)
        else:
            conn.rollback(); self.send_message_to_client({"type": "CHANGE_ROLE_FAIL",
"payload": {"group_id":
group_id,
"message": f"Failed
to update role for '{target_username}'."}})
            except sqlite3.Error as e:
                conn.rollback(); print(
                    f"[CHG_ROLE_DB_ERR] User {self.username}, GID {group_id}, Target
{target_username}: {e}"); self.send_message_to_client(
                    {"type": "CHANGE_ROLE_FAIL", "payload": {"group_id": group_id, "message": f"DB
error: {e}"}})
            except Exception as e_unhandled:
                conn.rollback(); print(
                    f"[CHG_ROLE_UNEX_ERR] User {self.username}, GID {group_id}: {e_unhandled}");
                self.send_message_to_client(
                    {"type": "CHANGE_ROLE_FAIL", "payload": {"group_id": group_id, "message":
"Unexpected server error."}})
            finally:
                conn.close()

```

```

def _handle_rename_group(self, payload):
    if not self.username: return self.send_message_to_client({"type": "AUTH_REQUIRED"})
    group_id = payload.get("group_id");
    new_group_name = payload.get("new_group_name", "").strip()
    if group_id is None or not new_group_name: return self.send_message_to_client(
        {"type": "RENAME_GROUP_FAIL", "payload": {"group_id": group_id, "message":
"Invalid payload."}})
    conn = self._get_db_connection();
    if not conn: return
    cursor = conn.cursor()
    try:
        if not self.server.db_manager.is_user_admin_in_group(cursor, self.username,
group_id):
            conn.rollback();
            return self.send_message_to_client(
                {"type": "RENAME_GROUP_FAIL", "payload": {"group_id": group_id, "message":
"Permission denied."}})
        group_details = self.server.db_manager.get_group_details_from_db(cursor, group_id)
        if not group_details: conn.rollback(); return self.send_message_to_client(
            {"type": "RENAME_GROUP_FAIL", "payload": {"group_id": group_id, "message":
"Group not found."}})
        current_group_name = group_details["group_name"]
        if current_group_name == new_group_name: conn.rollback(); return
    self.send_message_to_client(
        {"type": "RENAME_GROUP_FAIL",
         "payload": {"group_id": group_id, "new_group_name": new_group_name,
"message": "Name is same."}})
        ts = "";
        sys_msg = ""
        if self.server.db_manager.rename_group_in_db(cursor, group_id, new_group_name):
            sys_msg = f"'{self.username}' renamed group from '{current_group_name}' to
'(new group name)'.";
            ts = datetime.now().strftime('%Y-%m-%d %H:%M:%S.%f')[:-3]
            self.server.db_manager.store_message_in_db(cursor, "SYSTEM_EVENT", sys_msg,
recipient_group_id=group_id,
                                                    timestamp=ts, is_system=True)
            conn.commit()
            self.send_message_to_client({"type": "GROUP_RENAMED_SUCCESS",
                                         "payload": {"group_id": group_id,
"new_group_name": new_group_name,
                                         "message": f"Group renamed to
'(new_group_name)'."}})
            broadcast_payload = {"group_id": group_id, "sender_username": "SYSTEM_EVENT",
"text": sys_msg,
                                "timestamp": ts, "is_system_event": True,
"old_group_name": current_group_name,
                                "new_group_name": new_group_name}
            self.server.broadcast_message_to_group(group_id,
                                                    {"type": "RECEIVE_GROUP_MESSAGE",
"payload": broadcast_payload},
                                                    cursor)
        else:
            conn.rollback(); self.send_message_to_client({"type": "RENAME_GROUP_FAIL",
"payload": {"group_id":
group_id,
"message": "Failed
to rename in DB."}})
            except sqlite3.Error as e:
                conn.rollback(); print(
                    f"[REN_GRP_DB_ERR] User {self.username}, GID {group_id}: {e}");
                self.send_message_to_client(
                    {"type": "RENAME_GROUP_FAIL", "payload": {"group_id": group_id, "message":
f"DB error: {e}"}})
            except Exception as e_unhandled:
                conn.rollback(); print(
                    f"[REN_GRP_UNEX_ERR] User {self.username}, GID {group_id}: {e_unhandled}");
                self.send_message_to_client(
                    {"type": "RENAME_GROUP_FAIL", "payload": {"group_id": group_id, "message":
"Unexpected server error."}})
            finally:
                conn.close()

def _handle_leave_group(self, payload):
    if not self.username: return self.send_message_to_client({"type": "AUTH_REQUIRED"})
    group_id = payload.get("group_id")
    if group_id is None: return self.send_message_to_client(
        {"type": "LEAVE_GROUP_FAIL", "payload": {"message": "Group ID missing."}})

```

```

        conn = self._get_db_connection();
        if not conn: return
        cursor = conn.cursor()
        try:
            group_details = self.server.db_manager.get_group_details_from_db(cursor, group_id)
            if not group_details: conn.rollback(); return self.send_message_to_client(
                {"type": "LEAVE_GROUP_FAIL", "payload": {"group_id": group_id, "message":
"Group not found."}})
            group_name = group_details["group_name"];
            creator = group_details["creator_username"]
            if not self.server.db_manager.is_user_member_of_group(cursor, self.username,
group_id):
                conn.rollback();
                return self.send_message_to_client(
                    {"type": "LEAVE_GROUP_FAIL", "payload": {"group_id": group_id, "message":
"Not a member."}})
            member_count = self.server.db_manager.count_group_members_in_db(cursor, group_id)
            user_role = self.server.db_manager.get_group_member_role(cursor, self.username,
group_id)
            ts = datetime.now().strftime('%Y-%m-%d %H:%M:%S.%f')[:-3];
            sys_msg = ""
            if member_count == 1:
                self.server.db_manager.remove_member_from_group_in_db(cursor, group_id,
self.username)
                self.server.db_manager.delete_group_from_db(cursor, group_id)
                conn.commit()
                return self.send_message_to_client({"type": "LEAVE_GROUP_SUCCESS",
                    "payload": {"group_id": group_id,
"group_name": group_name,
"message": f"Left
'{group_name}'. Group deleted as last member."}})
                if user_role == 'admin' and
self.server.db_manager.count_group_admins_in_db(cursor,
group_id) == 1 and member_count > 1:
                    conn.rollback();
                    return self.send_message_to_client({"type": "LEAVE_GROUP_FAIL", "payload":
{"group_id": group_id,
"message": "Cannot leave as last admin with other members. Promote another or remove
others."}})
                    if self.server.db_manager.remove_member_from_group_in_db(cursor, group_id,
self.username):
                        sys_msg = f"'{self.username}' left the group." + (f" (the creator)" if
self.username == creator else "")
                        self.server.db_manager.store_message_in_db(cursor, "SYSTEM_EVENT", sys_msg,
recipient_group_id=group_id,
timestamp=ts, is_system=True)
                        conn.commit()
                        self.send_message_to_client({"type": "LEAVE_GROUP_SUCCESS",
                            "payload": {"group_id": group_id, "group_name":
group_name,
"message": f"You left
'{group_name}'."}})
                        broadcast_payload = {"group_id": group_id, "sender_username": "SYSTEM_EVENT",
"text": sys_msg,
"timestamp": ts, "is_system_event": True, "left_user":
self.username}
                        self.server.broadcast_message_to_group(group_id,
{"type": "RECEIVE_GROUP_MESSAGE",
"payload": broadcast_payload},
cursor)
                    else:
                        conn.rollback(); self.send_message_to_client({"type": "LEAVE_GROUP_FAIL",
                            "payload": {"group_id":
group_id,
"message": "Failed
to remove from DB."}})
                        except sqlite3.Error as e:
                            conn.rollback(); print(
                                f"[LEAVE_GRP_DB_ERR] User {self.username}, GID {group_id}: {e}");
                            self.send_message_to_client(
                                {"type": "LEAVE_GROUP_FAIL", "payload": {"group_id": group_id, "message": f"DB
error: {e}"}})
                        except Exception as e_unhandled:
                            conn.rollback(); print(
                                f"[LEAVE_GRP_UNEX_ERR] User {self.username}, GID {group_id}: {e_unhandled}");

```

```

self.send_message_to_client(
    {"type": "LEAVE_GROUP_FAIL", "payload": {"group_id": group_id, "message":
"Unexpected server error."}})
    finally:
        conn.close()

def _handle_upload_profile_picture(self, payload):
    image_b64 = payload.get("image_data base64");
    original_filename = payload.get("original_filename", "upload.jpg")
    if not self.username: return self.send_message_to_client({"type": "AUTH_REQUIRED"})
    if not image_b64: return self.send_message_to_client(
        {"type": "PROFILE_PICTURE_UPLOAD_STATUS", "payload": {"success": False, "message":
"No image data."}})
    try:
        image_data = base64.b64decode(image_b64)
        _, ext = os.path.splitext(original_filename);
        ext = ext.lower() if ext.lower() in ['.jpg', '.jpeg', '.png'] else ".jpg"
        new_filename = f"user {self.username} pfp{ext}"
        filepath = os.path.join(PROFILE_PICS_DIR, new_filename)
        with open(filepath, "wb") as f:
            f.write(image_data)
        conn = self._get_db_connection();
        if not conn: return
        cursor = conn.cursor()
        try:
            if self.server.db_manager.update_user_profile_picture_in_db(cursor,
self.username, new_filename):
                conn.commit();
                self.send_message_to_client({"type": "PROFILE_PICTURE_UPLOAD_STATUS",
                    "payload": {"success": True, "message": "PFP
updated."}})
            else:
                conn.rollback(); self.send_message_to_client({"type":
"PROFILE_PICTURE_UPLOAD_STATUS",
                    "payload": {"success":
False,
                    "message":
"Failed DB update."}})
        except sqlite3.Error as e:
            conn.rollback(); print(f"[UP_PFP_DB_ERR] User {self.username}: {e}");
self.send_message_to_client(
            {"type": "PROFILE_PICTURE_UPLOAD_STATUS", "payload": {"success": False,
"message": "DB error."}})
        finally:
            conn.close()
        except (base64.binascii.Error, ValueError) as e_decode:
            print(f"[UP_PFP_DEC_ERR] User {self.username}: {e_decode}");
self.send_message_to_client(
            {"type": "PROFILE_PICTURE_UPLOAD_STATUS",
                "payload": {"success": False, "message": f"Invalid image data: {e_decode}"}})
        except IOError as e_io:
            print(f"[UP_PFP_IO_ERR] User {self.username}: {e_io}");
self.send_message_to_client(
            {"type": "PROFILE_PICTURE_UPLOAD_STATUS",
                "payload": {"success": False, "message": "Server error saving image."}})
        except Exception as e:
            print(f"[UP_PFP_UNEX_ERR] User {self.username}: {e}");
self.send_message_to_client(
            {"type": "PROFILE_PICTURE_UPLOAD_STATUS",
                "payload": {"success": False, "message": "Unexpected error."}})

def _handle_get_profile_picture(self, payload):
    target_username = payload.get("username")
    if not target_username: return self.send_message_to_client(
        {"type": "PROFILE_PICTURE_DATA", "payload": {"username": None, "message":
"Username missing."}})
    conn = self._get_db_connection();
    if not conn: return
    cursor = conn.cursor();
    image_b64 = None;
    filename = None
    try:
        filename = self.server.db_manager.get_profile_picture_filename_from_db(cursor,
target_username)
        if filename:
            filepath = os.path.join(PROFILE_PICS_DIR, filename)
            if os.path.exists(filepath):

```

```

        with open(filepath, "rb") as f:
            image_b64 = base64.b64encode(f.read()).decode('utf-8')
        else:
            print(f"[GET_PFP_NOT_FOUND] File {filename} for {target_username}
missing."); filename = None
            self.send_message_to_client({"type": "PROFILE_PICTURE_DATA",
                                         "payload": {"username": target_username, "filename":
filename,
                                                    "image_data_base64": image_b64}})

        except sqlite3.Error as e:
            print(f"[GET_PFP_DB_ERR] User {target_username}: {e}");
            self.send_message_to_client(
                {"type": "PROFILE_PICTURE_DATA", "payload": {"username": target_username,
"message": "DB error."}})
        except IOError as e_io:
            print(f"[GET_PFP_IO_ERR] User {target_username}, File {filename}: {e_io}");
            self.send_message_to_client(
                {"type": "PROFILE_PICTURE_DATA",
                 "payload": {"username": target_username, "filename": filename, "message":
"Error reading image."}})
        finally:
            conn.close()

    def _handle_upload_group_picture(self, payload):
        if not self.username: # Check if the user is logged in
            return self.send_message_to_client(
                {"type": "AUTH_REQUIRED", "payload": {"message": "Authentication required."}})

        group_id = payload.get("group_id")
        image_b64 = payload.get("image_data_base64")
        original_filename = payload.get("original_filename", "group_icon.jpg") # Default
        filename if not provided

        if group_id is None or not image_b64:
            return self.send_message_to_client({
                "type": "GROUP_PICTURE_UPLOAD_STATUS",
                "payload": {"success": False, "group_id": group_id, "message": "Group ID or
image data missing."}
            })

        conn = self._get_db_connection()
        if not conn:
            # _get_db_connection already sends a SYSTEM_ERROR if it fails
            return

        cursor = conn.cursor()
        icon_filepath = None # Initialize to None
        new_icon_filename = None # Initialize to None

        try:
            # 1. Check if current user is an admin of the group
            if not self.server.db_manager.is_user_admin_in_group(cursor, self.username,
group_id):
                conn.rollback() # No database changes made yet, but good practice for clarity
                return self.send_message_to_client({
                    "type": "GROUP_PICTURE_UPLOAD_STATUS",
                    "payload": {"success": False, "group_id": group_id,
                                "message": "Permission denied: You are not an admin of this
group."}
                })

            # 2. Get group details (e.g., for group name in system message)
            group_details = self.server.db_manager.get_group_details_from_db(cursor, group_id)
            if not group_details:
                conn.rollback()
                return self.send_message_to_client({
                    "type": "GROUP_PICTURE_UPLOAD_STATUS",
                    "payload": {"success": False, "group_id": group_id, "message": "Group not
found."}
                })
            group_name = group_details["group_name"]

            # 3. Decode Base64 image data
            image_data = base64.b64decode(image_b64)

            # Optional: Add image validation (size, type) here if desired
            # Example:

```



```

# MAX_ICON_SIZE_BYTES = 1 * 1024 * 1024 # 1MB
# if len(image_data) > MAX_ICON_SIZE_BYTES:
#     raise ValueError("Group icon image is too large (max 1MB).")

_, ext = os.path.splitext(original_filename)
valid_extensions = ['.jpg', '.jpeg', '.png']
if not ext.lower() in valid_extensions:
    ext = ".jpg" # Default to .jpg if extension is missing or not allowed

new_icon_filename = f"group_{group_id}_icon{ext}"
# Ensure GROUP_ICONS_DIR is a globally defined constant or accessible via
self.server.config
icon_filepath = os.path.join(GROUP_ICONS_DIR, new_icon_filename)

# 4. Save the image file
with open(icon_filepath, "wb") as f:
    f.write(image_data)

# 5. Update database with new icon filename
if self.server.db_manager.update_group_icon_in_db(cursor, group_id,
new_icon_filename):
    system_message_text = f"'{self.username}' changed the icon for group
'{group_name}'."
    current_ts = datetime.now().strftime('%Y-%m-%d %H:%M:%S.%f')[:-3]
    self.server.db_manager.store_message_in_db(
        cursor, "SYSTEM EVENT", system_message_text,
        recipient_group_id=group_id, timestamp=current_ts, is_system=True
    )
    conn.commit()

    self.send_message_to_client({
        "type": "GROUP_PICTURE_UPLOAD_STATUS",
        "payload": {"success": True, "group_id": group_id, "new_filename":
new_icon_filename,
                    "message": "Group icon updated successfully."}
    })

# 6. Broadcast system message to group members
broadcast_payload = {
    "group_id": group_id, "sender_username": "SYSTEM EVENT",
    "text": system_message_text, "timestamp": current_ts,
    "is_system_event": True, "new_icon_filename": new_icon_filename
}
message_to_broadcast = {"type": "RECEIVE_GROUP_MESSAGE", "payload":
broadcast_payload}
# The cursor is still valid for read operations after commit on the same
connection
self.server.broadcast_message_to_group(group_id, message_to_broadcast, cursor)
else:
    # This else means update_group_icon_in_db returned False (e.g., group_id not
    found during update)
    conn.rollback()
    if icon_filepath and os.path.exists(icon_filepath): # Cleanup saved file if
    DB update failed
        try:
            os.remove(icon_filepath)
            print(f"[CLEANUP_FILE] Removed partially uploaded group icon:
{icon_filepath}")
        except OSError as e_remove:
            print(f"[CLEANUP_FILE_ERROR] Could not remove group icon
{icon_filepath}: {e_remove}")

    self.send_message_to_client({
        "type": "GROUP_PICTURE_UPLOAD_STATUS",
        "payload": {"success": False, "group_id": group_id,
                    "message": "Failed to update group icon in database (group
might not exist or no change made)."}
    })

except (base64.binascii.Error,
        ValueError) as e_decode: # Catches b64 errors and custom ValueErrors (like
size validation)
    # No DB transaction started or it's clean, so rollback is for safety but might not
    be strictly needed.
    if conn: conn.rollback()
    print(f"[UPLOAD_GROUP_ICON_VALIDATION_ERROR] User {self.username}, GID {group id}:
{e_decode}")

```

```

        self.send_message_to_client({"type": "GROUP_PICTURE_UPLOAD_STATUS",
                                     "payload": {"success": False, "group_id": group_id,
                                                  "message": f"Invalid image data or
format: {e_decode}}}})
    except IOError as e_io: # Error writing file
        if conn: conn.rollback()
        print(f"[UPLOAD_GROUP_ICON_IO_ERROR] User {self.username}, GID {group_id}:
{e_io}")
        self.send_message_to_client({"type": "GROUP_PICTURE_UPLOAD_STATUS",
                                     "payload": {"success": False, "group_id": group_id,
                                                  "message": "Server error saving group
icon."}))
    except sqlite3.Error as e_db: # Catch specific database errors during the process
        if conn: conn.rollback()
        print(f"[UPLOAD_GROUP_ICON_DB_ERROR] User {self.username}, GID {group_id}:
{e_db}")
        self.send_message_to_client({"type": "GROUP_PICTURE_UPLOAD_STATUS",
                                     "payload": {"success": False, "group_id": group_id,
                                                  "message": f"Database operation failed:
{e_db}}}})
    except Exception as e_unhandled: # Catch any other unexpected errors
        if conn: conn.rollback()
        print(f"[UPLOAD_GROUP_ICON_UNEXPECTED_ERROR] User {self.username}, GID {group_id}:
{e_unhandled}")
        self.send_message_to_client({"type": "GROUP_PICTURE_UPLOAD_STATUS",
                                     "payload": {"success": False, "group id": group id,
                                                  "message": "An unexpected server error
occurred."}))
    finally:
        if conn: conn.close()

    def _handle_get_group_icon(self, payload):
        group_id = payload.get("group_id")
        if group_id is None: return self.send_message_to_client(
            {"type": "GROUP_ICON_DATA", "payload": {"group_id": None, "message": "Group ID
missing."}))
        conn = self._get_db_connection()
        if not conn: return
        cursor = conn.cursor()
        image_b64 = None
        actual_filename = None
        try:
            actual_filename = self.server.db_manager.get_group_icon_filename_from_db(cursor,
group_id)
            if actual_filename:
                icon_filepath = os.path.join(GROUP_ICONS_DIR, actual_filename) # Ensure
GROUP_ICONS_DIR
                is defined
                if os.path.exists(icon_filepath):
                    with open(icon_filepath, "rb") as f:
                        image_b64 = base64.b64encode(f.read()).decode('utf-8')
                else:
                    print(
                        f"[GET_GRP_ICON_NOT_FOUND] File {actual_filename} for GID {group_id}
missing.")
                    actual_filename = None
            self.send_message_to_client({"type": "GROUP_ICON_DATA", "payload": {"group_id":
group_id,
                                     "filename":
actual_filename if image_b64 else None,
"image_data_base64": image_b64}})
        except sqlite3.Error as e_db:
            print(f"[GET_GRP_ICON_DB_ERR] GID {group_id}: {e_db}")
            self.send_message_to_client(
                {"type": "GROUP_ICON_DATA",
                 "payload": {"group_id": group_id, "filename": None, "image_data_base64":
None,
                             "message": "DB error."}})
        except IOError as e_io:
            print(f"[GET_GRP_ICON_IO_ERR] GID {group_id}, File {actual_filename}: {e_io}")
            self.send_message_to_client(
                {"type": "GROUP_ICON_DATA",
                 "payload": {"group_id": group_id, "filename": actual_filename,
"image_data_base64": None,
                             "message": "Error reading icon."}})
        except Exception as e_unhandled:

```

```

        print(f"[GET_GRP_ICON_UNEX_ERR] GID {group_id}: {e_unhandled}")
        self.send_message_to_client(
            {"type": "GROUP_ICON_DATA",
             "payload": {"group_id": group_id, "filename": None, "image_data_base64":
None,
                        "message": "Unexpected server error."}})

    finally:
        conn.close()

def _handle_distribute_group_key(self, payload):
    if not self.username: return self.send_error("Auth required for key distribution.")

    group_id = payload.get("group_id")
    target_username = payload.get("target_username")
    wrapped_key_b64 = payload.get("wrapped_group_key_b64")

    if not all([group_id is not None, target_username, wrapped_key_b64]):
        return self.send_error("Invalid DISTRIBUTE_GROUP_KEY payload.")

    # Basic check: is sender part of the group? (More robust: is sender an admin/allowed
to dist keys?)
    conn = self._get_db_connection()
    if not conn: return
    cursor = conn.cursor()
    is_member = self.server.db_manager.is_user_member_of_group(cursor, self.username,
group_id)
    target_is_member = self.server.db_manager.is_user_member_of_group(cursor,
target_username, group_id)
    conn.close()

    if not is_member or not target_is_member:
        return self.send_error(f"Sender or target not in group {group_id} for key
distribution.")

    target_handler = self.server.get_client_handler(target_username)
    if target_handler:
        key_message_to_target = {
            "type": "RECEIVE_WRAPPED_GROUP_KEY", # New client-side message type
            "payload": {
                "group_id": group_id,
                "sender_username": self.username, # User who initiated the distribution
                "wrapped_group_key_b64": wrapped_key_b64,
                "timestamp": datetime.now().strftime('%Y-%m-%d %H:%M:%S.%f')[:-3]
            }
        }
        target_handler.send_message_to_client(key_message_to_target)
        print(f"[SERVER] Relayed wrapped group key for GID {group_id} from {self.username}
to {target_username}")
    else:
        # What to do if target is offline? This wrapped key is crucial.
        # Option 1: Store it like a message (complex, needs new DB field/table or special
message type)
        # Option 2: Sender gets an error "target offline, key not delivered".
        # Option 3: Key distribution is part of adding member flow, new member gets it on
next login.
        # For now, let's assume live delivery. Offline key delivery is harder.
        print(
            f"[SERVER_WARN] Target {target_username} for group key (GID {group_id}) is
offline. Key not delivered live.")
        self.send_message_to_client({"type": "INFO", "payload": {
            "message": f"User {target_username} is offline. Group key may not have been
delivered live."}})

def _handle_e2ee_group_message(self, payload):
    if not self.username: return self.send_error("Auth required for E2EE group message.")

    group_id = payload.get("group_id")
    aes_payload = payload.get("aes_payload") # This is {"iv_b64": ..., "ciphertext_b64":
...}

    if group_id is None or not aes_payload or \
        not isinstance(aes_payload, dict) or \
        "iv_b64" not in aes_payload or "ciphertext_b64" not in aes_payload:
        return self.send_error("Invalid E2EE_GROUP_MESSAGE payload.")

    conn = self._get_db_connection()
    if not conn: return

```

```

        cursor = conn.cursor()

        try:
            if not self.server.db_manager.is_user_member_of_group(cursor, self.username,
group_id):
                conn.close()
                return self.send_error(f"You are not a member of group {group_id}.")

            message_content_to_store = json.dumps(aes_payload) # Store the encrypted payload
            current_ts = datetime.now().strftime('%Y-%m-%d %H:%M:%S.%f')[:-3]

            message_id_stored = self.server.db_manager.store_message_in_db(
                cursor,
                sender=self.username,
                message_text=message_content_to_store, # Storing the JSON of IV + Ciphertext
                recipient_group_id=group_id,
                timestamp=current_ts,
                is_system=False,
                e2ee_type='GROUP_AES' # New E2EE type for DB
            )
            conn.commit()
            print(f"[SERVER] Stored E2EE group message ID {message_id_stored} from
{self.username} for GID {group_id}")

            # Broadcast this E2EE payload to other group members
            # The payload for RECEIVE E2EE GROUP MESSAGE should be what clients need to
decrypt
            broadcast_payload_for_members = {
                "group_id": group_id,
                "sender_username": self.username,
                "aes_payload": aes_payload, # Forward the same encrypted payload
                "timestamp": current_ts,
                "e2ee_type": "GROUP_AES" # Let recipient know how to handle it
            }
            # New message type for client to receive encrypted group messages
            message_to_broadcast = {"type": "RECEIVE_E2EE_GROUP_MESSAGE", "payload":
broadcast_payload_for_members}

            # Server's broadcast_message_to_group just relays; ensure it sends to *other*
members if needed,
            # or all members (sender will ignore or already processed). Let's send to all for
now.
            self.server.broadcast_message_to_group(group_id, message_to_broadcast, cursor,
exclude_username=None) # or
exclude_username=self.username

        except sqlite3.Error as e_db:
            if conn: conn.rollback()
            print(f"[E2EE_GROUP_MSG_DB_ERROR] User {self.username}, GID {group_id}: {e_db}")
            self.send_error("Failed to store E2EE group message due to DB error.")
        except Exception as e_unhandled:
            print(f"[E2EE_GROUP_MSG_UNEXPECTED_ERROR] User {self.username}, GID {group_id}:
{e_unhandled}")
            self.send_error("Unexpected error sending E2EE group message.")
        finally:
            if conn: conn.close()

# =====
# Main Execution Guard
# =====
if __name__ == "__main__":
    # Ensure constants like PROFILE_PICS_DIR and GROUP_ICONS_DIR are defined globally
    # The ChatServer's __init__ passes DB_NAME to DatabaseManager.
    # DatabaseManager.initialize_database() uses global PROFILE_PICS_DIR and GROUP_ICONS_DIR.

    print("Initializing and starting Chat Server...")
    # Pass the ClientHandler class itself to the ChatServer constructor
    chat_server_instance = ChatServer(host=HOST, port=PORT, db_name=DB_NAME,
client_handler_class=ClientHandler)
    chat_server_instance.start()

```

בקובץ: "client.py"

```
# client.py (Complete Version with RSA E2EE for Private Messages)
import socket
import threading
import json
import customtkinter as ctk
from tkinter import messagebox, filedialog
from queue import Queue
from collections import defaultdict
from datetime import datetime, timedelta
import base64
import os
from PIL import Image, ImageTk
import io
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives import padding as sym_padding # Renamed to avoid clash
with rsa padding
from cryptography.hazmat.backends import default_backend

# --- Import NetworkNode ---
try:
    from shared modules import NetworkNode
except ImportError:
    print("CRITICAL: NetworkNode class could not be imported from shared_modules.py.")
    print("Ensure shared_modules.py is in the same directory or accessible in PYTHONPATH.")

    # Define a dummy NetworkNode if not found, so the script can be parsed,
    # but RSA features will fail.
    class NetworkNode:
        def __init__(self): self.public_key = None; self.private_key = None

        def get public key pem(self): return None

        def encrypt_message(self, *args): messagebox.showerror("Error", "NetworkNode not
loaded!"); return None

        def decrypt_message(self, *args): messagebox.showerror("Error", "NetworkNode not
loaded!"); return None

# --- Network Configuration ---
HOST = '127.0.0.1'
PORT = 65432

# --- Global Style and UI Constants ---
SENT_BUBBLE_COLOR = ("DCF8C6", "#056162")
RECEIVED_BUBBLE_COLOR = ("FFFFFF", "#2C2C2E")
GROUP_SENDER_TEXT_COLOR = ("404040", "#C0C0C0")
SYSTEM_EVENT_TEXT_COLOR = ("606060", "#909090")
SYSTEM_EVENT_BG_COLOR = ("F0F0F0", "#2B2B2B")
DATE_SEPARATOR_TEXT_COLOR = ("606060", "#909090")
MESSAGE_TEXT_FONT_FAMILY = "Arial"
MESSAGE_TEXT_FONT_SIZE = 14
TIMESTAMP_FONT_FAMILY = "Arial"
TIMESTAMP_FONT_SIZE = 10
TIMESTAMP_TEXT_COLOR = ("707070", "#A0A0A0")
BUBBLE_CORNER_RADIUS = 12
BUBBLE_PADDING_X = 10
BUBBLE_PADDING_Y = 6
MAX_BUBBLE_WIDTH_FACTOR = 0.75
DEFAULT_MESSAGES_TO_LOAD_INITIALLY = 30
MESSAGES_TO_LOAD_ON_SCROLL = 30
PROFILE_PIC_HEADER_SIZE = (40, 40)
PROFILE_PIC_LIST_SIZE = (28, 28)
GROUP_ICON_SETTINGS_SIZE = (60, 60)
MAX_UPLOAD_IMAGE_DIM = (300, 300) # Max dimensions for client-side image resizing before
upload

gui_running = True # Flag to control threads and periodic checks
ctk.set_appearance_mode("dark")
ctk.set_default_color_theme("blue")

class ChatClientApp(ctk.CTk, NetworkNode): # Inherit from CTk and NetworkNode
    def __init__(self, *args, **kwargs):
        ctk.CTk.__init__(self, *args, **kwargs)
        NetworkNode.__init__(self)
```

```

        if not self.public_key:
            messagebox.showerror("Fatal Error", "RSA Key generation failed. Secure features
unavailable.")

        self.title("SecureChat Client")
        self.geometry("800x600")

        self.client_socket = None
        self.receive_thread = None
        self.username = None
        self.message_queue = Queue()
        self.username_being_registered = None
        self.receive_buffer = b""

        self.server_public_key_pem = None
        self.user_public_keys_cache = {}
        self.current_chat_key_status = "unknown"
        self.unsent_e2ee_messages = defaultdict(list)
        self.current_chat_identifier = None
        self.current_chat_is_group = False
        self.current_chat_group_name = None
        self.chat_histories = defaultdict(list)
        self.all_users_list = []
        self.my_groups = []
        self.messages_currently_shown_count = defaultdict(lambda:
DEFAULT MESSAGES TO LOAD INITIALLY)
        self.is_loading_more_messages = False
        self.chat_all_older_loaded = defaultdict(bool)
        self.chat_session_keys = {} # For 1-to-1 E2EE AES keys {peer_username: {"key":
aes_bytes}}

        # For Group E2EE AES Keys
        self.group_aes_keys = {} # In-memory: {group_id: aes_key_bytes}
        self.GROUP_KEYS_FILE = "group_aes_keys.json" # Or make it user-specific after login

        self.profile_picture_cache = {}
        self.group_icon_cache = {}
        self.default_profile_image = None
        self.default_list_profile_image = None
        self.default_group_icon = None
        self._create_default_profile_image()

        self.auth_frame = ctk.CTkFrame(self, corner_radius=0, fg_color="transparent")
        self.auth_frame.pack(pady=20, padx=30, fill="both", expand=True)
        self.main_chat_frame = ctk.CTkFrame(self, corner_radius=0, fg_color="transparent")
        self.new_chat_frame = ctk.CTkFrame(self, corner_radius=0, fg_color="transparent")

        # Widget placeholders
        # ... (your existing widget placeholder initializations to None) ...
        self.login_user_entry = self.login_pass_entry = None
        self.create_group_name_entry = self.create_group_members_scroll_frame = None
        self.create_group_member_vars = {}
        self.add_members_scroll_frame = None
        self.add_members_selection_vars = {}
        self.group_settings_member_list_frame = self.group_settings_icon_display_label = None
        self.last_group_details_data = None
        self.chat_header_image_label = self.current_chat_label =
self.chat_header_actions_frame = None
        self.message_display_frame = self.msg_entry = self.send_button = None
        self.scrollable_chat_list = None
        self.new_chat_search_entry = self.all_users_scrollable_list = None

        self._setup_message_handlers() # Call the new method here

        self.connect_to_server()
        self.show_login_view()

        self.protocol("WM_DELETE_WINDOW", self.on_closing)
        self.after(100, self.check_queue)

    def _setup_message_handlers(self):
        self.message_handlers = {
            "SERVER_PUBLIC_KEY": self._handle_server_public_key,
            "USER_PUBLIC_KEY_INFO": self._handle_user_public_key_info,
            "USER_PUBLIC_KEY_FAIL": self._handle_user_public_key_fail,
            "REGISTER_SUCCESS": self._handle_register_success,
            "REGISTER_FAIL": self._handle_register_fail,

```

```

        "LOGIN_SUCCESS": self._handle_login_success,
        "LOGIN_FAIL": self._handle_login_fail,

        "MESSAGE": self._handle_message, # Handles historical messages, system messages
via history

        "RECEIVE_SECURE_SESSION_INIT": self._handle_receive_secure_session_init, # Live
1-to-1 E2EE
        "RECEIVE_AES_MESSAGE": self._handle_receive_aes_message, # Live 1-to-1 E2EE

        "RECEIVE_WRAPPED_GROUP_KEY": self._handle_receive_wrapped_group_key, # Live Group
Key Delivery
        "RECEIVE_E2EE_GROUP_MESSAGE": self._handle_receive_e2ee_group_message, # Live
E2EE Group Message
        "RECEIVE_GROUP_MESSAGE": self._handle_receive_plain_group_message, # Live
Plaintext Group Message

        "MESSAGE_STORED_FOR_OFFLINE_DELIVERY":
self._handle_message_stored_for_offline_delivery,

        "ALL_USERS_LIST": self._handle_all_users_list,
        "MY_GROUPS_LIST": self._handle_my_groups_list,
        "GROUP_DETAILS_DATA": self._handle_group_details_data,

        "GROUP_CREATE_SUCCESS": self._handle_group_create_success,
        "GROUP_CREATE_FAIL": self._handle_group_operation_fail, # Generic handler
        "ADD_MEMBERS_SUCCESS": self._handle_add_members_success,
        "ADD_MEMBERS_FAIL": self._handle_group_operation_fail, # Generic handler
        "REMOVE_MEMBER_SUCCESS": self._handle_remove_member_success,
        "REMOVE_MEMBER_FAIL": self._handle_group_operation_fail, # Generic handler
        "MEMBER_ROLE_CHANGED": self._handle_member_role_changed,
        "CHANGE_ROLE_FAIL": self._handle_group_operation_fail, # Generic handler
        "GROUP_RENAMED_SUCCESS": self._handle_group_renamed_success,
        "RENAME_GROUP_FAIL": self._handle_group_operation_fail, # Generic handler
        "LEAVE_GROUP_SUCCESS": self._handle_leave_group_success,
        "LEAVE_GROUP_FAIL": self._handle_group_operation_fail, # Generic handler

        "PROFILE_PICTURE_UPLOAD_STATUS": self._handle_profile_picture_upload_status,
        "PROFILE_PICTURE_DATA": self._handle_profile_picture_data,
        "GROUP_PICTURE_UPLOAD_STATUS": self._handle_group_picture_upload_status,
        "GROUP_ICON_DATA": self._handle_group_icon_data,

        # Generic server messages
        "SYSTEM": self._handle_generic_server_message,
        "SYSTEM_ERROR": self._handle_generic_server_message,
        "ERROR": self._handle_generic_server_message,
        "SERVER_ERROR": self._handle_generic_server_message,
        "AUTH_REQUIRED": self._handle_generic_server_message,
    }

# --- Core Helper Methods ---
def generate_aes_key(self):
    """Generates a random 256-bit (32 bytes) AES key."""
    return os.urandom(32)

# --- NEW: AES Encryption Method ---
def encrypt_aes(self, plaintext_bytes, key):
    """Encrypts plaintext_bytes using AES-256-CBC.
    Returns a dictionary containing 'iv_b64' and 'ciphertext_b64'."""
    if not isinstance(plaintext_bytes, bytes):
        raise TypeError("Plaintext for AES encryption must be bytes.")

    iv = os.urandom(16) # Generate a fresh IV for each encryption (for CBC mode)
    cipher = Cipher(algorithms.AES(key), modes.CBC(iv), backend=default_backend())
    encryptor = cipher.encryptor()

    # PKCS7 padding for AES block size (128 bits / 16 bytes)
    padder = sym_padding.PKCS7(algorithms.AES.block_size).padder()
    padded_data = padder.update(plaintext_bytes) + padder.finalize()

    ciphertext_bytes = encryptor.update(padded_data) + encryptor.finalize()

    return {
        "iv_b64": base64.b64encode(iv).decode('utf-8'),
        "ciphertext_b64": base64.b64encode(ciphertext_bytes).decode('utf-8')
    }

```

```
# --- NEW: AES Decryption Method ---
def decrypt_aes(self, ciphertext_b64, iv_b64, key):
    """Decrypts ciphertext_b64 using AES-256-CBC with the given key and IV.
    Returns decrypted plaintext_bytes or None on failure.
    """
    try:
        iv_bytes = base64.b64decode(iv_b64)
        ciphertext_bytes = base64.b64decode(ciphertext_b64)

        cipher = Cipher(algorithms.AES(key), modes.CBC(iv_bytes),
backend=default_backend())
        decryptor = cipher.decryptor()

        padded_plaintext = decryptor.update(ciphertext_bytes) + decryptor.finalize()

        unpadder = sym_padding.PKCS7(algorithms.AES.block_size).unpadder()
        plaintext_bytes = unpadder.update(padded_plaintext) + unpadder.finalize()
        return plaintext_bytes
    except Exception as e:
        print(f"[AES DECRYPT ERROR] {e}")
        return None

def _create_default_profile_image(self):
    # (Implementation as in your original code)
    try:
        mode_color = ("#E0E0E0", "#3A3A3A")[1 if ctk.get_appearance_mode().lower() ==
"dark" else 0]
        header_default_pil = Image.new('RGB', PROFILE_PIC_HEADER_SIZE, color=mode_color);
        self.default_profile_image = ctk.CTkImage(light_image=header_default_pil,
dark_image=header_default_pil,
size=PROFILE_PIC_HEADER_SIZE)
        list_default_pil = Image.new('RGB', PROFILE_PIC_LIST_SIZE, color=mode_color);
        self.default_list_profile_image = ctk.CTkImage(light_image=list_default_pil,
dark_image=list_default_pil,
size=PROFILE_PIC_LIST_SIZE)
        group_icon_pil = Image.new('RGB', GROUP_ICON_SETTINGS_SIZE, color=mode_color);
        self.default_group_icon = ctk.CTkImage(light_image=group_icon_pil,
dark_image=group_icon_pil,
size=GROUP_ICON_SETTINGS_SIZE)

    except Exception as e:
        print(f"Error creating default images: {e}")

def clear_auth_frame(self):
    # (Implementation as in your original code)
    for widget in self.auth_frame.winfo_children(): widget.destroy()

def toggle_password_visibility(self, entry_widget, button_widget):
    # (Implementation as in your original code)
    current_show_char = entry_widget.cget("show")
    if current_show_char == "*":
        entry_widget.configure(show=""); button_widget.configure(text="Hide")
    else:
        entry_widget.configure(show="*"); button_widget.configure(text="Show")

# In ChatClientApp class
def _update_chat_input_state(self):
    if not hasattr(self, 'msg_entry') or not self.msg_entry:
        return
    send_button_widget = getattr(self, 'send_button', None)

    # Default state for the textbox (it's usually always "normal" for typing)
    self.msg_entry.configure(state="normal")
    if send_button_widget:
        send_button_widget.configure(state="normal")

    # You can provide visual cues using a separate label if needed,
    # or by prepending/clearing text in the textbox itself,
    # but CTkTextbox does not have a 'placeholder_text' configure option.

    # For example, you might have a status label that you update:
    # status_text = ""
    # if self.current_chat_identifier and not self.current_chat_is_group: # Private chat
    #     if self.current_chat_key_status == "available":
    #         status_text = "Secure connection (Encrypted)"
    #     elif self.current_chat_key_status == "fetching":
    #         status_text = "Establishing secure connection..."
    #     elif self.current_chat_key_status == "failed":
```



```
#         status_text = "⚠ Key unavailable. Messages may be queued."
#     else: # unknown
#         status_text = "Initializing security..."
# else: # Group chat or no chat selected
#     status_text = "Group chat" if self.current_chat_is_group else "Select a chat"
#
# if hasattr(self, 'some_status_label_widget'):
#     self.some_status_label_widget.configure(text=status_text)

# The important part is to remove any attempts to set 'placeholder_text'
# on self.msg_entry.configure()

# Based on your original logic, the main goal was to enable/disable
# and set placeholder. Since CtkTextbox is generally always enabled
# for typing, we just ensure it's "normal". The send button can
# still be controlled if needed, but typically would also be normal
# if a chat is selected.

if self.current_chat_identifier:
    self.msg_entry.configure(state="normal")
    if send_button_widget:
        send_button_widget.configure(state="normal")
else: # No chat selected
    self.msg_entry.configure(state="disabled") # Or "normal" and just don't send
    if send_button_widget:
        send_button_widget.configure(state="disabled")

def show_register_view(self):
    self.title("Register - SecureChat") # Or your app name
    self.geometry("350x500") # Adjusted height slightly for the new field
    self.clear_auth_frame()
    self.main_chat_frame.pack_forget()
    self.new_chat_frame.pack_forget()
    self.auth_frame.pack(pady=20, padx=30, fill="both", expand=True)

    ctk.CTkLabel(self.auth_frame, text="Create Account", font=ctk.CTkFont(size=24,
weight="bold")).pack(
        pady=(20, 15)) # Adjusted padding

    # Username Entry
    self.register_user_entry = ctk.CTkEntry(self.auth_frame, placeholder_text="Choose
Username", width=250,
                                           height=40)

    self.register_user_entry.pack(pady=7)

    # Password Entry Frame 1
    password_frame1 = ctk.CTkFrame(self.auth_frame, fg_color="transparent")
    password_frame1.pack(pady=7)
    self.register_pass_entry = ctk.CTkEntry(password_frame1, placeholder_text="Choose
Password", show="*",
                                           width=200, height=40)

    self.register_pass_entry.pack(side="left", padx=(0, 5))
    show_pass_button1 = ctk.CTkButton(password_frame1, text="Show", width=40, height=40,
command=lambda e=self.register_pass_entry,
b=None:
self.toggle_password_visibility(e,
b if b else show_pass_button1)) # Pass button itself
    show_pass_button1.pack(side="left")
    # Associate button with command after it's defined
    # show_pass_button1.configure(command=lambda e=self.register_pass_entry,
b=show_pass_button1: self.toggle_password_visibility(e, b))

    # --- NEW: Confirm Password Entry Frame ---
    password_frame2 = ctk.CTkFrame(self.auth_frame, fg_color="transparent")
    password_frame2.pack(pady=7)
    self.register_confirm_pass_entry = ctk.CTkEntry(password_frame2,
placeholder_text="Confirm Password", show="*",
                                           width=200, height=40)

    self.register_confirm_pass_entry.pack(side="left", padx=(0, 5))
    show_pass_button2 = ctk.CTkButton(password_frame2, text="Show", width=40, height=40,
command=lambda e=self.register_confirm_pass_entry,
b=None:
self.toggle_password_visibility(e,
b if b else show_pass_button2))
    show_pass_button2.pack(side="left")
```

```

        # show_pass_button2.configure(command=lambda e=self.register_confirm_pass_entry,
b=show_pass_button2: self.toggle_password_visibility(e, b))
        # --- END NEW ---

        # Make sure toggle_password_visibility can handle button being None initially or
passed in lambda
        # Or, define button then configure its command with itself if needed.
        # For toggle_password_visibility, if button_widget is passed and not None:
        # if button_widget.cget("text") == "Show": button_widget.configure(text="Hide") else:
button_widget.configure(text="Show")

        # Update lambda for register button to get all three values
        register_action = lambda u=self.register_user_entry,
p1=self.register_pass_entry,p2=self.register_confirm_pass_entry: \
            self.send_registration(u.get(), p1.get(), p2.get())

        ctk.CTkButton(self.auth_frame, text="Register", width=250, height=40,
command=register_action).pack(
            pady=(15, 10))

        # Link to Login View
        back_to_login_frame = ctk.CTkFrame(self.auth_frame, fg_color="transparent")
        back_to_login_frame.pack(pady=10)
        ctk.CTkLabel(back_to_login_frame, text="Have an account? ").pack(side="left")
        ctk.CTkButton(back_to_login_frame, text="Login", fg_color="transparent",
            text_color=ctk.ThemeManager.theme["CTkButton"]["text_color"], # Use
theme color
            hover=False, width=40, command=self.show_login_view).pack(side="left",
padx=0)

    def show_chat_screen(self):
        self.title(f"SecureChat - {self.username}")
        self.geometry("800x600")
        self.auth_frame.pack_forget()
        self.new_chat_frame.pack_forget()
        self.main_chat_frame.pack(pady=0, padx=0, fill="both", expand=True)
        for widget in self.main_chat_frame.wininfo_children():
            widget.destroy()

        self.main_chat_frame.grid_columnconfigure(0, weight=0, minsize=250)
        self.main_chat_frame.grid_columnconfigure(1, weight=1)
        self.main_chat_frame.grid_rowconfigure(0, weight=1)

        # Chat List Frame (Left Pane)
        self.chat_list_frame = ctk.CTkFrame(self.main_chat_frame, width=250, corner_radius=0)
        self.chat_list_frame.grid(row=0, column=0, padx=0, pady=0, sticky="nsew")

        chat_list_top_frame = ctk.CTkFrame(self.chat_list_frame, corner_radius=0,
fg_color="transparent")
        chat_list_top_frame.pack(pady=10, padx=10, fill="x")
        ctk.CTkButton(chat_list_top_frame, text="New Chat",
command=self.show_new_chat_view).pack(fill="x", expand=True)
        ctk.CTkButton(chat_list_top_frame, text="My Picture",
command=self.prompt_upload_profile_picture).pack(fill="x", expand=True, pady=(5, 0))
        ctk.CTkButton(chat_list_top_frame, text="New Group",
command=self.show_create_group_view).pack(fill="x", expand=True, pady=(5, 0))

        self.scrollable_chat_list = ctk.CTkScrollableFrame(self.chat_list_frame,
corner_radius=0, fg_color="transparent")
        self.scrollable_chat_list.pack(fill="both", expand=True, padx=0, pady=0)

        # Current Chat Frame (Right Pane)
        self.current_chat_frame = ctk.CTkFrame(self.main_chat_frame, corner_radius=0)
        self.current_chat_frame.grid(row=0, column=1, padx=0, pady=0, sticky="nsew")

        self.current_chat_frame.grid_rowconfigure(0, weight=0) # Header
        self.current_chat_frame.grid_rowconfigure(1, weight=1) # Message Display
        self.current_chat_frame.grid_rowconfigure(2, weight=0) # Input Frame
        self.current_chat_frame.grid_columnconfigure(0, weight=1)

        # Chat Header
        chat_header_ui_frame = ctk.CTkFrame(self.current_chat_frame, height=60,
corner_radius=0, fg_color=ctk.ThemeManager.theme["CTkFrame"]["fg_color"])
        chat_header_ui_frame.grid(row=0, column=0, sticky="ew", padx=0, pady=0)
        chat_header_ui_frame.grid_columnconfigure(0, weight=0) # Image
        chat_header_ui_frame.grid_columnconfigure(1, weight=1) # Label
        chat_header_ui_frame.grid_columnconfigure(2, weight=0) # Actions

```

```

        if self.default_profile_image is None:
            self._create_default_profile_image() # Ensure default image exists

        self.chat_header_image_label = ctk.CTkLabel(chat_header_ui_frame, text="",
image=self.default_profile_image, width=PROFILE_PIC_HEADER_SIZE[0],
height=PROFILE_PIC_HEADER_SIZE[1])
        self.chat_header_image_label.grid(row=0, column=0, padx=(10, 5), pady=5, sticky="w")

        self.current_chat_label = ctk.CTkLabel(chat_header_ui_frame, text="Select a chat",
font=ctk.CTkFont(family=MESSAGE_TEXT_FONT_FAMILY, size=MESSAGE_TEXT_FONT_SIZE + 2,
weight="bold"))
        self.current_chat_label.grid(row=0, column=1, padx=(5, 10), pady=5, sticky="w")

        self.chat_header_actions_frame = ctk.CTkFrame(chat_header_ui_frame,
fg_color="transparent")
        self.chat_header_actions_frame.grid(row=0, column=2, padx=(5, 10), pady=5, sticky="e")

        # Message Display Area
        self.message_display_frame = ctk.CTkScrollableFrame(self.current_chat_frame,
corner radius=0, fg_color=ctk.ThemeManager.theme["CTkTextbox"]["fg_color"])
        self.message_display_frame.grid(row=1, column=0, sticky="nsew", padx=0, pady=0)
        if hasattr(self.message_display_frame, '_v_scrollbar') and
self.message_display_frame._v_scrollbar:
self.message_display_frame._v_scrollbar.configure(command=self.on_chat_scrollbar_command)

        # Input Frame
        input_frame = ctk.CTkFrame(self.current_chat_frame, corner_radius=0,
fg_color=ctk.ThemeManager.theme["CTkFrame"]["fg_color"])
        input_frame.grid(row=2, column=0, sticky="ew", padx=10, pady=10)
        input_frame.grid_columnconfigure(0, weight=1)

        self.msg_entry = ctk.CTkTextbox(input_frame,
font=ctk.CTkFont(family=MESSAGE_TEXT_FONT_FAMILY,
size=MESSAGE_TEXT_FONT_SIZE - 1),
height=50, # Initial height, can be adjusted or allow
auto-grow if CTkTextbox supports it easily
wrap="word") # Ensures words wrap automatically
        self.msg_entry.grid(row=0, column=0, sticky="nsew", padx=(0, 5)) # Use "nsew" for
textbox

        # Bind Enter for sending and Shift+Enter for newline
        self.msg_entry.bind("<Return>", self._on_msg_entry_return)
        self.msg_entry.bind("<Shift-Return>", self._on_msg_entry_shift_return)
        # --- END OF UPDATED MESSAGE INPUT ---

        self.send_button = ctk.CTkButton(input_frame, text="Send", width=60,
command=self.send_message_event)
        self.send_button.grid(row=0, column=1, sticky="e")

        self.populate_chat_list()
        if self.current_chat_identifier:
            self.select_chat(self.current_chat_identifier, self.current_chat_is_group,
group_name_for_header=self.current_chat_group_name if
self.current_chat_is_group else None)
        else:
            self.current_chat_label.configure(text="Select a chat")
            self.chat_header_image_label.configure(image=self.default_profile_image, text="")
            self._update_chat_input_state()

    def _on_msg_entry_return(self, event):
        """Handles Enter key press in the message textbox."""
        if not (event.state & 0x0001): # Check if Shift is NOT pressed (state mask for Shift
is 1)
            self.send_message_event()
            return "break"

    def _on_msg_entry_shift_return(self, event):
        """Handles Shift+Enter key press in the message textbox."""
        # Manually insert a newline character at the current cursor position
        self.msg_entry.insert(ctk.INSERT, "\n")
        return "break" # Prevents any other default behavior for Shift+Enter

    def send_registration(self, username, password, confirm_password): # Added
confirm password
        if not username or not password or not confirm_password:

```

```

        messagebox.showwarning("Input Required",
                                "Please fill in all fields: username, password, and confirm
password.")
        return

    if len(username) > 50: # Example length check
        messagebox.showwarning("Input Error", "Username is too long (max 50 characters).")
        return
    if len(password) > 50: # Example length check
        messagebox.showwarning("Input Error", "Password is too long (max 50 characters).")
        return

    if password != confirm_password:
        messagebox.showerror("Password Mismatch", "The passwords entered do not match.
Please try again.")
        # Optionally clear the password fields:
        # if hasattr(self, 'register_pass_entry'): self.register_pass_entry.delete(0,
'end')
        # if hasattr(self, 'register_confirm_pass_entry'):
self.register_confirm_pass_entry.delete(0, 'end')
        return

    # Ensure RSA keys are available (NetworkNode.__init__ should have handled this)
    if not self.public_key:
        messagebox.showerror("Key Error", "Client RSA public key not available. Cannot
register securely.")
        return

    client_pk_pem = self.get_public_key_pem()

    if not client_pk_pem:
        messagebox.showerror("Key Error", "Failed to get client public key PEM. Cannot
register.")
        return

    self.username_being_registered = username

    registration_payload = {
        "username": username,
        "password": password, # Send only the verified password
        "client_public_key": client_pk_pem
    }

    print(
        f"[CLIENT SEND_REGISTER] Sending registration for '{username}'. Public key
present: {client_pk_pem is not None}")

    if not self.send_message("REGISTER", registration_payload):
        self.username_being_registered = None
    def connect_to_server(self):
        try:
            self.client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM);
            self.client_socket.connect((HOST, PORT));
            print("[CLIENT] Connected to server.")
            self.receive_thread = threading.Thread(target=self.receive_messages_thread,
daemon=True);
            self.receive_thread.start()
        except socket.error as e:
            messagebox.showerror("Connection Error",
                                f"Could not connect to server: {e}"); self.on_closing() # Or
attempt reconnect
        except Exception as e:
            messagebox.showerror("Connection Error", f"Unexpected connection error: {e}");
self.on_closing()

    def receive_messages_thread(self):
        global gui_running
        self.receive_buffer = b""
        while gui_running and self.client_socket:
            try:
                data = self.client_socket.recv(8192) # Increased buffer
                if not data:
                    if gui_running: self.message_queue.put(
                        ("[SYSTEM]", "Connection lost: Server closed connection.)); break
                    self.receive_buffer += data
                while b'\n' in self.receive_buffer:
                    message_part, self.receive_buffer = self.receive_buffer.split(b'\n', 1);

```

```

        msg_str = message_part.decode('utf-8').strip()
        if not msg_str: continue
        try:
            self.message_queue.put(json.loads(msg_str))
        except json.JSONDecodeError:
            self.message_queue.put(["[SYSTEM]", f"Corrupted data from server:
{msg_str[:100]}"])
        except Exception as e:
            self.message_queue.put(["[SYSTEM]", f"Error processing server data:
{e}"])
    except (ConnectionResetError, ConnectionAbortedError, OSError) as e:
        if gui_running: self.message_queue.put(["[SYSTEM]", f"Connection issue:
{e}."]);
        try:
            if self.client_socket: self.client_socket.close()
        except:
            pass # Ignore errors on close if already broken
            self.client_socket = None;
            break
        except Exception as e: # Other unexpected errors
            if gui_running: self.message_queue.put(["[SYSTEM]", f"Critical receiving
error: {e}"]); break
        print("[CLIENT] Receive thread finished.")
        if gui_running: # If GUI is still running but socket died, inform user
            self.message_queue.put(["[SYSTEM]", "Disconnected from server.")]
        self.client_socket = None # Ensure socket is None if thread exits

    def send_message(self, message_type, payload): # (As in original)
        if not self.client_socket: messagebox.showerror("Connection Error", "Not connected.");
        return False
        try:
            message = {"type": message_type, "payload": payload};
            message_json = json.dumps(message)
            self.client_socket.sendall(message_json.encode('utf-8') + b'\n');
            return True
        except OSError as e:
            messagebox.showerror("Connection Error", f"Send failed: {e}. Disconnected.");
            self.message_queue.put(
                "[SYSTEM]",
                f"Send failed: {e}. Disconnected."); self.client_socket = None; self.show_login_view();
        return False # More robust disconnect handling
        except Exception as e:
            messagebox.showerror("Send Error", f"Unexpected error sending: {e}"); return False

    def _get_group_keys_filepath(self):
        return self.GROUP_KEYS_FILE

    def _load_group_aes_keys(self):
        filepath = self._get_group_keys_filepath()
        if os.path.exists(filepath):
            try:
                with open(filepath, 'r') as f:
                    keys_b64 = json.load(f)
                    for group_id, key_b64 in keys_b64.items():
                        try: # Ensure group_id can be int if needed, though string keys for
dict are fine
                            gid = int(group_id) if group_id.isdigit() else group_id
                            self.group_aes_keys[gid] = base64.b64decode(key_b64)
                        except (ValueError, TypeError) as e_inner:
                            print(f"[CLIENT_ERROR] Corrupt key for group {group_id} in
{filepath}: {e_inner}")
                        print(f"[CLIENT] Loaded {len(self.group_aes_keys)} group AES keys from
{filepath}")
            except Exception as e:
                print(f"[CLIENT_ERROR] Failed to load group AES keys from {filepath}: {e}")
                self.group_aes_keys = {} # Reset on error
        else:
            print(f"[CLIENT] Group AES key file '{filepath}' not found. Starting with empty
keys.")
            self.group_aes_keys = {}

    def _save_group_aes_keys(self):
        filepath = self._get_group_keys_filepath()
        keys_b64 = {}
        for group_id, key_bytes in self.group_aes_keys.items():

```

```

        keys_b64[str(group_id)] = base64.b64encode(key_bytes).decode('utf-8')
    try:
        with open(filepath, 'w') as f:
            json.dump(keys_b64, f)
        print(f"[CLIENT] Saved group AES keys to {filepath}")
    except Exception as e:
        print(f"[CLIENT_ERROR] Failed to save group AES keys to {filepath}: {e}")

def _store_and_save_group_key(self, group_id, aes_key_bytes):
    """Helper to store in memory and then save all keys to file."""
    if group_id is None or aes_key_bytes is None: return
    self.group_aes_keys[group_id] = aes_key_bytes
    self._save_group_aes_keys()

def send_message_event(self, event=None):
    if not self.username:
        messagebox.showerror("Login Required", "Please login to send messages.")
        return
    if not self.current_chat_identifier:
        messagebox.showwarning("No Chat Selected", "Please select a chat to send a
message.")
        return
    if not hasattr(self, 'msg_entry') or not self.msg_entry:
        return

    # Get text from CTKTextbox (1.0 to end-1c to exclude the automatic trailing newline)
    # Also strip leading/trailing whitespace that might result from just newlines.
    message_text = self.msg_entry.get("1.0", "end-1c").strip()

    if not message_text: # Check if message is empty *after* stripping
        # Optionally clear the textbox if it only contained whitespace
        # self.msg_entry.delete("1.0", "end")
        return

    now_ts_str = datetime.now().strftime('%Y-%m-%d %H:%M:%S.%f')[:-3]
    # For local history, generate a unique-ish local ID
    local_msg_id = f"local_{int(datetime.now().timestamp() *
1000)}_{len(self.chat_histories.get(self.current_chat_identifier, []))}"

    if self.current_chat_is_group:
        group_id = self.current_chat_identifier
        group_aes_key = self.group_aes_keys.get(group_id)

        if not group_aes_key:
            messagebox.showerror("Group Encryption Error",
                                f"No AES key found for group
{self.current_chat_group_name or group_id}.\nCannot send encrypted message.")
            return

    try:
        message_bytes = message_text.encode('utf-8')
        aes_encrypted_payload = self.encrypt_aes(message_bytes, group_aes_key)

        e2ee_group_payload = {
            "group_id": group_id,
            "aes_payload": aes_encrypted_payload
        }

        if self.send_message("E2EE_GROUP_MESSAGE", e2ee_group_payload):
            print(f"[CLIENT SEND_GROUP_E2EE] Sent E2EE_GROUP_MESSAGE to GID
{group_id}")

            self.chat_histories[group_id].append({
                "sender_username": self.username,
                "text": message_text, # Store plaintext for self
                "timestamp": now_ts_str,
                "is_group_message": True,
                "group_id": group_id,
                "is_system_event": False,
                "is_encrypted": True,
                "status": "sent_e2ee_group_aes",
                "local_id": local_msg_id
            })
            if hasattr(self, 'load_chat_history'):
                self.load_chat_history(group_id, True, True)
            self.msg_entry.delete("1.0", "end") # Clear CTKTextbox
    except Exception as e:
        messagebox.showerror("Encryption Error", f"Failed to encrypt or send group

```

```

message: {e}")
    print(f"[CLIENT_ERROR] Encrypt/Send E2EE Group Message: {e}")

else: # Private E2EE message
    recipient = self.current_chat_identifier
    if not self.public_key or not self.private_key:
        messagebox.showerror("Security Error", "Client RSA keys not initialized.")
        return

    recipient_rsa_public_key_pem = self.user_public_keys_cache.get(recipient)

    if not recipient_rsa_public_key_pem:
        print(f"[CLIENT_QUEUE_MSG] RSA key for {recipient} not available for private
E2EE. Queuing message: '{message_text[:30]}...'")
        self.unsent_e2ee_messages[recipient].append(
            {'local_id': local_msg_id, 'text': message_text, 'timestamp': now_ts_str})
        self.chat_histories[recipient].append({
            "sender": self.username, "recipient": recipient, "text": message_text,
            "timestamp": now_ts_str, "is_group_message": False, "is_system_event":
False,
            "is encrypted": False, # Not yet encrypted if key is missing for recipient
            "status": "queued_for_key_exchange", "local_id": local_msg_id
        })
        if hasattr(self, 'load_chat_history'): self.load_chat_history(recipient,
False, True)
        if hasattr(self, 'populate chat list'): self.populate chat list()
        self.msg_entry.delete("1.0", "end") # Clear CTKTextbox after queuing
        if self.current_chat_key_status != "fetching": # Avoid repeated requests if
already fetching
            if hasattr(self, 'request_other_user_public_key'):
self.request_other_user_public_key(recipient)
            return

        current_aes_session = self.chat_session_keys.get(recipient)

        if not current_aes_session:
            print(f"[CLIENT_HYBRID] No AES session for {recipient}. Initiating key
exchange for message: '{message_text[:30]}...'")
            try:
                aes_key = self.generate_aes_key()
                # Temporarily store, commit only if send is successful
                # self.chat_session_keys[recipient] = {"key": aes_key} # Store key for
this recipient

                recipient_encrypted_aes_key_bytes = self.encrypt_message(aes_key,
recipient_rsa_public_key_pem)
                if not recipient_encrypted_aes_key_bytes:
                    messagebox.showerror("Security Error", f"Failed to RSA-encrypt session
key for {recipient}.")
                    return # Don't clear textbox if RSA encryption failed before send
attempt

                recipient_encrypted_aes_key_b64 =
base64.b64encode(recipient_encrypted_aes_key_bytes).decode('utf-8')

                own_public_key_pem = self.get_public_key_pem()
                if not own_public_key_pem:
                    messagebox.showerror("Security Error", "Own public key not found for
self-encryption.")
                    return

                sender_self_encrypted_aes_key_bytes = self.encrypt_message(aes_key,
own_public_key_pem)
                if not sender_self_encrypted_aes_key_bytes:
                    messagebox.showerror("Security Error", "Failed to self-encrypt session
key.")
                    return

                sender_self_encrypted_aes_key_b64 =
base64.b64encode(sender_self_encrypted_aes_key_bytes).decode('utf-8')

                message_bytes_to_send = message_text.encode('utf-8')
                aes_encrypted_payload_for_initial_message =
self.encrypt_aes(message_bytes_to_send, aes_key)

                session_init_payload = {
                    "recipient": recipient,

```

```

        "encrypted_session_key_b64": recipient_encrypted_aes_key_b64,
        "sender_encrypted_session_key_b64": sender_self_encrypted_aes_key_b64,
        "initial_message": aes_encrypted_payload_for_initial_message
    }
    print(f"[CLIENT SEND_INIT_SESS] Payload for {recipient}:
{session_init_payload['encrypted_session_key_b64'][:10]}...,
{session_init_payload['sender_encrypted_session_key_b64'][:10]}..., initial_msg_iv:
{session_init_payload['initial_message']['iv_b64'][:10]}...")

    if self.send_message("INITIATE_SECURE_SESSION", session_init_payload):
        self.chat_session_keys[recipient] = {"key": aes_key} # Commit key
storage after successful send
        self.chat_histories[recipient].append({
            "sender": self.username, "recipient": recipient, "text":
message_text,
            "timestamp": now_ts_str, "is_group_message": False,
"is_system_event": False,
            "is_encrypted": True, "status": "sent_e2ee_aes", "local_id":
local_msg_id
        })
        if hasattr(self, 'load_chat_history'):
self.load_chat_history(recipient, False, True)
        if hasattr(self, 'populate_chat_list'): self.populate_chat_list()
        self.msg_entry.delete("1.0", "end") # Clear CTKTextbox
    else:
        messagebox.showerror("Send Error", f"Failed to send session initiation
to {recipient}.")
        # Do not store the key if send failed:
self.chat_session_keys.pop(recipient, None)
    except Exception as e_sess_init:
        messagebox.showerror("Session Init Error", f"Error initiating session with
{recipient}: {e_sess_init}")
        # Ensure key is not stored if an error occurred before successful send
        self.chat_session_keys.pop(recipient, None)

    else: # AES session already exists
        print(f"[CLIENT HYBRID] AES session exists for {recipient}. Sending AES
message: '{message_text[:30]}...'")
        try:
            aes_key = current_aes_session["key"]
            message_bytes_to_send = message_text.encode('utf-8')
            aes_encrypted_payload = self.encrypt_aes(message_bytes_to_send, aes_key)
            payload_to_send = {"recipient": recipient, "aes_payload":
aes_encrypted_payload}

            if self.send_message("AES_ENCRYPTED_MESSAGE", payload_to_send):
                self.chat_histories[recipient].append({
                    "sender": self.username, "recipient": recipient, "text":
message_text,
                    "timestamp": now_ts_str, "is_group_message": False,
"is_system_event": False,
                    "is_encrypted": True, "status": "sent_e2ee_aes", "local_id":
local_msg_id
                })
                if hasattr(self, 'load_chat_history'):
self.load_chat_history(recipient, False, True)
                if hasattr(self, 'populate_chat_list'): self.populate_chat_list()
                self.msg_entry.delete("1.0", "end") # Clear CTKTextbox
            else:
                messagebox.showerror("Send Error", f"Failed to send AES message to
{recipient}.")
        except Exception as e_aes_send:
            messagebox.showerror("AES Error", f"Error sending AES message:
{e_aes_send}")

    def check_queue(self):
        history_needs_update_for_cycle = False
        try:
            while not self.message_queue.empty():
                message_obj = self.message_queue.get_nowait()

                is_internal_system_msg = isinstance(message_obj, tuple) and message_obj[0] ==
"[SYSTEM]"

                if is_internal_system_msg:
                    system_text = message_obj[1]
                    if hasattr(self, 'display_system_message'):
```



```

        self.display_system_message(system_text)

        connection_lost_keywords = ["Connection issue", "Connection lost",
                                     "Connection closed by server", "Disconnected
from server"]

        if any(keyword.lower() in system_text.lower() for keyword in
connection_lost_keywords):
            if self.username:
                messagebox.showwarning("Connection", "Lost connection to the
server.")

                self.username = None
                self.client_socket = None
                self.current_chat_identifier = None
                self.current_chat_is_group = False
                self.my_groups = []
                self.chat_histories.clear()
                self.profile_picture_cache.clear()
                self.group_icon_cache.clear()
                self.last_group_details_data = None
                self.all_users_list = []
                self.server_public_key_pem = None
                self.user_public_keys_cache.clear()
                self.current_chat_key_status = "unknown"
                self.unsent_e2ee_messages.clear()
                self.chat_session_keys.clear()

                # Clear and attempt to save group keys (though unlikely to have new
ones if disconnected)
                current_group_keys_before_clear = self.group_aes_keys.copy()
                self.group_aes_keys.clear()
                if hasattr(self, '_save_group_aes_keys') and
current group keys before clear:
                    # This might be problematic if called when username is None for
user-specific files
                    # For now, it implies a generic key file or needs username check
for filename
                    # self._save_group_aes_keys() # Let's defer saving on disconnect
to avoid complexity
                    print("[CLIENT] Connection lost, group AES keys cleared from
memory.")

                    if hasattr(self, 'show_login_view'): self.show_login_view()
                    if hasattr(self, 'update_chat_input_state'):
self._update_chat_input_state()
                        continue
                    # --- End of internal system message handling ---

                msg_type = message_obj.get("type")
                payload = message_obj.get("payload", {})

                # For generic handlers that might want to know the original type
                if msg_type in self.message_handlers and \
                    self.message_handlers[msg_type] ==
self._handle_generic_server_message:
                    payload["original_msg_type"] = msg_type # Pass original type to generic
handler

                handler_method = self.message_handlers.get(msg_type)

                if handler_method:
                    # Pass payload, and for generic handler, also the original_msg_type
                    if handler_method == self._handle_generic_server_message:
                        if handler_method(payload, msg_type): # Pass original_msg_type
                            history_needs_update_for_cycle = True
                        elif handler_method(payload): # Standard handlers
                            history_needs_update_for_cycle = True
                    else:
                        self._handle_unhandled_message(msg_type, payload) # Call specific
unhandled method

                if history_needs_update_for_cycle:
                    if hasattr(self, 'main_chat_frame') and \
                        self.main_chat_frame.wininfo_exists() and
self.main_chat_frame.wininfo_ismapped() and \
                        hasattr(self, 'populate_chat_list'):
                        print("[CLIENT] Calling populate chat list after processing queue cycle.")
                        self.populate_chat_list()

```

```

except Queue.Empty:
    pass
except Exception as e:
    print(f"CRITICAL Error in check_queue processing loop: {e}")
    import traceback
    traceback.print_exc()
finally:
    global gui_running
    if gui_running:
        self.after(100, self.check_queue)

def _handle_unhandled_message(self, original_msg_type, payload):
    print(f"[CLIENT WARNING] Unhandled server message type: '{original_msg_type}' -
Payload: {str(payload)[:200]}")
    if hasattr(self, 'display_system_message'):
        self.display_system_message(f"[Server/Unknown Type '{original_msg_type}']:
{str(payload)[:150]}")
    return False # No history update

# --- Authentication & Key Exchange Handlers ---

def _handle_server_public_key(self, payload):
    print("[CLIENT] Handling SERVER PUBLIC KEY")
    self.server_public_key_pem = payload.get("key")
    if self.server_public_key_pem:
        print("[CLIENT] Stored server's public key.")
    else:
        print("[CLIENT_ERROR] SERVER_PUBLIC_KEY: Key missing in payload.")
        messagebox.showerror("Security Error", "Could not receive server's public key.")
    return False # Does not directly require chat list refresh

def _handle_user_public_key_info(self, payload):
    target_username = payload.get("username")
    rsa_key_pem = payload.get("key")
    print(f"[CLIENT] Handling USER_PUBLIC_KEY_INFO for {target_username}")
    if target_username and rsa_key_pem:
        self.user_public_keys_cache[target_username] = rsa_key_pem
        print(f"[CLIENT] Cached RSA PK for {target_username}.")

    # Process unsent E2EE messages for this user
    if target_username in self.unsent_e2ee_messages and
self.unsent_e2ee_messages[target_username]:
        messages_to_send_now = list(self.unsent_e2ee_messages[target_username])
        self.unsent_e2ee_messages[target_username] = [] # Optimistically clear

        print(
            f"[CLIENT] Found {len(messages_to_send_now)} queued messages for
{target_username}. Attempting to send with newly received key.")
        aes_key_for_session = self.chat_session_keys.get(target_username,
{}).get("key")
        session_initiated_this_cycle = False # Only init session once per batch

        for queued_msg_data in messages_to_send_now:
            local_id = queued_msg_data.get('local_id');
            text_to_send = queued_msg_data['text'];
            orig_ts = queued_msg_data['timestamp']
            message_sent_successfully = False
            try:
                if not aes_key_for_session and not session_initiated_this_cycle:
                    aes_key_for_session = self.generate_aes_key()
                    # Encrypt AES key for recipient
                    recipient_enc_aes_key_bytes =
self.encrypt_message(aes_key_for_session,
                                                                rsa_key_pem) #
Use newly received rsa_key_pem
                    if not recipient_enc_aes_key_bytes:
self.unsent_e2ee_messages[target_username].append(queued_msg_data);
                    continue
                    recipient_enc_aes_key_b64 =
base64.b64encode(recipient_enc_aes_key_bytes).decode('utf-8')
                    # Self-encrypt AES key
                    own_pk_pem = self.get_public_key_pem()
                    if not own_pk_pem:
self.unsent_e2ee_messages[target_username].append(
                    queued_msg_data); continue # Cannot self-encrypt

```

```

        sender_self_enc_aes_key_bytes =
self.encrypt_message(aes_key_for_session, own_pk_pem)
        if not sender_self_enc_aes_key_bytes:

self.unsent_e2ee_messages[target_username].append(queued_msg_data);
        continue
        sender_self_enc_aes_key_b64 =
base64.b64encode(sender_self_enc_aes_key_bytes).decode(
        'utf-8')

        self.chat_session_keys[target_username] = {"key":
aes_key_for_session} # Store new key
        aes_enc_payload = self.encrypt_aes(text_to_send.encode('utf-8'),
aes_key_for_session)

        session_init_payload = {
            "recipient": target_username,
            "encrypted_session_key_b64": recipient_enc_aes_key_b64,
            "sender_encrypted_session_key_b64":
sender_self_enc_aes_key_b64,
            "initial message": aes enc payload
        }
        if self.send_message("INITIATE_SECURE_SESSION",
session_init_payload):
            session_initiated_this_cycle = True;
            message sent successfully = True
        else: # Failed to send session init

self.unsent_e2ee_messages[target_username].append(queued_msg_data)
        self.chat_session_keys.pop(target_username, None);
        aes_key_for_session = None # Reset

        elif aes_key_for_session: # Session key exists (either old or just
initiated)
            aes_enc_payload = self.encrypt_aes(text_to_send.encode('utf-8'),
aes_key_for_session)
            send_payload = {"recipient": target_username, "aes_payload":
aes_enc_payload}
            if self.send_message("AES_ENCRYPTED_MESSAGE", send_payload):
                message_sent_successfully = True
            else:
                self.unsent_e2ee_messages[target_username].append(
                    queued_msg_data) # Re-queue on send fail
        else: # Should not happen if first condition logic is right, re-queue
self.unsent_e2ee_messages[target_username].append(queued_msg_data)

        if message_sent_successfully:
            print(
                f"[CLIENT] Queued message (local_id: {local_id}) sent
successfully to {target_username} after key received.")
            for hist_msg in self.chat_histories.get(target_username, []):
                if hist_msg.get("local_id") == local_id and
hist_msg.get("status", "").startswith(
                    "queued"):
                    hist_msg["status"] = "sent_e2ee_aes";
                    hist_msg["is_encrypted"] = True;
                    hist_msg["timestamp"] = orig_ts;
                    break
            except Exception as e:
                print(
                    f"[CLIENT_ERROR] Processing queued msg (local_id:{local_id}) for
{target_username} after key RX: {e}")
                self.unsent_e2ee_messages[target_username].append(queued_msg_data) #
Re-add on error

            if not self.unsent_e2ee_messages.get(target_username): # If list became empty
self.unsent_e2ee_messages.pop(target_username, None)

            if self.current_chat_identifier == target_username and not
self.current_chat_is_group:
                self.current_chat_key_status = "available"
                if hasattr(self, 'select_chat'):
self.select_chat(self.current_chat_identifier,
False) # Refresh chat UI
state

        return True # History might need update due to sent messages

```

```

        return False # No direct history update if key was just cached without sending queued

    def _handle_user_public_key_fail(self, payload):
        target_username = payload.get("username")
        error_message = payload.get("message", "Failed to get Public Key.")
        print(f"[CLIENT] Handling USER_PUBLIC_KEY_FAIL for {target_username}: {error_message}")
        messagebox.showwarning("Key Exchange Failed",
                               f"Could not get Public Key for {target_username}: {error_message}")
        if self.current_chat_identifier == target_username and not self.current_chat_is_group:
            self.current_chat_key_status = "failed"
            if hasattr(self, 'select_chat'): self.select_chat(self.current_chat_identifier,
                                                              False) # Refresh chat UI state
        return False

    def _handle_register_success(self, payload):
        print("[CLIENT] Handling REGISTER_SUCCESS")
        messagebox.showinfo("Registration Successful", payload.get("message", "Registered! Please log in. "))
        self.username_being_registered = None
        if hasattr(self, 'show_login_view'): self.show_login_view()
        return False

    def _handle_register_fail(self, payload):
        print("[CLIENT] Handling REGISTER_FAIL")
        messagebox.showerror("Registration Failed", payload.get("message", "Registration failed. "))
        self.username_being_registered = None
        return False

    def handle_login_success(self, payload):
        print("[CLIENT] Handling LOGIN_SUCCESS")
        self.username = payload.get("username")
        if self.username:
            print(f"[CLIENT] Logged in as: {self.username}")
            # Clear session-specific data
            self.chat_histories.clear()
            self.messages_currently_shown_count.clear()
            self.chat_all_older_loaded.clear()
            self.profile_picture_cache.clear()
            self.group_icon_cache.clear()
            self.user_public_keys_cache.clear()
            self.current_chat_key_status = "unknown"
            self.my_groups = []
            self.last_group_details_data = None
            self.all_users_list = []
            self.unsent_e2ee_messages.clear()
            self.chat_session_keys.clear()

            # Clear and load group AES keys
            self.group_aes_keys.clear()
            print(f"[CLIENT] {self.username}] LOGIN_SUCCESS: Attempting to load group AES keys.")
            self._load_group_aes_keys() # Load persistent group keys

            if hasattr(self, 'show_chat_screen'): self.show_chat_screen()
            self.send_message("GET_MY_GROUPS", {});
            self.send_message("GET_ALL_USERS", {});
            return True # Requires chat list refresh
        else:
            messagebox.showerror("Login Error", "Login successful but user data missing.")
            return False

    def _handle_login_fail(self, payload):
        print("[CLIENT] Handling LOGIN_FAIL")
        messagebox.showerror("Login Failed", payload.get("message", "Invalid credentials. "))
        return False

    # --- Core Messaging Handlers ---

    def _handle_message(self, payload):
        # This handler processes messages of type "MESSAGE" from the server,
        # which are typically historical messages, but could also be live messages
        # if the server uses this type for general message relay.

        sender = payload.get("sender")

```

```

recipient = payload.get("recipient") # For private messages
text_from_payload = payload.get("text", "") # Raw content from DB (plain text or JSON
for E2EE)
    ts = payload.get("timestamp", datetime.now().strftime('%Y-%m-%d %H:%M:%S.%f')[:-3])
    gid = payload.get("recipient_group_id") # Group ID if it's a group message
    is_sys = payload.get("is_system_message", False)
    e2ee_type_hist = payload.get("e2ee_type") # E.g., AES_SESSION_INIT, AES_MSG,
GROUP_AES, GROUP_KEY_DELIVERY

    # For self-sent E2EE 1-to-1 messages (AES_SESSION_INIT type)
    sender_e2ee_key_data_b64 = payload.get("sender_e2ee_session_data")

    # For group messages, sender_username from payload is preferred if available
    sender_uname_grp = payload.get("sender_username", sender)

    chat_partner_key = None # Key for self.chat_histories dictionary (group_id or
other_user_username)
    text_to_display = text_from_payload # Default, will be overridden by decryption if
E2EE

    is_encrypted_render = False
    message_render_status = "unknown"

    print(
        f"[CLIENT HIST] _handle message: Processing. From:{sender}, To:{recipient},
GID:{gid}, E2EE_Type:{e2ee_type_hist}, Sys:{is_sys}")
    print(f"[CLIENT HIST] _handle message: Full payload received: {payload}")

    if gid is not None: # --- This is a Group Message (from history) ---
        chat_partner_key = gid
        # Default status for group messages from history (can be refined by E2EE type)
        message_render_status = "received" if sender_uname_grp != self.username else
"sent"

        if e2ee_type_hist == 'GROUP_AES':
            is_encrypted_render = True
            message_render_status = "received_e2ee_group_aes" if sender_uname_grp !=
self.username else "sent_e2ee_group_aes"
            group_aes_key = self.group_aes_keys.get(gid) # Use int(gid) if gid is stored
as int key

            print(
                f"[CLIENT HIST_GROUP {self.username}] _handle_message: Attempting to
decrypt GID {gid}. Key found: {group_aes_key is not None}. Sender: {sender_uname_grp}")

            if not group_aes_key:
                text_to_display = "[Encrypted group history - Key unavailable]"
                message_render_status = "failed_decrypt_group_aes"
            else:
                try:
                    aes_payload_hist = json.loads(text_from_payload) # text_from_payload
is the stored JSON
                    iv = aes_payload_hist.get("iv_b64")
                    ct = aes_payload_hist.get("ciphertext_b64")
                    if iv and ct:
                        dec_bytes = self.decrypt_aes(ct, iv, group_aes_key)
                        if dec_bytes:
                            text_to_display = dec_bytes.decode('utf-8')
                        else:
                            text_to_display = "[Group Hist Decrypt Fail]"
                            message_render_status = "failed_decrypt_group_aes"
                    else:
                        text_to_display = "[Group Hist Malformed AES Payload]"
                        message_render_status = "failed_decrypt_group_aes"
                except json.JSONDecodeError:
                    text_to_display = f"[Group Hist Invalid JSON:
{text_from_payload[:60]}...]"
                    message_render_status = "failed_decrypt_group_aes"
                    print(
                        f"[CLIENT HIST_GROUP_ERROR] text_from_payload for GROUP_AES GID
{gid} was not valid JSON.")
                    except Exception as e:
                        text_to_display = "[Group Hist Decrypt Error]"
                        message_render_status = "failed_decrypt_group_aes"
                        print(f"[CLIENT HIST_GROUP_ERROR] Decrypting GID {gid} (sender
{sender_uname_grp}): {e}")

        elif e2ee_type_hist == "GROUP_KEY_DELIVERY":

```

```

        # This is a control message, not for direct display as a chat bubble.
        # It processes the key and then we might skip adding it to chat_histories for
display,

        # or mark it clearly as a processed system event.
        print(
            f"[CLIENT {self.username}] _handle_message: Processing historical
GROUP_KEY_DELIVERY for GID {gid}")
        try:
            key_delivery_payload = json.loads(
                text_from_payload) # text from payload IS the key_delivery_payload
            group_id_from_delivery = key_delivery_payload.get("group_id")
            wrapped_key_b64_from_delivery =
key_delivery_payload.get("wrapped_group_key_b64")
            distributor_from_delivery = key_delivery_payload.get("sender_username")

            if group_id_from_delivery is not None and wrapped_key_b64_from_delivery
and self.private_key:
                wrapped_key_bytes = base64.b64decode(wrapped_key_b64_from_delivery)
                group_aes_key_bytes = self.decrypt_message(wrapped_key_bytes) # RSA
Decrypt

                if group_aes_key_bytes:
                    self._store_and_save_group_key(group_id_from_delivery,
group_aes_key_bytes)

                    print(
                        f"[CLIENT {self.username}] Successfully processed and stored
historical AES key for group {group_id_from_delivery} (distributed by
{distributor_from_delivery}). Keys: {list(self.group_aes_keys.keys())}")
                    else:
                        print(
                            f"[CLIENT_ERROR {self.username}] Failed to decrypt historical
wrapped group key for GID {group_id_from_delivery}")
                        else:
                            print(f"[CLIENT_ERROR {self.username}] Invalid historical
GROUP_KEY_DELIVERY payload content.")
                        except Exception as e_hist_key:
                            print(
                                f"[CLIENT_ERROR {self.username}] Error processing historical
GROUP_KEY_DELIVERY: {e_hist_key}")
                            # This type of message should not be added to chat_histories for display as a
user message.
                            # After processing the key, the UI should be refreshed so other messages can
use the key.
                            if hasattr(self, 'load_chat_history') and self.current_chat_identifier == gid:
self.load_chat_history(
                    gid, True, True)
                            return True # Return True to trigger populate_chat_list

                            # Append group message to history
                            self.chat_histories[chat_partner_key].append({
                                "sender_username": sender_username_grp, "text": text_to_display, "timestamp": ts,
                                "is_group_message": True, "group_id": gid, "is_system_event": is_sys,
                                "is_encrypted": is_encrypted_render, "status": message_render_status
                            })

                        else: # --- This is a Private Message (from history) ---
                            if not sender or not recipient: # Should not happen for private messages from DB
                                print(
                                    f"[CLIENT_ERROR] _handle_message: Private message missing sender or
recipient. Payload: {payload}")
                                return True # Refresh UI

                            chat_partner_key = sender if recipient == self.username else recipient

                            if sender == self.username: # Historical message I SENT (1-to-1 E2EE)
                                message_render_status = "sent"
                                print(
                                    f"[CLIENT HIST_SELF_SENT] Processing my own historical 1-to-1 message to
{recipient}. E2EE Type: {e2ee_type_hist}.")
                                # (DEBUG 1 was: Full payload: {payload} - payload is already printed above)

                                if e2ee_type_hist == "AES_SESSION_INIT":
                                    is_encrypted_render = True;
                                    message_render_status = "sent_e2ee_aes"
                                    print(
                                        f"[CLIENT HIST_SELF_SENT] AES_SESSION_INIT: sender_e2ee_session_data
from payload is present: {sender_e2ee_key_data_b64 is not None}") # DEBUG 2
                                    if sender_e2ee_key_data_b64: print(

```

```

        f"[CLIENT HIST_SELF_SENT] AES_SESSION_INIT: Self-encrypted key data
(b64 preview): {str(sender_e2ee_key_data_b64)[:50]}..." # DEBUG 2.1
        print(
            f"[CLIENT HIST_SELF_SENT] AES_SESSION_INIT: self.private_key
available: {self.private_key is not None}") # DEBUG 3

        if sender_e2ee_key_data_b64 and self.private_key:
            try:
                print(
                    "[CLIENT HIST_SELF_SENT] AES_SESSION_INIT: Attempting to RSA-
decrypt self-encrypted AES key..." # DEBUG 4
                self_decrypted_aes_key =
self.decrypt_message(base64.b64decode(sender_e2ee_key_data_b64))
                if self_decrypted_aes_key: # RSA Decrypt OK
                    print(
                        f"[CLIENT HIST_SELF_SENT] AES_SESSION_INIT: Successfully
RSA-decrypt self_aes_key. Storing for recipient: {recipient}") # DEBUG 5
                    self.chat_session_keys[recipient] = {"key":
self_decrypted_aes_key}

                    e2ee_init_structure = json.loads(text_from_payload)
                    print(
                        f"[CLIENT HIST_SELF_SENT] AES_SESSION_INIT: Parsed
e2ee_init_structure: {e2ee_init_structure is not None}") # DEBUG 6
                    initial_msg_json = e2ee_init_structure.get("initial_message")
                    if initial_msg_json and isinstance(initial_msg_json, dict):
                        iv = initial_msg_json.get("iv_b64");
                        ct = initial_msg_json.get("ciphertext_b64")
                        print(
                            f"[CLIENT HIST_SELF_SENT] AES_SESSION_INIT: IV: {iv is
not None}, CT: {ct is not None}") # DEBUG 7
                        if iv and ct:
                            dec_b = self.decrypt_aes(ct, iv,
self_decrypted_aes_key)

                            if dec_b:
                                text_to_display = dec_b.decode('utf-8'); print(
                                    f"[CLIENT HIST_SELF_SENT] AES_SESSION_INIT:
Decrypted initial message: '{text_to_display}'") # DEBUG 8
                                else:
                                    text_to_display = "[E2EE Init: AES Decrypt Fail
(Self)]"; print(
                                        "[CLIENT HIST_SELF_SENT] AES_SESSION_INIT: AES
decrypt initial message failed.") # DEBUG 8.1
                                else:
                                    text_to_display = "[E2EE Init: Missing IV/CT (Self)]";
print(
                                        "[CLIENT HIST_SELF_SENT] AES_SESSION_INIT: Missing
IV/CT.") # DEBUG 7.1
                                else:
                                    text_to_display = "[E2EE Init: Malformed initial_message
(Self)]"; print(
                                        "[CLIENT HIST_SELF_SENT] AES_SESSION_INIT: Malformed
initial_message.") # DEBUG 6.1
                                else:
                                    text_to_display = "[E2EE Init: Self-Key RSA Decrypt Fail]";
print(
                                        "[CLIENT HIST_SELF_SENT] AES_SESSION_INIT: RSA decrypt
self-key failed.") # DEBUG 5.1
                                except Exception as e:
                                    text_to_display = "[E2EE Init: Error self-data]"; print(
                                        f"[CLIENT HIST_ERROR] Self-sent AES_SESSION_INIT: {e}") #
DEBUG 4.1
                                else:
                                    text_to_display = "[You started a secure session. Details
unavailable.]"; print(
                                        f"[CLIENT HIST_SELF_SENT] AES_SESSION_INIT: Fallback. SelfData:
{sender_e2ee_key_data_b64 is not None}, PrivKey: {self.private_key is not None}") # DEBUG 3.1

                                elif e2ee_type_hist == "AES_MSG":
                                    is_encrypted_render = True;
                                    message_render_status = "sent_e2ee_aes"
                                    print(
                                        f"[CLIENT HIST_SELF_SENT] Processing self-sent AES_MSG to {recipient}.
text_from_payload: {text_from_payload}") # DEBUG 9
                                    session = self.chat_session_keys.get(recipient)
                                    if session and session.get("key"):
                                        print(f"[CLIENT HIST_SELF_SENT] AES MSG: Found session key for
{recipient}.") # DEBUG 10

```

```

        try:
            aes_msg_structure = json.loads(text_from_payload)
            iv = aes_msg_structure.get("iv_b64");
            ct = aes_msg_structure.get("ciphertext_b64")
            print(
                f"[CLIENT HIST_SELF_SENT] AES_MSG: IV: {iv is not None}, CT:
{ct is not None}") # DEBUG 11
            if iv and ct:
                dec_b = self.decrypt_aes(ct, iv, session["key"])
                if dec_b:
                    text_to_display = dec_b.decode('utf-8'); print(
                        f"[CLIENT HIST_SELF_SENT] AES_MSG: Decrypted:
'{text_to_display}'") # DEBUG 12
                else:
                    text_to_display = "[E2EE Msg: AES Decrypt Fail (Self)]";
print(
                    "[CLIENT HIST_SELF_SENT] AES_MSG: AES decrypt
failed.") # DEBUG 12.1
            else:
                text_to_display = "[E2EE Msg: Missing IV/CT (Self)]"; print(
                    "[CLIENT HIST_SELF_SENT] AES_MSG: Missing IV/CT.") #
DEBUG 11.1
        except Exception as e:
            text_to_display = "[E2EE Msg: Error self-data (AES_MSG)]"; print(
                f"[CLIENT HIST_ERROR] Self-sent AES_MSG: {e}") # DEBUG 10.1
        else:
            text_to_display = "[Encrypted message you sent. Session key not
recovered.]"; print(
                f"[CLIENT HIST_SELF_SENT] AES_MSG: Session key for {recipient} not
found. Keys: {list(self.chat_session_keys.keys())}") # DEBUG 10.2
            # else: text_to_display is already text_from_payload for plain messages

            else: # Historical message TO ME (sender != self.username) - E2EE 1-to-1
                message_render_status = "received"
                print(
                    f"[CLIENT HIST_RECV] Processing 1-to-1 message TO ME from {sender}. E2EE
Type: {e2ee_type_hist}.") # DEBUG R0
                # (DEBUG R1 was full payload, already printed at start of _handle_message)

                if isinstance(text_from_payload, str) and e2ee_type_hist:
                    if e2ee_type_hist == "AES_SESSION_INIT":
                        print(
                            f"[CLIENT HIST_RECV] AES_SESSION_INIT from {sender}: Attempting to
decrypt.") # DEBUG R1.1
                        try:
                            session_data = json.loads(text_from_payload)
                            recipient_encrypted_aes_key_b64 =
session_data.get("encrypted_session_key_b64")
                            initial_msg_payload_json = session_data.get("initial_message")
                            print(
                                f"[CLIENT HIST_RECV] AES_SESSION_INIT: RecKey:
{recipient_encrypted_aes_key_b64 is not None}, InitMsg: {initial_msg_payload_json is not
None}, PrivKey: {self.private_key is not None}") # DEBUG R2
                                if recipient_encrypted_aes_key_b64 and initial_msg_payload_json
and self.private_key:
                                    aes_key_bytes =
self.decrypt_message(base64.b64decode(recipient_encrypted_aes_key_b64))
                                    if aes_key_bytes: # RSA Decrypt OK
                                        print(
                                            f"[CLIENT HIST_RECV] AES_SESSION_INIT: Successfully
RSA-decrypted AES key from {sender}.") # DEBUG R3
                                            self.chat_session_keys[sender] = {"key": aes_key_bytes}
                                            iv = initial_msg_payload_json.get("iv_b64");
                                            cip = initial_msg_payload_json.get("ciphertext_b64")
                                            print(
                                                f"[CLIENT HIST_RECV] AES_SESSION_INIT: IV: {iv is not
None}, CT: {cip is not None}") # DEBUG R4
                                                if iv and cip:
                                                    dec_b = self.decrypt_aes(cip, iv, aes_key_bytes)
                                                    if dec_b:
                                                        text_to_display = dec_b.decode('utf-8'); print(
                                                            f"[CLIENT HIST_RECV] AES_SESSION_INIT:
Decrypted message: '{text_to_display}'") # DEBUG R5
                                                        else:
                                                            text_to_display = "[E2EE Init Decrypt Fail
(Incoming)]"; print(
                                                            f"[CLIENT HIST_RECV] AES_SESSION_INIT: AES

```



```

decrypt failed for {sender}.") # DEBUG R5.1
        else:
            text_to_display = "[E2EE Init Missing IV/CT
(Incoming)]"; print(
                                f"[CLIENT HIST_RECV] AES_SESSION_INIT: Missing
IV/CT from {sender}.") # DEBUG R4.1
        else:
            text_to_display = "[E2EE Init RSA Key Decrypt Fail
(Incoming)]"; print(
                                f"[CLIENT HIST_RECV] AES_SESSION_INIT: RSA key decrypt
failed for {sender}.") # DEBUG R3.1
        else:
            text_to_display = "[E2EE Init Invalid Data/Client Key
(Incoming)]"; print(
                                f"[CLIENT HIST_RECV] AES_SESSION_INIT: Invalid data from
{sender}.") # DEBUG R2.1
            is_encrypted_render = True;
            message_render_status = "received_e2ee_aes" if text_to_display and
not text_to_display.startswith(
                "[E2EE") else "failed_decrypt_aes"
            except Exception as e:
                text_to_display = f"[E2EE Init Parse/Decrypt Err (Incoming)]";
print(
                                f"E2EE Hist Init Err for {sender}: {e}"); is_encrypted_render
= True; message_render_status = "failed_decrypt_aes"

            elif e2ee_type_hist == "AES_MSG":
                print(f"[CLIENT HIST_RECV] AES_MSG from {sender}: Attempting to
decrypt.") # DEBUG R1.2
                try:
                    aes_p_hist = json.loads(text_from_payload)
                    iv = aes_p_hist.get("iv_b64");
                    cip = aes_p_hist.get("ciphertext_b64")
                    sess = self.chat_session_keys.get(sender)
                    print(
                        f"[CLIENT HIST_RECV] AES MSG: IV: {iv is not None}, CT: {cip
is not None}, SessionKey for {sender}: {sess and sess.get('key') is not None}") # DEBUG R6
                    if sess and sess.get("key") and iv and cip:
                        dec_b = self.decrypt_aes(cip, iv, sess["key"])
                        if dec_b:
                            text_to_display = dec_b.decode('utf-8'); print(
                                f"[CLIENT HIST_RECV] AES_MSG: Decrypted:
'{text_to_display}'))" # DEBUG R7
                        else:
                            text_to_display = "[E2EE Msg Decrypt Fail (Incoming)]";
print(
                                f"[CLIENT HIST_RECV] AES_MSG: AES decrypt failed for
{sender}.") # DEBUG R7.1
                        else:
                            text_to_display = "[E2EE Msg Missing Key/Payload (Incoming)]";
print(
                                f"[CLIENT HIST_RECV] AES_MSG: Missing key/payload for
{sender}.") # DEBUG R6.1
                            is_encrypted_render = True;
                            message_render_status = "received_e2ee_aes" if text_to_display and
not text_to_display.startswith(
                                "[E2EE") else "failed_decrypt_aes"
                            except Exception as e:
                                text_to_display = f"[E2EE Msg Parse/Decrypt Err (Incoming)]";
print(
                                    f"E2EE Hist AES_MSG Err for {sender}: {e}");
is_encrypted_render = True; message_render_status = "failed_decrypt_aes"
                            # else (not E2EE), text_to_display is already text_from_payload

                            self.chat_histories[chat_partner_key].append({
                                "sender": sender, "recipient": recipient, "text": text_to_display,
                                "timestamp": ts,
                                "is_group_message": False, "is_system_event": is_sys,
                                "is_encrypted": is_encrypted_render, "status": message_render_status
                            })

                            # Common UI update logic at the end of processing a "MESSAGE"
                            if chat_partner_key:
                                self.chat_all_older_loaded[
                                    chat_partner_key] = False # New message implies older history might be
                                relatively different
                                if self.current_chat_identifier == chat_partner_key:

```

```

        if hasattr(self, 'load_chat_history'):
            self.load_chat_history(self.current_chat_identifier, (gid is not None),
                                  True) # Scroll to bottom for new history items
    return True # Indicates history was updated and chat list might need refresh

def _handle_receive_secure_session_init(self, payload):
    """
    Handles a live E2EE session initiation from another user.
    Decrypts the AES session key using own RSA private key,
    then decrypts the initial message using the AES key.
    """
    sender_username = payload.get("sender_username")
    # This is the AES session key, RSA-encrypted by the sender using THIS client's public
    key
    recipient_encrypted_session_key_b64 = payload.get("encrypted_session_key_b64")
    initial_message_json = payload.get("initial_message") # Contains IV and ciphertext of
    the first message
    timestamp = payload.get("timestamp", datetime.now().strftime('%Y-%m-%d
    %H:%M:%S.%f'))[:-3])

    print(f"[CLIENT {self.username}] Received live SECURE SESSION INIT from
    {sender_username}.")
    print(f"[CLIENT {self.username}] Payload: {payload}")

    if not sender_username or not recipient_encrypted_session_key_b64 or \
        not initial_message_json or not isinstance(initial_message_json, dict) or \
        "iv_b64" not in initial_message_json or "ciphertext_b64" not in
    initial_message_json:
        print(
            f"[CLIENT_ERROR {self.username}] Invalid RECEIVE_SECURE_SESSION_INIT payload
            from {sender_username}.")
        return False # No history update if payload is invalid

    if not self.private_key:
        print(
            f"[CLIENT_ERROR {self.username}] Own private key not available. Cannot decrypt
            session key from {sender_username}.")
        # Optionally, display a generic error for the user, but don't add to history
        # self.chat_histories[sender_username].append({ ... placeholder ... })
        return False

    try:
        # Step 1: RSA-decrypt the AES session key
        encrypted_aes_key_bytes = base64.b64decode(recipient_encrypted_session_key_b64)
        aes_key_bytes = self.decrypt_message(encrypted_aes_key_bytes) # Uses
        self.private_key

        if not aes_key_bytes:
            print(f"[CLIENT_ERROR {self.username}] Failed to RSA-decrypt AES session key
            from {sender_username}.")
            # Store a placeholder or error message in history for this interaction
            text_to_display = "[E2EE Error: Could not establish secure session - Key
            Decryption Failed]"
            self.chat_histories[sender_username].append({
                "sender": sender_username, "recipient": self.username, "text":
            text_to_display,
                "timestamp": timestamp, "is_group_message": False, "is_system_event":
            False,
                "is_encrypted": True, "status": "failed_decrypt_session_init"
            })
            if self.current_chat_identifier == sender_username and not
            self.current_chat_is_group:
                if hasattr(self, 'load_chat_history'):
                    self.load_chat_history(sender_username, False, True)
                    return True # History updated (with error message)

        # Store the successfully decrypted AES key for this peer
        self.chat_session_keys[sender_username] = {"key": aes_key_bytes}
        print(
            f"[CLIENT {self.username}] Successfully decrypted and stored AES session key
            for chat with {sender_username}.")

        # Step 2: AES-decrypt the initial message
        iv_b64 = initial_message_json.get("iv_b64")
        ciphertext_b64 = initial_message_json.get("ciphertext_b64")

        decrypted_initial_message_bytes = self.decrypt_aes(ciphertext_b64, iv_b64,

```

```

aes_key_bytes)

    if decrypted_initial_message_bytes:
        text_to_display = decrypted_initial_message_bytes.decode('utf-8')
        print(
            f"[CLIENT {self.username}] Successfully decrypted initial E2EE message
from {sender_username}: '{text_to_display}'")
        status = "received_e2ee_aes"
    else:
        text_to_display = "[E2EE Error: Failed to decrypt initial message]"
        print(
            f"[CLIENT_ERROR {self.username}] Failed to AES-decrypt initial message
from {sender_username} even after getting session key.")
        status = "failed_decrypt_initial_message"

    # Add the processed message to chat history
    self.chat_histories[sender_username].append({
        "sender": sender_username,
        "recipient": self.username,
        "text": text_to_display,
        "timestamp": timestamp,
        "is_group_message": False,
        "is_system_event": False,
        "is_encrypted": True, # It was an E2EE message
        "status": status
    })

    # If this is the currently active chat, refresh the display
    if self.current_chat_identifier == sender_username and not
self.current_chat_is_group:
        if hasattr(self, 'load_chat_history'):
            self.load_chat_history(sender_username, False, True) # is group=False,
scroll_to_bottom=True

    return True # History was updated, chat list might need refresh if it's a new
chat partner

except Exception as e:
    print(
        f"[CLIENT_ERROR {self.username}] Exception processing
RECEIVE_SECURE_SESSION_INIT from {sender_username}: {e}")
    # Store a generic error message in history
    self.chat_histories[sender_username].append({
        "sender": sender_username, "recipient": self.username,
        "text": "[E2EE Error: Critical failure processing initial secure message]",
        "timestamp": timestamp, "is_group_message": False, "is_system_event": True, #
Mark as system event
        "is_encrypted": True, "status": "failed_critical_session_init"
    })
    if self.current_chat_identifier == sender_username and not
self.current_chat_is_group:
        if hasattr(self, 'load_chat_history'): self.load_chat_history(sender_username,
False, True)
    return True # History updated (with error message)

def _handle_receive_aes_message(self, payload):
    """
    Handles a live, subsequent E2EE message from another user in an established AES
session.
    Uses the previously stored AES session key to decrypt the message.
    """
    sender_username = payload.get("sender_username")
    aes_payload_data = payload.get("aes_payload") # This is {'iv_b64': ...,
'ciphertext_b64': ...}
    timestamp = payload.get("timestamp", datetime.now().strftime('%Y-%m-%d
%H:%M:%S.%f')[:-3])

    print(f"[CLIENT {self.username}] Received live AES_MESSAGE from {sender_username}.")
    # print(f"[CLIENT {_self.username}] AES_MESSAGE Payload: {payload}") # Optional: for
more detailed debugging

    if not sender_username or not aes_payload_data or \
        not isinstance(aes_payload_data, dict) or \
        "iv_b64" not in aes_payload_data or "ciphertext_b64" not in aes_payload_data:
        print(f"[CLIENT_ERROR {self.username}] Invalid RECEIVE_AES_MESSAGE payload from
{sender_username}.")
        return False # No history update if payload is invalid

```

```

text_to_display = "[E2EE Error: Could not decrypt message]" # Default placeholder
is_decrypted_successfully = False
current_status = "failed_decrypt_aes" # Default status for failure

# Retrieve the AES session key for this sender
session = self.chat_session_keys.get(sender_username)
if not session or not session.get("key"):
    print(f"[CLIENT_ERROR {self.username}] No AES session key found for {sender_username} to decrypt message.")
    text_to_display = "[E2EE Error: Session key missing for this user]"
    # Note: Unlike session init, if the key is missing here, it's a more definite
error
    # as a session should have been established.
else:
    aes_key = session.get("key")
    iv_b64 = aes_payload_data.get("iv_b64")
    ciphertext_b64 = aes_payload_data.get("ciphertext_b64")

    try:
        decrypted_message_bytes = self.decrypt_aes(ciphertext_b64, iv_b64, aes_key)
        if decrypted_message_bytes:
            text_to_display = decrypted_message_bytes.decode('utf-8')
            is_decrypted_successfully = True
            current_status = "received_e2ee_aes"
            print(
                f"[CLIENT {self.username}] Successfully decrypted AES message from {sender_username}: '{text_to_display}'")
        else:
            print(f"[CLIENT_ERROR {self.username}] Failed to AES-decrypt message from {sender_username}.")
            text_to_display = "[E2EE Error: Message decryption failed]"
    except Exception as e:
        print(
            f"[CLIENT_ERROR {self.username}] Exception during AES decryption for message from {sender_username}: {e}")
        text_to_display = "[E2EE Error: Critical decryption error]"

    # Add the processed message to chat history
    self.chat_histories[sender_username].append({
        "sender": sender_username,
        "recipient": self.username,
        "text": text_to_display,
        "timestamp": timestamp,
        "is_group_message": False,
        "is_system_event": False,
        "is_encrypted": True, # It was an E2EE message
        "status": current_status
    })

    # If this is the currently active chat, refresh the display
    if self.current_chat_identifier == sender_username and not self.current_chat_is_group:
        if hasattr(self, 'load_chat_history'):
            self.load_chat_history(sender_username, False, True) # is_group=False,
scroll_to_bottom=True

    return True # History was updated, chat list might need refresh if it's a new chat
partner

def _handle_receive_wrapped_group_key(self, payload):
    """
    Handles receiving a wrapped (RSA-encrypted) group AES key.
    Decrypts the key using the client's private RSA key and stores it.
    """
    group_id = payload.get("group_id")
    wrapped_key_b64 = payload.get("wrapped_group_key_b64")
    distributor_username = payload.get("sender_username") # User who initiated the key
distribution
    timestamp = payload.get("timestamp", datetime.now().strftime('%Y-%m-%d
%H:%M:%S.%f')[:-3]) # Optional

    print(
        f"[CLIENT {self.username}] Received RECEIVE_WRAPPED_GROUP_KEY for GID {group_id}
from distributor {distributor_username}.")
    # print(f"[CLIENT {self.username}] Wrapped Key Payload: {payload}") # For more
detailed debugging if needed

```

```

        if group_id is None or not wrapped_key_b64:
            print(
                f"[CLIENT_ERROR {self.username}] Invalid RECEIVE_WRAPPED_GROUP_KEY payload: Missing group_id or wrapped_key_b64 for GID {group_id}.")
            messagebox.showerror("Group Key Error", f"Received incomplete key information for group {group_id}.")
            return False # No history update needed for invalid payload

        if not self.private_key:
            print(
                f"[CLIENT_ERROR {self.username}] Own private RSA key not available. Cannot decrypt group key for GID {group_id}.")
            messagebox.showerror("Security Error", "Your private key is not available. Cannot process group key.")
            return False

        try:
            wrapped_key_bytes = base64.b64decode(wrapped_key_b64)
            # Decrypt the group AES key using this client's private RSA key
            group_aes_key_bytes = self.decrypt_message(wrapped_key_bytes)

            if group_aes_key_bytes:
                # Store the decrypted key in memory and save it persistently
                self.store_and_save_group_key(group_id, group_aes_key_bytes)
                print(
                    f"[CLIENT {self.username}] Successfully decrypted and stored AES key for group {group_id}. Current in-memory group keys for: {list(self.group_aes_keys.keys())}")

                # If this group is the currently viewed chat, refresh its history
                # as messages might now become decryptable.
                if self.current_chat_identifier == group_id and self.current_chat_is_group:
                    print(f"[CLIENT {self.username}] Group {group_id} is current chat, reloading history.")
                    if hasattr(self, 'load_chat_history'):
                        self.load_chat_history(group_id, True, True) # is_group=True, scroll_to_bottom=True

                # Even if not current chat, other UI elements (like chat list unread counts or icons) might depend on this.
                # Returning True will trigger populate_chat_list if needed by the main check_queue loop.
                return True
            else:
                print(
                    f"[CLIENT_ERROR {self.username}] Failed to RSA-decrypt wrapped group key for GID {group_id} from distributor {distributor_username}.")
                messagebox.showerror("Group Key Decryption Error",
                    f"Failed to decrypt key for group {group_id}.\n\nThe key material might be corrupted or not intended for you.")
                return False
        except base64.binascii.Error as b64_error:
            print(
                f"[CLIENT_ERROR {self.username}] Base64 decode error for wrapped group key (GID {group_id}): {b64_error}")
            messagebox.showerror("Group Key Error", f"Received corrupted key data for group {group_id}.")
            return False
        except Exception as e:
            print(f"[CLIENT_ERROR {self.username}] Error processing wrapped group key for GID {group_id}: {e}")
            messagebox.showerror("Group Key Processing Error",
                f"An error occurred while processing the key for group {group_id}.")
            return False

    def _handle_receive_e2ee_group_message(self, payload):
        """
        Handles a live E2EE group message received from the server.
        If the message is from another user, it decrypts it using the stored group AES key.
        If the message is a broadcast of the current user's own sent message, it's ignored
        here
        as it was already added to history by send_message_event.
        """
        group_id = payload.get("group_id")
        sender_username = payload.get("sender_username") # User who originally sent the message
        aes_payload = payload.get("aes_payload") # This is {"iv_b64": ..., "ciphertext_b64":

```

```

...}
    timestamp = payload.get("timestamp", datetime.now().strftime('%Y-%m-%d
%H:%M:%S.%f')[:-3])
    # e2ee_type = payload.get("e2ee_type") # Server might send 'GROUP_AES' for context if
needed

    # --- Check if this is a broadcast of the current user's own message ---
    if sender_username == self.username:
        print(
            f"[CLIENT {self.username}] Ignored own broadcasted E2EE group message for GID
{group_id}. (Already added on send)")
        # No need to process further or add to history again.
        # No UI refresh is strictly needed from *this specific handler call* for this
ignored message,
        # as the UI was likely updated when the message was first sent.
        return False
        # --- End of check for own message ---

    print(
        f"[CLIENT {self.username}] Received live E2EE_GROUP_MESSAGE for GID {group_id}
from OTHER user: {sender_username}.")
    # print(f"[CLIENT {self.username}] Live E2EE Group Message Payload from
{sender_username}: {payload}") # For detailed debugging

    if group_id is None or not aes_payload or not sender_username or \
        not isinstance(aes_payload, dict) or \
        "iv_b64" not in aes_payload or "ciphertext_b64" not in aes_payload:
        print(
            f"[CLIENT_ERROR {self.username}] Invalid RECEIVE_E2EE_GROUP_MESSAGE payload
from {sender_username} for GID {group_id}.")
        return False # No history update for invalid payload

    text_to_display = "[Encrypted group message - Decryption error or key unavailable]" #
Default placeholder
    is_decrypted_successfully = False
    current_status = "failed_decrypt_group_aes" # Default status for failure

    group_aes_key = self.group_aes_keys.get(group_id)

    if not group_aes_key:
        print(
            f"[CLIENT_ERROR {self.username}] No AES key found for group {group_id} to
decrypt live message from {sender_username}.")
        text_to_display = "[Encrypted group message - Key unavailable to decrypt]"
    else:
        try:
            iv = aes_payload.get("iv_b64")
            ct = aes_payload.get("ciphertext_b64")

            if not iv or not ct:
                text_to_display = "[Group E2EE Malformed AES Payload (Live) - IV/CT
missing]"

            print(
                f"[CLIENT_ERROR {self.username}] Live group message GID {group_id}
from {sender_username} had missing IV or Ciphertext within aes_payload.")
            else:
                dec_bytes = self.decrypt_aes(ct, iv, group_aes_key)
                if dec_bytes:
                    text_to_display = dec_bytes.decode('utf-8')
                    is_decrypted_successfully = True
                    current_status = "received_e2ee_group_aes"
                    print(
                        f"[CLIENT {self.username}] Successfully decrypted live E2EE group
message from {sender_username} in GID {group_id}: '{text_to_display}'")
                else:
                    text_to_display = "[Group E2EE Decrypt Fail (Live)]"
                    print(
                        f"[CLIENT_ERROR {self.username}] AES decryption of live group
message from {sender_username} in GID {group_id} FAILED.")
            except Exception as e:
                text_to_display = "[Group E2EE Decrypt Error (Live)]"
                print(
                    f"[CLIENT_ERROR {self.username}] Exception decrypting live group message
GID {group_id} from {sender_username}: {e}")

        # Add the processed message to this client's local chat history
        # This part is now only executed for messages from OTHERS due to the check at the

```

```

beginning
    self.chat_histories[group_id].append({
        "sender_username": sender_username,
        "text": text_to_display,
        "timestamp": timestamp,
        "is_group_message": True,
        "group_id": group_id,
        "is_system_event": False,
        "is_encrypted": True,
        "status": current_status
    })

    # If this group is the currently active chat, refresh its display
    if self.current_chat_is_group and self.current_chat_identifier == group_id:
        if hasattr(self, 'load_chat_history'):
            self.load_chat_history(group_id, True, True)

    return True # History was updated (with a message from another user)
def _handle_receive_plain_group_message(self, payload): # Was RECEIVE_GROUP_MESSAGE
    gid = payload.get("group_id")
    sender_username = payload.get("sender_username")
    text = payload.get("text", "")
    ts = payload.get("timestamp")
    is_sys = payload.get("is_system_event", False)
    print(f"[CLIENT] Handling plain RECEIVE_GROUP_MESSAGE for GID {gid}")
    if gid is not None:
        status = "received" if sender_username != self.username else "sent"
        self.chat_histories[gid].append(
            {"sender_username": sender_username, "text": text, "timestamp": ts,
            "is_group_message": True,
            "group_id": gid, "is_system_event": is_sys, "is_encrypted": False, "status":
            status})
        if self.current_chat_is_group and self.current_chat_identifier == gid:
            self.load_chat_history(gid, True,
            True)

        return True
    return False

def _handle_message_stored_for_offline_delivery(self, payload):
    """
    Handles confirmation from the server that a message sent by this client
    to an offline user has been successfully stored by the server.
    """
    recipient_username = payload.get("recipient_username")
    timestamp_of_storage = payload.get("timestamp_of_storage") # Server's timestamp when
    it stored it
    message_preview = payload.get("message_preview", "[Message]") # Optional preview

    print(
        f"[CLIENT {self.username}] Received MESSAGE_STORED_FOR_OFFLINE_DELIVERY for
    recipient: {recipient_username}, Preview: '{message_preview}' at {timestamp_of_storage}")

    if not recipient_username:
        print(f"[CLIENT_ERROR {self.username}] MESSAGE_STORED_FOR_OFFLINE_DELIVERY missing
    recipient_username.")
        return False # No specific history update if recipient is unknown

    # Try to find the corresponding sent message in local history and update its status.
    # This is a bit heuristic as we don't have a unique ID echoed back in this specific
    message type.
    # We'll look for the most recent message sent by us to this recipient that might be
    pending server confirmation.
    # A common status for a message just sent by the client might be "sent_e2ee_aes",
    "sent_e2ee_group_aes", or just "sent".
    # We want to update it to something like "sent_to_server" or "delivered_to_server".

    chat_history_with_recipient = self.chat_histories.get(recipient_username)
    message_updated = False

    if chat_history_with_recipient:
        for i in range(len(chat_history_with_recipient) - 1, -1, -1): # Iterate backwards
            msg_data = chat_history_with_recipient[i]
            # Check if it's a message sent by the current user to this specific recipient
            is_private_chat_to_recipient = not msg_data.get("is_group_message") and \
            msg_data.get("sender") == self.username and \
            msg_data.get("recipient") == recipient_username

```

```

        # Define statuses that indicate the message was just sent from client and
        # awaits server ack
        # For 1-to-1 E2EE, these would be "sent_e2ee_aes"
        # For plain private messages, it might just be "sent" (if you have such a
        status)
        # We assume a status like "sent_e2ee_aes" or a generic "pending_server_ack"
        pending_statuses = ["sent_e2ee_aes",
                            "sent_encrypted"] # Add any other relevant client-side
        "just sent" statuses

        if is_private_chat_to_recipient and msg_data.get("status") in
        pending_statuses:
            # This is likely the message. Update its status.
            msg_data["status"] = "sent_to_server" # Or "delivered_to_server"
            print(
                f"[CLIENT {self.username}] Updated status of sent message to
{recipient_username} to 'sent_to_server'. Original text hint: '{msg_data.get('text',
'')[:30]}...'")
            message_updated = True
            break # Update only the most recent matching message

        if message_updated:
            # If the chat with this recipient is currently open, refresh its display
            if self.current_chat_identifier == recipient_username and not
            self.current_chat_is_group:
                if hasattr(self, 'load_chat_history'):
                    self.load_chat_history(recipient_username, False,
                                           True) # is_group=False, scroll_to_bottom=True (or
            False if you don't want to jump)
                return True # Indicates a potential UI update for message status

            return False # No specific chat history list refresh needed if no message status was
            updated
        def _handle_all_users_list(self, payload):
            print("[CLIENT] Handling ALL_USERS_LIST")
            self.all_users_list = payload.get("users", [])
            # Update UI if specific views are active
            if hasattr(self, 'new_chat_frame') and self.new_chat_frame.wininfo.ismapped():
                if hasattr(self, 'populate_all_users_list'): self.populate_all_users_list(
                    getattr(self.new_chat_search_entry, 'get', lambda: " ")())
                if hasattr(self, '_populate_create_group_member_list'):
                    self._populate_create_group_member_list()
            return False # Does not directly refresh main chat list

        def _handle_my_groups_list(self, payload):
            print("[CLIENT] Handling MY_GROUPS_LIST")
            self.my_groups = payload.get("groups", [])
            return True # Cause populate_chat_list

        def _handle_group_details_data(self, payload):
            """
            Handles receiving detailed information about a group from the server.
            Typically used to populate a group settings/info view.
            """
            group_id = payload.get("group_id")
            group_name = payload.get("group_name")
            creator_username = payload.get("creator_username")
            members = payload.get("members", []) # List of {'username': '...', 'role': '...'}
            group_icon_filename = payload.get("group_icon_filename")

            print(
                f"[CLIENT {self.username}] Received GROUP_DETAILS_DATA for GID {group_id}
('{group_name}'). Members: {len(members)}")

            if group_id is None:
                print(f"[CLIENT_ERROR {self.username}] GROUP_DETAILS_DATA received with missing
group_id.")
                return False

            self.last_group_details_data = payload # Store the latest details

            # If the group settings view is currently active for this group, refresh it
            # We check if the 'new_chat_frame' is active and if the title matches a settings view
            pattern.
            # This check might need to be more robust based on how you manage your views.
            if hasattr(self, 'new_chat_frame') and self.new_chat_frame.wininfo.ismapped() and \

```



```

        hasattr(self, 'title') and self.title().startswith(f"Settings: {group_name}")
and \
        hasattr(self, '_populate_group_settings_member_list'):

        print(
            f"[CLIENT {self.username}] Group settings view for GID {group_id} is active.
Refreshing member list and icon.")
        self._populate_group_settings_member_list(group_id, group_name, creator_username,
members)

        # Update the icon in the settings view
        if hasattr(self, 'group_settings_icon_display_label'):
            cache_key_settings = f"group_{group_id}_settings"
            if cache_key_settings in self.group_icon_cache:

self.group_settings_icon_display_label.configure(image=self.group_icon_cache[cache_key_setting
s])

            elif group_icon_filename:
                self.request_group_icon(group_id, group_icon_filename) # Request if not
cached

            elif self.default_group_icon: # Fallback to default group icon if available

self.group_settings_icon_display_label.configure(image=self.default_group_icon)
            # else: leave as is or use a more generic default

        # This handler usually doesn't require a full refresh of the main chat list,
        # unless a group name change or something similar also needs to be reflected there
immediately
        # (which is handled by GROUP_RENAMED_SUCCESS typically).
        return False

    def handle_group_create_success(self, payload):
        """
        Handles confirmation from the server that group creation was successful.
        The creator's client will now generate the group AES key and distribute it.
        """
        group_id = payload.get("group_id")
        group_name = payload.get("group_name")
        # Server should ideally send back the confirmed list of initial members, including the
creator.
        initial_members_usernames = payload.get("members", [])

        if group_id is None or not group_name:
            print(f"[CLIENT_ERROR {self.username}] GROUP_CREATE_SUCCESS payload missing
group_id or group_name.")
            messagebox.showerror("Group Creation Error", "Received incomplete confirmation
from server.")
            return False

        print(
            f"[CLIENT {self.username}] Successfully created group '{group_name}' (GID
{group_id}) with initial members: {initial_members_usernames}")
            messagebox.showinfo("Group Created", payload.get("message",
                f"Group '{group_name}' created
successfully! You will now set up its secure channel.))

        # --- CREATOR generates and distributes the group AES key ---
        # Ensure the current user (creator) is in the initial_members_usernames list
        if self.username not in initial_members_usernames:
            initial_members_usernames.append(self.username) # Creator must be a member

        print(
            f"[CREATOR_CLIENT GID {group_id}] Now generating and distributing initial group
AES key to members: {initial_members_usernames}...")

        group_aes_key = self.generate_aes_key() # Generate a new AES key for this group

        if not group_aes_key:
            print(f"[CREATOR_CLIENT GID {group_id}] CRITICAL ERROR: Failed to generate group
AES key.")
            messagebox.showerror("Group Encryption Setup Failed",
                f"Could not generate secure key for group '{group_name}'.")
            # Requesting MY_GROUPS anyway to update the list, but E2EE will fail for this
group.

            self.send_message("GET_MY_GROUPS", {})
            return True # history needs update to show the new group (even if E2EE setup
failed)

```

```

# 1. Store and save the key for the creator immediately
self._store_and_save_group_key(group_id, group_aes_key)
print(
    f"[CREATOR_CLIENT GID {group_id}] Generated and stored own group AES key. In-
memory keys for groups: {list(self.group_aes_keys.keys())}")

# 2. Distribute to all members (including self for consistent handling and testing of
the receive path)
for member_username in initial_members_usernames:
    member_public_key_pem = None
    if member_username == self.username:
        member_public_key_pem = self.get_public_key_pem() # Creator uses their own
loaded public key
    else:
        member_public_key_pem = self.user_public_keys_cache.get(member_username)

    if not member_public_key_pem:
        print(
            f"[CREATOR_CLIENT GID {group_id}] WARNING: Public RSA key for member
'{member_username}' not in local cache. Attempting to request it for key distribution.")
        self.request_other_user_public_key(
            member_username) # Request it, but it won't be available for this
iteration
        continue # Skip wrapping for this member for now

    try:
        print(f"[CREATOR_CLIENT GID {group_id}] Wrapping group AES key for member:
{member_username}")
        wrapped_key_bytes = self.encrypt_message(group_aes_key, member_public_key_pem)
# RSA encrypt
        if wrapped key bytes:
            wrapped_key_b64 = base64.b64encode(wrapped_key_bytes).decode('utf-8')
            key_dist_payload = {
                "group_id": group_id,
                "target_username": member_username,
                "wrapped_group_key_b64": wrapped_key_b64,
                "distributor_username": self.username # Let recipient know who sent
the key
            }
            print(
                f"[CREATOR_CLIENT GID {group_id}] Sending wrapped group key to server
for member: {member_username}")
            self.send_message("DISTRIBUTE_GROUP_KEY", key_dist_payload)
        else:
            print(
                f"[CREATOR_CLIENT GID {group_id}] ERROR: Failed to RSA-encrypt group
key for {member_username}.")
            messagebox.showerror("Key Wrapping Error",
                f"Could not prepare secure key for member
{member_username}.")
            except Exception as e_wrap:
                print(
                    f"[CREATOR_CLIENT GID {group_id}] ERROR: Exception during key wrapping for
{member_username}: {e_wrap}")
                messagebox.showerror("Key Wrapping Exception", f"Error preparing key for
{member_username}: {e_wrap}")

            print(f"[CREATOR_CLIENT GID {group_id}] Finished initiating group key distribution to
{len(initial_members_usernames)} members.")

# Refresh the user's list of groups & potentially switch to the new group view
self.send_message("GET_MY_GROUPS", {})

# Optionally, automatically select and open the new group chat
# if hasattr(self, 'show_chat_screen'): self.show_chat_screen() # Go to main chat
screen
# if hasattr(self, 'select_chat'): self.select_chat(group_id, True, group_name) # Then
select the new group

return True # history_needs_update will be True because GET_MY_GROUPS will update the
list

def _handle_group_operation_fail(self, payload, original_msg_type="GROUP_OPERATION_FAIL"):
# Generic handler for various group operation failures
message = payload.get("message", f"{original_msg_type.replace(' ', ' ')} failed.")
print(f"[CLIENT] Handling {original_msg_type}: {message}")

```

```

        messagebox.showerror(original_msg_type.replace('_', ' ').title(), message)
        return False

    # Implement other specific success handlers similarly, returning True if they affect
    the main chat list

    def _handle_add_members_success(self, payload):
        # ... return False (or True if group list needs refresh based on new member count
        display)
        print(f"[CLIENT] Handling ADD MEMBERS SUCCESS for GID {payload.get('group_id')}")
        messagebox.showinfo("Add Members", payload.get("message", "Members processed."))
        # Potentially refresh group details if that view is open
        if hasattr(self, 'new_chat_frame') and self.new_chat_frame.wininfo_ismapped() and \
            self.last_group_details_data and self.last_group_details_data.get("group_id")
        == payload.get(
            "group_id"):
            self.send_message("GET_GROUP_DETAILS", {"group_id": payload.get("group_id")})
            return False

    def _handle_remove_member_success(self, payload):
        group_id_rem = payload.get("group_id")
        removed_username = payload.get("removed_username")
        message = payload.get("message", f"User {removed_username} removed from group
        {group_id_rem}.")
        print(
            f"[CLIENT {self.username}] Handling REMOVE MEMBER SUCCESS for GID {group_id_rem},
            removed: {removed_username}")
        messagebox.showinfo("Member Removed", message)

        history_updated = False
        if removed_username == self.username: # Current user was removed
            messagebox.showwarning("Removed from Group",
                f"You were removed from group '{payload.get('group_name',
                group_id_rem)}'.")
            self.my_groups = [g for g in self.my_groups if g.get("group_id") != group_id_rem]
            if group_id_rem in self.chat_histories: del self.chat_histories[group_id_rem]
            # Clear relevant caches
            if f"group_{group_id_rem}_list" in self.group_icon_cache: del
            self.group_icon_cache[
                f"group_{group_id_rem}_list"]
            if f"group_{group_id_rem}_settings" in self.group_icon_cache: del
            self.group_icon_cache[
                f"group_{group_id_rem}_settings"]
            if group_id_rem in self.group_aes_keys: del self.group_aes_keys[group_id_rem];
            self._save_group_aes_keys()

            if self.current_chat_identifier == group_id_rem and self.current_chat_is_group:
                self.current_chat_identifier = None
                self.current_chat_is_group = False
                if hasattr(self, 'show_chat_screen'): self.show_chat_screen() # Go back to
                default chat screen
            history_updated = True
            elif hasattr(self, 'new_chat_frame') and self.new_chat_frame.wininfo_ismapped() and \
                self.last_group_details_data and self.last_group_details_data.get("group_id")
        == group_id_rem:
            # If current user is admin and viewing settings for this group, refresh member
            list
            self.send_message("GET_GROUP_DETAILS", {"group_id": group_id_rem})

            return history_updated

    def _handle_member_role_changed(self, payload):
        group_id_role = payload.get("group_id")
        target_username = payload.get("username")
        new_role = payload.get("new_role")
        message = payload.get("message", f"Role of {target_username} in group {group_id_role}
        changed to {new_role}.")
        print(
            f"[CLIENT {self.username}] Handling MEMBER_ROLE_CHANGED for GID {group_id_role},
            User: {target_username}, New Role: {new_role}")
        messagebox.showinfo("Member Role Changed", message)

        # If group settings view is open for this group, refresh its member list
        if hasattr(self, 'new_chat_frame') and self.new_chat_frame.wininfo_ismapped() and \
            self.last_group_details_data and self.last_group_details_data.get("group_id")
        == group_id_role:
            self.send_message("GET_GROUP_DETAILS", {"group_id": group_id_role})

```

```

        return False # Usually doesn't require main chat list refresh

    def _handle_group_renamed_success(self, payload):
        group_id_ren = payload.get("group_id")
        new_group_name = payload.get("new_group_name")
        message = payload.get("message", f"Group renamed to '{new_group_name}'")
        print(f"[CLIENT {self.username}] Handling GROUP_RENAMED_SUCCESS for GID {group_id_ren}
to '{new_group_name}'")
        messagebox.showinfo("Group Renamed", message)

        updated = False
        for g_info in self.my_groups:
            if g_info.get("group_id") == group_id_ren:
                g_info["group_name"] = new_group_name
                updated = True
                break

        if self.current_chat_is_group and self.current_chat_identifier == group_id_ren:
            self.current_chat_group_name = new_group_name
            # select chat will update header and reload history if needed
            if hasattr(self, 'select chat'): self.select chat(group_id_ren, True,
new_group_name)
            updated = True # Current chat header changed

        # If group settings view is open for this group, update its title and refresh
        if hasattr(self, 'new chat frame') and self.new chat frame.wininfo.ismapped() and \
            self.last_group_details_data and self.last_group_details_data.get("group_id")
== group_id_ren:
            if hasattr(self, 'title') and self.title().startswith("Settings:"): self.title(
f"Settings: {new_group_name}")
            self.send_message("GET_GROUP_DETAILS", {"group_id": group_id_ren}) # Refresh
details

        return updated # True if group name changed in my_groups or current chat

    def _handle_leave_group_success(self, payload):
        group_id_left = payload.get("group_id")
        group_name_left = payload.get("group_name", f"ID {group_id_left}")
        message = payload.get("message", f"You left group '{group_name_left}'.")
        print(f"[CLIENT {self.username}] Handling LEAVE_GROUP_SUCCESS for GID
{group_id_left}")
        messagebox.showinfo("Left Group", message)

        self.my_groups = [g for g in self.my_groups if g.get("group_id") != group_id_left]
        if group_id_left in self.chat_histories: del self.chat_histories[group_id_left]
        # Clear relevant caches
        if f"group_{group_id_left}_list" in self.group_icon_cache: del self.group_icon_cache[
f"group_{group_id_left}_list"]
        if f"group_{group_id_left}_settings" in self.group_icon_cache: del
self.group_icon_cache[
f"group_{group_id_left}_settings"]
        if group_id_left in self.group_aes_keys: del self.group_aes_keys[group_id_left];
self._save_group_aes_keys()

        if self.current_chat_identifier == group_id_left and self.current_chat_is_group:
            self.current_chat_identifier = None
            self.current_chat_is_group = False
            if hasattr(self, 'show_chat_screen'): self.show_chat_screen() # Go back to
default

        return True # my_groups changed, requires chat list refres

    def _handle_group_renamed_success(self, payload):
        group_id_ren = payload.get("group_id")
        new_group_name = payload.get("new_group_name")
        message = payload.get("message", f"Group renamed to '{new_group_name}'")
        print(f"[CLIENT {self.username}] Handling GROUP_RENAMED_SUCCESS for GID {group_id_ren}
to '{new_group_name}'")
        messagebox.showinfo("Group Renamed", message)

        updated = False
        for g_info in self.my_groups:
            if g_info.get("group_id") == group_id_ren:
                g_info["group_name"] = new_group_name
                updated = True
                break

```

```

        if self.current_chat_is_group and self.current_chat_identifier == group_id_ren:
            self.current_chat_group_name = new_group_name
            # select_chat will update header and reload history if needed
            if hasattr(self, 'select_chat'): self.select_chat(group_id_ren, True,
new_group_name)
            updated = True # Current chat header changed

        # If group settings view is open for this group, update its title and refresh
        if hasattr(self, 'new_chat_frame') and self.new_chat_frame.wininfo_ismapped() and \
            self.last_group_details_data and self.last_group_details_data.get("group_id")
== group_id_ren:
            if hasattr(self, 'title') and self.title().startswith("Settings:"): self.title(
                f"Settings: {new_group_name}")
            self.send_message("GET_GROUP_DETAILS", {"group_id": group_id_ren}) # Refresh
details

        return updated # True if group name changed in my_groups or current chat

    def _handle_leave_group_success(self, payload):
        group_id_left = payload.get("group_id")
        group_name_left = payload.get("group name", f"ID {group_id_left}")
        message = payload.get("message", f"You left group '{group_name_left}'.")
        print(f"[CLIENT {self.username}] Handling LEAVE_GROUP_SUCCESS for GID
{group_id_left}")
        messagebox.showinfo("Left Group", message)

        self.my_groups = [g for g in self.my_groups if g.get("group_id") != group_id_left]
        if group_id_left in self.chat_histories: del self.chat_histories[group_id_left]
        # Clear relevant caches
        if f"group_{group_id_left}_list" in self.group_icon_cache: del self.group_icon_cache[
            f"group_{group_id_left}_list"]
        if f"group {group_id_left} settings" in self.group icon cache: del
self.group_icon_cache[
            f"group_{group_id_left} settings"]
        if group_id_left in self.group_aes_keys: del self.group_aes_keys[group_id_left];
self._save_group_aes_keys()

        if self.current_chat_identifier == group_id_left and self.current_chat_is_group:
            self.current_chat_identifier = None
            self.current_chat_is_group = False
            if hasattr(self, 'show_chat_screen'): self.show_chat_screen() # Go back to
default

        return True # my_groups changed, requires chat list refresh

    # Picture Handling

    def _handle_profile_picture_upload_status(self, payload):
        success = payload.get("success")
        message = payload.get("message", "Profile picture update status unknown.")
        print(f"[CLIENT {self.username}] Handling PROFILE_PICTURE_UPLOAD_STATUS. Success:
{success}")

        if success:
            messagebox.showinfo("Profile Picture Upload", message)
            # Clear cache for own profile picture to force a refresh if viewed
            if self.username in self.profile_picture_cache:
                del self.profile_picture_cache[self.username]

            # If current chat is with self (e.g. "Notes to self" or if header shows own pic),
refresh it
            if self.current_chat_identifier == self.username and not
self.current_chat_is_group:
                if hasattr(self, 'chat_header_image_label') and self.chat_header_image_label:
                    self.chat_header_image_label.configure(image=self.default_profile_image,
text="") # Reset
                if hasattr(self, 'request_profile_picture'):
                    self.request_profile_picture(self.username) # Re-fetch
                # If profile pictures are shown in the chat list itself, then a refresh is needed.
                # For now, let's assume it doesn't directly change the chat list items enough for
a full refresh.
                return False # Or True if your chat list items display user PFPs that need
refreshing
            else:
                messagebox.showerror("Profile Picture Upload Failed", message)
                return False

```

```
def _handle_profile_picture_data(self, payload):
    # This is the original logic from your client.py's _handle_profile_picture_data
    data_uname = payload.get("username")
    fname = payload.get("filename") # original filename from server if any
    img_b64 = payload.get("image_data_base64")
    error_message = payload.get("message") # Server might send an error message

    print(
        f"[CLIENT {self.username}] Handling PROFILE_PICTURE_DATA for user: {data_uname}.
Image data present: {img_b64 is not None}")

    if error_message and not img_b64: # Server explicitly sent an error
        print(f"[CLIENT_ERROR {self.username}] Server sent error for PFP data of
{data_uname}: {error_message}")
        # Optionally, show a subtle error or just use default. No messagebox here to avoid
spam.

        if data_uname in self.profile_picture_cache:
            del self.profile_picture_cache[data_uname] # Remove potentially outdated
cache

        # If this is the current chat header, ensure it shows default
        if data_uname == self.current_chat_identifier and not self.current_chat_is_group
and \
            hasattr(self, 'chat_header_image_label') and self.default_profile_image:
            self.chat_header_image_label.configure(image=self.default_profile_image,
text="")

        return True # Chat list might need update if it shows PFP placeholders

    cache_updated = False
    if img_b64:
        try:
            img_bytes = base64.b64decode(img_b64)
            pil_img = Image.open(io.BytesIO(img_bytes))

            # Cache for header size
            header_img_resized = pil_img.copy().resize(PROFILE_PIC_HEADER_SIZE,
Image.Resampling.LANCZOS)
            self.profile_picture_cache[data_uname] =
ctk.CTkImage(light_image=header_img_resized,
dark_image=header_img_resized,
size=PROFILE_PIC_HEADER_SIZE)

            # Cache for list size (if you use a different size for chat list items)
            list_img_resized = pil_img.copy().resize(PROFILE_PIC_LIST_SIZE,
Image.Resampling.LANCZOS)
            self.profile_picture_cache[f"{data_uname}_list"] =
ctk.CTkImage(light_image=list_img_resized,
dark_image=list_img_resized,
size=PROFILE_PIC_LIST_SIZE)

            cache_updated = True
            print(f"[CLIENT {self.username}] Cached profile picture for {data_uname}.")
        except Exception as e:
            print(f"[CLIENT_ERROR {self.username}] Error processing/caching PFP for
{data_uname}: {e}")
            if data_uname in self.profile_picture_cache: del
self.profile_picture_cache[data_uname]
            if f"{data_uname}_list" in self.profile_picture_cache: del
self.profile_picture_cache[
                f"{data_uname}_list"]
            else: # No image data received, ensure cache is cleared
                if data_uname in self.profile_picture_cache:
                    del self.profile_picture_cache[data_uname];
                    cache_updated = True
                if f"{data_uname}_list" in self.profile_picture_cache:
                    del self.profile_picture_cache[f"{data_uname}_list"];
                    cache_updated = True

            # Update UI if this PFP is for the currently selected chat header
            if data_uname == self.current_chat_identifier and not self.current_chat_is_group:
                if hasattr(self, 'load_and_display_profile_picture'): # This method handles
header
                    self.load_and_display_profile_picture(data_uname, img_b64, fname) # fname is
more like a hint here
```

```

        return cache_updated # True if cache changed, might trigger populate_chat_list

    def _handle_group_picture_upload_status(self, payload):
        success = payload.get("success")
        message = payload.get("message", "Group icon update status unknown.")
        gid_gpik = payload.get("group_id")
        new_filename = payload.get("new_filename") # Server might send back the stored
filename
        print(f"[CLIENT {self.username}] Handling GROUP_PICTURE_UPLOAD_STATUS for GID
{gid_gpik}. Success: {success}")

        if success:
            messagebox.showinfo("Group Icon Upload", message)
            if gid_gpik is not None:
                # Clear local cache for this group icon to force a refresh
                if f"group_{gid_gpik}_list" in self.group_icon_cache: del
self.group_icon_cache[
                    f"group_{gid_gpik}_list"]
                if f"group_{gid_gpik}_settings" in self.group_icon_cache: del
self.group icon cache[
                    f"group_{gid_gpik}_settings"]

                # If group settings view is open for this group, refresh it by fetching
details again
                if hasattr(self, 'new chat frame') and self.new chat frame.wininfo ismapped()
and \
                    self.last_group_details_data and
self.last_group_details_data.get("group_id") == gid_gpik:
                    self.send_message("GET_GROUP_DETAILS", {"group_id": gid_gpik})
                    # Also refresh current chat header if it's this group
                    elif self.current chat identifier == gid_gpik and self.current chat is group:
                        if hasattr(self, 'select_chat'): self.select_chat(gid_gpik, True,
self.current_chat_group_name) # Re-select to refresh icon
                        return True # Icon changed, chat list likely needs refresh
            else:
                messagebox.showerror("Group Icon Upload Failed", message)
                return False

    def _handle_group_icon_data(self, payload):
        # This is the original logic from your client.py's _handle_group_icon_data
        gid_icon = payload.get("group_id")
        fname_icon = payload.get("filename") # The actual filename stored on server
        img_b64_icon = payload.get("image_data_base64")
        error_message = payload.get("message")
        print(
            f"[CLIENT {self.username}] Handling GROUP_ICON_DATA for GID {gid_icon}. Image data
present: {img_b64_icon is not None}")

        if error_message and not img_b64_icon:
            print(
                f"[CLIENT_ERROR {self.username}] Server sent error for group icon data of GID
{gid_icon}: {error_message}")
            # Clear cache if error
            if f"group_{gid_icon}_list" in self.group_icon_cache: del
self.group_icon_cache[f"group_{gid_icon}_list"]
            if f"group_{gid_icon}_settings" in self.group_icon_cache: del
self.group_icon_cache[
                f"group_{gid_icon}_settings"]
            return True # To refresh list possibly showing old/default icon

        cache_updated = False
        if img_b64_icon and gid_icon is not None:
            try:
                img_bytes = base64.b64decode(img_b64_icon)
                pil_img = Image.open(io.BytesIO(img_bytes))

                # Cache for list size
                res_list_pil = pil_img.copy().resize(PROFILE_PIC_LIST_SIZE,
                                                    Image.Resampling.LANCZOS) # Use
PROFILE_PIC_LIST_SIZE for consistency in lists
                self.group_icon_cache[f"group_{gid_icon}_list"] =
                ctk.CTkImage(light_image=res_list_pil,
dark image=res list pil,

```

```

size=PROFILE_PIC_LIST_SIZE)

        # Cache for settings view size
        res_settings_pil = pil_img.copy().resize(GROUP_ICON_SETTINGS_SIZE,
Image.Resampling.LANCZOS)
        self.group_icon_cache[f"group_{gid_icon}_settings"] =
        ctk.CTkImage(light_image=res_settings_pil,

dark_image=res_settings_pil,

size=GROUP_ICON_SETTINGS_SIZE)

        cache_updated = True
        print(f"[CLIENT {self.username}] Cached group icon for GID {gid_icon}.")

        # Update group settings view if it's currently showing this group
        if hasattr(self, 'group_settings_icon_display_label') and \
            hasattr(self, 'new_chat_frame') and \
self.new_chat_frame.winfo_ismapped() and \
            self.last_group_details_data and
self.last_group_details_data.get("group_id") == gid_icon:
            self.group_settings_icon_display_label.configure(
                image=self.group_icon_cache[f"group_{gid_icon}_settings"])

        # Update chat header if this group is the current chat
        if self.current_chat_identifier == gid_icon and self.current_chat_is_group and \
            hasattr(self, 'chat_header_image_label'):
            self.chat_header_image_label.configure(
                image=self.group_icon_cache[f"group_{gid_icon}_list"]) # Or a header-
specific size if different

        except Exception as e:
            print(f"[CLIENT_ERROR {self.username}] Error processing/caching group icon for
GID {gid_icon}: {e}")
            if f"group_{gid_icon}_list" in self.group_icon_cache: del
self.group_icon_cache[
                f"group_{gid_icon}_list"]
            if f"group_{gid_icon}_settings" in self.group_icon_cache: del
self.group_icon_cache[
                f"group_{gid_icon}_settings"]
        else: # No image data, clear cache
            if f"group_{gid_icon}_list" in self.group_icon_cache:
                del self.group_icon_cache[f"group_{gid_icon}_list"];
                cache_updated = True
            if f"group_{gid_icon}_settings" in self.group_icon_cache:
                del self.group_icon_cache[f"group_{gid_icon}_settings"];
                cache_updated = True

        return cache_updated # True if cache changed, might trigger populate_chat_list

# --- Generic Server Message Handler ---
def _handle_generic_server_message(self, payload, original_msg_type): # Now takes
original_msg_type
    message_text = payload.get("message", payload.get("text", "Unknown server
notification."))
    # Use original_msg_type for a more accurate title
    title = f"Server Info: {original_msg_type.replace('_', ' ').title()}"

    print(f"[CLIENT {self.username}] Generic Server Msg: Type '{original_msg_type}', Msg:
'{message_text}'")

    is_error_type = "error" in original_msg_type.lower() or \
        "_FAIL" in original_msg_type.upper() or \
        original_msg_type == "AUTH_REQUIRED" or \
        original_msg_type == "SERVER_ERROR" # Standardized check

    if is_error_type:
        messagebox.showerror(title, message_text)
    else:
        messagebox.showinfo(title, message_text)
    return False # These usually don't require a full chat list refresh

def on_closing(self):

```



```

global gui_running
if messagebox.askokcancel("Quit", "Are you sure you want to quit?"):
    gui_running = False;
    print("[CLIENT] Closing application...")
    if self.client_socket:
        try:
            self.client_socket.shutdown(socket.SHUT_RDWR)
        except OSError:
            pass # Socket might already be closed
        except Exception as e:
            print(f"Error during socket shutdown: {e}")
        finally:
            try:
                self.client_socket.close()
            except:
                pass
            self.client_socket = None;
            print("[CLIENT] Socket closed.")
    self.destroy()

def select_chat(self, identifier, is_group=False, group_name_for_header=None):
    print(
        f"[CLIENT SELECT CHAT] Selecting: ID='{identifier}', Group={is_group}. Previous
key status for this ID (if same): {self.current_chat_key_status if
self.current_chat_identifier == identifier else 'N/A'}")

    # Only reset key_status to "unknown" if we are selecting a *different* chat
    if self.current_chat_identifier != identifier: # Or different type (group vs private)
        self.current_chat_key_status = "unknown"
    print(
        f"[CLIENT SELECT CHAT] Switched to new chat '{identifier}', resetting its key
status to unknown initially.")

    self.current_chat_identifier = identifier
    self.current_chat_is_group = is_group
    self.current_chat_group_name = group_name_for_header if is_group else None

    header_text = ""
    header_status_suffix = ""

    # --- Reset UI elements for the new chat view ---
    if hasattr(self, 'chat_header_image_label') and self.chat_header_image_label:
        if not self.default_profile_image and hasattr(self,
'_create_default_profile_image'):
            self._create_default_profile_image()
            self.chat_header_image_label.configure(image=self.default_profile_image, text="")

    if hasattr(self, 'chat_header_actions_frame') and self.chat_header_actions_frame:
        for widget in self.chat_header_actions_frame.winfo_children():
            widget.destroy()
    # --- End of UI element reset ---

    if is_group:
        header_text = self.current_chat_group_name if self.current_chat_group_name else
f"Group ({identifier})"
        group_icon_filename = None
        group_id_for_icon = identifier # identifier is group_id for groups
        for g_info in self.my_groups:
            if g_info.get("group_id") == group_id_for_icon:
                group_icon_filename = g_info.get("group_icon_filename")
                # If header_text was generic, try to get specific name
                if not self.current_chat_group_name and g_info.get("group_name"):
                    header_text = g_info.get("group_name")
                    self.current_chat_group_name = header_text # Update if we found a
more specific one
                break

        cache_key_header = f"group_{group_id_for_icon}_list" # Using list size for header
too, or define a header cache key
        if hasattr(self, 'chat_header_image_label') and self.chat_header_image_label:
            if cache_key_header in self.group_icon_cache:
self.chat_header_image_label.configure(image=self.group_icon_cache[cache_key_header])
            elif group_icon_filename and hasattr(self, 'request_group_icon'):
                self.request_group_icon(group_id_for_icon, group_icon_filename)
            elif self.default_group_icon:

```

```

        self.chat_header_image_label.configure(image=self.default_group_icon)
    elif self.default_profile_image: # Fallback
        self.chat_header_image_label.configure(image=self.default_profile_image)

    if hasattr(self, 'chat_header_actions_frame'):
        ctk.CTkButton(self.chat_header_actions_frame, text="Group Settings",
width=120,
                        command=lambda gid=identifier, gname=header_text:
self.show_group_settings_view(gid,
gname)).pack(
                        side="right", padx=5)
        self.current_chat_key_status = "not_applicable_group" # E2EE key status for group
handled by group_aes_keys

    else: # Private chat with another user (identifier is username)
        header_text = identifier
        if hasattr(self, 'request_profile_picture'): self.request_profile_picture(
            identifier) # This will update header via its handler

        if self.public key and identifier != self.username:
            if identifier in self.user_public_keys_cache:
                self.current_chat_key_status = "available"
                header_status_suffix = " (🔒 Encrypted)"
            elif self.current_chat_key_status == "failed": # Status set by
_handle_user_public_key_fail
                header_status_suffix = " (🔒 Key Unavailable)"
            elif self.current_chat_key_status == "fetching": # Status set by this method
below if not cached
                header_status_suffix = " (🔄 Fetching Key...)"
            else: # Status is "unknown", initiate fetch
                self.current_chat_key_status = "fetching"
                header_status_suffix = " (🔄 Fetching Key...)"
            if hasattr(self, 'request_other_user_public_key'):
                self.request_other_user_public_key(identifier)
            elif identifier == self.username:
                self.current_chat_key_status = "not_applicable_self"
                header_status_suffix = " (Self)"
            else:
                self.current_chat_key_status = "failed_client_key_error"
                header_status_suffix = " (🔒 Client Key Error / Unencrypted)"

        if hasattr(self, 'current_chat_label') and self.current_chat_label:
            self.current_chat_label.configure(text=header_text + header_status_suffix)

        if hasattr(self, '_update_chat_input_state'): self._update_chat_input_state()

        if hasattr(self, 'load_chat_history'):
            self.messages_currently_shown_count[identifier] =
DEFAULT_MESSAGES_TO_LOAD_INITIALLY
            self.is_loading_more_messages = False
            # Explicitly pass preserve_scroll_info=None for a new chat selection
            self.load_chat_history(identifier, is_group=is_group, scroll_to_bottom=True,
preserve_scroll_info=None)

            if hasattr(self, 'populate_chat_list'): self.populate_chat_list() # Refresh chat list
to show selection
            print(
                f"[CLIENT SELECT_CHAT] Finished selecting '{identifier}'. Current key status:
{self.current_chat_key_status}")

            def load_chat_history(self, identifier, is_group=False, scroll_to_bottom=False,
preserve_scroll_info=None):
                if not hasattr(self, 'message_display_frame') or \
                    not self.message_display_frame or \
                    not self.message_display_frame.winfo_exists() or \
                    identifier is None:
                    print(
                        f"[LOAD_HISTORY DEBUG] Called but message_display_frame not ready or
identifier ({identifier}) is None.")
                    return

                print(
                    f"[LOAD_HISTORY DEBUG] Loading for Identifier: {identifier}, Is Group: {is_group},
ScrollToBottom: {scroll_to_bottom}, Preserve: {preserve_scroll_info is not None}")

```

```

all_messages_for_identifier = self.chat_histories.get(identifier, [])

old_top_message_data_for_scroll = None
# Only use preserve_scroll_info if scroll_to_bottom is False
if not scroll_to_bottom and preserve_scroll_info and
preserve_scroll_info.get("first_visible_message_data"):
    old_top_message_data_for_scroll =
preserve_scroll_info["first_visible_message_data"]
    print(
        f"[LOAD HISTORY DEBUG] Scroll preservation active. Target message data
(local_id if exists): {old_top_message_data_for_scroll.get('local_id', 'N/A')}")

    for widget in self.message_display_frame.winfo_children():
        widget.destroy()

    num_to_display = self.messages_currently_shown_count.get(identifier,
DEFAULT_MESSAGES_TO_LOAD_INITIALLY)
    num_to_display = min(num_to_display, len(all_messages_for_identifier))
    messages_to_display = all_messages_for_identifier[-num_to_display:]

    self.chat_all_older_loaded[identifier] = (num_to_display >=
len(all_messages_for_identifier))

    last_date_displayed = None
    self.message_display_frame.update_idletasks()
    available_width = self.message_display_frame.winfo_width() - \
        (getattr(self.message_display_frame, '_scrollbar_width', 20) if
hasattr(
            self.message_display_frame, '_scrollbar_width') else 20) - 20 #
Padding

    newly_rendered_widgets = [] # To find target widget for scroll preservation

    self.message_display_frame.update_idletasks() # Ensure all new widgets are drawn and
have geometry

    if scroll_to_bottom:
        if hasattr(self, 'scroll_chat_to_bottom_delayed'):
            print(
                f"[LOAD HISTORY DEBUG] Calling scroll_chat_to_bottom_delayed for
{identifier} (scroll_to_bottom=True)")
            self.scroll_chat_to_bottom_delayed(delay=50)
        elif old_top_message_data_for_scroll:
            print(
                f"[LOAD HISTORY DEBUG] Attempting to preserve scroll for {identifier}. Target
msg local_id: {old_top_message_data_for_scroll.get('local_id', 'N/A')}")
            target_widget_to_scroll_to = None
            for widget_iter in newly_rendered_widgets:
                if hasattr(widget_iter, '_message_data_for_scroll') and \
                    getattr(widget_iter, '_message_data_for_scroll') ==
old_top_message_data_for_scroll:
                    target_widget_to_scroll_to = widget_iter
                    print(f"[LOAD HISTORY DEBUG] Found target widget for scroll
preservation.")
                    break

            if target_widget_to_scroll_to:
                # Using a closure to correctly capture target_widget for the self.after call
                def
_do_preserve_scroll_to_widget_closure(target_widget=target_widget_to_scroll_to):
                    if hasattr(self, 'message_display_frame') and self.message_display_frame
and \
                        self.message_display_frame.winfo_exists() and
target_widget.winfo_exists() and \
                            hasattr(self.message_display_frame, '_parent_canvas'):
                                self.message_display_frame.update_idletasks()

                                bbox_all = self.message_display_frame._parent_canvas.bbox("all")
                                if bbox_all:

self.message_display_frame._parent_canvas.configure(scrollregion=bbox_all)

                                widget_y_abs = target_widget.winfo_y() # y-coordinate relative to
parent (the scrollable
frame's inner frame)
                                canvas_height =

```

```

self.message_display_frame._parent_canvas.winfo_height()
    content_start_y_offset = bbox_all[1] if bbox_all else 0
    content_height = (bbox_all[3] - bbox_all[1]) if bbox_all else
canvas_height

    if content_height > 0:
viewport.

        scroll_fraction = (
            widget_y_abs +
content_start_y_offset) / content_height if content_height > canvas_height else 0.0
        scroll_fraction = max(0.0, min(scroll_fraction, 1.0 - (
            canvas_height / content_height if content_height > 0 else
1.0)))

        scroll_fraction_yview = 0.0
        if content_height > 0:
            # scroll_fraction_yview = (widget_y_abs +
content_start_y_offset) / content_height
            # Let's use the widget's position directly, assuming it's
within the scrollable content.
            # The `yview moveto` takes a fraction from 0.0 to 1.0.
            # We want the top of the `target_widget` to be at the top of
the view.
            scroll_fraction_yview = (
                target_widget.winfo_y() +
self.message_display_frame._parent_canvas.canvassy(
                0)) / content_height if
content_height > 0 else 0.0

            target_y_in_scroll_content = target_widget.winfo_y()
            current_scroll_region =
self.message_display_frame._parent_canvas.cget("scrollregion")
            if current_scroll_region:
                _, sr_y1, _, sr_y2 = map(float, current_scroll_region.split())
                scrollable_height = sr_y2 - sr_y1
                if scrollable_height > 0:
                    fraction = (target_y_in_scroll_content - sr_y1) /
scrollable_height
                    fraction = max(0.0, min(fraction, 1.0))

self.message_display_frame._parent_canvas.yview_moveto(fraction)
            print(
                f"[LOAD_HISTORY DEBUG] Preserved scroll to widget
Y:{target_y_in_scroll_content}, frac:{fraction} within SR: {current_scroll_region}")
            else:
self.message_display_frame._parent_canvas.yview_moveto(0.0)
            else:
                self.message_display_frame._parent_canvas.yview_moveto(0.0)

            else: # No content or bbox not found
                self.message_display_frame._parent_canvas.yview_moveto(0.0)
            else:
                print("[LOAD_HISTORY DEBUG] Could not preserve scroll,
frame/widget/canvas missing.")

                self.after(150, _do_preserve_scroll_to_widget_closure)
            else:
                if hasattr(self.message_display_frame, '_parent_canvas'):
                    print(
                        f"[LOAD_HISTORY DEBUG] Target widget for scroll preservation not found
for {identifier}, or not preserving. Defaulting scroll to top.")
                    self.after(100, lambda:
self.message_display_frame._parent_canvas.yview_moveto(0.0) if hasattr(
self.message_display_frame, '_parent_canvas') else None)

                self.is_loading_more_messages = False

            def scroll_chat_to_bottom_delayed(self, delay=50):
                if hasattr(self, 'message_display_frame') and
self.message_display_frame.winfo_exists():
                    def _do_scroll():
                        if hasattr(self, 'message_display_frame') and \
self.message_display_frame.winfo_exists() and \
hasattr(self.message_display_frame, '_parent_canvas'): # Check if
parent canvas exists
                            try:

```

```

        self.message_display_frame.update_idletasks() # Process all pending
geometry changes

        # Explicitly update the scrollregion of the internal canvas
        # This tells the canvas its new total size based on its current
content.

        bbox = self.message_display_frame._parent_canvas.bbox("all")
        if bbox: # bbox returns (x1, y1, x2, y2) or None

self.message_display_frame._parent_canvas.configure(scrollregion=bbox)
        print(
            f"[SCROLL_DEBUG] Updated scrollregion for
{self.current_chat_identifier or 'N/A'}: {bbox}")
        else:
            print(
                f"[SCROLL_DEBUG] No bbox for {self.current_chat_identifier or
'N/A'}, scrollregion not updated effectively.")
            # If no content, setting a minimal scrollregion might help
yview_moveto(1.0) not error out
            # or behave unexpectedly, though often it defaults to 0,0,1,1
internally or similar.

            #
self.message_display_frame._parent_canvas.configure(scrollregion=(0,0,self.message_display_fra
me._parent_canvas.winfo_width(), self.message_display_frame._parent_canvas.winfo_height()))

            self.message_display_frame._parent_canvas.yview_moveto(1.0)
            print(f"[SCROLL_DEBUG] Scrolled to bottom (1.0) for
{self.current_chat_identifier or 'N/A'}")
            except Exception as e:
                print(f"[SCROLL_DEBUG] Error in _do_scroll for
{self.current_chat_identifier or 'N/A'}: {e}")
            else:
                print(
                    f"[SCROLL_DEBUG] _do_scroll: message_display_frame or _parent_canvas
not available for {self.current_chat_identifier or 'N/A'}.")

                self.after(delay, _do_scroll)
            else:
                print(
                    f"[SCROLL_DEBUG] scroll_chat_to_bottom_delayed: message_display_frame not
available for {self.current_chat_identifier or 'N/A'}.")

        def request_other_user_public_key(self, target_username): # (As previously defined)
            if not self.username or target_username == self.username or not self.public_key:
return False
            print(f"[CLIENT] Requesting PK for: {target_username}");
            return self.send_message("REQUEST_USER_PUBLIC_KEY", {"username": target_username})

        def load_chat_history(self, identifier, is_group=False, scroll_to_bottom=False,
preserve_scroll_info=None):
            if not hasattr(self, 'message_display_frame') or \
                not self.message_display_frame or \
                not self.message_display_frame.winfo_exists() or \
                identifier is None:
                print(
                    f"[LOAD_HISTORY_DEBUG] Called but message_display_frame not ready or
identifier ({identifier}) is None.")
                return

            print(f"[LOAD_HISTORY_DEBUG] Loading for Identifier: {identifier}, Is Group:
{is_group}")
            all_messages_for_identifier = self.chat_histories.get(identifier, [])
            # print(f"[LOAD_HISTORY_DEBUG] Full history for '{identifier}' from
self.chat_histories ({len(all_messages_for_identifier)} messages):")
            # for i, msg_debug in enumerate(all_messages_for_identifier): # Can be very verbose
            #     print(f" [LOAD_HISTORY_DEBUG] HistMsg {i}: {msg_debug}")

            old_top_message_data_for_scroll = None
            if preserve_scroll_info and preserve_scroll_info.get("first_visible_message_data"):
                old_top_message_data_for_scroll =
preserve_scroll_info["first_visible_message_data"]

            for widget in self.message_display_frame.winfo_children():
                widget.destroy()

            num_to_display = self.messages_currently_shown_count.get(identifier,
DEFAULT_MESSAGES_TO_LOAD_INITIALLY)

```

```

num_to_display = min(num_to_display, len(all_messages_for_identifier))
messages_to_display = all_messages_for_identifier[-num_to_display:]

self.chat_all_older_loaded[identifier] = (num_to_display >=
len(all_messages_for_identifier))
# print(f"[LOAD HISTORY DEBUG] Displaying last {len(messages_to_display)} messages.
All older loaded: {self.chat_all_older_loaded[identifier]}")

last_date_displayed = None
self.message_display_frame.update_idletasks()
available_width = self.message_display_frame.winfo_width() - \
    (getattr(self.message_display_frame, '_scrollbar_width', 20) if
hasattr(
        self.message_display_frame, '_scrollbar_width') else 20) - 20

newly_rendered_widgets = []

for msg_data in messages_to_display:
    text = msg_data.get("text", "")
    timestamp_str = msg_data.get("timestamp")
    is_system_event_msg = msg_data.get("is system event", False)

    # --- Get message status flags ---
    is_encrypted_msg = msg_data.get("is_encrypted", False)
    message_status = msg_data.get(
        "status") # e.g., "queued", "sent to server", "sent encrypted",
"received_decrypted"

    current_msg_time_obj = None
    if timestamp_str:
        try:
            ts_processed = timestamp_str.replace('T', ' ')
            current_msg_time_obj = datetime.strptime(ts_processed, '%Y-%m-%d
%H:%M:%S.%f') \
                if '.' in ts_processed else datetime.strptime(ts_processed, '%Y-%m-%d
%H:%M:%S')

            msg_date_obj = current_msg_time_obj.date()
            if msg_date_obj != last_date_displayed:
                date_label_text = msg_date_obj.strftime('%B %d, %Y')
                today_date = datetime.today().date()
                yesterday_date = today_date - timedelta(days=1)
                if msg_date_obj == today_date:
                    date_label_text = "Today"
                elif msg_date_obj == yesterday_date:
                    date_label_text = "Yesterday"

            date_label = ctk.CTkLabel(self.message_display_frame,
text=date_label_text,
font=ctk.CTkFont(family=TIMESTAMP_FONT_FAMILY,
size=TIMESTAMP_FONT_SIZE -
1, slant="italic"),
text_color=DATE_SEPARATOR_TEXT_COLOR,
anchor="center")

            date_label.pack(pady=(10, 5), fill="x")
            newly_rendered_widgets.append(date_label)
            last_date_displayed = msg_date_obj
        except ValueError as ve:
            print(f"Error parsing timestamp '{timestamp_str}': {ve}")

    if is_system_event_msg:
        print(f"[LOAD HISTORY DEBUG] Rendering system event: '{text}' for identifier
'{identifier}'")
        event_frame = ctk.CTkFrame(self.message_display_frame, fg_color="transparent")
        event_frame.pack(fill="x", padx=20, pady=(3, 3))
        event_label = ctk.CTkLabel(event_frame, text=text,
font=ctk.CTkFont(family=MESSAGE_TEXT_FONT_FAMILY,
size=MESSAGE_TEXT_FONT_SIZE - 2,
slant="italic"),
text_color=SYSTEM_EVENT_TEXT_COLOR,
fg_color=SYSTEM_EVENT_BG_COLOR,
corner_radius=6, wraplength=max(100,
available_width * 0.7))
        event_label.pack(pady=2, ipady=3, side="top", anchor="center")
        newly_rendered_widgets.append(event_frame)
        continue

```

```

actual_sender = "Unknown";
is_sent_by_me = False
if is_group:
    actual_sender = msg_data.get("sender_username", "Unknown Group Member")
    is_sent_by_me = (actual_sender == self.username)
else:
    actual_sender = msg_data.get("sender", "Unknown User")
    is_sent_by_me = (actual_sender == self.username)

line_frame = ctk.CTkFrame(self.message_display_frame, fg_color="transparent")
setattr(line_frame, 'message_data_for_scroll', msg_data)
line_frame.pack(fill="x", padx=5, pady=(1, 2))
newly_rendered_widgets.append(line_frame)

bubble_fg_color = SENT_BUBBLE_COLOR if is_sent_by_me else RECEIVED_BUBBLE_COLOR
bubble_anchor = "e" if is_sent_by_me else "w"
bubble_frame = ctk.CTkFrame(line_frame, fg_color=bubble_fg_color,
corner_radius=BUBBLE_CORNER_RADIUS)
bubble_frame.pack(anchor=bubble_anchor, padx=5, pady=1, ipadx=BUBBLE_PADDING_X,
ipady=BUBBLE_PADDING_Y)

if is_group and not is_sent_by_me:
    ctk.CTkLabel(bubble_frame, text=actual_sender,
font=ctk.CTkFont(family=MESSAGE_TEXT_FONT_FAMILY,
size=MESSAGE_TEXT_FONT_SIZE - 3,
weight="bold"),
text_color=GROUP_SENDER_TEXT_COLOR).pack(anchor="nw", padx=(0,
5), pady=(0, 1))

max_text_label_width = int(available_width * MAX_BUBBLE_WIDTH_FACTOR);
max_text_label_width = max(50, max_text_label_width)
ctk.CTkLabel(bubble_frame, text=text,
font=ctk.CTkFont(family=MESSAGE_TEXT_FONT_FAMILY,
size=MESSAGE_TEXT_FONT_SIZE),
wraplength=max_text_label_width, justify="left").pack(anchor="w")

# --- Timestamp and Message Status Icons ---
timestamp_info_frame = ctk.CTkFrame(bubble_frame, fg_color="transparent")
timestamp_info_frame.pack(anchor="se")

status_icon_text = ""
if is_sent_by_me: # Status icons are usually for sent messages
    if message_status == "queued":
        status_icon_text = "🕒 " # Clock icon for queued
    elif message_status == "sent_to_server":
        status_icon_text = "✓ " # Single tick for sent to server
    elif message_status == "sent_encrypted" or (
        is_encrypted_msg and message_status != "queued"): # Successfully sent
E2EE
        status_icon_text = "🔒 " # Lock for E2EE sent
    # Add more statuses like "delivered" (double tick), "read" (blue ticks) if
    implemented later

    elif is_encrypted_msg and not is_sent_by_me and message_status ==
"received_decrypted":
        # For received E2EE messages, we can also show a lock
        status_icon_text = "🔒 "

    if status_icon_text: # Only create label if there's an icon
        ctk.CTkLabel(timestamp_info_frame, text=status_icon_text,
font=ctk.CTkFont(family=TIMESTAMP_FONT_FAMILY,
size=TIMESTAMP_FONT_SIZE),
# Adjust size if needed
text_color=TIMESTAMP_TEXT_COLOR).pack(side="left", padx=(0, 2))

time_str = current_msg_time_obj.strftime('%I:%M %p').lstrip('0') if
current_msg_time_obj else "--:--"
ctk.CTkLabel(timestamp_info_frame, text=time_str,
font=ctk.CTkFont(family=TIMESTAMP_FONT_FAMILY,
size=TIMESTAMP_FONT_SIZE),
text_color=TIMESTAMP_TEXT_COLOR).pack(side="left")

self.message_display_frame.update_idletasks()
if scroll_to_bottom:
    if hasattr(self, 'scroll_chat_to_bottom_delayed'):

```

```

self.scroll_chat_to_bottom_delayed()
    elif preserve_scroll_info and old_top_message_data_for_scroll:
        # (Scroll preservation logic as previously provided - try/except block with
        yview_moveto)
        target_widget_to_scroll_to = None
        for widget in newly_rendered_widgets:
            if hasattr(widget, '_message_data_for_scroll') and \
                getattr(widget, '_message_data_for_scroll') ==
old_top_message_data_for_scroll:
                target_widget_to_scroll_to = widget;
                break
        if target_widget_to_scroll_to:
            def _do_preserve_scroll_to_widget():
                if hasattr(self,
                    'message_display_frame') and self.message_display_frame and
self.message_display_frame.wininfo_exists() and \
                    target_widget_to_scroll_to.wininfo_exists() and
hasattr(self.message_display_frame,
'_parent_canvas'):
                    self.message_display_frame.update_idletasks()
                    widget_y = target_widget_to_scroll_to.wininfo_y();
                    canvas_bbox = self.message_display_frame._parent_canvas.bbox("all")
                    if canvas_bbox:
                        content_start_y = canvas_bbox[1];
                        content_height = canvas_bbox[3] - content_start_y
                        if content_height > 0:
                            scroll_fraction = (
                                widget_y - content_start_y) /
content_height if content_start_y < widget_y else 0
                            scroll_fraction = max(0.0, min(scroll_fraction, 1.0))
                    self.message_display_frame._parent_canvas.yview_moveto(scroll_fraction)
                    else:
                        self.message_display_frame._parent_canvas.yview_moveto(0.0)
                    else:
                        self.message_display_frame._parent_canvas.yview_moveto(0.0)

                    self.after(150, _do_preserve_scroll_to_widget)
                else:
                    if hasattr(self.message_display_frame, '_parent_canvas'):
                        self.after(100, lambda:
self.message_display_frame._parent_canvas.yview_moveto(0.0))

            self.is_loading_more_messages = False
            def populate_chat_list(self):
                if not hasattr(self, 'scrollable_chat_list') or not
self.scrollable_chat_list.wininfo_exists(): return
                current_sel_id = self.current_chat_identifier;
                current_is_group_sel = self.current_chat_is_group
                for w in self.scrollable_chat_list.wininfo_children(): w.destroy()
                if self.default_list_profile_image is None: self._create_default_profile_image()
                combined_list_items = []
                user_partners = [p for p in self.chat_histories.keys() if isinstance(p, str)]
                for partner_name in user_partners:
                    last_msg_time = self.get_last_message_time(partner_name, is_group=False);
                    partner_image = self.default_list_profile_image
                    if partner_name in self.profile_picture_cache:
                        cached_header_ctk_image = self.profile_picture_cache[partner_name];
                        pil_light = cached_header_ctk_image._light_image;
                        pil_dark = cached_header_ctk_image._dark_image
                        if pil_light and pil_dark:
                            try:
                                r_pil_l = pil_light.resize(PROFILE_PIC_LIST_SIZE,
                                                            Image.Resampling.LANCZOS);
                                r_pil_d = pil_dark.resize(
                                    PROFILE_PIC_LIST_SIZE, Image.Resampling.LANCZOS);
                                partner_image = ctk.CTkImage(
                                    light_image=r_pil_l, dark_image=r_pil_d,
size=PROFILE_PIC_LIST_SIZE)
                            except:
                                pass
                    else:
                        self.request_profile_picture(partner_name)
                    combined_list_items.append(
                        {"last_time": last_msg_time, "name": partner_name, "id": partner_name,
"is_group": False,

```



```

        "image": partner_image))
    has_printed_groups_header_this_run = False
    if self.my_groups:
        sorted_groups_only = sorted(self.my_groups,
                                     key=lambda g:
self.get_last_message_time(g["group_id"], is_group=True),
                                     reverse=True)
        for group_info in sorted_groups_only:
            group_id = group_info["group_id"];
            group_name = group_info["group name"];
            group_icon_filename = group_info.get("group_icon_filename")
            last_msg_time = self.get_last_message_time(group_id, is_group=True)
            group_display_icon = self.default_list_profile_image
            cache_key_list = f"group {group_id}_list"
            if cache_key_list in self.group_icon_cache:
                group_display_icon = self.group_icon_cache[cache_key_list]
            elif group_icon_filename:
                self.request_group_icon(group_id, group_icon_filename)
            combined_list_items.append(
                {"last_time": last_msg_time, "name": group_name, "id": group_id,
"is group": True,
                "image": group_display_icon})
        sorted_list_items = sorted(combined_list_items, key=lambda item: item["last_time"],
reverse=True)
        if not sorted_list_items: ctk.CTkLabel(self.scrollable_chat_list, text="No active
chats or groups.").pack(
            padx=5, pady=10)
        has_printed_user_chats_in_sorted_list = any(not item["is_group"] for item in
sorted_list_items)
        for item in sorted_list_items:
            if item["is_group"] and not has_printed_groups_header_this_run and any(
                g item["is_group"] for g item in sorted_list_items):
                has_printed_groups_header_this_run = True
            is_selected = (item["id"] == current_sel_id and item["is_group"] ==
current_is_group_sel)
            fg = ("dce4ee", "#343638") if is_selected else "transparent";
            font = ctk.CTkFont(family=MESSAGE_TEXT_FONT_FAMILY, size=MESSAGE_TEXT_FONT_SIZE -
1);
            display_name = item["name"]
            btn = ctk.CTkButton(self.scrollable_chat_list, text=display_name,
image=item["image"], compound="left",
                                font=font, anchor="w", fg_color=fg, hover_color=("e0e0e0",
"#3a3a3a"),
                                command=lambda i=item["id"], ig=item["is_group"],
                                    name_for_header=item["name"]):
self.select_chat(i, is_group=ig,
group_name_for_header=name_for_header if ig else None));
            btn.pack(fill="x", padx=5, pady=(1, 2))

        def display_system_message(self, message_text): # DEFINED HERE
            if not hasattr(self, 'message_display_frame') or not
self.message_display_frame.winfo_exists(): print(
                f"SysMsg (No UI): {message_text}"); return
            try:
                if self.default_list_profile_image is None: self.create_default_profile_image()
                ctk.CTkLabel(self.message_display_frame, text=message_text,
                    font=ctk.CTkFont(family=TIMESTAMP_FONT_FAMILY,
size=TIMESTAMP_FONT_SIZE, slant="italic"),
                    text_color=SYSTEM_EVENT_TEXT_COLOR, fg_color=SYSTEM_EVENT_BG_COLOR,
corner_radius=6).pack(
                    pady=5, padx=20, ipady=3, fill="x", anchor="center")
                self.scroll_chat_to_bottom_delayed()
            except Exception as e:
                print(f"Error displaying system message: {e}")

        def _create_default_profile_image(self):
            try:
                mode_color = ("e0e0e0", "#3a3a3a")[1 if ctk.get_appearance_mode().lower() ==
"dark" else 0]
                header_default_pil = Image.new('RGB', PROFILE_PIC_HEADER_SIZE, color=mode_color)
                self.default_profile_image = ctk.CTkImage(light_image=header_default_pil,
dark_image=header_default_pil,
                                                            size=PROFILE_PIC_HEADER_SIZE)

                list_default_pil = Image.new('RGB', PROFILE_PIC_LIST_SIZE, color=mode_color)
                self.default_list_profile_image = ctk.CTkImage(light_image=list_default_pil,

```

```

dark_image=list_default_pil,
                                                    size=PROFILE_PIC_LIST_SIZE)

    group_icon_pil = Image.new('RGB', GROUP_ICON_SETTINGS_SIZE, color=mode_color)
    self.default_group_icon = ctk.CTkImage(light_image=group_icon_pil,
dark_image=group_icon_pil,
                                                    size=GROUP_ICON_SETTINGS_SIZE)

    except Exception as e:
        print(f"Error creating default profile images: {e}")
        self.default_profile_image = None
        self.default_list_profile_image = None
        self.default_group_icon = None

    def clear_auth_frame(self):
        for widget in self.auth_frame.wininfo_children(): widget.destroy()

    def toggle_password_visibility(self, entry_widget, button_widget):
        current_show_char = entry_widget.cget("show")
        if current_show_char == "*":
            entry_widget.configure(show="");
            button_widget.configure(text="Hide")
        else:
            entry_widget.configure(show="*");
            button_widget.configure(text="Show")

    # --- View Switching Methods ---

    def show_login_view(self):
        self.title("Login");
        self.geometry("350x450");
        self.clear_auth_frame()
        self.main_chat_frame.pack_forget();
        self.new_chat_frame.pack_forget()
        self.auth_frame.pack(pady=20, padx=30, fill="both", expand=True)
        ctk.CTkLabel(self.auth_frame, text="Welcome Back", font=ctk.CTkFont(size=24,
weight="bold")).pack(pady=(30, 10))
        sf = ctk.CTkFrame(self.auth_frame, fg_color="transparent");
        sf.pack(pady=(0, 20));
        ctk.CTkLabel(sf, text="Don't have an account yet? ").pack(side="left");
        ctk.CTkButton(sf, text="Sign up", fg_color="transparent", text_color=("#3a7ebf",
"#1f6aa5"), hover=False,
                        width=50, command=self.show_register_view).pack(side="left", padx=0)
        self.login_user_entry = ctk.CTkEntry(self.auth_frame, placeholder_text="Username",
width=250, height=40);
        self.login_user_entry.pack(pady=10)
        pf = ctk.CTkFrame(self.auth_frame, fg_color="transparent");
        pf.pack(pady=10);
        self.login_pass_entry = ctk.CTkEntry(pf, placeholder_text="Password", show="*",
width=200, height=40);
        self.login_pass_entry.pack(side="left", padx=(0, 5));
        self.login_pass_entry.bind("<Return>", self.login_user_event);
        sb = ctk.CTkButton(pf, text="Show", width=40, height=40);
        sb.configure(command=lambda e=self.login_pass_entry, b=sb:
self.toggle_password_visibility(e, b));
        sb.pack(side="left");
        ctk.CTkButton(self.auth_frame, text="Login", width=250, height=40,
command=self.login_user).pack(pady=(20, 30))

    def populate_create_group_member_list(self):
        if not hasattr(self,
            'create_group_members_scroll_frame') or not
self.create_group_members_scroll_frame.wininfo_exists(): return
        for widget in self.create_group_members_scroll_frame.wininfo_children():
            widget.destroy(); self.create_group_member_vars = {}
            if not self.all_users_list: ctk.CTkLabel(self.create_group_members_scroll_frame,
text="No users loaded.").pack(
                padx=5, pady=5); return
            dc = 0
            for user in self.all_users_list:
                if user == self.username: continue
                var = ctk.StringVar(value="off");
                cb = ctk.CTkCheckBox(self.create_group_members_scroll_frame, text=user,
variable=var, onvalue="on",
                                offvalue="off");
                cb.pack(anchor="w", padx=5, pady=2)
                self.create_group_member_vars[user] = var;
                dc += 1

```

```

        if dc == 0: ctk.CTkLabel(self.create_group_members_scroll_frame, text="No other users
to add.").pack(padx=5,
pady=5)

    def show_create_group_view(self):
        self.title("Create New Group");
        self.auth_frame.pack_forget();
        self.main_chat_frame.pack_forget();
        self.new_chat_frame.pack(pady=10, padx=10, fill="both", expand=True)
        for widget in self.new_chat_frame.winfo_children(): widget.destroy()
        tf = ctk.CTkFrame(self.new_chat_frame, fg_color="transparent");
        tf.pack(fill="x", pady=(0, 10), padx=5);
        ctk.CTkLabel(tf, text="Create Group", font=ctk.CTkFont(size=18,
weight="bold")).pack(side="left");
        ctk.CTkButton(tf, text="Back to Chats", width=100,
command=self.show_chat_screen).pack(side="right")
        gnf = ctk.CTkFrame(self.new_chat_frame, fg_color="transparent");
        gnf.pack(fill="x", padx=10, pady=5);
        ctk.CTkLabel(gnf, text="Group Name:").pack(side="left", padx=(0, 5));
        self.create_group_name_entry = ctk.CTkEntry(gnf, placeholder_text="Enter group
name...");
        self.create_group_name_entry.pack(side="left", fill="x", expand=True)
        ctk.CTkLabel(self.new_chat_frame, text="Select Members:",
font=ctk.CTkFont(size=14)).pack(anchor="w", padx=10,
pady=(10, 0))
        self.create_group_members_scroll_frame = ctk.CTkScrollableFrame(self.new_chat_frame);
        self.create_group_members_scroll_frame.pack(fill="both", expand=True, padx=10, pady=5)
        if not self.all_users_list:
            self.send_message("GET_ALL_USERS", {});
            ctk.CTkLabel(self.create_group_members_scroll_frame,
text="Loading user list...").pack()
        else:
            self._populate_create_group_member_list()
            ctk.CTkButton(self.new_chat_frame, text="Create Group",
command=self.handle_create_group_action).pack(
pady=(10, 0))

    def handle_create_group_action(self):
        group_name = self.create_group_name_entry.get();
        if not group_name.strip(): messagebox.showerror("Error", "Group name cannot be
empty."); return
        selected_members = [self.username]
        for user, var in self.create_group_member_vars.items():
            if var.get() == "on": selected_members.append(user)
        final_members = list(set(selected_members))
        payload = {"group_name": group_name, "member_usernames": final_members}
        self.send_message("CREATE_GROUP", payload)

    def show_new_chat_view(self):
        self.title("Start New Chat");
        self.geometry("400x500");
        self.auth_frame.pack_forget();
        self.main_chat_frame.pack_forget()
        self.new_chat_frame.pack(pady=10, padx=10, fill="both", expand=True);
        for widget in self.new_chat_frame.winfo_children(): widget.destroy()
        ctk.CTkButton(self.new_chat_frame, text="Back to Chats", width=100,
command=self.show_chat_screen).pack(
pady=(5, 10), padx=10, anchor="w")
        self.new_chat_search_entry = ctk.CTkEntry(self.new_chat_frame,
placeholder_text="Search username...");
        self.new_chat_search_entry.pack(pady=5, padx=10, fill="x");
        self.new_chat_search_entry.bind("<KeyRelease>", self.filter_user_list_display)
        self.all_users_scrollable_list = ctk.CTkScrollableFrame(self.new_chat_frame,
corner_radius=0);
        self.all_users_scrollable_list.pack(pady=10, padx=10, fill="both", expand=True)
        if self.all_users_list:
            self.populate_all_users_list()
        else:
            self.send_message("GET_ALL_USERS", {})

    def show_group_settings_view(self, group_id, group_name):
        self.title(f"Settings: {group_name}");
        self.auth_frame.pack_forget();
        self.main_chat_frame.pack_forget()
        self.new_chat_frame.pack(pady=10, padx=10, fill="both", expand=True)

```

```

        for widget in self.new_chat_frame.winfo_children(): widget.destroy()
        settings_header_frame = ctk.CTkFrame(self.new_chat_frame, fg_color="transparent");
        settings_header_frame.pack(fill="x", pady=(0, 10))
        ctk.CTkLabel(settings_header_frame, text=f"Settings for {group_name}",
                      font=ctk.CTkFont(size=18, weight="bold")).pack(side="left", padx=5)
        back_to_chat_command = lambda: (
            self.show_chat_screen(), self.select_chat(group_id, is_group=True,
            group_name_for_header=group_name))
        ctk.CTkButton(settings_header_frame, text="Back to Chat", width=100,
            command=back_to_chat_command).pack(
            side="right", padx=5)

        self.group_settings_icon_display_label = ctk.CTkLabel(self.new_chat_frame, text="",
            image=self.default_group_icon,
            width=GROUP_ICON_SETTINGS_SIZE[0],
            height=GROUP_ICON_SETTINGS_SIZE[1])
        self.group_settings_icon_display_label.pack(pady=5)

        actions_frame = ctk.CTkFrame(self.new_chat_frame, fg_color="transparent");
        actions_frame.pack(fill="x", pady=5, padx=10)
        ctk.CTkButton(actions_frame, text="Rename Group",
            command=lambda gid=group_id, current_gname=group_name:
            self.prompt_rename_group(gid,
            current_gname)).pack(
            side="left", padx=(0, 5), expand=True, fill="x")
        ctk.CTkButton(actions_frame, text="Change Group Picture",
            command=lambda gid=group_id, gname=group_name:
            self.prompt_upload_group_picture(gid, gname)).pack(
            side="left", padx=(5, 0), expand=True, fill="x")

        ctk.CTkLabel(self.new_chat_frame, text="Members:", font=ctk.CTkFont(size=14,
            weight="bold")).pack(anchor="w",
            padx=10,
            pady=(5, 0))
        self.group_settings_member_list_frame = ctk.CTkScrollableFrame(self.new_chat_frame,
            height=150)
        self.group_settings_member_list_frame.pack(fill="x", expand=False, padx=10, pady=5)

        bottom_actions_frame = ctk.CTkFrame(self.new_chat_frame, fg_color="transparent");
        bottom_actions_frame.pack(fill="x", pady=5, padx=10)
        ctk.CTkButton(bottom_actions_frame, text="Add Members",
            command=lambda gid=group_id, gname=group_name:
            self.prepare_add_members_view(gid, gname)).pack(
            side="left", padx=(0, 5), expand=True, fill="x")
        ctk.CTkButton(bottom_actions_frame, text="Leave Group",
            command=lambda gid=group_id:
            self.confirm_leave_group(gid)).pack(side="left", padx=(5, 0),
            expand=True, fill="x")

        self.send_message("GET_GROUP_DETAILS", {"group_id": group_id})

    def _populate_group_settings_member_list(self, group_id, group_name, creator_username,
        members_data_list):
        if not hasattr(self,
            'group_settings_member_list_frame') or not
            self.group_settings_member_list_frame.winfo_exists(): return
        if not (self.new_chat_frame.winfo_ismapped() and self.title().startswith(f"Settings:
        {group_name}")): return
        for widget in self.group_settings_member_list_frame.winfo_children(): widget.destroy()
        if not members_data_list: ctk.CTkLabel(self.group_settings_member_list_frame,
            text="No members found.").pack(); return

        current_user_is_admin = any(
            m.get("username") == self.username and m.get("role") == 'admin' for m in
            members_data_list)
        for member_data in members_data_list:
            member_username = member_data.get("username");
            member_role = member_data.get("role", "member")
            member_frame = ctk.CTkFrame(self.group_settings_member_list_frame,
            fg_color="transparent");
            member_frame.pack(fill="x", pady=1)
            display_text = f"{member_username} ({member_role.capitalize()})"

```

```

        if member_username == creator_username: display_text = f"{member_username}
(Creator, Admin)" if member_role == 'admin' else f"{member_username} (Creator)"
        ctk.CTkLabel(member_frame, text=display_text, anchor="w").pack(side="left",
padx=5)

        if current_user_is_admin and member_username != self.username:
            action_buttons_frame = ctk.CTkFrame(member_frame, fg_color="transparent");
            action_buttons_frame.pack(side="right", padx=2)
            if member_role == 'member':
                ctk.CTkButton(action_buttons_frame, text="Make Admin", width=80,
height=24,
                                command=lambda gid=group_id, uname=member_username:
self.handle_change_member_role(
                                    gid, uname, "admin")).pack(side="left", padx=2)
            elif member_role == 'admin' and member_username != creator_username:
                ctk.CTkButton(action_buttons_frame, text="Demote", width=70, height=24,
                                command=lambda gid=group_id, uname=member_username:
self.handle_change_member_role(
                                    gid, uname, "member")).pack(side="left", padx=2)
            ctk.CTkButton(action_buttons_frame, text="Remove", width=70, height=24,
fg_color="red",
                                command=lambda gid=group_id, uname=member_username:
self.confirm_remove_member(gid,
uname)).pack(
                                side="left", padx=2)

        def confirm_leave_group(self, group_id_to_leave):
            if group_id_to_leave is None: messagebox.showerror("Error", "No group selected.");
            return
            group_name_display = f"Group ID {group_id_to_leave}";
            _ = [(group_name_display := g.get("group_name", group_name_display)) for g in
self.my_groups if
                    g.get("group_id") == group_id_to_leave]
            if messagebox.askyesno("Leave Group",
                                f"Are you sure you want to leave '{group_name_display}'?"):
self.send_message(
    "LEAVE_GROUP", {"group_id": group_id_to_leave})

        def prepare_add_members_view(self, group_id, group_name):
            if self.last_group_details_data and self.last_group_details_data.get("group_id") ==
group_id:
                current_members = [m.get("username") for m in
self.last_group_details_data.get("members", []) if
                    isinstance(m, dict)]
                self.show_add_members_to_group_view(group_id, group_name, current_members)
            else:
                messagebox.showinfo("Info",
                                "Fetching group details. Click 'Add Members' again shortly.");
                self.send_message(
                    "GET_GROUP_DETAILS", {"group_id": group_id})

        def show_add_members_to_group_view(self, group_id, group_name, current_members_list):
            self.title(f"Add Members to {group_name}");
            self.auth_frame.pack_forget();
            self.main_chat_frame.pack_forget();
            self.new_chat_frame.pack(pady=10, padx=10, fill="both", expand=True)
            for widget in self.new_chat_frame.winfo_children(): widget.destroy()
            add_header_frame = ctk.CTkFrame(self.new_chat_frame, fg_color="transparent");
            add_header_frame.pack(fill="x", pady=(0, 10));
            ctk.CTkLabel(add_header_frame, text=f"Add to '{group_name}'",
font=ctk.CTkFont(size=16, weight="bold")).pack(
                side="left");
            back_command = lambda: self.show_group_settings_view(group_id, group_name);
            ctk.CTkButton(add_header_frame, text="Back to Settings", width=120,
command=back_command).pack(side="right")
            ctk.CTkLabel(self.new_chat_frame, text="Select users to add:",
font=ctk.CTkFont(size=14)).pack(anchor="w",
padx=10,
pady=(5, 0))
            self.add_members_scroll_frame = ctk.CTkScrollableFrame(self.new_chat_frame)
            self.add_members_scroll_frame.pack(fill="both", expand=True, padx=10, pady=5);
            self.add_members_selection_vars = {}
            if not self.all_users_list: ctk.CTkLabel(self.add_members_scroll_frame,
text="User list not available.").pack();
            self.send_message(

```

```

        "GET_ALL_USERS", {})); return
    displayed_count = 0
    for user in self.all_users_list:
        if user == self.username or user in current_members_list: continue
        var = ctk.StringVar(value="off");
        cb = ctk.CTkCheckBox(self.add_members_scroll_frame, text=user, variable=var,
onvalue="on", offvalue="off");
        cb.pack(anchor="w", padx=5, pady=2)
        self.add_members_selection_vars[user] = var;
        displayed_count += 1
    if displayed_count == 0: ctk.CTkLabel(self.add_members_scroll_frame,
        text="No new users available to add.").pack()
    ctk.CTkButton(self.new_chat_frame, text="Add Selected Members",
        command=lambda gid=group_id, gname=group_name:
self.handle_add_selected_members(gid, gname)).pack(
        pady=10)

    def handle_add_selected_members(self, group_id, group_name_for_return):
        newly_selected_members = [user for user, var in
self.add_members_selection_vars.items() if var.get() == "on"]
        if not newly_selected_members: messagebox.showinfo("No Selection", "No new members
selected."); return
        payload = {"group_id": group_id, "new_member_usernames": newly_selected_members};
        self.send_message("ADD_GROUP_MEMBERS", payload)
        self.show_group_settings_view(group_id, group_name_for_return)

    def handle_change_member_role(self, group_id, target_username, new_role):
        payload = {"group_id": group_id, "target_username": target_username, "new_role":
new_role};
        self.send_message("CHANGE_MEMBER_ROLE", payload)

    def confirm_remove_member(self, group_id, username_to_remove):
        group_name_display = f"Group ID {group_id}";
        _ = [(group_name_display := g.get("group_name", group_name_display)) for g in
self.my_groups if
            g.get("group_id") == group_id]
        if messagebox.askyesno("Remove Member",
            f"Are you sure you want to remove {username_to_remove} from
'{group_name_display}'?"):
            self.send_message("REMOVE_GROUP_MEMBER", {"group_id": group_id,
"username_to_remove": username_to_remove})

    def prompt_rename_group(self, group_id, current_group_name): # New Method
        if not self.username: return
        dialog = ctk.CTkInputDialog(text="Enter new group name:", title="Rename Group",
            entry_fg_color=("#F9F9FA", "#343638"),
entry_border_color=("#979DA2", "#565B5E"),
            entry_text_color=("#1A1A1A", "#DCE4EE"))
        dialog.geometry("300x200")
        new_name_input = dialog.get_input()
        if new_name_input and new_name_input.strip() and new_name_input.strip() !=
current_group_name:
            self.send_message("RENAME_GROUP", {"group_id": group_id, "new_group_name":
new_name_input.strip()})
        elif new_name_input and new_name_input.strip() == current_group_name:
            messagebox.showinfo("Rename Group", "New name is the same as the current name.")

    # --- Network and Core Logic Methods ---

    def login_user(self):
        username_val = self.login_user_entry.get()
        password_val = self.login_pass_entry.get()
        if not username_val or not password_val:
            messagebox.showwarning("Input Required", "Please enter both username and
password.")
            return
        if not self.client_socket: # Check if socket connected
            messagebox.showerror("Connection Error",
                "Not connected to the server. Please try restarting the
application.")
            return

        # Ensure RSA keys are generated and NetworkNode is functional
        if not self.public_key: # NetworkNode. init sets this
            messagebox.showerror("Key Error", "Client RSA public key not available. Cannot

```

```

login securely.")
    return

    client_pk_pem = self.get_public_key_pem() # Method from NetworkNode

    # --- ADD THIS DEBUG PRINT ---
    print(f"[CLIENT LOGIN] Attempting to send client_public_key: {client_pk_pem is not
None}")
    if client_pk_pem:
        print(f"[CLIENT LOGIN] Client PK (first 60 char): {client_pk_pem[:60]}...")
    else:
        messagebox.showerror("Key Error", "Failed to get client public key PEM. Cannot
login.")
        return # Stop if key is None

    payload = {
        "username": username_val,
        "password": password_val,
        "client_public_key": client_pk_pem # Ensure this key is exactly
"client_public_key"
    }
    print(f"[CLIENT LOGIN] Sending payload: {payload}") # See the full payload being sent
    self.send_message("LOGIN", payload)

def login_user_event(self, event=None): # Make sure this calls the updated login_user
    self.login_user()

def load_and_display_profile_picture(self, username, image_data_base64, filename):
    target_label = self.chat_header_image_label
    target_size = PROFILE_PIC_HEADER_SIZE
    default_image = self.default_profile_image
    cache to use = self.profile_picture_cache
    cache_key = username
    if self.current_chat_is_group and username == self.current_chat_identifider: # It's
group icon for header
        cache_key = f"group_{username}_header" # username is group id here
        default_image = self.default_group_icon if self.default_group_icon else
self.default_profile_image
        if not target_label:
            return
        ctk_image_to_display = default_image
        if image_data_base64:
            try:
                if cache_key in cache_to_use:
                    ctk_image_to_display = cache_to_use[cache_key]
                else:
                    img_bytes = base64.b64decode(image_data_base64)
                    img = Image.open(io.BytesIO(img_bytes))
                    img = img.resize(target_size, Image.Resampling.LANCZOS)
                    ctk_image_to_display = ctk.CTkImage(light_image=img, dark_image=img,
size=target_size)
                    cache_to_use[cache_key] = ctk_image_to_display
                    target_label.configure(image=ctk_image_to_display, text="")
            except Exception as e:
                print(f"Error processing image for {username}: {e}")
                target_label.configure(image=default_image,
text="")
            _ = cache_to_use.pop(
                cache_key, None)
        else:
            target_label.configure(image=default_image, text="");
            _ = cache_to_use.pop(cache_key, None)

def request_profile_picture(self, target_username): # For user PFPs
    if target_username in self.profile_picture_cache and hasattr(self,
'chat_header_image_label') and target_username == self.current_chat_identifider and not
self.current_chat_is_group:
        self.chat_header_image_label.configure(image=self.profile_picture_cache[target_username],
text="")
        return
    self.send_message("GET_PROFILE_PICTURE", {"username": target_username})

def request_group_icon(self, group_id, icon_filename):
    if not icon_filename:
        return
    # Using distinct cache keys for list and settings view of group icons

```



```

        cache_key_list = f"group_{group_id}_list"
        cache_key_settings = f"group_{group_id}_settings"
        if cache_key_list in self.group_icon_cache and cache_key_settings in
self.group_icon_cache:
            return # Both sizes cached
        self.send_message("GET_GROUP_ICON", {"group_id": group_id, "icon_filename":
icon_filename})

    def prompt_upload_profile_picture(self):
        if not self.username:
            messagebox.showerror("Error", "Login to upload.")
            return
        filepath = filedialog.askopenfilename(title="Select PFP",
                                              filetype=(("Images", "*.jpg *.jpeg *.png"),
("All", "*.*")))
        if not filepath:
            return
        try:
            filename = os.path.basename(filepath)
            if not filename.lower().endswith(('.png', '.jpg', '.jpeg')):
                messagebox.showerror("Invalid File", "Use PNG or JPG.")
                return
            with open(filepath, "rb") as image_file:
                image_data = image_file.read()
                processed_image_data = image_data
                processed_filename = filename
            try:
                img = Image.open(io.BytesIO(image_data))
                img.thumbnail(MAX_UPLOAD_IMAGE_DIM, Image.Resampling.LANCZOS)
                output_buffer = io.BytesIO()
                target_format = img.format if img.format in ['JPEG', 'PNG'] else 'JPEG'
                if target_format == 'PNG':
                    img.save(output_buffer, format="PNG")
                else:
                    if img.mode in ('RGBA', 'P'): img = img.convert('RGB')
                    img.save(output_buffer, format="JPEG", quality=85)
                processed_image_data = output_buffer.getvalue()
                processed_filename = os.path.splitext(filename)[0] + "." +
target_format.lower()
            except Exception as e:
                messagebox.showwarning("Image Processing Warning", f"Could not fully process
image. Error: {e}")
                image_data_base64 = base64.b64encode(processed_image_data).decode('utf-8')
                payload = {"original_filename": processed_filename, "image_data_base64":
image_data_base64}
                self.send_message("UPLOAD_PROFILE_PICTURE", payload)
            except Exception as e:
                messagebox.showerror("Error", f"Could not read/encode image: {e}")

    def prompt_upload_group_picture(self, group_id, group_name_for_title):
        if not self.username:
            messagebox.showerror("Error", "You must be logged in.")
            return
        filepath = filedialog.askopenfilename(title=f"Select Icon for {group_name_for_title}",
                                              filetype=(("Images", "*.jpg *.jpeg *.png"),
("All", "*.*")))
        if not filepath:
            return
        try:
            filename = os.path.basename(filepath)
            if not filename.lower().endswith(('.png', '.jpg', '.jpeg')):
                messagebox.showerror("Invalid File",
"Use PNG or JPG."); return
            with open(filepath, "rb") as image_file:
                image_data = image_file.read()
                processed_image_data = image_data
                processed_filename = filename
            try:
                img = Image.open(io.BytesIO(image_data))
                img.thumbnail(MAX_UPLOAD_IMAGE_DIM, Image.Resampling.LANCZOS)
                output_buffer = io.BytesIO()
                target_format = img.format if img.format in ['JPEG', 'PNG'] else 'JPEG'
                if target_format == 'PNG':
                    img.save(output_buffer, format="PNG")
                else:
                    if img.mode in ('RGBA', 'P'): img = img.convert('RGB')

```



```

        img.save(output_buffer, format="JPEG", quality=85)
        processed_image_data = output_buffer.getvalue();
        processed_filename = os.path.splitext(filename)[0] + "." +
target_format.lower()
    except Exception as e:
        messagebox.showwarning("Image Processing Warning", f"Could not fully process
image: {e}")
        image_data_base64 = base64.b64encode(processed_image_data).decode('utf-8')
        payload = {"group_id": group_id, "original_filename": processed_filename,
                  "image_data_base64": image_data_base64}
        self.send_message("UPLOAD_GROUP_PICTURE", payload)
    except Exception as e:
        messagebox.showerror("Error", f"Could not read or encode group icon: {e}")

def get_last_message_time(self, item_id, is_group=False):
    history = self.chat_histories.get(item_id, [])
    if not history:
        if is_group:
            for group_info in self.my_groups:
                if group_info["group_id"] == item_id and "created at" in group_info:
                    try:
                        return datetime.strptime(group_info["created_at"].split(".")[0],
'%Y-%m-%d %H:%M:%S')
                    except:
                        return datetime.now()
                elif group_info["group_id"] == item_id:
                    return datetime.now()
            return datetime.min
        try:
            last_ts_str = history[-1].get("timestamp", "").replace('T', ' ')
            if not last_ts_str: return datetime.min
            try:
                return datetime.strptime(last_ts_str, '%Y-%m-%d %H:%M:%S.%f')
            except ValueError:
                return datetime.strptime(last_ts_str, '%Y-%m-%d %H:%M:%S')
        except Exception as e:
            print(f"Err parsing last_msg_time for {item_id} (group={is_group}): {e}");
            return datetime.min

def on_chat_scrollbar_command(self, command, *args):
    self.message_display_frame.v_scrollbar_callback(command, *args)
    current_pos = self.message_display_frame.v_scrollbar.get()
    if current_pos[0] < 0.001 and current_pos[1] != 1.0:
        if self.current_chat_identifier and not
self.chat_all_older_loaded.get(self.current_chat_identifier, False):
            self.trigger_load_older_messages()

def trigger_load_older_messages(self):
    if not self.current_chat_identifier or self.is_loading_more_messages or \
        self.chat_all_older_loaded.get(self.current_chat_identifier, False):
        return

    self.is_loading_more_messages = True

    preserve_info = {}
    # Try to get the data of the first currently visible message bubble
    if hasattr(self, 'message_display_frame') and
self.message_display_frame.wininfo_children():
        first_bubble_widget = None
        for child in self.message_display_frame.wininfo_children():
            if hasattr(child, '_message_data_for_scroll'): # Check if it's one of our
message line_frames
                first_bubble_widget = child
                break
        if first_bubble_widget:
            preserve_info["first_visible_message_data"] = getattr(first_bubble_widget,
'_message_data_for_scroll')
            # You could also try to get the current scrollbar position (fraction)
            # scroll_fraction_top, _ = self.message_display_frame.v_scrollbar.get()
            # preserve_info["scroll_fraction_top"] = scroll_fraction_top

    current_shown = self.messages_currently_shown_count.get(self.current_chat_identifier,
DEFAULT_MESSAGES_TO_LOAD_INITIALLY)
    self.messages_currently_shown_count[self.current_chat_identifier] = current_shown +
MESSAGES_TO_LOAD_ON_SCROLL

```

```

        self.load_chat_history(
            self.current_chat_identifier,
            is_group=self.current_chat_is_group,
            scroll_to_bottom=False, # We are loading older messages on top
            preserve_scroll_info=preserve_info
        )
    def populate_all_users_list(self, filter_text=""):
        if not hasattr(self, 'all_users_scrollable_list') or not
self.all_users_scrollable_list.winfo_exists():
            return
        for w in self.all_users_scrollable_list.winfo_children():
            w.destroy()
        f_lower = filter_text.lower()
        for user in self.all_users_list:
            if user == self.username: continue
            if f_lower in user.lower():
                font = ctk.CTkFont(family=MESSAGE_TEXT_FONT_FAMILY,
size=MESSAGE_TEXT_FONT_SIZE - 1)
                btn = ctk.CTkButton(self.all_users_scrollable_list, text=user, anchor="w",
font=font,
                                command=lambda u=user: self.select_new_chat_partner(u))
                btn.pack(fill="x", padx=5, pady=2)

    def filter_user_list_display(self, event=None):
        if hasattr(self, 'new_chat_search_entry'):
self.populate_all_users_list(self.new_chat_search_entry.get())

    def select_new_chat_partner(self, partner_username):
        print(f"[CLIENT] select_new_chat_partner: Attempting to start chat with
'({partner_username})'") # Debug
        if not self.username:
            messagebox.showerror("Error", "You must be logged in to start a new chat.")
            return

        if partner_username == self.username:
            messagebox.showinfo("Info", "You cannot start a chat with yourself.")
            return

        # Ensure the partner is added to chat_histories if new
        if partner_username not in self.chat_histories:
            print(f"[CLIENT] Adding '{partner_username}' to chat_histories.")
            self.chat_histories[partner_username] = [] # Initialize with empty message list
            self.messages_currently_shown_count[partner_username] =
DEFAULT_MESSAGES_TO_LOAD_INITIALLY
            self.chat_all_older_loaded[partner_username] = True # No history to load
initially

        # --- Critical Change: Set current chat context BEFORE switching screens ---
        self.current_chat_identifier = partner_username
        self.current_chat_is_group = False
        self.current_chat_group_name = None # Not a group
        # The key status will be handled by select_chat called within show_chat_screen

        # Now, show_chat_screen will use the updated current_chat_identifier
        self.show_chat_screen()

        print(
            f"[CLIENT] select_new_chat_partner: Explicitly calling select_chat for
'({partner_username})' after show_chat_screen.")
        self.select_chat(partner_username, is_group=False)

# --- Main Execution ---
if __name__ == "__main__":
    app = ChatClientApp()
    app.mainloop()

```